

Data Engineering on Google Cloud — Day 2 of 2

Iowa Department of Management, Department of IT · October 6, 2026 · Student slide pack

- Slides follow the class agenda
- Hands-on activities: <https://roittraining.github.io/gcp-demos/bigquery.html>
- Labs are in the Google Skills classroom

Custom slide · added by Imran Ahmad

Iowa DOM DoIT · Data Engineering on Google Cloud

INSTRUCTOR NOTES — PAGE 1 OF 208 · COVER & AGENDA

FACILITATOR NOTE

- Day 2 turns yesterday's mechanics into their target operating model: lakehouse vocabulary, the medallion, and the Power BI last mile.
- Before starting, point at the logistics lines: hands-on demos at roittraining.github.io/gcp-demos/bigquery.html, labs in the Google Skills classroom.
- Collect overnight parking-lot questions and confirm any M5/M6 spillover to absorb during the 09:00 slot.

Flow: First slide of Day 2; next come the two agenda cards, then the 09:00 recap quiz.

Agenda — Tuesday, October 6 (1 of 2) · morning

09:00–09:15	Recap quiz — which GCP tool replaces which piece of your stack? (+ Day-1 spillover)	QUIZ
09:15–09:45	Module 7 — Intro to Modern Data Engineering: lake vs warehouse vs lakehouse	
09:45–10:30	Module 8 — Building a Data Lakehouse (break 10:30–10:45)	
10:45–11:15	Federated Query with BigQuery — the linked-server pattern, live	LAB
11:15–11:35	Exploring external tables — Bronze made tangible	DIN
11:35–12:00	Module 9 — Modernizing Data Warehouses with BigQuery & BigLake	

Custom slide · added by Imran Ahmad

Iowa DOM DoIT · Data Engineering on Google Cloud

INSTRUCTOR NOTES — PAGE 2 OF 208 · COVER & AGENDA

Hook

Yesterday you loaded and transformed inside BigQuery; this morning we name that architecture — lake, warehouse, lakehouse — in your own architecture doc's vocabulary.

Say · cover

- 09:15 M7: lake vs warehouse vs lakehouse isn't trivia — your Bronze/Silver/Gold doc IS a lakehouse; run the deck's built-in quiz as a group.
- 09:45 M8: GCS + open formats = Bronze; compress AlloyDB to one line — OLTP sibling, not your path.
- 10:45 lab: federated query = the linked-server pattern, live — `EXTERNAL_QUERY` to Cloud SQL; read-only, source takes the load, cross-region cap 1 TB/day.
- 11:15 DIN: external tables on hive-partitioned Parquet make Bronze tangible — create, query, filter one partition.
- 11:35 M9: read slides 4–5 'traditional DW challenges' as your SQL Server pain points; partitioning/clustering previews the 3-day DW course.

Iowa tie

Morning = the Bronze half of the migration: GCS landings, open formats, query-in-place before modeling — replacing county file drops and linked servers.

Design patterns

- Anchor every lakehouse term to their doc: Bronze=GCS/open formats, Silver/Gold=BigQuery.
- State the federate-vs-load trade-off: no copy, always current vs BigQuery storage optimizations.

Anti-patterns

- Don't pitch federation as a load replacement — the source absorbs every query.
- Don't drill AlloyDB; not their path.

History & what's new

In November 2025 the BigLake metastore's Iceberg REST Catalog went GA, letting Spark and other engines share one Iceberg catalog with BigQuery (Google Cloud blog, 2025).

Flow

After the cover; previews 09:15–12:00, next page covers the afternoon, then the 09:00 quiz.

Links

<https://cloud.google.com/bigquery/docs/biglake-intro>

<https://cloud.google.com/bigquery/docs/federated-queries-intro>

<https://cloud.google.com/bigquery/docs/external-data-sources>

Agenda — Tuesday, October 6 (2 of 2) · afternoon

12:00–13:00	Lunch	
13:00–13:45	Querying External Data & Apache Iceberg Tables — the Bronze-layer lab	LAB
13:45–14:00	Iceberg deep-dive + the BigLake table upgrade	DEMO
14:00–14:35	Module 10 — Governance + the Medallion: Google's Bronze/Silver/Gold next to yours (break 14:35–14:50)	
14:50–15:20	Module 11 — BQML + the Power BI last mile (compressed)	
15:20–16:30	Capstone workshop — map one of YOUR extract jobs; course summary; flex, Q&A & evaluations	WORKSHOP

Custom slide · added by Imran Ahmad

Iowa DOM DoIT · Data Engineering on Google Cloud

INSTRUCTOR NOTES — PAGE 3 OF 208 · COVER & AGENDA

Hook

This afternoon projects your architecture doc onto Google's slides: Bronze/Silver/Gold zones, and Gold marts landing in the Power BI you already own.

Say · cover

- 13:00 lab — External Data & Iceberg tables: the Bronze-layer lab that matters most; roam for T-SQL->GoogleSQL dialect questions.
- 13:45 demo: Iceberg managed tables (formerly BigLake Iceberg) — BigQuery-managed, data stays Parquet in your bucket; verify tutorial status, fallback is a BigLake upgrade demo.
- 14:00 M10: slides 43–48 are the moment — Google's medallion zones next to your doc, 'your design, in their deck'; governance slides = the single-source-of-truth requirement.
- 14:50 M11 compressed: show BQML exists, don't drill; spend the minutes on Power BI — Import vs DirectQuery, service-account auth, Gold marts only.
- 15:20 capstone: groups of 3–4 map one real DOM extract job source-to-Power-BI; 20 min build, 10 share-out — the artifact they take back to work.

Iowa tie

Afternoon = Silver->Gold discipline plus the last mile: governed single source of truth, Dataform-built marts, and the connector closing extracts->BQ->medallion->Power BI.

Design patterns

- Power BI touches Gold only — never Bronze or Silver.
- Capstone uses single-org safety: real job names, real tables.

Anti-patterns

- Don't let BQML eat the Power BI segment — nothing in the brief points at ML.
- Don't burn the 16:10 flex/Q&A/evaluations buffer.

History & what's new

Microsoft's Power BI connector for Google BigQuery (Entra ID, GA) moved to a new BigQuery ADBC-driver-based implementation in July 2025 (Microsoft Learn) — a better DirectQuery story for their last mile.

Flow

Second agenda card, previewing 13:00–16:30; the next page's 09:00 quiz opens the day.

Links

<https://cloud.google.com/bigquery/docs/iceberg-tables>

<https://cloud.google.com/dataplex/docs/overview>

<https://learn.microsoft.com/en-us/power-query/connectors/google-bigquery-aad>

QUIZ

Day-1 recap — which GCP tool replaces which piece of your stack?

15 minutes — 09:00–09:15 · Group warm-up — confer at your table, answer aloud; nothing scored

1. Nightly extract jobs → which service? SQL Server Agent → which scheduler? SSIS → what?
2. File drops → where do they land first, what loads them? Linked servers → the BigQuery way?
3. T-SQL procedures → where does that discipline live? SSAS cubes → the analogue?
4. The 60M-row race: CSV ~2 min vs Parquet seconds — the lesson? A query bills on what?
5. Bronze, Silver, Gold — one sentence each, in your target design

Why it matters — retrieval practice in your own vocabulary: name what each piece of your SQL Server stack becomes on Google Cloud — the map for the next two days.

Custom slide · added by Imran Ahmad

Iowa DOM DoIT · Data Engineering on Google Cloud

INSTRUCTOR NOTES — PAGE 4 OF 208 · 09:00 · QUIZ — DAY-1 RECAP: YOUR STACK -> GCP (15 MIN)

Hook

Fifteen no-stakes minutes: prove Day 1 stuck — name what each piece of your SQL Server stack becomes on Google Cloud.

Say · cover

- Mechanics: question aloud, ~30 seconds table talk, one answer per group; nothing scored — retrieval practice.
- Q1–3: extracts->Datastream (log-based CDC, no production load); SQL Agent->Cloud Scheduler or a one-statement scheduled query; SSIS->Dataform for SQL ELT, Dataflow as escape hatch.
- Q4–5: file drops land in Cloud Storage (Bronze) via bq load / LOAD DATA / DTS; linked servers->federated queries or external tables — no copy, but the source takes the load.
- Q6–7: T-SQL discipline->Dataform (SQLX, assertions, git, your CI/CD); SSAS cubes->materialized views and Dataform-built Gold marts.
- Q8–10: columnar formats win (CSV ~2 min vs Parquet seconds); queries bill on bytes scanned, loads are free; Bronze raw, Silver conformed, Gold marts for Power BI.
- Absorb any M5/M6 spillover here; park deep questions for their module.

Iowa tie

The migration map spoken aloud — Agent->Scheduler, SSIS->Dataform, cubes->marts — the service mapping their extract-kill project needs before the lakehouse modules.

Design patterns

- Reuse the answer key as the capstone's service-mapping table.
- Retrieve in the audience's vocabulary first, product names second.

Anti-patterns

- Don't answer for the tables — wait out the silence.
- Rotate who answers; don't let one voice carry the room.

History & what's new

Datastream supports SQL Server as a CDC source; its November 17, 2025 release notes added partitioning and clustering for BigQuery destinations — replicas land pre-optimized.

Flow

Opens Day 2 at 09:00 after the agenda; hands off to M7 (lake vs warehouse vs lakehouse) at 09:15.

Links

<https://cloud.google.com/datastream/docs/overview>

<https://cloud.google.com/bigquery/docs/scheduling-queries>

<https://cloud.google.com/dataform/docs/overview>

M7 — What this block gets you

Lake, warehouse, lakehouse: the vocabulary of your own architecture doc.

- ✓ The industry words for what you are already building— lake, warehouse, lakehouse
- ✓ Why warehouses alone and lakes alone both fail— and why the lakehouse won
- ✓ Every term mapped to your target design— SQL Server DW → BigQuery, Bronze/Silver/Gold
- ✓ Why 'open' formats (Parquet, Iceberg) are your insurance against lock-in
- ✓ The deck's built-in quiz, run as a group— if you can answer it, you can defend the design doc

Custom slide · added by Imran Ahmad

Iowa DOM DoIT · Data Engineering on Google Cloud

INSTRUCTOR NOTES — PAGE 5 OF 208 · 09:15 · MODULE 7 — INTRO TO MODERN DATA ENGINEERING

Hook

You already chose a lakehouse — Bronze/Silver/Gold on BigQuery. This block hands you the industry vocabulary to defend that choice in your architecture doc.

Say · cover

- Lake, warehouse, lakehouse: the three nouns your target-design document must define precisely.
- Pure lakes and pure warehouses both failed at full scope — that failure is why the lakehouse won.
- Map every term live: SQL Server DW -> BigQuery; file drops -> GCS Bronze; cubes -> Gold marts.
- Open formats (Parquet, Iceberg) are the anti-lock-in insurance clause in your design.
- The deck's built-in quiz runs as a group — answering it means you can defend the doc.

Iowa tie

Your Bronze->Silver->Gold target gets its industry name here: a lakehouse — GCS raw landings, BigQuery curated layers, Dataform transforming between.

Design patterns

- Name the pattern in the design doc before naming products.
- Tie every vocabulary term to a component you will actually build.

Anti-patterns

- Buzzword architecture: 'lakehouse' on a slide with no mapped components behind it.

History & what's new

Databricks popularized 'lakehouse' in January 2020; Armbrust et al. formalized it in the CIDR 2021 paper. Google's Iceberg-based open lakehouse added a GA Iceberg REST catalog in BigLake metastore in late 2025.

Flow

The 09:00 recap quiz just proved Day 1 stuck; next, the morning roadmap, the quiz key, then deck content.

Links

<https://cloud.google.com/learn/what-is-a-data-lake>

<https://cloud.google.com/learn/what-is-a-data-warehouse>

<https://cloud.google.com/blog/products/data-analytics/biglake-metastore-now-supports-iceberg-rest-catalog>

Day 2, first block — 09:00 to 10:30

09:00–09:15	Recap quiz — which GCP tool replaces which piece of your current stack?	QUIZ
09:15–09:45	M7 — Intro to modern DE: lake vs warehouse vs lakehouse (deck quiz pages at the end run as a group)	
09:45–10:30	M8 — Building a data lakehouse: GCS, open formats, Iceberg, federated queries	
10:30–10:45	Break	

Custom slide · added by Imran Ahmad

Iowa DOM DoIT · Data Engineering on Google Cloud

INSTRUCTOR NOTES — PAGE 6 OF 208 · 09:15 · MODULE 7 — INTRO TO MODERN DATA ENGINEERING

Hook

Yesterday you touched the tools; this morning you name the architecture — then touch GCS, Iceberg, and federated queries before the break.

Say · cover

- 09:00 recap quiz: which GCP service replaces each piece of your SQL Server stack — retrieval practice, nothing scored.
- 09:15 M7: lake vs warehouse vs lakehouse — the vocabulary of your target design doc.
- 09:45 M8: building the lakehouse — GCS, open formats, Iceberg, federated queries; that is your Bronze foundation.
- The deck's quiz pages at the end of M7 run as a group — one org, sixteen people, safe and lively.
- 10:30 break, then straight into the Federated Query lab — the lakehouse promise made hands-on.

Iowa tie

These 90 minutes are the theory half of your Bronze design: M8's GCS and open-formats content is literally the layer your extracts will land in.

Design patterns

- Vocabulary before products, products before hands-on — the morning is sequenced that way deliberately.

Anti-patterns

- Don't skip the deck quiz to save time — it is the confidence check before GCS gets hands-on.

History & what's new

The arc you teach this morning is current-generation: Databricks coined 'lakehouse' in 2020, and Google's BigLake metastore shipped a GA Iceberg REST catalog in November 2025 — this is not legacy material.

Flow

Module overview slide just set the frame; this roadmap previews M7 and M8 before the quiz key and the deck pages.

Links

<https://docs.cloud.google.com/bigquery/docs/biglake-iceberg-tables-in-bigquery>
<https://docs.cloud.google.com/bigquery/docs/federated-queries-intro>
<https://docs.cloud.google.com/storage/docs/introduction>

QUIZ

Which GCP tool replaces which piece of your current stack?

15 minutes · Whole room — confer at your table

1. Instructor reads each question aloud — 10 questions, no trick ones
2. Confer with your table for about 30 seconds per question
3. One person answers for the group — nothing is scored
4. Name, out loud, what each piece of your SQL Server stack becomes on Google Cloud
5. Jot your answers down — the key is on the next slide

Why it matters — retrieval practice before the lakehouse modules start — proving Day 1 stuck, in your own vocabulary, not Google's.

Custom slide · added by Imran Ahmad

Iowa DOM DoIT · Data Engineering on Google Cloud

INSTRUCTOR NOTES — PAGE 7 OF 208 · 09:15 · MODULE 7 — INTRO TO MODERN DATA ENGINEERING

Hook

Ten questions, no tricks: name what each piece of your SQL Server stack becomes on Google Cloud — in your vocabulary, not Google's.

Say · cover

- Read each question aloud; give tables about 30 seconds to confer, then take one answer for the group.
- Nothing is scored — saying the mapping out loud is the retention mechanism, so cold-call every table at least once.
- Push for their words first ('nightly extracts', 'the Agent jobs'), then attach the GCP name (Datastream, Cloud Scheduler).
- The 60M-row race question recalls yesterday's demo: CSV ~2 minutes vs Parquet in seconds — and a query bills on bytes scanned.
- Have them jot answers; the key is the next slide — debrief each row in one breath, no re-teaching.

Iowa tie

Every answer is a line from your migration: extracts->Datastream, SQL Agent->Scheduler, SSIS->Dataform, linked servers->federated queries, SSAS->materialized views.

Design patterns

- Retrieval practice before new content; whole-room conferring suits a 16-person single-org class.

Anti-patterns

- Don't lecture the answers — let tables produce them, then confirm.

History & what's new

The replacements are production-proven: Datastream went generally available in December 2021 for exactly these CDC workloads, and Google acquired Dataform in December 2020 to make SQL-first ELT native to BigQuery.

Flow

Quiz questions are on p004; this page is the running mechanics — the answer key follows.

Links

<https://docs.cloud.google.com/datastream/docs/overview>
<https://docs.cloud.google.com/dataform/docs/overview>
<https://docs.cloud.google.com/scheduler/docs/overview>

Quiz key — your stack, translated

The short version of yesterday; the deck quiz at the end of this module checks the new vocabulary.

- Nightly extract jobs— Datastream change data capture, replicating continuously
- SQL Server Agent— Cloud Scheduler; cheapest version is a BigQuery scheduled query
- SSIS packages— Dataform for SQL-first ELT; Dataflow only when SQL is not enough
- Staging file drops— land in Cloud Storage (Bronze), load via bq load, LOAD DATA, or DTS
- Linked servers— federated queries and external tables, no copy, always current
- SSAS cubes— materialized views and Gold marts, built and scheduled by Dataform

Custom slide · added by Imran Ahmad

Iowa DOM DoIT · Data Engineering on Google Cloud

INSTRUCTOR NOTES — PAGE 8 OF 208 · 09:15 · MODULE 7 — INTRO TO MODERN DATA ENGINEERING

Hook

Six rows, one message: everything in your SQL Server stack has a named GCP successor — nothing you do today is orphaned.

Say · cover

- Nightly extracts -> Datastream CDC: continuous replication replaces the batch window.
- SQL Server Agent -> Cloud Scheduler; the cheapest scheduled job is a BigQuery scheduled query.
- SSIS -> Dataform for SQL-first ELT; Dataflow only when SQL genuinely cannot express the transform.
- Staging file drops -> land in Cloud Storage (Bronze); load via bq load, LOAD DATA, or DTS.
- Linked servers -> federated queries and external tables: no copy, always current.
- SSAS cubes -> materialized views and Gold marts, built and scheduled by Dataform.

Iowa tie

Pin this slide — it is the migration map for your whole project and the seed of the 15:20 capstone workshop on one of your own extract jobs.

Design patterns

- Keep a one-to-one service mapping table in the design doc; prefer serverless successors over lifted VMs.

Anti-patterns

- Don't reach for Dataflow first — SQL-first Dataform keeps your ten SQL developers productive.

History & what's new

Google's own migration guides use this same row-by-row mapping style, and the Datastream GA announcement (December 2021) made SQL Server a first-class CDC source — your exact scenario.

Flow

Quiz closed and stack map confirmed — now the deck's own narrative starts: two storage methods.

Links

<https://docs.cloud.google.com/bigquery/docs/federated-queries-intro>
<https://docs.cloud.google.com/bigquery/docs/materialized-views-intro>
<https://docs.cloud.google.com/bigquery/docs/batch-loading-data>

For years, organizations have relied on two primary methods for large-scale data storage: **data lakes and data warehouses.**

Google Cloud

INSTRUCTOR NOTES — PAGE 9 OF 208 · 09:15 · MODULE 7 — INTRO TO MODERN DATA ENGINEERING

Hook

You have lived one of these for a decade — the SQL Server warehouse. The other, the lake, is what your Bronze layer is about to become.

Say · cover

- The deck's framing: large-scale storage has meant data lakes and data warehouses — two answers to two different problems.
- Warehouses answer known business questions fast; lakes keep everything cheap for questions not yet asked.
- Your nightly extracts exist precisely because one system — the DW — was asked to do both jobs.
- Cymbal, the deck's running example, is an e-commerce company; swap in a state agency as we go.
- Preview: the next dozen slides give each model its strengths, then its failure mode — keep score.

Iowa tie

Your SQL Server DW is the warehouse half; GCS Bronze is the lake half — this module explains why the target connects both instead of picking one.

Design patterns

- Use the two-model framing to explain to stakeholders why the target design contains both GCS and BigQuery.

Anti-patterns

- Don't present lake vs warehouse as either/or — each model's solo failure mode fills the next slides.

History & what's new

Warehouses came first: Devlin and Murphy coined the 'business data warehouse' at IBM in 1988. The data lake arrived 22 years later — Pentaho CTO James Dixon coined it in an October 2010 blog post.

Flow

Quiz key closed; this is the first deck content page — the two-model framing everything else unpacks.

Links

<https://cloud.google.com/learn/what-is-a-data-lake>

<https://cloud.google.com/learn/what-is-a-data-warehouse>

<https://docs.cloud.google.com/bigquery/docs/introduction>

This is the vocabulary of your own architecture doc.

Frame for the next 30 minutes — lake, warehouse and lakehouse are the words in the DOM DoIT target design, not textbook terms.

Custom slide · added by Imran Ahmad

Iowa DOM DoIT · Data Engineering on Google Cloud

INSTRUCTOR NOTES — PAGE 10 OF 208 · 09:15 · MODULE 7 — INTRO TO MODERN DATA ENGINEERING

Hook

For the next 30 minutes, every slide is a paragraph in the design doc you will write for the BigQuery migration — treat it as dictation, not theory.

Say · cover

- Lake, warehouse, lakehouse: the three nouns your target-state document must define before any product names appear.
- When the deck says 'data lake', hear 'GCS Bronze with lifecycle policies'.
- When it says 'data warehouse', hear 'BigQuery Silver and Gold, queried by Power BI'.
- When it says 'lakehouse', hear your whole Bronze->Silver->Gold pattern running on open formats.
- Write the mapping down as we go — you will reuse it in the 15:20 capstone workshop.

Iowa tie

This frame is the entire point of M7 for DOM DoIT: your Bronze/Silver/Gold plan is a lakehouse, and these slides are its justification section.

Design patterns

- Architecture docs should define terms against a shared industry vocabulary, not invent local ones.

Anti-patterns

- Don't let the Cymbal e-commerce examples stay retail — translate each scenario to a state-agency one aloud.

History & what's new

Google formalized this same vocabulary in its own docs: the 2026 'borderless Lakehouse' documentation describes one governed interface across lake and warehouse, built on Apache Iceberg.

Flow

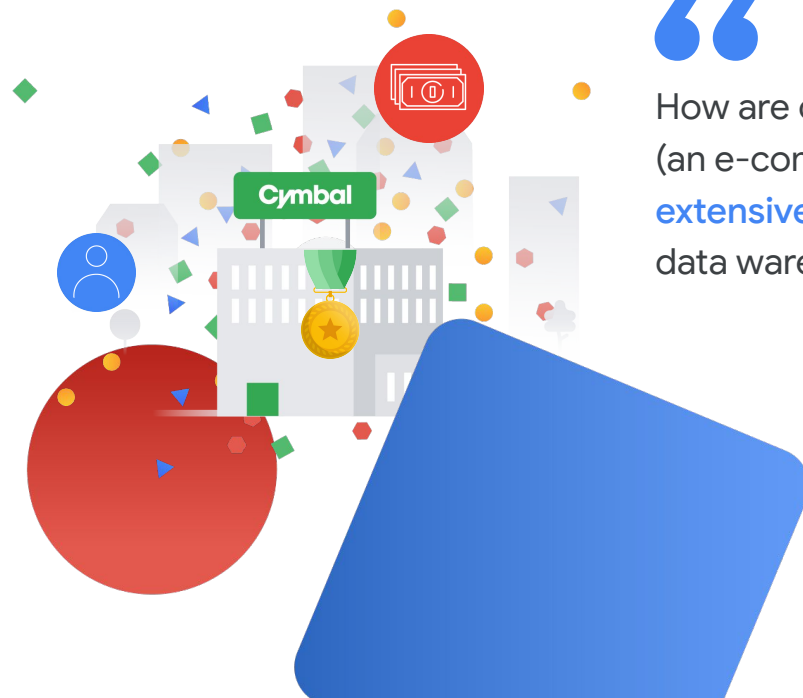
Custom framing page after the two-model opener; next, the deck's Cymbal discussion prompt.

Links

<https://docs.cloud.google.com/bigquery/docs/lakehouse-in-bigquery>

<https://cloud.google.com/products/lakehouse>

<https://cloud.google.com/learn/what-is-a-data-lake>



How are organizations, such as Cymbal (an e-commerce company), **managing extensive datasets** with data lakes and data warehouses?



Google Cloud

INSTRUCTOR NOTES — PAGE 11 OF 208 · 09:15 · MODULE 7 — INTRO TO MODERN DATA ENGINEERING

Hook

Swap 'Cymbal e-commerce' for 'a state agency with decades of program data' — the question is identical: what goes in the lake, what goes in the warehouse?

Say · cover

- This is a discussion prompt, not a quiz — ask one table to argue lake-first, another warehouse-first.
- Cymbal is Google Cloud's stock fictional company, the way AdventureWorks was Microsoft's — don't linger on it.
- 'Extensive datasets' means mixed shapes: transactions, logs, documents — no single store fits all of them well.
- Steer the room to the synthesis: raw landings in the lake, curated reporting in the warehouse.
- Hold discussion to 60–90 seconds — the next slides hand them the answer in deck language.

Iowa tie

Your version of the answer: legacy extract files and agency source systems land raw in Bronze (GCS); certified reporting tables live in Gold (BigQuery) for Power BI.

Design patterns

- Scenario-first teaching: let the room derive the lake/warehouse split before the deck shows it.

Anti-patterns

- Don't let the debate run past two minutes — the deck resolves it; your job is steering, not judging.

History & what's new

The two-answer problem is genuinely old: warehouses descend from IBM's 1988 'business data warehouse' paper, lakes from Dixon's 2010 post — Cymbal just inherits both traditions at once.

Flow

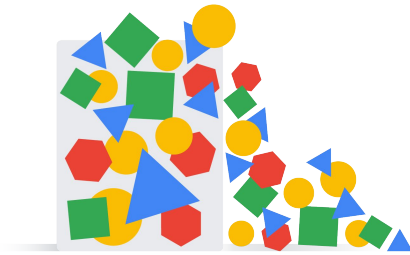
Framing slide done; this prompt opens the deck's lake section — the reservoir definition is next.

Links

<https://cloud.google.com/learn/what-is-a-data-lake>
<https://cloud.google.com/learn/what-is-a-data-warehouse>
<https://cloud.google.com/customers>

Data lakes

Think of a data lake as a vast reservoir of data. It stores enormous amounts of raw data in its native format.



01 Structured data

02 Semistructured data

03 Unstructured data

Google Cloud

INSTRUCTOR NOTES — PAGE 12 OF 208 · 09:15 · MODULE 7 — INTRO TO MODERN DATA ENGINEERING

Hook

This slide is your Bronze layer's job description: a vast reservoir holding raw data in native format — structured, semistructured, unstructured.

Say · cover

- A lake stores enormous volumes of raw data in native format — no cleansing contract at the door.
- Three tenants: structured rows, semistructured JSON/CSV logs, unstructured PDFs and images.
- On Google Cloud the reservoir is Cloud Storage: buckets, objects, lifecycle rules — show it in the Console later.
- Full fidelity is the point: raw survives, so you can re-derive Silver differently in five years.
- 'Native format' today usually means Parquet — not the CSVs your current extracts produce.

Iowa tie

Bronze = a GCS bucket per domain landing raw extracts as-is — the reservoir your Dataform pipelines then refine into Silver and Gold.

Design patterns

- Land raw first and keep it immutable; one bucket per domain and environment, with lifecycle policies.

Anti-patterns

- Don't transform on landing — any cleansing in Bronze destroys the reprocessing option.

History & what's new

Dixon's 2010 metaphor was deliberate: data marts are bottled water — cleansed and packaged; the lake is a large body of water in its natural state. Cloud Storage's ancestor, Google Storage for Developers, previewed that same year (May 2010).

Flow

Cymbal prompt discussed; the lake is now defined — schema-on-read mechanics come next.

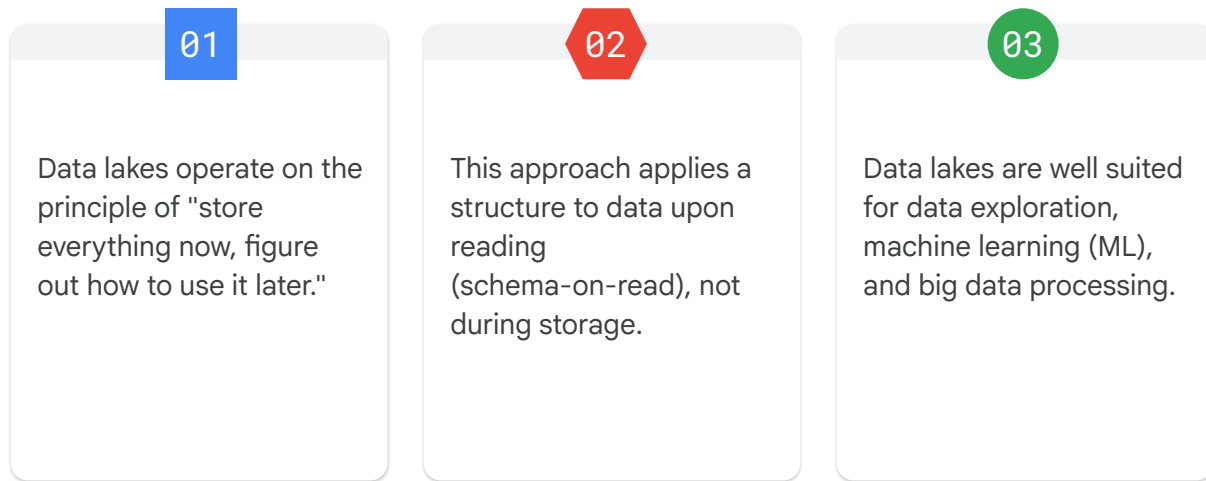
Links

<https://cloud.google.com/learn/what-is-a-data-lake>

<https://docs.cloud.google.com/storage/docs/introduction>

<https://docs.cloud.google.com/storage/docs/storage-classes>

Data lakes



Google Cloud

INSTRUCTOR NOTES — PAGE 13 OF 208 · 09:15 · MODULE 7 — INTRO TO MODERN DATA ENGINEERING

Hook

Schema-on-read is the opposite of your SQL Server discipline — and that is exactly why Bronze can land anything without a design meeting first.

Say · cover

- 'Store everything now, figure out how to use it later': structure is applied at query time, not at write time.
- No upfront modeling means ingestion is fast and nothing is rejected — that agility is the point of the lake.
- The schema lives in the reader: BigQuery external tables, autodetect on load, or a Spark job.
- Contrast coming next section: the warehouse is schema-on-write — it validates before storing.
- Recall yesterday's autodetect pitfall DIN: schema-on-read still needs a reader that guesses well.

Iowa tie

Your Bronze files get structure when Dataform declares them as sources — the .sqlx definitions are your schema-on-read layer, versioned in Git.

Design patterns

- Declare schemas explicitly in the transform layer; never trust autodetect for production reads.

Anti-patterns

- Schema-on-read is not schema-never — undecoded raw piles are how swamps form.

History & what's new

The idea traveled with Hadoop: Google's GFS and MapReduce papers inspired Hadoop's store-first era (2006–2012), normalizing schema-on-read years before Dixon named the lake in 2010.

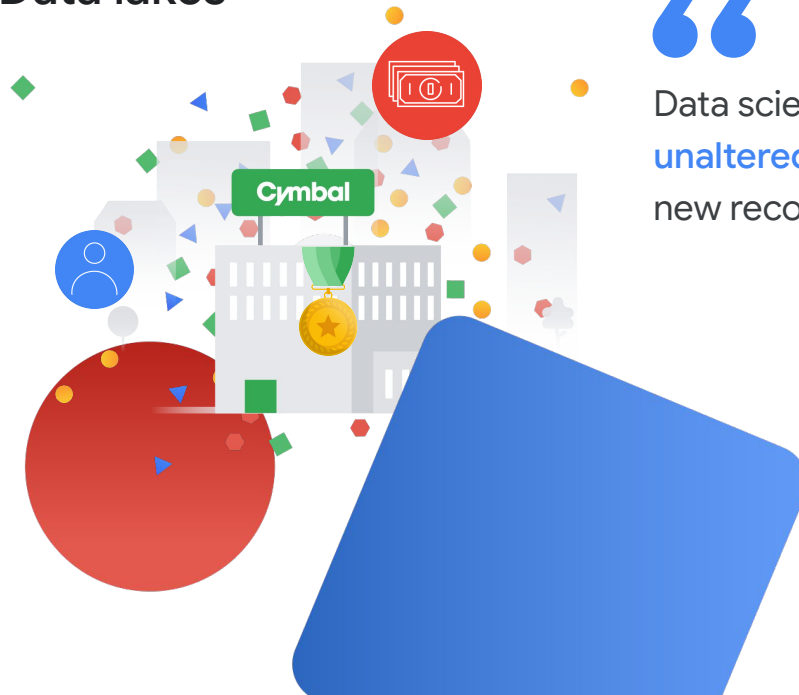
Flow

Lake definition and read mechanics done; next, the Cymbal scenario that justifies keeping raw clickstream.

Links

<https://docs.cloud.google.com/bigquery/docs/external-data-sources>
<https://docs.cloud.google.com/bigquery/docs/federated-queries-intro>
<https://cloud.google.com/learn/what-is-a-data-lake>

Data lakes



Data scientists need access to original, **unaltered clickstream data** to build a new recommendation engine.



Google Cloud

INSTRUCTOR NOTES — PAGE 14 OF 208 · 09:15 · MODULE 7 — INTRO TO MODERN DATA ENGINEERING

Hook

Your analogue: a data science team asking for unaltered case-level data, not the aggregated tables the DW publishes — the lake is where that request lands.

Say · cover

- Cymbal's data scientists need original, unaltered clickstream data to build a recommendation engine.
- Aggregation destroys signal: models need the very events the warehouse discarded as noise.
- Lakes serve exploration and ML training precisely because raw fidelity is preserved.
- On GCP: read Bronze straight from BigQuery via external tables, or load to managed tables for BigQuery ML.
- Same split as your consumers: Power BI wants curated Gold; a future ML or audit question wants governed raw.

Iowa tie

Bronze keeps full-fidelity landings so tomorrow's ML, audit, or open-records questions don't depend on what today's extract job decided to keep.

Design patterns

- Keep raw alongside curated: ML reads Bronze, BI reads Gold, one governance umbrella over both.

Anti-patterns

- Don't force ML consumers to reverse-engineer aggregated marts — give them governed access to raw instead.

History & what's new

The canonical case: Netflix offered \$1M in October 2006 for a 10% better recommendation algorithm — the Netflix Prize, won September 2009 by BellKor's Pragmatic Chaos. Raw viewing data was the fuel.

Flow

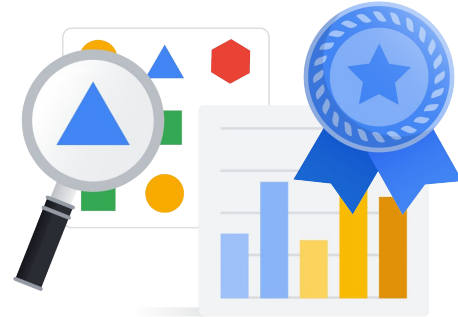
The lake's purpose is now concrete; next slide itemizes its five advantages.

Links

<https://docs.cloud.google.com/bigquery/docs/bqml-introduction>
<https://docs.cloud.google.com/bigquery/docs/introduction>
<https://cloud.google.com/learn/what-is-a-data-lake>

Data lakes: Advantages

- 01 **Flexibility:** Stores all data types.
- 02 **Agility:** Fast to ingest data.
- 03 **Scalability:** Can grow to exabyte scale.
- 04 **Cost-effectiveness:** Uses low-cost object storage.
- 05 **Support for advanced analytics:** Ideal for AI/ML model training.



Google Cloud

 INSTRUCTOR NOTES — PAGE 15 OF 208 · 09:15 · MODULE 7 — INTRO TO MODERN DATA ENGINEERING

Hook

Read these five as the argument for Bronze: cheap, schema-free object storage that ingests anything today and feeds AI tomorrow.

Say · cover

- Flexibility: every data type is welcome — no redesign when a new source system appears.
- Agility: ingestion is a file drop — no modeling gate, so landings happen the same day.
- Scalability: object storage grows to exabytes with zero capacity planning on your side.
- Cost-effectiveness: GCS storage classes are far cheaper than database storage — match the class to access frequency.
- Advanced analytics: raw data in place is exactly what ML training (BigQuery ML, Spark) consumes.

Iowa tie

This is why the extracts die: landing raw in GCS is cheaper and faster than maintaining per-agency staging databases — and nothing is thrown away.

Design patterns

- Use lifecycle rules (Standard->Nearline->Coldline->Archive) so cost follows access frequency automatically.

Anti-patterns

- Cheap storage tempts hoarding — pair every landing with an owner and a retention rule.

History & what's new

Object storage economics made lakes viable: Google Storage for Developers previewed in May 2010 and became Google Cloud Storage in 2011 — the same substrate that now underpins Google's Iceberg lakehouse.

Flow

The lake story peaks here — next slide is the hangover: swamps, management overhead, security risk.

Links

<https://docs.cloud.google.com/storage/docs/introduction>
<https://docs.cloud.google.com/storage/docs/storage-classes>
<https://cloud.google.com/storage/pricing>

Data lakes: Disadvantages

01

Risk of becoming a "data swamp": Can become a disorganized collection of unusable data.

02

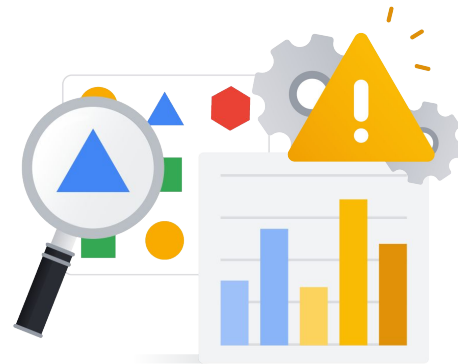
Management complexity: Requires significant overhead to maintain.

03

Time-consuming analysis: Data needs to be cleaned and wrangled before it is usable.

04

Security risks: Raw data formats can increase security and compliance risks.



Google Cloud

 INSTRUCTOR NOTES — PAGE 16 OF 208 · 09:15 · MODULE 7 — INTRO TO MODERN DATA ENGINEERING

Hook

This slide is the cautionary tale your M&O team already suspects: without governance, Bronze becomes a swamp nobody dares drink from.

Say · cover

- Data swamp: unowned, undocumented files accumulate until the lake is unusable.
- Management complexity: partitioning, retention, formats, permissions — someone has to run all of it.
- Time-consuming analysis: every question starts by cleaning and wrangling raw files.
- Security and compliance: raw copies multiply sensitive data; PII needs discovery and policy, not hope.
- Google's answer: Knowledge Catalog (formerly Dataplex) for discovery, quality, and policy across GCS and BigQuery.

Iowa tie

This is why your design has Silver and Gold plus a Dataform pipeline — and why M&O owns lifecycle rules and access policy, not just the storage bill.

Design patterns

- Catalog everything at landing, assign owners, and automate quality checks before data reaches Silver.

Anti-patterns

- 'Dump it in the lake, govern later' — later never comes; that is the swamp recipe.

History & what's new

Gartner's 2014 'Data Lake Fallacy' note warned of exactly this, and in 2016 Gartner predicted 90% of deployed lakes would be useless through 2018. The governance tooling that answered it — Dataplex — was renamed Knowledge Catalog in April 2026.

Flow

Lake case closed, good and bad — now pivot to the world they know: the warehouse as organized library.

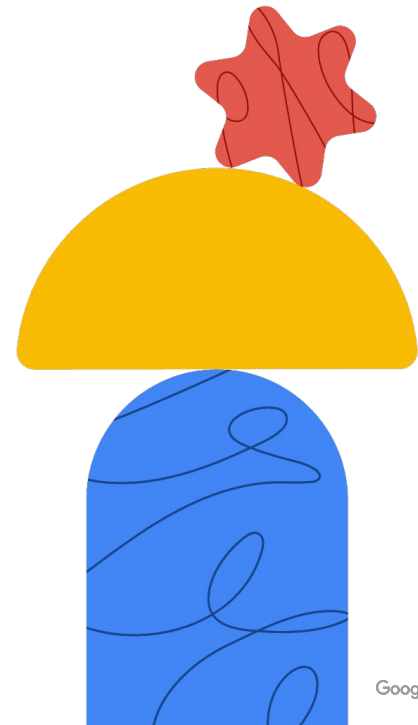
Links

<https://docs.cloud.google.com/knowledge-catalog/docs/introduction>

<https://cloud.google.com/blog/products/data-analytics/introducing-the-google-cloud-knowledge-catalog>

<https://cloud.google.com/learn/what-is-a-data-lake>

A **data warehouse** is like a highly organized library.



INSTRUCTOR NOTES — PAGE 17 OF 208 · 09:15 · MODULE 7 — INTRO TO MODERN DATA ENGINEERING

Hook

Home turf — your SQL Server DW is this slide: every book cataloged, shelved, and findable in seconds, because someone did the work up front.

Say · cover

- The library metaphor: books (data) are cleaned, cataloged, and shelved before anyone reads them.
- The cataloging work is your ETL — the cost is paid before the first query ever runs.
- Readers get speed and reliability; what they cannot do is shelve a random manuscript (a raw file).
- Ask the room: what is your library's catalog? Answer: the DW schema plus the SSIS packages that enforce it.
- BigQuery keeps the library promise but shelves itself — serverless, no indexes or hardware to babysit.

Iowa tie

Your current DW is the library; the migration moves it to BigQuery — same organized promise, without the M&O hardware burden.

Design patterns

- Curated, documented, contract-backed tables are the only surface BI tools should ever see.

Anti-patterns

- Don't let analysts use the warehouse as a landing zone — that is the lake's job.

History & what's new

The term predates most tooling: IBM's Barry Devlin and Paul Murphy published the 'business data warehouse' architecture in 1988; Bill Inmon's 'Building the Data Warehouse' (1992) turned it into a discipline.

Flow

From the lake's risks to the familiar warehouse — schema-on-write mechanics are next.

Links

<https://cloud.google.com/learn/what-is-a-data-warehouse>
<https://docs.cloud.google.com/bigquery/docs/introduction>
<https://docs.cloud.google.com/bigquery/docs/migration-intro>

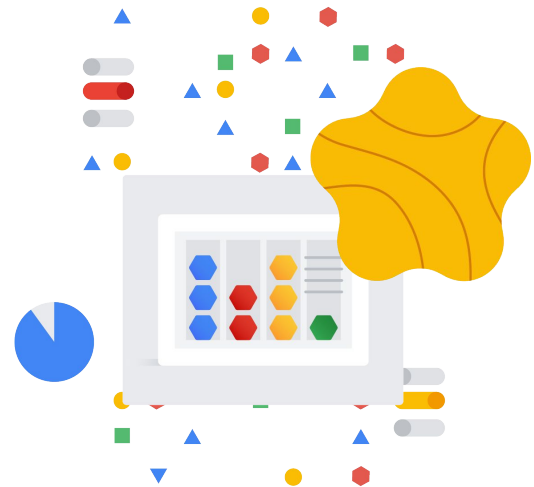
Data warehouses

01

Data is cleaned, transformed, and structured before storage.

02

This process, schema-on-write, optimizes data for analysis and business intelligence (BI).



Google Cloud

INSTRUCTOR NOTES — PAGE 18 OF 208 · 09:15 · MODULE 7 — INTRO TO MODERN DATA ENGINEERING

Hook

Schema-on-write is the discipline your SSIS packages enforce every night — validate first, store second; BigQuery keeps the same contract.

Say · cover

- Data is cleaned, transformed, and structured before storage — nothing enters unvalidated.
- Schema-on-write optimizes for analysis and BI: every query can trust the shape of every table.
- The transform cost is front-loaded; query time is where you get paid back.
- In BigQuery: declare schemas explicitly — or enforce them through Dataform declarations — not autodetect.
- Yesterday's CSV-vs-Parquet race proved it: typed, columnar storage is why the same query took seconds, not minutes.

Iowa tie

Silver->Gold in Dataform is schema-on-write: declared columns, types, and assertions before Power BI ever sees a table.

Design patterns

- Explicit schemas plus Dataform assertions as the write-time gate for everything in Gold.

Anti-patterns

- Relying on load-time autodetect for production tables — yesterday's pitfall DIN showed how that breaks.

History & what's new

Schema-on-write is Inmon's 1992 discipline industrialized. Google's twist was separating storage from compute: BigQuery (announced May 2010, GA November 2011) made the library's shelf-space effectively infinite.

Flow

Warehouse defined; next, why design must start from known questions — Cymbal's metrics example.

Links

<https://cloud.google.com/learn/what-is-a-data-warehouse>
<https://docs.cloud.google.com/bigquery/docs/schemas>
<https://docs.cloud.google.com/dataform/docs/overview>

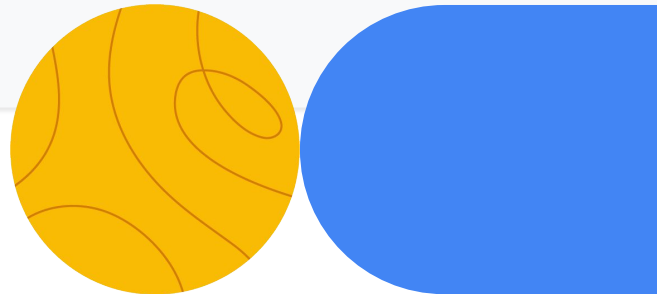
Data warehouses

Structure and quality first

The intended use of the data must be determined before the data warehouse is designed.

Cymbal example

- Key metrics, such as customer lifetime value and daily sales figures, are already defined.
- The data warehouse is then built to provide fast and accurate reports on these specific metrics.



INSTRUCTOR NOTES — PAGE 19 OF 208 · 09:15 · MODULE 7 — INTRO TO MODERN DATA ENGINEERING

Hook

You cannot build the warehouse before you know the questions — that is why your data-mart design workshops matter more than any tool choice.

Say · cover

- The intended use of the data must be determined before the warehouse is designed — questions first, schema second.
- Cymbal's example: customer lifetime value and daily sales are defined up front; the DW is then built to answer them fast.
- That is the star-schema instinct your team already has: facts and dimensions arranged around known metrics.
- The trade-off: genuinely new question types arrive slowly — that is the inflexibility slide coming up.
- Your Gold marts are exactly this: one mart per program area, Power BI-ready, shaped by Dataform.

Iowa tie

Gold is Cymbal's warehouse: certified metrics per agency and program, versioned in Git and deployed through your existing CI/CD pipelines.

Design patterns

- Model Gold marts backward from the BI questions; keep the models versioned in Git.

Anti-patterns

- Don't build marts shaped like source systems — source-shaped marts rot as soon as sources change.

History & what's new

Ralph Kimball's 'The Data Warehouse Toolkit' (1996) codified this question-first dimensional method — the same star-schema thinking your Power BI semantic models already assume.

Flow

Design-first principle stated; next slide is the payoff — speed and accuracy for reporting.

Links

<https://docs.cloud.google.com/dataform/docs/overview>
<https://docs.cloud.google.com/bigquery/docs/materialized-views-intro>
<https://cloud.google.com/learn/what-is-a-data-warehouse>

Data warehouses are perfect for generating business reports and dashboards where **speed and accuracy are critical**.

Google Cloud

INSTRUCTOR NOTES — PAGE 20 OF 208 · 09:15 · MODULE 7 — INTRO TO MODERN DATA ENGINEERING

Hook

This is your Power BI estate's reason to exist: dashboards where speed and accuracy are non-negotiable — the warehouse's home game.

Say · cover

- Reports and dashboards are the warehouse's design center: repeatable, trusted, fast.
- Speed comes from pre-modeled data plus an engine built for scans; accuracy comes from one certified copy.
- Your Power BI semantic models stay as-is; only the data source swaps SQL Server -> BigQuery.
- BigQuery feeds Power BI through the built-in connector; BI Engine can accelerate repeated dashboard queries.
- Single source of truth means every dashboard reads Gold — never a local extract copy.

Iowa tie

Kill-the-extracts ends on this slide: Power BI reads BigQuery Gold marts directly — no more CSV drops feeding reports.

Design patterns

- One certified Gold mart per domain, and every BI tool points at it.

Anti-patterns

- Letting departments keep private extract copies 'just in case' — that rebuilds the silo you are killing.

History & what's new

Power BI went generally available on July 24, 2015; its native Google BigQuery connector supports both Import and DirectQuery modes and is documented by Microsoft today.

Flow

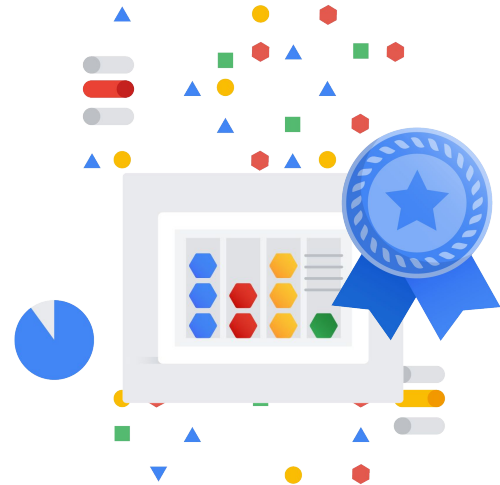
The warehouse's promise stated; next, its four advantages itemized.

Links

<https://cloud.google.com/learn/what-is-a-data-warehouse>
<https://learn.microsoft.com/en-us/power-query/connectors/google-bigquery>
<https://cloud.google.com/looker>

Data warehouses: Advantages

- 01 **Speed:** Optimized for high-performance queries.
- 02 **High-quality data:** Provides consistent and reliable information.
- 03 **Business focused:** Directly answers common business questions.
- 04 **Historical intelligence:** Enables deep analysis of trends over time.



Google Cloud

INSTRUCTOR NOTES — PAGE 21 OF 208 · 09:15 · MODULE 7 — INTRO TO MODERN DATA ENGINEERING

Hook

Read these four as your current DW's resume — the things BigQuery must match or beat for the migration to count as a win.

Say · cover

- Speed: engines optimized for high-performance analytical queries — BigQuery runs them serverless, at any concurrency.
- High-quality data: consistent, validated, one version of every number.
- Business focused: tables answer business questions directly, not source-system questions.
- Historical intelligence: years of trends in one place — your longitudinal program reporting depends on it.
- BigQuery extras: BI Engine for sub-second dashboards, materialized views for pre-computed aggregates.

Iowa tie

Make these four bullets your migration acceptance criteria: if BigQuery Gold does not beat the old DW on speed, quality, focus, and history, the project failed.

Design patterns

- Bake the four advantages into acceptance tests: query SLAs plus reconciliation checks against the old DW.

Anti-patterns

- Don't trade quality for speed at cutover — dual-run reconciliation before decommissioning anything.

History & what's new

BigQuery's speed descends from Dremel, Google's internal engine first presented at VLDB 2010; the 2020 'Dremel: A Decade' paper recounts how it became BigQuery, GA in November 2011.

Flow

Warehouse strengths banked; next, its four weaknesses — which are your current pain points.

Links

<https://docs.cloud.google.com/bigquery/docs/introduction>
<https://docs.cloud.google.com/bigquery/docs/bi-engine-intro>
<https://cloud.google.com/learn/what-is-a-data-warehouse>

Data warehouses: Disadvantages

01

Inflexibility: Cannot easily accommodate new data types or unstructured data.

02

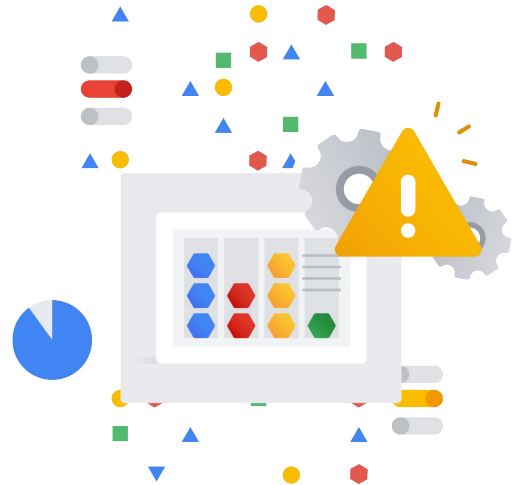
Cost: Often expensive to build and maintain.

03

Limited data types: Primarily designed for structured data only.

04

Long development time: Can take a long time to design and implement.



Google Cloud

INSTRUCTOR NOTES — PAGE 22 OF 208 · 09:15 · MODULE 7 — INTRO TO MODERN DATA ENGINEERING

Hook

Read this as your SQL Server DW's performance review: inflexible, expensive, structured-only, slow to change — the reasons you are in this room.

Say · cover

- Inflexibility: a new data type or source means redesign — change requests take months.
- Cost: licensing, hardware, and the M&O hours required just to keep it alive.
- Structured data only: PDFs, logs, and images never fit — so they live in extracts and file shares.
- Long development time: design-first means new questions queue behind projects.
- A cloud DW fixes cost and scale; the type-flexibility problem needs the lake half — that is the next slide's point.

Iowa tie

These four bullets are your legacy-extract problem statement: every nightly job exists because the DW alone could not hold or shape the data.

Design patterns

- Reuse this slide verbatim in the business case as the current-state pain list.

Anti-patterns

- Don't assume BigQuery alone fixes inflexibility — that takes the lake half and open formats.

History & what's new

Google's own 'What is a data warehouse?' page lists exactly these on-prem limits — rigid schemas, hardware cost, batch-only capture — as the forces pushing enterprises to cloud warehouses like BigQuery (GA 2011).

Flow

Both models' trade-offs are now on the table — next slide lands the punchline: neither alone is enough.

Links

<https://cloud.google.com/learn/what-is-a-data-warehouse>

<https://cloud.google.com/bigquery/pricing>

<https://docs.cloud.google.com/bigquery/docs/migration-intro>

Neither data lakes or data warehouses excel at supporting the **full range of an organization's data needs.**

Google Cloud

INSTRUCTOR NOTES — PAGE 23 OF 208 · 09:15 · MODULE 7 — INTRO TO MODERN DATA ENGINEERING

Hook

Here is the module's thesis and your project's justification: the lake cannot serve the board, the warehouse cannot hold the world — you need both, fused.

Say · cover

- Lakes lack governance and query speed; warehouses lack flexibility and cheap raw storage.
- The old fix — ETL copies from lake into the DW — is precisely the extract sprawl you live with today.
- Two copies mean drift, double storage, and a doubled security surface.
- The lakehouse answer: one open copy of the data with warehouse-grade management on top — M8 builds it next.
- Your Bronze->Silver->Gold design is this slide's resolution, drawn as an architecture.

Iowa tie

This single slide is the 'why' section of your migration business case: the SQL Server DW alone failed, a lake alone would fail — hence the lakehouse target.

Design patterns

- Position the lakehouse as removing copies, not adding a third silo.

Anti-patterns

- Beware 'lake plus warehouse with ETL between them' — that architecture is the problem, not the solution.

History & what's new

The industry named the fix when Databricks popularized 'lakehouse' in 2020; Armbrust, Ghodsi, Xin and Zaharia formalized it in the CIDR 2021 paper. Google's Iceberg-based borderless Lakehouse (2025–2026) is its current GCP form.

Flow

Part A closes on the problem statement; part B opens with the lakehouse definition, the deck quiz, then M8 builds it on GCS and Iceberg.

Links

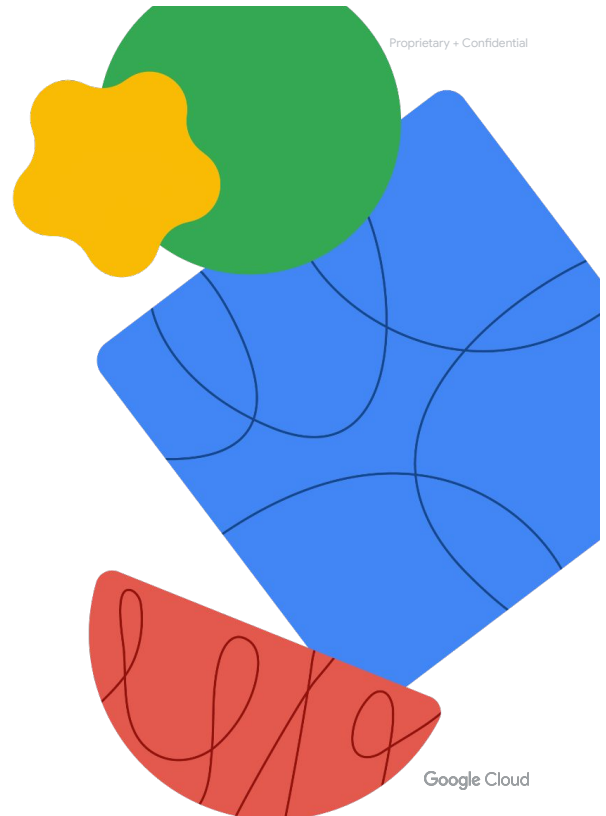
<https://cloud.google.com/products/lakehouse>

<https://cloud.google.com/blog/products/data-analytics/biglake-metastore-now-supports-iceberg-rest-catalog>

<https://docs.cloud.google.com/bigquery/docs/lakehouse-in-bigquery>

The modern approach: Data lakehouse

Combines the flexibility and low cost of a data lake with the speed and accuracy of a data warehouse.



INSTRUCTOR NOTES — PAGE 24 OF 208 · 09:15 · MODULE 7 — INTRO TO MODERN DATA ENGINEERING

Hook

You have spent careers modeling everything into SQL Server before anyone could touch it. The lakehouse is the industry's answer to that rigidity — and it is literally your target architecture.

Say · cover

- Definition: a lakehouse combines the lake's low-cost, flexible storage with the warehouse's query speed and data accuracy — one platform, not two.
- It is an architecture pattern, not a product: on Google Cloud it means Cloud Storage and BigQuery acting as one system.
- The economic point: stop copying data between a raw zone and a curated zone — one copy serves BI, data science, and AI.
- The course's running example (Cymbal) shows the join that silos make impossible.

Iowa tie

Your Bronze-on-GCS plus Silver/Gold-in-BigQuery design IS a lakehouse — this module hands you the vocabulary to defend it.

Design patterns

- One copy of data, many workloads
- Land raw, model when ready

Anti-patterns

- Keeping lake and warehouse as two silos with nightly copies between them

History & what's new

Databricks coined 'data lakehouse' in a January 30, 2020 blog post (Lorica, Armbrust, Ghodsi, Xin, Zaharia), formalized in a CIDR 2021 paper — the term is barely six years old.

Flow

Part A contrasted lake vs warehouse; this slide names the synthesis. Next: the vocabulary mapped to their own project.

Links

<https://cloud.google.com/discover/what-is-a-data-lakehouse>
<https://www.databricks.com/blog/2020/01/30/what-is-a-data-lakehouse.html>
<https://cloud.google.com/discover/data-lake-vs-data-warehouse>

Lake, warehouse, lakehouse — in your project's terms

Your target design IS a lakehouse design; this module gives you the words to defend it.

- The lake— Cloud Storage buckets; your Bronze layer, raw data landed as-is
- The warehouse— BigQuery; your Silver and Gold, cleaned and business-ready
- The lakehouse— the two acting as one system, BigQuery querying both
- Schema-on-write— your world today: everything modeled before it lands
- Schema-on-read— land raw in Bronze first, model into Silver/Gold when ready
- One engine owning the data— replaced by storage and compute scaled independently

Custom slide · added by Imran Ahmad

Iowa DOM DoIT · Data Engineering on Google Cloud

INSTRUCTOR NOTES — PAGE 25 OF 208 · 09:15 · MODULE 7 — INTRO TO MODERN DATA ENGINEERING

Hook

Screenshot this slide — it is the paragraph your architecture document needs, and the answer to 'why not just rebuild the warehouse in BigQuery?'

Say · cover

- The lake: Cloud Storage buckets — your Bronze layer, data landed as-is.
- The warehouse: BigQuery — your Silver and Gold, cleaned and business-ready.
- The lakehouse: the two acting as one system, BigQuery querying both.
- Schema-on-write is your world today — everything modeled before it lands; schema-on-read means land raw in Bronze, model into Silver/Gold when ready.
- One engine owning the data is replaced by storage and compute scaled independently.

Iowa tie

This is the migration in one table: Bronze lands raw on GCS, Dataform models Silver-to-Gold in BigQuery, Power BI reads Gold.

Design patterns

- Translate vendor vocabulary into your project's own layers
- Scale storage and compute independently

Anti-patterns

- Reading schema-on-read as 'never model anything' — Gold is still schema-on-write

History & what's new

The CIDR 2021 lakehouse paper argued the two-tier lake-plus-warehouse design was unsustainable — two copies to reconcile, warehouse always stale against the lake. That critique is why the Bronze-Silver-Gold plan keeps one copy.

Flow

Custom bridge slide between the concept (p024) and Google's three-part definition, next.

Links

<https://cloud.google.com/discover/what-is-a-data-lakehouse>
<https://cloud.google.com/discover/data-lake-vs-data-warehouse>
<https://cloud.google.com/storage/docs/introduction>

Data lakehouse

01

Unified platform

Combines low-cost storage of a data lake with data warehouse management and query performance.

02

Versatile workloads

Supports traditional BI, modern data science, and AI without data movement or duplication.

03

Optimized architecture

Implements metadata and governance on open-format files in low-cost object storage (such as Google Cloud Storage) for a hybrid approach.

Google Cloud

INSTRUCTOR NOTES — PAGE 26 OF 208 · 09:15 · MODULE 7 — INTRO TO MODERN DATA ENGINEERING

Hook

Read trait 03 twice — it is the only one that tells you HOW the trick works.

Say · cover

- 01 Unified platform: lake-grade storage costs with warehouse-grade management and query performance.
- 02 Versatile workloads: traditional BI, modern data science, and AI on the same data — no movement, no duplication.
- 03 Optimized architecture: a metadata and governance layer over open-format files on cheap object storage such as GCS — that is the mechanism.
- The 'house' is metadata, not a new database: governance, schema, and performance layered onto files you already own.
- Trait 02 is why Power BI and any future ML pilot can read the same governed copy.

Iowa tie

'Without data movement or duplication' is your kill-the-extracts mandate restated by Google — one governed copy, queried where it lives.

Design patterns

- Metadata/governance layer over open files; one engine over raw and curated

Anti-patterns

- Re-duplicating Bronze into BigQuery for every query instead of querying in place

History & what's new

Google now brands this stack 'Lakehouse' — BigLake was renamed Lakehouse in April 2026, so docs and console show both names during the transition.

Flow

Concept defined; next, the Cymbal story makes it concrete.

Links

<https://cloud.google.com/discover/what-is-a-data-lakehouse>

<https://cloud.google.com/bigquery/docs/biglake-intro>

<https://cloud.google.com/blog/products/data-analytics/announcing-bigquery-tables-for-apache-iceberg>

Data lakehouse

Data silo challenges

Cymbal's sales data was in a data warehouse and its customer review data was in a data lake. The two could not be analyzed together easily.

Integrated data analysis

With a data lakehouse, Cymbal can now run a single query to correlate customer sentiment from text reviews with sales trends from its transaction database.

Google Cloud

INSTRUCTOR NOTES — PAGE 27 OF 208 · 09:15 · MODULE 7 — INTRO TO MODERN DATA ENGINEERING

Hook

Cymbal's split — sales in the warehouse, reviews in the lake — is every agency's split: line-of-business data in SQL Server, everything else stranded in file shares and exports nobody can join.

Say · cover

- Cymbal is the course's running example company: sales data in a data warehouse, customer-review text in a data lake.
- The silo cost: the two could not be analyzed together — no single engine reached both.
- The lakehouse fix: ONE query correlating review sentiment with sales trends.
- The win is the join across raw and curated — without a migration project first.

Iowa tie

Your Cymbal moment: extracts scattered across SQL Server instances and file drops collapse into one queryable estate.

Design patterns

- Join raw text/JSON to curated tables in one engine
- Land messy data in Bronze instead of rejecting it

Anti-patterns

- Building a second warehouse for 'the other data' instead of unifying

History & what's new

Google's answer shipped as BigLake: previewed April 2022, generally available July 2022 (Google Cloud blog: 'Unify data lakes and warehouses with BigLake, now generally available').

Flow

Story done; next, the six key features that make it work.

Links

<https://cloud.google.com/discover/what-is-a-data-lakehouse>

<https://cloud.google.com/blog/products/data-analytics/unify-data-lakes-and-warehouses-with-biglake-now-generally-available>

<https://cloud.google.com/bigquery/docs/biglake-intro>

Data lakehouse: Key features

01 Support for most data formats

04 Cost flexibility based on needs

02 Flexible schema-on-read or schema-on-write

05 Unified data governance

03 Access for all types of data users

06 ACID transaction support

Google Cloud

INSTRUCTOR NOTES — PAGE 28 OF 208 · 09:15 · MODULE 7 — INTRO TO MODERN DATA ENGINEERING

Hook

Feature 06 — ACID on files — is the one that was impossible when your SQL Server warehouse was built. It changes what a lake can be.

Say · cover

- 01-03: support for most data formats, schema-on-read OR schema-on-write, access for all data users — the lake's openness kept.
- 04 Cost flexibility: object-storage prices for raw data; pay for compute only when you query.
- 05 Unified governance: one permission and audit model across Bronze, Silver, and Gold.
- 06 ACID transactions: safe updates and deletes on lake files — warehouse behavior without the warehouse.
- ACID is what lets Dataform-style ELT run safely against lake-resident tables.

Iowa tie

For M&O: one governance plane replaces per-instance SQL logins plus file-share permissions — one place to grant and audit.

Design patterns

- ACID table format on the lake
- Single governance plane across all medallion layers

Anti-patterns

- Concurrent writers on raw files with no transaction layer — silent corruption

History & what's new

Apache Iceberg — created at Netflix in 2017, donated to the Apache Software Foundation in November 2018, a top-level Apache project since May 2020 — brought ACID and time travel to object-storage files.

Flow

Features listed; next, the Google technology that delivers them.

Links

<https://iceberg.apache.org/>

<https://cloud.google.com/bigquery/docs/iceberg-tables>

<https://cloud.google.com/discover/what-is-a-data-lakehouse>



Lakehouse technology

- Google Cloud uses Lakehouse to implement data lakehouses.
- Lakehouse extends Google Cloud's governance and query features to data in Google Cloud or other major clouds.
- Manage and query data in its original location using open formats, such as Parquet, ORC, and Avro.
- Utilize BigQuery's powerful query engine.

Google Cloud

INSTRUCTOR NOTES — PAGE 29 OF 208 · 09:15 · MODULE 7 — INTRO TO MODERN DATA ENGINEERING

Hook

Naming alert: this slide's 'Lakehouse' was 'BigLake' until April 2026 — you will meet both names in docs, the console, and every blog post older than six months.

Say · cover

- Lakehouse (formerly BigLake) is Google's product family for extending BigQuery's governance and query engine to data outside BigQuery-managed storage.
- It reaches data in its original location: Cloud Storage, and AWS S3 / Azure blobs via BigQuery Omni — no copies.
- Open formats only: Parquet, ORC, Avro (plus Iceberg tables) — the bytes stay readable by other engines.
- The query engine is BigQuery's — no second engine to provision, patch, or scale.
- BigLake/Lakehouse tables add table-level security — row and column policies — over files sitting in buckets.

Iowa tie

Your Bronze Parquet on GCS becomes queryable with row/column security without being loaded — M&O keeps one permission model.

Design patterns

- Query in place with fine-grained security via BigLake/Lakehouse tables

Anti-patterns

- Granting broad bucket-level access because table-level security felt hard

History & what's new

BigLake previewed at Google's Data Cloud Summit in April 2022 and went GA that July; Google renamed BigLake to Lakehouse in April 2026.

Flow

The technology named; next, why 'open' is doing the heavy lifting.

Links

<https://cloud.google.com/bigquery/docs/biglake-intro>

<https://cloud.google.com/bigquery/docs/omni-introduction>

<https://cloud.google.com/blog/products/data-analytics/unify-data-lakes-and-warehouses-with-biglake-now-generally-available>

'Open' is the load-bearing word

- Open file formats— Parquet, ORC, Avro: no single engine owns your bytes
- Open table formats— Apache Iceberg adds ACID, schema evolution, time travel to files
- Your Bronze files stay readable by non-Google engines— insurance against lock-in
- Multiple engines— BigQuery, Spark, others — work safely on the same tables
- This is why the Bronze decision (Parquet on GCS, not CSV) was yesterday's real lesson

Custom slide · added by Imran Ahmad

Iowa DOM DoIT · Data Engineering on Google Cloud

INSTRUCTOR NOTES — PAGE 30 OF 208 · 09:15 · MODULE 7 — INTRO TO MODERN DATA ENGINEERING

Hook

One sentence: if your bytes are Parquet on GCS, no vendor — Google included — can hold your data hostage.

Say · cover

- Open FILE formats — Parquet, ORC, Avro: no single engine owns your bytes.
- Open TABLE formats — Apache Iceberg — add ACID, schema evolution, and time travel to those files.
- Bronze stays readable by Spark, Trino, and others: insurance against lock-in, in either direction.
- Multiple engines can work the same tables safely because the table format coordinates them.
- This is why yesterday's CSV-vs-Parquet DIN mattered: Parquet in Bronze was the real lesson.

Iowa tie

Bronze standard: Parquet on GCS, never CSV — one decision that keeps every future engine option open, including Spark if a workload ever needs it.

Design patterns

- Parquet as the canonical Bronze format; Iceberg when you need ACID on the lake

Anti-patterns

- CSV as an interchange standard — no types, no compression, no column pruning

History & what's new

BigQuery tables for Apache Iceberg entered preview in October 2024; by 2026 Google shipped GA cross-engine Iceberg features (ObjectRefs, per InfoQ, May 2026).

Flow

Openness argued; next, the benefits recap before the quiz.

Links

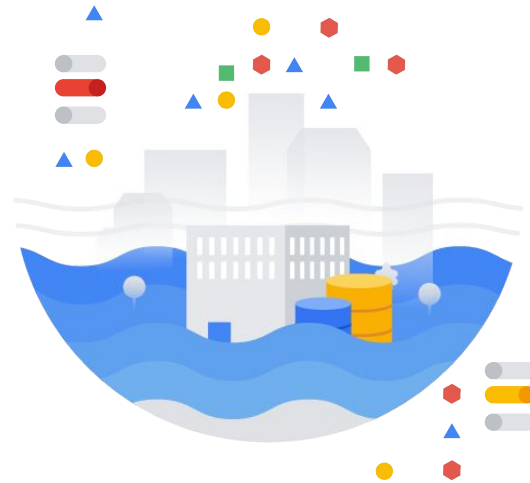
<https://iceberg.apache.org/>

<https://parquet.apache.org/>

<https://cloud.google.com/bigquery/docs/iceberg-tables>

Data lakehouse benefits

- Unified data storage and access
- Reduced data redundancy
- Unified governance
- Broken-down data silos
- Improved flexibility and scalability



Google Cloud

INSTRUCTOR NOTES — PAGE 31 OF 208 · 09:15 · MODULE 7 — INTRO TO MODERN DATA ENGINEERING

Hook

Five benefits — but they only cash out if you resist the urge to re-copy everything 'just in case.'

Say · cover

- Unified storage and access: one estate to secure, catalog, and back up.
- Reduced data redundancy: the copies die — that is the business case, in four words.
- Unified governance: policies follow the data across Bronze, Silver, and Gold.
- Silos broken: raw and curated become joinable — the Cymbal lesson.
- Flexibility and scalability: storage scales on GCS, compute scales on BigQuery — independently.
- The caveat: benefits evaporate if teams keep private copies — this is governance plus habit change, not just architecture.

Iowa tie

'Reduced data redundancy' is the slide you quote when justifying the extract-kill list to leadership: one copy, one truth, Power BI reads Gold.

Design patterns

- Measure the migration by number of copies eliminated

Anti-patterns

- Keeping lake and warehouse with nightly sync jobs — the two-tier failure mode in a new costume

History & what's new

The CIDR 2021 paper made 'two copies, always stale' the canonical anti-pattern; by 2026 industry commentary jokes that 'every vendor is a lakehouse' — the pattern won.

Flow

Benefits banked; next slide sets up the group quiz.

Links

<https://cloud.google.com/discover/what-is-a-data-lakehouse>

<https://www.databricks.com/blog/2020/01/30/what-is-a-data-lakehouse.html>

<https://cloud.google.com/discover/data-lake-vs-data-warehouse>

QUIZ

Built-in quiz — run it as a group

Last 5–8 minutes of the block · Whole room — the quiz pages that follow in this deck

1. Instructor shows each question page and reads it aloud
2. Confer at your table for about 30 seconds, then answer for the group
3. Treat wrong answers as vocabulary gaps to fix now, not as grades
4. Four questions with answer pages — lake, warehouse, lakehouse, and the Cymbal scenario

Why it matters — if you can answer this quiz out loud, you can narrate your own architecture doc to anyone who asks 'why not just a warehouse?'

Custom slide · added by Imran Ahmad

Iowa DOM DoIT · Data Engineering on Google Cloud

INSTRUCTOR NOTES — PAGE 32 OF 208 · 09:15 · MODULE 7 — INTRO TO MODERN DATA ENGINEERING

Hook

Sixteen people, one org, no grades — this is the safest room this vocabulary will ever get tested in.

Say · cover

- Budget the last 5-8 minutes of the block; four questions: lake, warehouse, lakehouse, Cymbal scenario.
- Read each question aloud, give tables about 30 seconds to confer, then one table answers for the room.
- Treat wrong answers as vocabulary gaps to fix now, not as grades.
- The meta-point: anyone who can answer these out loud can narrate the architecture doc to leadership.
- Pocket answer key: Q1=C, Q2=B, Q3=C, Q4=C.
- Keep the pace brisk — four questions plus short discussion fit comfortably inside eight minutes.

Iowa tie

The reusable sentence they walk out with: 'we are building a lakehouse — GCS Bronze, BigQuery Silver/Gold, one governed copy.'

Design patterns

- Cold-call gently; park long debates until after the reveal

Anti-patterns

- Lecturing the answer before the room commits to one

History & what's new

The vocabulary under quiz is young: 'lakehouse' dates to Databricks' January 2020 post, and Google's 'Lakehouse' product name is only from April 2026.

Flow

Quiz protocol set; next pages are the four Q&A pairs, then the M7 wrap on p042.

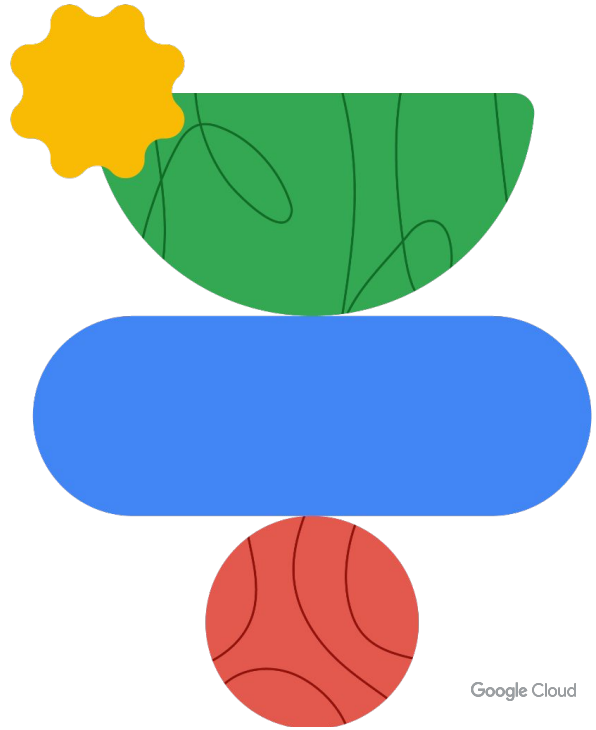
Links

<https://cloud.google.com/discover/what-is-a-data-lakehouse>

<https://cloud.google.com/learn/what-is-a-data-warehouse>

<https://cloud.google.com/discover/data-lake-vs-data-warehouse>

Quiz



Google Cloud

INSTRUCTOR NOTES — PAGE 33 OF 208 · 09:15 · MODULE 7 — INTRO TO MODERN DATA ENGINEERING

FACILITATOR NOTE

- Section divider for the built-in quiz. Run it as a group per the previous slide: read each question aloud, give tables about 30 seconds to confer, one table answers for the room.
- Pocket answer key: Q1=C (lake = raw, schema-on-read), Q2=B (warehouse inflexibility), Q3=C (metadata/governance layer over open files), Q4=C (lakehouse with BigQuery and Lakehouse). Wrong answers are vocabulary gaps, not grades.

Flow: Q1 next; answer/reveal pages follow each question; M7 wraps on p042, then M8 at 09:45.

Quiz | Question 01

Question

Which of the following best describes the primary characteristic of a data lake?

- A. Data is cleaned, transformed, and structured before storage, optimized for BI.
- B. It is a highly organized library designed for fast and accurate reports based on predefined metrics.
- C. It stores enormous amounts of raw data in its native format with schema-on-read, ideal for exploration and ML.
- D. It primarily handles structured data and is optimized for high-performance queries on consistent information.

Google Cloud

INSTRUCTOR NOTES — PAGE 34 OF 208 · 09:15 · MODULE 7 — INTRO TO MODERN DATA ENGINEERING

Hook

First question, easiest one — but listen for anyone answering A; that person is still thinking like a warehouse.

Say + cover

- Correct: C — enormous raw data in native format, schema-on-read, ideal for exploration and ML.
- A, B, and D all describe the warehouse: cleaned before storage, the organized library, structured and consistent.
- The discriminator to teach: WHERE the schema is applied — at write time or at read time.
- If someone says 'data swamp,' park it — that is the lake's failure mode, and it is Q2's trap answer.
- Tie it back: their Bronze landing zone is option C in production clothing.

lowa tie

C is your ingest pattern verbatim: Bronze lands raw on GCS, and the schema arrives in Silver/Gold via Dataform.

Design patterns

- Quiz the discriminator (schema timing), not the definition

Anti-patterns

- Letting 'raw' be heard as 'unmanaged' — Bronze still gets lifecycle and access rules

History & what's new

James Dixon, Pentaho's CTO, coined 'data lake' in 2010 — a 'large body of water in a natural state,' versus the data mart's 'bottled water — cleansed, packaged, structured.'

Flow

Q1 of 4; answer reveal on the next page.

Links

<https://cloud.google.com/discover/data-lake-vs-data-warehouse>
<https://cloud.google.com/storage/docs/introduction>
<https://cloud.google.com/discover/what-is-a-data-lakehouse>

Quiz | Question 01

Answer

Which of the following best describes the primary characteristic of a data lake?

- A. Data is cleaned, transformed, and structured before storage, optimized for BI.
- B. It is a highly organized library designed for fast and accurate reports based on predefined metrics.
- C. It stores enormous amounts of raw data in its native format with schema-on-read, ideal for exploration and ML.
- D. It primarily handles structured data and is optimized for high-performance queries on consistent information.



Google Cloud

INSTRUCTOR NOTES — PAGE 35 OF 208 · 09:15 · MODULE 7 — INTRO TO MODERN DATA ENGINEERING

FACILITATOR NOTE

- Answer: C. A, B, and D each describe a data warehouse — cleaned-before-storage, the organized library, structured and consistent.
- Drive home the schema-on-read vs schema-on-write discriminator: that one idea separates every architecture discussed today.

Flow: Q2 next — now from the warehouse side.

Quiz | Question 02

Question

What is a key disadvantage of a traditional data warehouse?

- A. It risks becoming a "data swamp" without proper governance.
- B. It is inflexible and cannot easily accommodate new data types or unstructured data.
- C. It primarily uses low-cost object storage, leading to security risks with raw data.
- D. It is ideal for AI/ML model training due to its support for advanced analytics.

Google Cloud

INSTRUCTOR NOTES — PAGE 36 OF 208 · 09:15 · MODULE 7 — INTRO TO MODERN DATA ENGINEERING

Hook

The trap is A — half the room will pick it, and it is a LAKE problem, not a warehouse one.

Say + cover

- Correct: B — traditional warehouses are inflexible and cannot easily take new data types or unstructured data.
- A ('data swamp') is the lake's governance failure — the deliberate trap; C is backwards (low-cost object storage is the lake's trait); D is false (ML training is not the warehouse's strength).
- Root cause of B: schema-on-write means every new data type is a modeling project before ingestion.
- Their lived version: adding a column to the SQL Server warehouse means change tickets and SSIS package edits.

Iowa tie

B is the honest summary of why the legacy warehouse became the bottleneck — and the justification line for landing raw in Bronze first.

Design patterns

- Name the trap answer explicitly so the room argues about the right thing

Anti-patterns

- Trash-talking warehouses generally — Gold IS a warehouse pattern, and that is fine

History & what's new

Bill Inmon formalized the warehouse in 'Building the Data Warehouse' (1992) — subject-oriented, integrated, time-variant, non-volatile. Thirty-plus years of schema-on-write discipline is what the lake relaxes.

Flow

Q2 of 4; answer reveal on the next page.

Links

<https://cloud.google.com/learn/what-is-a-data-warehouse>
<https://cloud.google.com/discover/data-lake-vs-data-warehouse>
<https://cloud.google.com/discover/what-is-a-data-lakehouse>

Quiz | Question 02

Answer

What is a key disadvantage of a traditional data warehouse?

- A. It risks becoming a "data swamp" without proper governance.
- B. It is inflexible and cannot easily accommodate new data types or unstructured data.
- C. It primarily uses low-cost object storage, leading to security risks with raw data.
- D. It is ideal for AI/ML model training due to its support for advanced analytics.



Google Cloud

INSTRUCTOR NOTES — PAGE 37 OF 208 · 09:15 · MODULE 7 — INTRO TO MODERN DATA ENGINEERING

FACILITATOR NOTE

- Answer: B. A is the planted trap — the swamp is the lake's governance failure, not a warehouse weakness; C reverses the storage economics; D misassigns ML training.
- Tie B to their world: schema-on-write makes every new data type a change-ticket project in SQL Server.

Flow: Q3 next — the lakehouse mechanism itself.

Quiz | Question 03

Question

How does a data lakehouse architecture combine the best features of data lakes and data warehouses?

- A. By enforcing a strict schema-on-write approach for all data types to ensure high data quality.
- B. By creating separate, specialized platforms for BI and AI workloads to maximize performance.
- C. By implementing a metadata and governance layer on top of open-format files in low-cost object storage.
- D. By always requiring data to be moved and duplicated between a data lake and a data warehouse.

Google Cloud

INSTRUCTOR NOTES — PAGE 38 OF 208 · 09:15 · MODULE 7 — INTRO TO MODERN DATA ENGINEERING

Hook

This is the only question whose answer is a mechanism, not a definition — and it is the mechanism of your whole target design.

Say + cover

- Correct: C — a metadata and governance layer on top of open-format files in low-cost object storage.
- A is wrong: strict schema-on-write for all data would throw away the lake's flexibility.
- B is wrong: separate specialized platforms for BI and AI re-creates the silos.
- D is wrong: mandatory movement and duplication is precisely what the lakehouse abolishes.
- Have the room say C's mechanism in their own words — it is the sentence their architecture doc needs.

Iowa tie

C is your stack in one line: Dataform plus BigQuery (metadata/governance) over Parquet in Bronze (open files) on GCS (cheap object storage).

Design patterns

- Make learners restate the mechanism — retention beats recognition

Anti-patterns

- Accepting 'it combines both' as an answer — that is the slide title, not the how

History & what's new

Google shipped exactly this mechanism as BigLake (GA July 2022): BigQuery's engine and governance extended over open files — renamed Lakehouse in April 2026.

Flow

Q3 of 4; answer reveal on the next page.

Links

<https://cloud.google.com/discover/what-is-a-data-lakehouse>

<https://cloud.google.com/bigquery/docs/biglake-intro>

<https://iceberg.apache.org/>

Quiz | Question 03

Answer

How does a data lakehouse architecture combine the best features of data lakes and data warehouses?

- A. By enforcing a strict schema-on-write approach for all data types to ensure high data quality.
- B. By creating separate, specialized platforms for BI and AI workloads to maximize performance.
- C. By implementing a metadata and governance layer on top of open-format files in low-cost object storage. 
- D. By always requiring data to be moved and duplicated between a data lake and a data warehouse.

Google Cloud

INSTRUCTOR NOTES — PAGE 39 OF 208 · 09:15 · MODULE 7 — INTRO TO MODERN DATA ENGINEERING

FACILITATOR NOTE

- Answer: C. The how is a metadata and governance layer over open-format files on cheap object storage — not stricter schema (A), not separate platforms (B), not more copies (D).
- Have someone map C onto Bronze/Silver/Gold in their own words before you move on.

Flow: Q4 next — the Cymbal scenario, everything combined.

Quiz | Question 04

Question

Cymbal, an e-commerce company, needs a solution to support traditional BI, modern data science, and AI workloads on a single, governed copy of its data while also breaking down data silos between sales and customer review data. Which architecture is best suited for this need?

- A. A data warehouse, such as BigQuery, for high-speed, interactive BI on structured data.
- B. A data lake, using Cloud Storage, for initial, low-cost storage of massive volumes of raw data for AI/ML research.
- C. A data lakehouse architecture with BigQuery and Lakehouse.
- D. Separate data lakes and data warehouses, with manual data integration as needed.

Google Cloud

INSTRUCTOR NOTES — PAGE 40 OF 208 · 09:15 · MODULE 7 — INTRO TO MODERN DATA ENGINEERING

Hook

The scenario question — and the one most likely to be quoted back at you next week when someone asks 'so what are we building?'

Say · cover

- Correct: C — a data lakehouse architecture with BigQuery and Lakehouse (formerly BigLake).
- The tells in the prompt: 'BI, data science, AND AI' (breadth of workloads), 'single, governed copy' (no duplication), 'breaking down silos' (sales plus reviews joined).
- A covers only BI; B covers only cheap raw storage for ML; D rebuilds the silo with manual integration as the tax.
- The option text literally pairs the two products — that pairing IS the Google lakehouse.

Iowa tie

C is your justification paragraph: one governed copy serving Power BI today, and the data-science workloads that come after go-live.

Design patterns

- Teach the three keyword tells (workloads / single copy / silos) for scenario questions

Anti-patterns

- Choosing A 'because BigQuery is fast' — speed was never the question

History & what's new

The 'single governed copy' promise got concrete on GCP when BigQuery tables for Apache Iceberg entered preview in October 2024 — open table format, BigQuery governance, no copies.

Flow

Q4 of 4; reveal next, then the M7 wrap-up.

Links

<https://cloud.google.com/blog/products/data-analytics/announcing-bigquery-tables-for-apache-iceberg>
<https://cloud.google.com/bigquery/docs/biglake-intro>
<https://cloud.google.com/discover/what-is-a-data-lakehouse>

Quiz | Question 04

Question

Cymbal, an e-commerce company, needs a solution to support traditional BI, modern data science, and AI workloads on a single, governed copy of its data while also breaking down data silos between sales and customer review data. Which architecture is best suited for this need?

- A. A data warehouse, such as BigQuery, for high-speed, interactive BI on structured data.
- B. A data lake, using Cloud Storage, for initial, low-cost storage of massive volumes of raw data for AI/ML research.
- C. A data lakehouse architecture with BigQuery and Lakehouse.
- D. Separate data lakes and data warehouses, with manual data integration as needed.



Google Cloud

INSTRUCTOR NOTES — PAGE 41 OF 208 · 09:15 · MODULE 7 — INTRO TO MODERN DATA ENGINEERING

FACILITATOR NOTE

- Answer: C. Only the lakehouse covers traditional BI, data science, and AI on one governed copy while joining the sales and review silos. A and B each solve half; D re-creates the problem with manual integration.
- If time is short, spend it on this question — it restates the whole module as a decision.

Flow: Quiz done; M7 wrap-up next, then M8 at 09:45.

M7 — What you now have

- ✓ The lake/warehouse/lakehouse vocabulary— and why the lakehouse collapses both silos into one platform
- ✓ Your design in those terms— GCS is the lake (Bronze), BigQuery the warehouse (Silver/Gold), one system
- ✓ The case for open formats— Parquet and Iceberg keep your Bronze free of lock-in
- ✓ The group quiz answered— vocabulary you can reuse when you defend the architecture doc
- ✓ Next: M8 builds the lakehouse itself— storage options, Iceberg, federated queries (09:45–10:30)

Custom slide · added by Imran Ahmad

Iowa DOM DoIT · Data Engineering on Google Cloud

INSTRUCTOR NOTES — PAGE 42 OF 208 · 09:15 · MODULE 7 — INTRO TO MODERN DATA ENGINEERING

Hook

Thirty minutes, one module — and you now hold the vocabulary of your own architecture document.

Say · cover

- Recap 1: lake vs warehouse vs lakehouse — the lakehouse collapses two silos into one platform.
- Recap 2: your design in those terms — GCS is the lake (Bronze), BigQuery the warehouse (Silver/Gold), one system.
- Recap 3: the case for open formats — Parquet and Iceberg keep Bronze free of lock-in.
- Recap 4: the quiz answers are reusable sentences for the architecture doc.
- Tease M8: next block builds it — storage options, Iceberg tables, federated queries, 09:45-10:30.

Iowa tie

Every recap bullet maps to a section of their migration design doc — assign that mapping as mental homework over the break.

Design patterns

- Close a module by re-speaking the audience's own design back in the new vocabulary

Anti-patterns

- Letting 'lakehouse' stay a buzzword — without open formats underneath, it is marketing

History & what's new

Next hour's tech is genuinely current: Iceberg tables previewed in BigQuery in October 2024, and the BigLake-to-Lakehouse rename (April 2026) means console and docs may disagree on names.

Flow

M7 closes here; M8 'Building a Data Lakehouse' starts at 09:45 — the how-to block.

Links

<https://cloud.google.com/discover/what-is-a-data-lakehouse>

<https://cloud.google.com/bigquery/docs/iceberg-tables>

<https://cloud.google.com/blog/products/data-analytics/announcing-bigquery-tables-for-apache-iceberg>

WATCH FIRST · 3:27 · GOOGLE CLOUD TECH

Bucket Options in Cloud Storage

- ✓ Naming, locations, storage classes — the three decisions you make once per bucket, and the foundation of your Bronze layer.
- ✓ Everything this morning lands in a bucket: Parquet files, Hive partitions, Iceberg tables. This is the vocabulary for all of it.



Scan to open on YouTube
youtu.be/8DMOJ6Lgm7s

Custom slide · added by Imran Ahmad

Iowa DOM DoIT · Data Engineering on Google Cloud

INSTRUCTOR NOTES — PAGE 43 OF 208 · 09:45 · MODULE 8 — BUILDING A DATA LAKEHOUSE**Hook**

- Play as the M8 opener — 'bucket' stops being an abstract noun before the lakehouse build starts.

Say

- “A bucket is a file share with a region and a price class — you choose both deliberately, once, like filegroups and drives.”

Cue

- All 3:27; the storage-classes table (~2:20) maps to hot/cool/archive instincts — point at it.

Link

- <https://www.youtube.com/watch?v=8DMOJ6Lgm7s>

M8 — What this block gets you

- ✓ GCS + open formats (Parquet, Iceberg)— the Bronze foundation in your own architecture doc
- ✓ Apache Iceberg— warehouse behavior (ACID, schema evolution, time travel) on raw files in a bucket
- ✓ BigQuery as the one SQL engine over Bronze, Silver and Gold— managed tables and lake files alike
- ✓ Federated queries— your linked-server / OPENQUERY pattern, from BigQuery, without moving data
- ✓ Hands-on: the federated-query lab, then external tables on Hive-partitioned Parquet— Bronze made tangible

Custom slide · added by Imran Ahmad

Iowa DOM DoIT · Data Engineering on Google Cloud

INSTRUCTOR NOTES — PAGE 44 OF 208 · 09:45 · MODULE 8 — BUILDING A DATA LAKEHOUSE

Hook

Your architecture doc already says Bronze, Silver, Gold — this 45 minutes is where each word becomes a specific GCP service.

Say · cover

- GCS plus open formats (Parquet, Iceberg) is the Bronze foundation: files in your bucket, queryable in place.
- Apache Iceberg puts warehouse behavior — ACID, schema evolution, time travel — onto raw lake files.
- BigQuery is the one SQL engine over managed tables and lake files alike: no second tool per zone.
- Federated queries are your linked-server and OPENQUERY pattern, run from BigQuery without moving data.
- After the break: the federated-query lab, then external tables on Hive-partitioned Parquet.

Iowa tie

This block is the literal middle of their migration design: Bronze in GCS, one BigQuery SQL surface, federation so SQL Server systems stay reachable mid-migration.

Design patterns

- Open formats for landing, managed tables for serving
- One SQL surface over all three zones

Anti-patterns

- A separate engine per zone (Spark for the lake, BigQuery for the warehouse) doubles M&O work

History & what's new

BigLake launched in 2022 (TechCrunch); at Next '26 (April 2026) Google rebranded it Google Cloud Lakehouse, and BigQuery's Iceberg tables are GA per the 2026 release notes.

Flow

M7 supplied the vocabulary; this block builds the lakehouse, then break and the 10:45 lab.

Links

<https://cloud.google.com/architecture/build-a-data-lake-on-gcp>
<https://cloud.google.com/bigquery/docs/iceberg-tables>
<https://cloud.google.com/bigquery/docs/federated-queries-intro>

This block and the hands-on morning

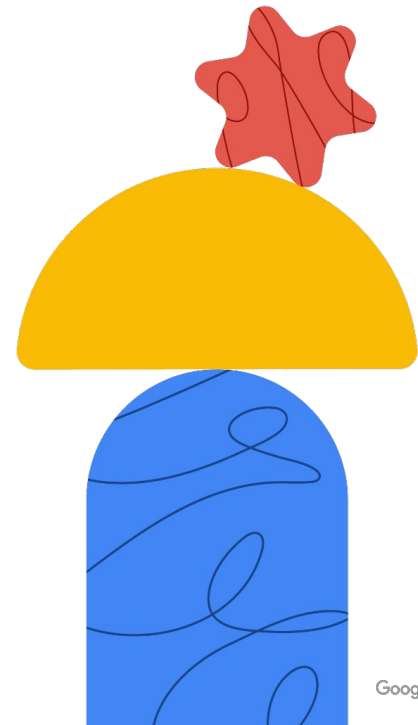
09:45–10:30	M8 — Building a data lakehouse: GCS, open formats, Iceberg, federated queries	
10:30–10:45	Break	
10:45–11:15	Lab: Federated Query with BigQuery — live operational data, no copy	LAB
11:15–11:35	DIN: Exploring external tables — Hive-partitioned Parquet on GCS	DIN
11:35–12:00	M9 — Modernizing data warehouses with BigQuery and BigLake	

FACILITATOR NOTE

- Roadmap: 45 minutes of M8 concepts, break, then the 10:45 federated-query lab and the 11:15 external-tables DIN — every concept lands hands-on within the hour.
- Tell them to hold two ideas for the hands-on: EXTERNAL_QUERY as OPENQUERY, and Iceberg as warehouse behavior on files.

Flow: Next: where lakehouse storage physically lives on Google Cloud.

Data storage for data lakehouses on Google Cloud



Google Cloud

INSTRUCTOR NOTES — PAGE 46 OF 208 · 09:45 · MODULE 8 — BUILDING A DATA LAKEHOUSE

Hook

First question your M&O team will ask: where do the bytes actually sit? Answer: buckets, not file shares.

Say · cover

- The lakehouse storage layer is Cloud Storage object storage: objects in buckets, addressed by gs:// URIs.
- Open file formats (Parquet) make those objects engine-readable; open table formats (Iceberg) make them tables.
- BigQuery managed storage stays in the picture for hot, frequently queried datasets — your Silver and Gold.
- Console location: Cloud Storage > Buckets; the same buckets appear later as BigQuery external sources.

Iowa tie

Bronze for county and vendor drops equals Parquet in GCS buckets; for M&O, operations become IAM and lifecycle policy instead of patching servers.

Design patterns

- Separate storage from compute so each scales independently
- Lay out prefixes so paths carry metadata (order_date=...)

Anti-patterns

- Treating buckets like a network share of CSVs nobody can query

History & what's new

Cloud Storage keeps absorbing lake duties: at Next '26 (April 2026) Google announced AI-focused throughput gains and better metadata generation for storage (blocksandfiles).

Flow

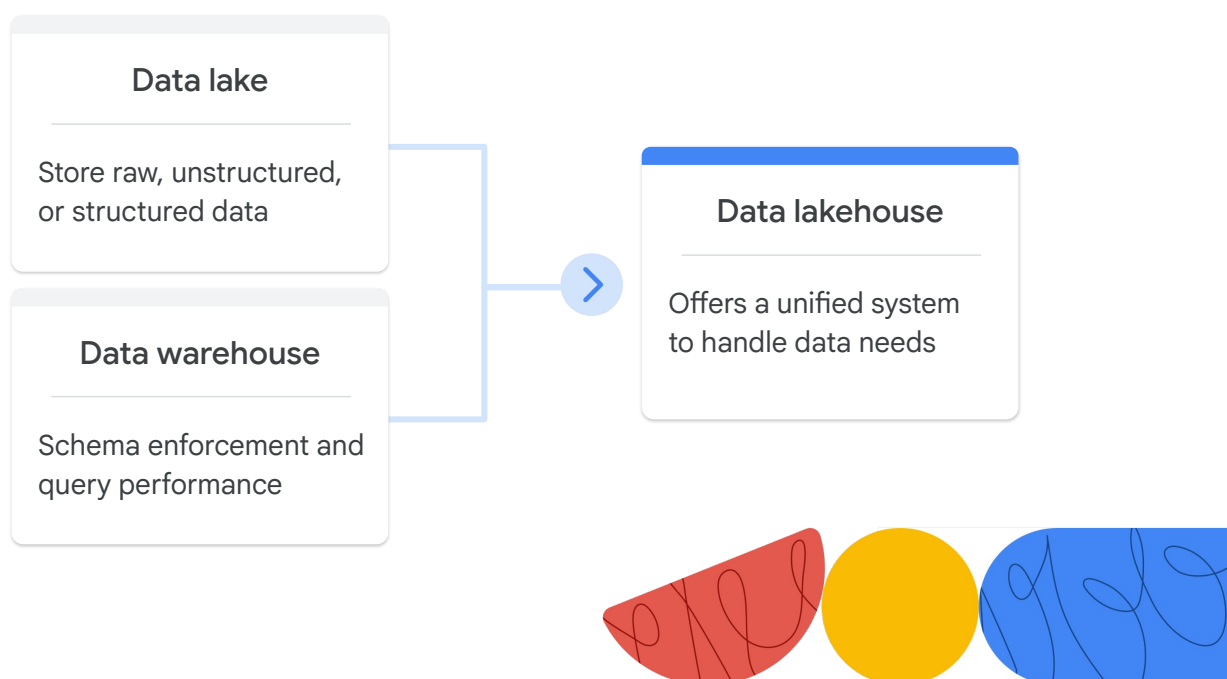
Next: the 60-second lake versus warehouse versus lakehouse framing.

Links

<https://cloud.google.com/storage/docs/introduction>

<https://cloud.google.com/storage/docs/storage-classes>

<https://cloud.google.com/architecture/build-a-data-lake-on-gcp>



INSTRUCTOR NOTES — PAGE 47 OF 208 · 09:45 · MODULE 8 — BUILDING A DATA LAKEHOUSE

Hook

You live the split today: cheap-but-dumb file drops versus governed-but-rigid warehouse tables. The lakehouse promise is not having to choose.

Say · cover

- Data lake: stores raw, unstructured, or structured data cheaply, in any format.
- Data warehouse: schema enforcement and query performance, at warehouse cost and rigidity.
- Lakehouse: warehouse behavior layered onto lake files through an open table format — one copy of data.
- Google's assembly: Cloud Storage for the lake, BigQuery as the engine, BigLake and Iceberg as the table layer.

Iowa tie

Their single-source-of-truth goal: Bronze lands as lake files, Silver and Gold are warehouse tables, and one engine spans both.

Design patterns

- Medallion layers over a single copy, not copied-per-layer systems

Anti-patterns

- Copying the whole lake into the warehouse before anyone may query it

History & what's new

Industry coverage in 2026 describes the warehouse market as converged on lakehouse architecture with Apache Iceberg as the open table format (eidosoft); Google's answer was BigLake in 2022 (TechCrunch), rebranded Google Cloud Lakehouse at Next '26.

Flow

Next: the storage workhorse — Cloud Storage.

Links

<https://cloud.google.com/architecture/build-a-data-lake-on-gcp>
<https://cloud.google.com/bigquery/docs/biglake-intro>
<https://cloud.google.com/bigquery/docs/introduction>

Cloud Storage

A massively scalable, highly durable, and cost-effective object storage service.



Google Cloud

INSTRUCTOR NOTES — PAGE 48 OF 208 · 09:45 · MODULE 8 — BUILDING A DATA LAKEHOUSE

Hook

GCS is the file share that never fills up and never needs patching — and BigQuery reads it directly, no load required.

Say · cover

- Objects (immutable blobs of any format) live in buckets; folders are naming conventions, with hierarchical namespace optional.
- Durability is eleven-nines; dual-region and multi-region options replicate data for availability.
- Storage classes run Standard, Nearline, Coldline, Archive, plus Rapid; Autoclass moves objects by access pattern.
- Soft delete (default 7-day retention) and Object Lifecycle Management protect and age data automatically.

Iowa tie

Bronze buckets for the file drops; M&O owns IAM, lifecycle, and retention policy instead of server maintenance.

Design patterns

- Hive-style key=value prefixes from day one
- Lifecycle rules age cold Bronze data into cheaper classes

Anti-patterns

- Random deep layouts that make partition pruning impossible later

History & what's new

Classes and Autoclass are established; at Next '26 (April 2026) Google announced AI-driven storage speed increases and smarter metadata generation (blocksandfiles, April 2026).

Flow

Next: what if some data has to stay in AWS or Azure?

Links

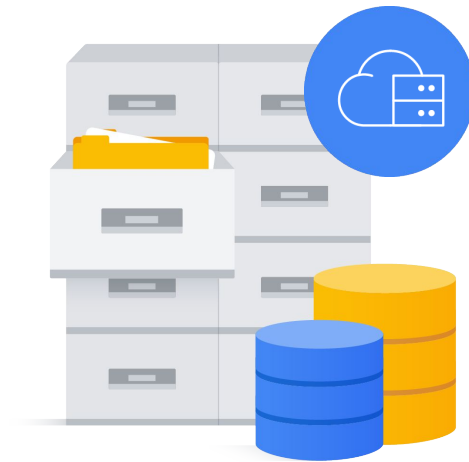
<https://cloud.google.com/storage/docs/introduction>

<https://cloud.google.com/storage/docs/storage-classes>

<https://cloud.google.com/storage/docs/key-terms>

Other storage options

If your strategy is multicloud, you can keep your data in another major provider's cloud storage.



Google Cloud

INSTRUCTOR NOTES — PAGE 49 OF 208 · 09:45 · MODULE 8 — BUILDING A DATA LAKEHOUSE

Hook

If a partner agency's data already sits in AWS or Azure, you don't have to drag it to Google before querying it.

Say · cover

- BigQuery Omni runs BigQuery analytics on data stored in Amazon S3 or Azure Blob Storage, via BigLake tables.
- Cross-cloud joins combine Google-resident tables with S3 or Azure tables in one query.
- The remote part of a join is staged as a temporary CTAS table and transfers are billed — so filter hard.
- For Iowa this is a footnote: home base is GCS, but know Omni exists so design reviews don't stall.

Iowa tie

Their migration targets GCS plus BigQuery; Omni matters only if county or vendor data is already cloud-hosted elsewhere.

Design patterns

- Query in place when data gravity sits in another cloud

Anti-patterns

- Assuming cross-cloud querying is free — egress and transfer costs are real

History & what's new

Omni was announced in July 2020 for AWS S3 (HPCwire); at Next '26 (April 2026) Google extended the story with a Cross-cloud Lakehouse capability for S3 and Azure Data Lake.

Flow

Next: the open table format that makes bucket files behave like warehouse tables — Iceberg.

Links

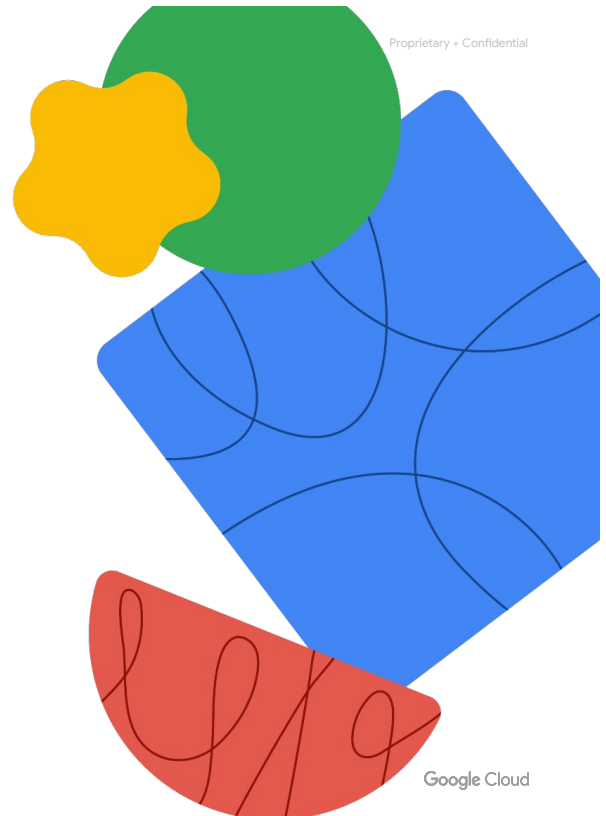
<https://cloud.google.com/bigquery/docs/omni-introduction>

<https://cloud.google.com/bigquery/docs/biglake-intro>

<https://cloud.google.com/storage/docs/introduction>

Introduction to Apache Iceberg open table format

A data lakehouse needs structure and performance. Open table formats, such as Apache Iceberg provide this.



INSTRUCTOR NOTES — PAGE 50 OF 208 · 09:45 · MODULE 8 — BUILDING A DATA LAKEHOUSE

Hook

Parquet in a bucket is a pile of files; Iceberg is the open standard that turns the pile into a table with SQL Server-grade guarantees.

Say · cover

- Iceberg is an open table format: a metadata and catalog layer over data files in object storage.
- It provides ACID transactions, schema evolution, hidden partitioning, and time travel on raw files.
- Multi-engine by design: BigQuery, Spark, and Trino can read the same table — no format lock-in.
- BigQuery managed Iceberg tables keep the bytes in your bucket while BigQuery does the bookkeeping.

Iowa tie

Insurance policy for their Bronze: readable by non-Google engines if strategy ever shifts; the 13:00 lab builds one hands-on.

Design patterns

- Open table formats where multiple engines or long time horizons are in play

Anti-patterns

- Homegrown 'tables' that are just naming conventions over raw CSV — no ACID, no evolution

History & what's new

Iceberg was built at Netflix in 2017, donated to Apache in 2018, and became a top-level Apache project in May 2020 (Wikipedia/ASF); BigQuery's managed Iceberg tables are GA per the 2026 release notes.

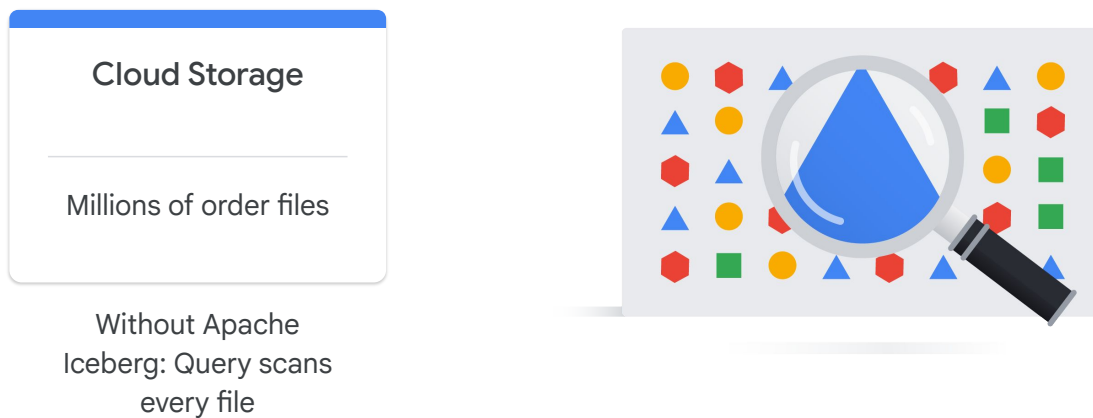
Flow

Next: a two-slide visual on why Iceberg makes queries cheaper.

Links

<https://iceberg.apache.org/>
<https://cloud.google.com/bigquery/docs/iceberg-tables>
<https://cloud.google.com/bigquery/docs/biglake-intro>

Efficient querying with Apache Iceberg (1 of 2)



Google Cloud

INSTRUCTOR NOTES — PAGE 51 OF 208 · 09:45 · MODULE 8 — BUILDING A DATA LAKEHOUSE

Hook

Ask for the January 1st orders in a bucket holding a million files — with no metadata, the engine's only move is opening every file.

Say · cover

- Object storage can list prefixes but cannot index contents; scan planning means listing, then opening files.
- Every query scans every file: bytes processed equals the whole dataset, every time.
- On-demand pricing bills bytes scanned — a structureless lake is slow AND expensive.
- Same lesson as Day 1's CSV demo: file layout and format decide cost before any SQL runs.

Iowa tie

Mirrors their today: county drops are unqueryable until SSIS loads them, and even queryable files would full-scan without structure.

Design patterns

- Columnar formats like Parquet shrink bytes scanned even before metadata helps

Anti-patterns

- Millions of tiny files: listing alone starts to dominate query time

History & what's new

Netflix created Iceberg in 2017 precisely because Hive-style tables degraded into full scans at their scale (documented project history); BigLake later added CMETA metadata caching for external tables (docs).

Flow

Next slide: same bucket, but now metadata directs the query.

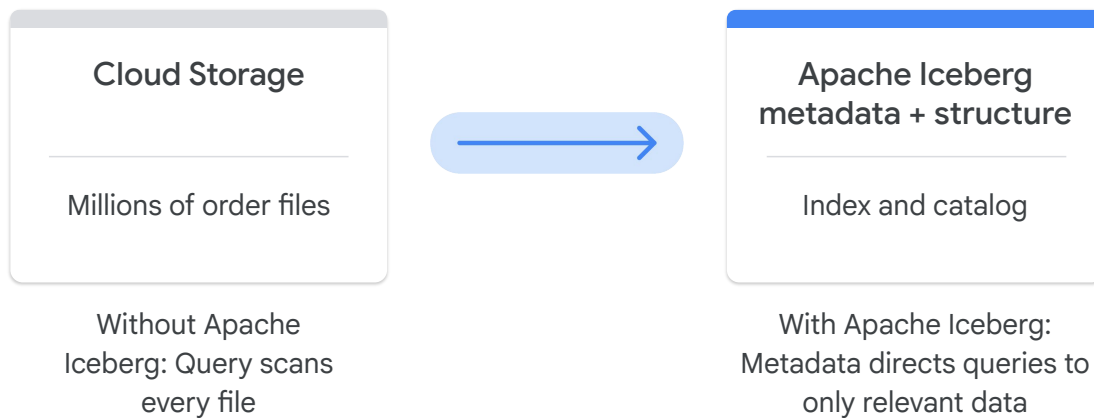
Links

<https://iceberg.apache.org/>

<https://cloud.google.com/bigquery/docs/hive-partitioned-queries>

<https://cloud.google.com/bigquery/docs/iceberg-tables>

Efficient querying with Apache Iceberg (2 of 2)



Google Cloud

INSTRUCTOR NOTES — PAGE 52 OF 208 · 09:45 · MODULE 8 — BUILDING A DATA LAKEHOUSE

Hook

Same million files, but now a catalog knows which twelve matter — the query opens those and nothing else.

Say · cover

- Iceberg tracks each table state as a snapshot: manifests list files plus per-file partition values and stats.
- Engines plan scans from metadata — no file listing at query time, irrelevant files skipped outright.
- This is partition elimination (your SQL Server reflex) and BigQuery pruning, implemented inside the lake itself.
- The 'index and catalog' in the diagram is just more files in the bucket — no service for M&O to run.

Iowa tie

Pruning on raw files is exactly what the 11:15 DIN demonstrates live — bytes processed drops hard on the partition key.

Design patterns

- Let table metadata drive pruning; compact small files so planning stays fast

Anti-patterns

- Streaming tiny files continuously: metadata explodes and planning slows

History & what's new

The snapshot and manifest design dates to Iceberg's Netflix era (2017-2018); BigQuery's GA managed Iceberg tables add automatic compaction and clustering (2026 release notes).

Flow

Next: the four guarantees Iceberg gives you on top of this.

Links

<https://iceberg.apache.org/>
<https://cloud.google.com/bigquery/docs/iceberg-tables>
<https://cloud.google.com/bigquery/docs/biglake-intro>

What does Apache Iceberg provide?

01

Schema evolution

Allows schema changes (such as adding and renaming columns) without impacting existing data or applications.

02

Hidden partitioning

Automatically manages how data is partitioned for efficient querying.

03

Time travel

Enables querying past data versions for auditing and debugging, allowing users to reproduce exact data states from previous points in time.

04

Atomic transactions

Enable reliable, concurrent data operations, preventing corruption when multiple processes read/write to the same tables.

Google Cloud

INSTRUCTOR NOTES — PAGE 53 OF 208 · 09:45 · MODULE 8 — BUILDING A DATA LAKEHOUSE

Hook

Rename a column in SQL Server and half your SSIS packages break — Iceberg was designed so schema change is a metadata edit, not surgery.

Say · cover

- Schema evolution: add, rename, or reorder columns without rewriting data or breaking existing readers.
- Hidden partitioning: the format tracks partition values per file — users stop writing path hacks in SQL.
- Time travel: query a prior snapshot to reproduce exactly what a report saw — built for audit and debugging.
- Atomic transactions: concurrent readers and writers can't corrupt the table; commits are all-or-nothing.

Iowa tie

Schema drift across county file formats and 'reproduce what the auditor saw' are two everyday pains — both answered here.

Design patterns

- Use snapshots and time travel for reproducible audits

Anti-patterns

- Schema-on-read over raw CSV: 'evolution' by silently renaming columns in new files

History & what's new

These guarantees are why Iceberg — open-sourced by Netflix in 2018, an Apache top-level project since May 2020 — became the default: by early 2026 every major vendor reads and writes it (eidosoft, 2026).

Flow

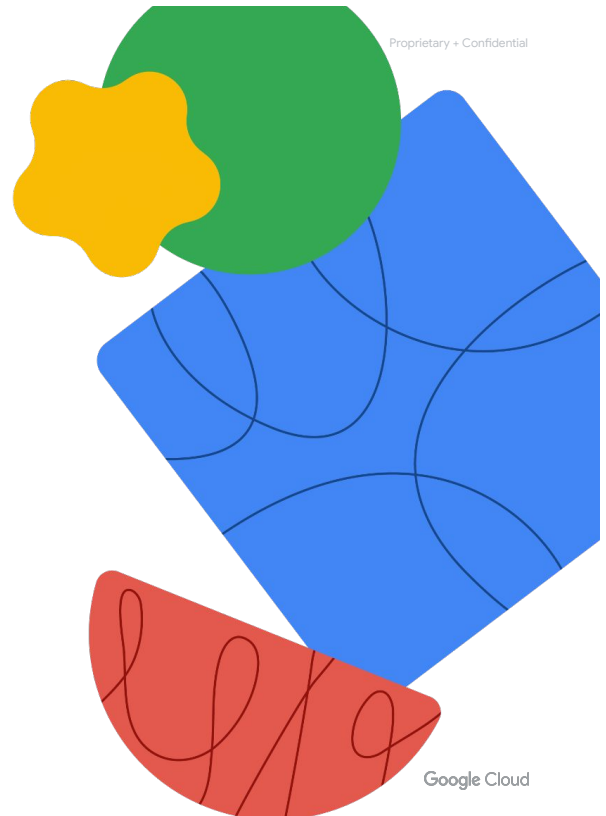
Next: Iceberg concretely on Cloud Storage — who writes what, where.

Links

<https://iceberg.apache.org/>
<https://cloud.google.com/bigquery/docs/iceberg-tables>
<https://cloud.google.com/bigquery/docs/biglake-intro>

Using Apache Iceberg with Cloud Storage

Treat a collection of data files as performant, queryable tables, bringing the benefits of a data warehouse to a flexible data lake.



INSTRUCTOR NOTES — PAGE 54 OF 208 · 09:45 · MODULE 8 — BUILDING A DATA LAKEHOUSE

Hook

The recipe: your bucket, Parquet data files, Iceberg metadata — BigQuery does the bookkeeping while the bytes stay yours.

Say · cover

- Managed Iceberg tables: `CREATE TABLE ... WITH CONNECTION, OPTIONS(table_format='ICEBERG', file_format='PARQUET', storage_uri='gs://...')`.
- BigQuery runs compaction, clustering, and garbage collection; Iceberg V2 snapshots export so Spark and Trino read the same table.
- Hard rule: only BigQuery writes to that prefix — untracked files get garbage-collected and external edits corrupt the table.
- Dataform workflows are supported, so Silver-to-Gold style pipelines can target Iceberg tables too.

Iowa tie

This is Bronze that their Dataform ELT can write to directly; bucket config (same region, Autoclass, IAM) is the M&O checklist.

Design patterns

- One writer (BigQuery), a dedicated bucket, and an empty prefix per table

Anti-patterns

- Letting other tools drop files into a managed-Iceberg prefix — data loss by garbage collection

History & what's new

Previewed as 'BigQuery tables for Apache Iceberg' (2025); GA and renamed BigLake/managed Iceberg tables per the 2026 release notes, with cross-engine support covered by InfoQ (May 2026).

Flow

Next: the custom slide that maps all of this onto YOUR Bronze.

Links

<https://cloud.google.com/bigquery/docs/iceberg-tables>
<https://iceberg.apache.org/>
<https://cloud.google.com/bigquery/docs/biglake-intro>

GCS + open formats = your Bronze foundation

- Today— county and vendor file drops sit unqueryable until an SSIS load runs
- Bronze target— Parquet files in a GCS bucket, Hive-partitioned by date, queryable immediately
- Iceberg on top— ACID, schema evolution, time travel over those same files, multi-engine readable
- Silver and Gold— BigQuery-managed tables, built by Dataform ELT from Bronze (this afternoon)
- One SQL surface— BigQuery reads managed tables and lake files alike; no staging hop
- AlloyDB— operational/OLTP sibling (managed PostgreSQL); not your path, one line only

Custom slide · added by Imran Ahmad

Iowa DOM DoIT · Data Engineering on Google Cloud

INSTRUCTOR NOTES — PAGE 55 OF 208 · 09:45 · MODULE 8 — BUILDING A DATA LAKEHOUSE

Hook

This is your architecture doc on one slide — read every line against the county and vendor file drops you run today.

Say · cover

- Today: drops sit unqueryable until an SSIS load runs — the file is invisible to SQL.
- Bronze target: Parquet in a GCS bucket, Hive-partitioned by date, queryable the moment it lands.
- Iceberg on top adds ACID, schema evolution, and time travel — still multi-engine readable.
- Silver and Gold are BigQuery-managed tables built by Dataform ELT — this afternoon's modules.
- AlloyDB one-liner: managed PostgreSQL for the operational sibling — not your path.

Iowa tie

Every line is theirs: kill-the-extracts for file drops, one SQL surface over Bronze, Silver, and Gold, Power BI served from Gold.

Design patterns

- Bronze open and immutable; Silver and Gold managed; one SQL surface

Anti-patterns

- The reflex copy: loading Bronze into the warehouse just to look at it

History & what's new

Google's lakehouse stack consolidated fast: BigLake (2022, TechCrunch) became Google Cloud Lakehouse at Next '26, with managed Iceberg tables GA the same year (release notes).

Flow

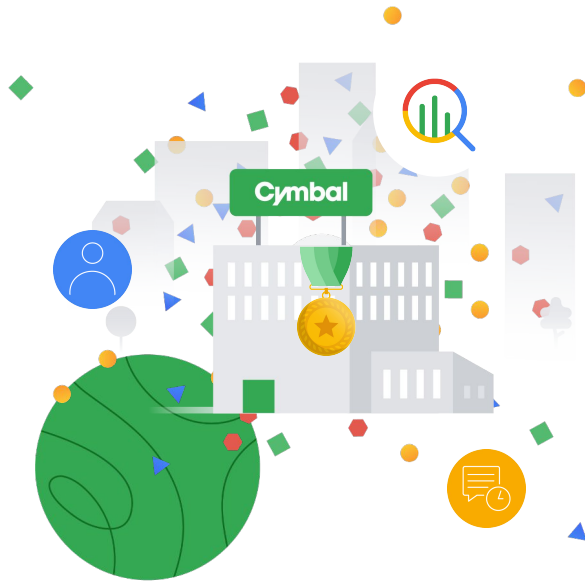
Next: BigQuery as the central engine over all of it.

Links

<https://cloud.google.com/bigquery/docs/hive-partitioned-queries>

<https://cloud.google.com/bigquery/docs/external-data-cloud-storage>

<https://cloud.google.com/bigquery/docs/iceberg-tables>



BigQuery as the central processing engine

Whereas Cloud Storage and Apache Iceberg provide a flexible, open-standard foundation for storing vast amounts of raw and structured data, BigQuery is the high-performance engine that activates it.

Google Cloud

INSTRUCTOR NOTES — PAGE 56 OF 208 · 09:45 · MODULE 8 — BUILDING A DATA LAKEHOUSE

Hook

One engine to learn, one dialect to govern: the same SQL hits managed warehouse tables and raw lake files — no staging hop.

Say · cover

- Storage and compute scale independently; you never size clusters separately for lake versus warehouse.
- BigQuery reads managed tables, BigLake tables, external tables, and Iceberg — one IAM surface, one SQL dialect.
- GCS plus Iceberg is the flexible foundation; BigQuery is what 'activates' it — the slide's word is right.
- The Power BI last mile (M11, this afternoon) connects to this same single surface.

Iowa tie

Their single-source-of-truth objective is exactly this architectural move: one query plane over Bronze, Silver, and Gold.

Design patterns

- Central engine plus open storage; govern once through IAM and Dataplex

Anti-patterns

- Per-zone engines — Spark for the lake, BigQuery for the warehouse — means double pipelines, double ops

History & what's new

BigQuery keeps absorbing the lake: BigLake in 2022 (TechCrunch), managed Iceberg tables GA in 2026 (release notes) — the central-engine story is current, not legacy.

Flow

Next: the Cymbal case — warehouse-grade SQL directly on lake data.

Links

<https://cloud.google.com/bigquery/docs/introduction>
<https://cloud.google.com/bigquery/docs/iceberg-tables>
<https://cloud.google.com/bigquery/docs/biglake-intro>

BigQuery

Seamless data querying with BigQuery and Lakehouse

By creating Lakehouse tables, Cymbal can use BigQuery's familiar SQL interface to directly and securely query the Apache Iceberg-formatted data in their Cloud Storage data lake.

Eliminating data duplication and ETL processes

No need to duplicate data or perform costly extract, transform, load (ETL) processes to run analytics.

Enhanced analytics performance for data lake

Analysts get the performance and features of a premier data warehouse while querying data directly in their open-format data lake.

Google Cloud

INSTRUCTOR NOTES — PAGE 57 OF 208 · 09:45 · MODULE 8 — BUILDING A DATA LAKEHOUSE

Hook

Cymbal's scenario in one line: analysts query Iceberg data in the lake with the same SQL they would run on the warehouse.

Say · cover

- Lakehouse tables equal BigQuery querying Iceberg-formatted data in Cloud Storage directly and securely.
- No duplication and no ETL built just to make data analyzable — query where it lives.
- Analysts still get warehouse-grade performance and features on open-format lake data.
- Vendor case study, but the mechanics are real: the BigLake and managed Iceberg tables from two slides ago.

Iowa tie

This is the kill-the-extracts slide for leadership: copy data when there is a reason — Silver and Gold serving — not as a reflex.

Design patterns

- Query in place for exploration; promote to managed tables for serving

Anti-patterns

- ETL-to-copy as the default first step for every new dataset

History & what's new

BigLake was announced at Google Cloud Next in April 2022 (TechCrunch); Iceberg-format tables reached GA in BigQuery in 2026 (release notes).

Flow

Next: when hot data belongs in BigQuery native storage instead.

Links

<https://cloud.google.com/bigquery/docs/biglake-intro>

<https://cloud.google.com/bigquery/docs/iceberg-tables>

<https://cloud.google.com/bigquery/docs/external-data-cloud-storage>

BigQuery

Native storage for “hot” datasets

BigQuery offers optimized, managed storage for frequently accessed or “hot” datasets. This is ideal for data requiring the fastest query performance, such as key marketing performance dashboards.

Unified data analysis platform

Cymbal can leverage a unified platform to analyze data located in their data lake, BigQuery's native storage, or a combination of both, all accessible through a single interface.

Google Cloud

INSTRUCTOR NOTES — PAGE 58 OF 208 · 09:45 · MODULE 8 — BUILDING A DATA LAKEHOUSE

Hook

Open formats are for landing and exploring; when a dashboard hammers the same data hourly, managed storage earns its keep.

Say · cover

- BigQuery managed storage is optimized, columnar, and self-tuning — the fastest scans BigQuery offers.
- Reserve it for hot datasets: key dashboards and the marts people query all day.
- The platform is unified: one SQL interface over lake files, native storage, or a mix — decided per table.
- Decision rule to repeat out loud: land and explore on files; serve on managed tables.

Iowa tie

Their Power BI data marts — Gold, built by Dataform from Silver — are exactly this slide's hot datasets: managed tables.

Design patterns

- Tier by access pattern: hot goes to managed storage, cold and raw stay in open formats on GCS

Anti-patterns

- Serving daily dashboards off raw external files — slower, and no storage optimizations apply

History & what's new

BigQuery documents managed storage as its performance tier (storage overview, updated 2026); managed Iceberg tables now get automatic compaction and clustering too (2026 release notes) — the gap keeps closing.

Flow

Next: a one-line AlloyDB detour before federation.

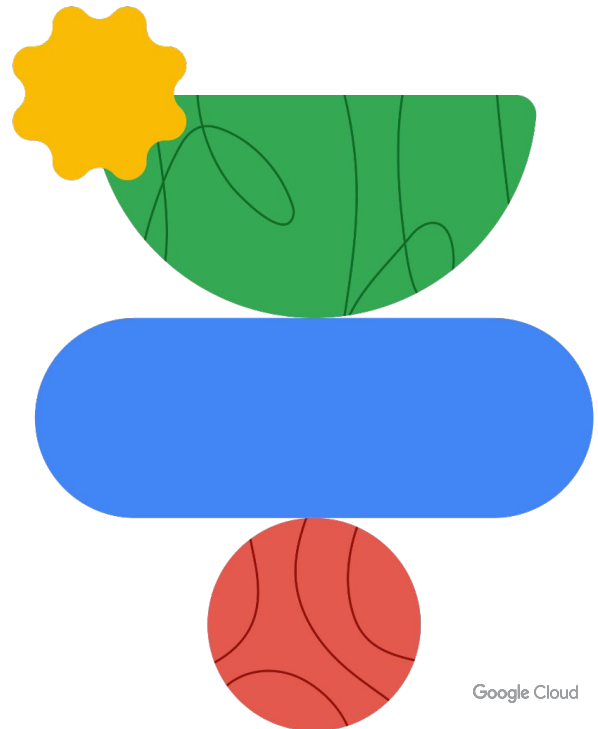
Links

https://cloud.google.com/bigquery/docs/storage_overview

<https://cloud.google.com/bigquery/docs/introduction>

<https://cloud.google.com/bigquery/docs/biglake-intro>

Combining operational data in AlloyDB



INSTRUCTOR NOTES — PAGE 59 OF 208 · 09:45 · MODULE 8 — BUILDING A DATA LAKEHOUSE

Hook

AlloyDB in one line, as promised: Google's fully managed PostgreSQL for operational apps — in this course it plays 'the SQL Server system that is still alive'.

Say · cover

- AlloyDB is fully managed, PostgreSQL-compatible OLTP: high availability and high throughput for transactions.
- It is not your migration target — the warehouse goes to BigQuery; operational stores stay operational.
- It appears here because federation needs a live operational source to query.
- Cloud SQL is the sibling for lift-and-shift MySQL, PostgreSQL, or SQL Server instances.

Iowa tie

Their SQL Server OLTP systems stay put during migration — AlloyDB is only today's lab stand-in for live operational data.

Design patterns

- Keep OLTP and analytics on separate systems; bridge with federation or Datastream CDC (Day 1)

Anti-patterns

- Running analytical queries against the production OLTP box — their current pain behind extracts

History & what's new

At Next '25 (April 2025) Google announced Database Migration Service support for SQL Server-to-PostgreSQL migrations into Cloud SQL and AlloyDB, with Gemini-assisted T-SQL conversion (SiliconANGLE) — directly relevant to their estate.

Flow

Next: federated queries — reach that operational data without moving it.

Links

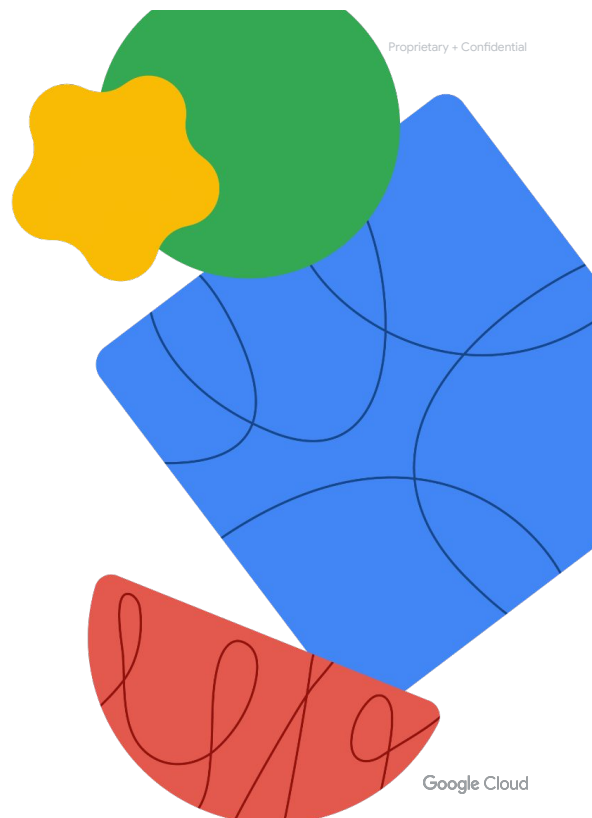
<https://cloud.google.com/alloydb/docs/overview>

<https://cloud.google.com/alloydb/docs/omni/overview>

<https://cloud.google.com/bigquery/docs/federated-queries-intro>

Federated queries: Combining operational and analytical data in a single query

Query data in external systems, such as an operational AlloyDB database, in real time without moving or copying it into BigQuery.



INSTRUCTOR NOTES — PAGE 60 OF 208 · 09:45 · MODULE 8 — BUILDING A DATA LAKEHOUSE

Hook

You have done this for years with linked servers and OPENQUERY — federation is the BigQuery-native version, and after break you will build one.

Say · cover

- Query AlloyDB, Cloud SQL, or Spanner in real time from BigQuery — no copy, no movement.
- The connection resource (BigQuery Connection API) is your linked-server definition: one-time admin setup.
- EXTERNAL_QUERY ships the inner SQL to the source, which executes it in its own dialect; results return as a temporary table typed to GoogleSQL.
- Federation is read-only, and on-demand billing charges for the bytes the external side returns.

Iowa tie

This is migrate-in-phases: join live operational data to warehouse data during the migration without building extract jobs first.

Design patterns

- Federate for freshness and ad-hoc joins; load when access becomes repeated

Anti-patterns

- Federation as permanent architecture: the source takes the load and gets no BigQuery storage optimizations

History & what's new

AlloyDB federated queries reached GA in 2024 (release notes, cited in hashicorp/terraform-provider-google issue 18663); SAP Datasphere federation is in preview per the current docs.

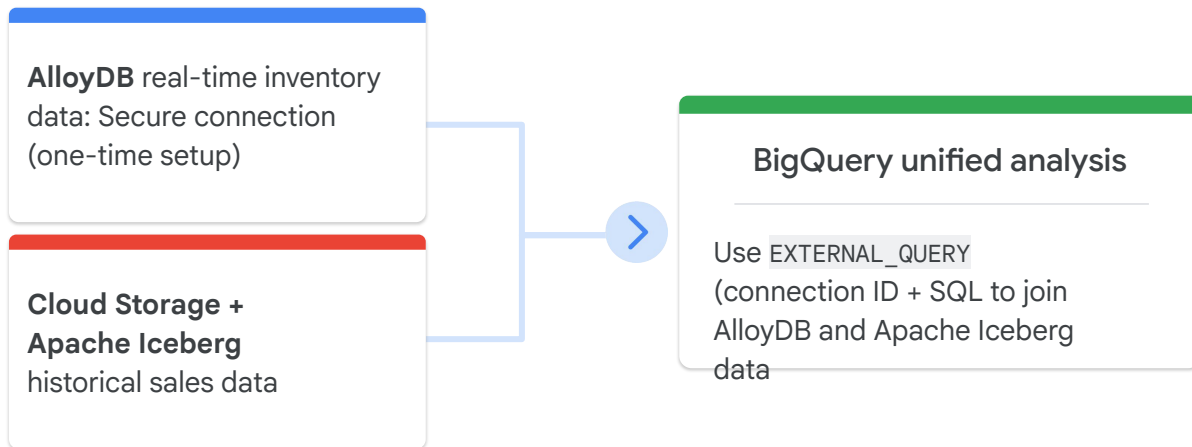
Flow

Next: the mechanics diagram — EXTERNAL_QUERY joining two worlds.

Links

<https://cloud.google.com/bigquery/docs/federated-queries-intro>
<https://cloud.google.com/bigquery/docs/alloydb-federated-queries>
<https://cloud.google.com/alloydb/docs/overview>

Combining data



Google Cloud

INSTRUCTOR NOTES — PAGE 61 OF 208 · 09:45 · MODULE 8 — BUILDING A DATA LAKEHOUSE

Hook

Read this diagram as your migration in miniature: history in the lake, live rows in the operational system, one SELECT.

Say · cover

- Anatomy: `EXTERNAL_QUERY('region.connection-id', "PostgreSQL here")` — the quoted string runs on AlloyDB, in PostgreSQL.
- One-time secure connection setup; then the remote result behaves like a table you can join.
- SQL pushdowns: BigQuery rewrites the query so column pruning and WHERE filters execute at the source — less data crosses the wire.
- Join the federated result to BigQuery history — or to Iceberg tables on GCS — in standard SQL.

Iowa tie

Their pattern exactly: warehouse history in BigQuery joined to live transactional rows still on the SQL Server side, no nightly extract.

Design patterns

- Put filters inside the inner query and keep returned sets small; point heavy federation at read replicas

Anti-patterns

- `SELECT *` inside `EXTERNAL_QUERY` and filtering outside — defeats pushdown and ships the whole table

History & what's new

Pushdowns (column pruning, filter pushdown) for AlloyDB, Cloud SQL, and Spanner are documented in the current docs (2026); cross-region federation carries a 1 TB per day per project quota.

Flow

Next: the custom lab intro — what you will actually do at 10:45.

Links

<https://cloud.google.com/bigquery/docs/federated-queries-intro>

<https://cloud.google.com/bigquery/docs/alloydb-federated-queries>

<https://cloud.google.com/bigquery/docs/external-data-sources>

WATCH FIRST · 2:30 · GOOGLE CLOUD TECH

Is AlloyDB Compatible with PostgreSQL?

- ✓ Yes — fully. AlloyDB is Google's managed, PostgreSQL-compatible OLTP engine, built for high availability and speed.
- ✓ Why it matters in the next 30 minutes: the federated-query lab joins BigQuery to a live AlloyDB database — your 'linked server' pattern, working.



Scan to open on YouTube
youtu.be/88k-eB3Eon8

Custom slide · added by Imran Ahmad

Iowa DOM DoIT · Data Engineering on Google Cloud

INSTRUCTOR NOTES — PAGE 62 OF 208 · 09:45 · MODULE 8 — BUILDING A DATA LAKEHOUSE**Hook**

- Play right before the federated-query lab brief — the lab's 'operational database' has a name after this.

Say

- “Think of it as the SQL Server instance your warehouse queries without migrating — federation instead of ETL.”

Cue

- ~2:30, plays in full; the compatibility answer lands by 1:00 — the rest is performance story you can trim.

Link

- <https://www.youtube.com/watch?v=88k-eB3Eon8>

Lab: Federated Query with BigQuery

30 minutes · 10:45–11:15 · skills.google lab seat — pre-warmed before the break ends

1. Launch the lab; create the BigQuery connection resource — your linked-server definition, one-time setup
2. Run your first EXTERNAL_QUERY — the inner SQL is PostgreSQL on the AlloyDB side, just like OPENQUERY
3. Join historical analytical data in BigQuery with live transactional rows — one query, two systems
4. Debrief: querying data where it lives — that is the lakehouse promise

Why it matters — your SQL Server systems stay operational through the migration — federation means you query them in place instead of moving everything first.

Custom slide · added by Imran Ahmad

Iowa DOM DoIT · Data Engineering on Google Cloud

INSTRUCTOR NOTES — PAGE 63 OF 208 · 09:45 · MODULE 8 — BUILDING A DATA LAKEHOUSE

Hook

Everything from the last twenty minutes becomes muscle memory in the next thirty: one connection, one EXTERNAL_QUERY, one join.

Say · cover

- Step 1: create the connection resource — your linked-server definition; region choice matters, one-time setup.
- Step 2: first EXTERNAL_QUERY — the inner SQL is PostgreSQL executed on AlloyDB; BigQuery syntax inside the quotes is the number-one failure.
- Step 3: join BigQuery historical data with live AlloyDB transactional rows — one query, two systems.
- Debrief line: querying data where it lives — that is the lakehouse promise.
- Seats are pre-warmed on skills.google; check the project picker — everything goes in the lab's temporary project.

Iowa tie

Federation is how their SQL Server systems stay reachable mid-migration — query in place instead of a big-bang move.

Design patterns

- Match the connection region to the query location; triple-quote inner SQL to escape single quotes

Anti-patterns

- Wrong project in the picker, or connection-ID typos — it must match region.connection-id exactly

History & what's new

The official 'Federated Query with BigQuery' lab runs on skills.google; its current form uses AlloyDB, whose federation went GA in 2024 (release notes).

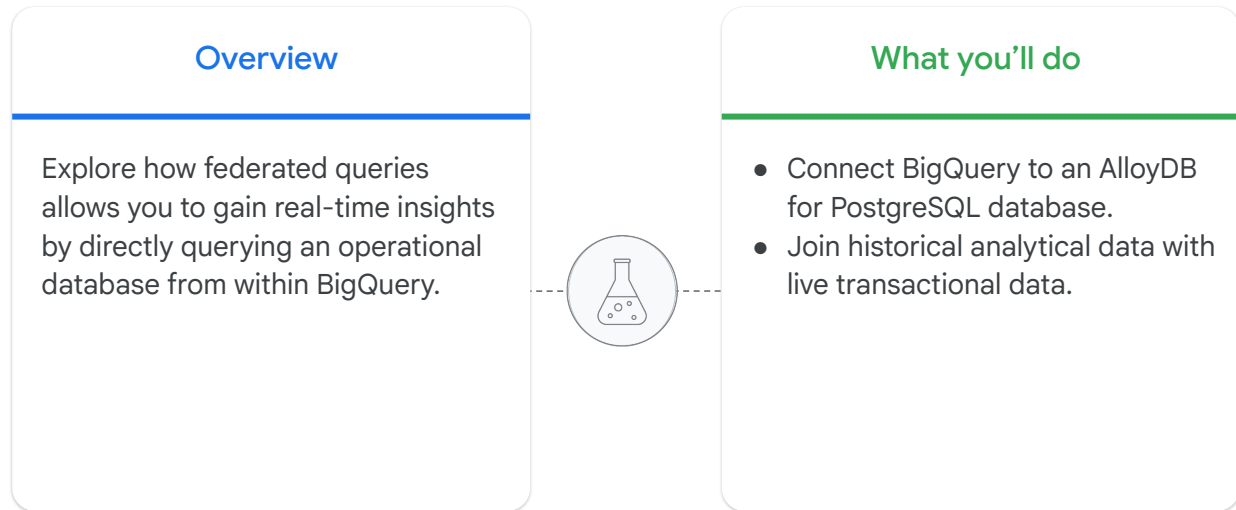
Flow

Next: the vendor's own lab-overview slide — then break, and seats go live at 10:45.

Links

<https://cloud.google.com/bigquery/docs/federated-queries-intro>
<https://cloud.google.com/bigquery/docs/alloydb-federated-queries>
<https://cloud.google.com/alloydb/docs/overview>

Lab: Federated query with BigQuery and AlloyDB



Google Cloud

INSTRUCTOR NOTES — PAGE 64 OF 208 · 09:45 · MODULE 8 — BUILDING A DATA LAKEHOUSE

Hook

The lab's own framing: real-time insight by querying the operational database from inside BigQuery.

Say · cover

- Two tasks: connect BigQuery to an AlloyDB for PostgreSQL database, then join historical analytical data with live transactional rows.
- Keep the mapping in your head: connection equals linked server, EXTERNAL_QUERY equals OPENQUERY, inner dialect equals PostgreSQL.
- API-enable and consent prompts are normal in a fresh lab project — click through.
- Early finishers: query information_schema through the same connection to browse the remote metadata blind.

Iowa tie

The skill transfers one-to-one: Cloud SQL (including SQL Server) and Spanner are federation sources too — same mechanics against their estate.

Design patterns

- Prove which side executes what: run the inner query alone, then add a WHERE inside it

Anti-patterns

- Trying to browse AlloyDB tables in the Explorer pane — federation is query-only, just like OPENQUERY

History & what's new

Federation targets have grown over time: Cloud SQL first, then Spanner, AlloyDB (GA 2024), and SAP Datasphere now in preview (docs, 2026).

Flow

Last slide before the 10:30 break; the lab launches at 10:45 sharp.

Links

<https://cloud.google.com/bigquery/docs/alloydb-federated-queries>
<https://cloud.google.com/bigquery/docs/federated-queries-intro>
<https://cloud.google.com/alloydb/docs/overview>

DIN: Exploring external tables

20 minutes · 11:15–11:35 · BigQuery console + GCS bucket jwd-gcp-demos/orders_partitioned

1. Browse the bucket — directories named order_date=YYYY-MM-DD, each holding Parquet files: Hive layout
2. Create external table ext_part in your class dataset — Parquet, source-data partitioning, auto-detect schema
3. Query the full table — every file scanned; note the bytes processed
4. Filter on order_num (a data column) — still a full scan; a WHERE clause alone saves nothing
5. Filter on order_date (the partition key) — bytes drop hard: partition pruning on raw files, no load job ran

Why it matters — Bronze made tangible — open-format files in a bucket, queryable in standard SQL with zero loading; delete the table and the files are untouched.

Custom slide · added by Imran Ahmad

Iowa DOM DoIT · Data Engineering on Google Cloud

INSTRUCTOR NOTES — PAGE 65 OF 208 · 09:45 · MODULE 8 — BUILDING A DATA LAKEHOUSE

Hook

After the lab, the same magic trick on a different source: not a database — a bucket of Parquet files, queried with zero loading.

Say · cover

- Browse gs://jwd-gcp-demos/orders_partitioned first: folders named order_date=YYYY-MM-DD are the Hive layout — the path is metadata.
- Create external table ext_part: Parquet plus Source data partitioning plus autodetect — safe for Parquet, schemas live inside the files.
- Three queries: full scan; WHERE on order_num still full-scans; WHERE on order_date drops bytes hard.
- That is partition pruning on raw files with no load job; delete the table and every file is untouched.

Iowa tie

Bronze made tangible — the county file-drop replacement; for M&O there is no server, just bucket IAM and lifecycle policy.

Design patterns

- Hive-partition Bronze by date; watch bytes processed on every query

Anti-patterns

- A WHERE clause alone saves nothing — only partition-key filters prune; mind the two URI formats (wildcard versus gs:// prefix)

History & what's new

External tables predate BigLake (launched 2022, TechCrunch); the governed BigLake upgrade path — row and column security over the same files — is the afternoon demo's topic (docs).

Flow

This DIN runs 11:15, right after the lab; next slide wraps M8.

Links

<https://cloud.google.com/bigquery/docs/external-data-cloud-storage>

<https://cloud.google.com/bigquery/docs/hive-partitioned-queries>

<https://cloud.google.com/bigquery/docs/biglake-intro>

M8 — What you now have

- ✓ The Bronze blueprint— Parquet/Iceberg files in GCS, Hive-partitioned, queryable in place from BigQuery
- ✓ Iceberg's warehouse behavior on lake files— ACID, schema evolution, time travel — for the 13:00 lab
- ✓ Federation done hands-on— live operational data joined from BigQuery without an extract
- ✓ An external table over partitioned Parquet— you watched partition pruning cut bytes on raw files
- ✓ Next: M9 (11:35)— modernizing the warehouse itself: partitioning, clustering, BigLake

Custom slide · added by Imran Ahmad

Iowa DOM DoIT · Data Engineering on Google Cloud

INSTRUCTOR NOTES — PAGE 66 OF 208 · 09:45 · MODULE 8 — BUILDING A DATA LAKEHOUSE

Hook

Forty-five minutes ago 'lakehouse' was vocabulary; now it is a blueprint you have seen working end to end.

Say · cover

- Bronze blueprint: Parquet and Iceberg files in GCS, Hive-partitioned, queryable in place from BigQuery.
- Iceberg gives warehouse behavior — ACID, schema evolution, time travel — on lake files; the 13:00 lab goes deep.
- Federation, done hands-on: live operational data joined from BigQuery without an extract.
- An external table over partitioned Parquet: you watched partition pruning cut bytes on raw files.
- Next at 11:35, M9: modernizing the warehouse itself — partitioning, clustering, BigLake.

Iowa tie

Each bullet is a line of their architecture doc: Bronze foundation, single SQL surface, phased migration via federation.

Design patterns

- Land open, serve managed, federate only while migrating

Anti-patterns

- Skipping Bronze discipline (unpartitioned dumps), then wondering why scans are expensive

History & what's new

Everything shown is 2026-current: the Google Cloud Lakehouse rebrand at Next '26 and managed Iceberg tables GA per the 2026 BigQuery release notes.

Flow

Now the 10:30 break — 15 minutes; lab seats are pre-warmed for 10:45.

Links

<https://cloud.google.com/bigquery/docs/iceberg-tables>

<https://cloud.google.com/bigquery/docs/hive-partitioned-queries>

<https://cloud.google.com/bigquery/docs/federated-queries-intro>

Morning Break

15 minutes · 10:30–10:45

Custom slide · added by Imran Ahmad

Iowa DOM DoIT · Data Engineering on Google Cloud

INSTRUCTOR NOTES — PAGE 67 OF 208 · 10:30 · MORNING BREAK (15 MIN)

FACILITATOR NOTE

- Fifteen minutes. Before students step away, have them reopen the Google Skills classroom tab and confirm the lab sign-in works — the 10:45 lab burns time fast when credentials fail.
- If M8's lakehouse vocabulary (Iceberg, federation, open formats) sparked questions, invite them back with those — the next hour answers them hands-on in the console, not on slides.

Flow: M8 (Building a Data Lakehouse) just ended on federated queries; after the break: LAB Federated Query at 10:45, then DIN §30 external tables at 11:15.

Lab — Federated Query with BigQuery

30 minutes — 10:45–11:15 · Google Skills classroom

1. Launch the lab; note the region and use it everywhere — check the project picker
2. Create the connection resource to AlloyDB for PostgreSQL — the linked-server definition
3. Run `EXTERNAL_QUERY(connection_id, '...')` — the inner string is PostgreSQL, executed remotely
4. The payoff: join BigQuery historical data with live AlloyDB rows in one statement
5. Early? List the remote `information_schema.tables` through the same connection

Why it matters — during migration, systems that haven't moved yet stay queryable — no extract job built just to join operational tables against the warehouse.

Custom slide · added by Imran Ahmad

Iowa DOM DoIT · Data Engineering on Google Cloud

INSTRUCTOR NOTES — PAGE 68 OF 208 · 10:45 · LAB — FEDERATED QUERY WITH BIGQUERY (30 MIN)

Hook

Everyone here has registered a linked server and written `OPENQUERY`; today they build the BigQuery equivalent in thirty minutes.

Say · cover

- The connection resource is the linked-server definition — one-time setup, listed under External connections; `EXTERNAL_QUERY(connection_id, '...')` is `OPENQUERY`.
- The inner string runs remotely in the source's dialect — PostgreSQL on AlloyDB. T-SQL habits and backticks fail in the quotes; escape single quotes by doubling.
- Pre-empt the stalls: wrong project in the picker, connection ID not exactly region.connection-id, connection/query region mismatch.
- Insist everyone reaches step 4, the join — BigQuery history JOIN live AlloyDB rows. Early finishers query remote `information_schema.tables`.
- Trade-offs: read-only, source takes the load, on-demand bills bytes returned — a bridge, not a destination.

Iowa tie

The phased-migration bridge: unmigrated systems stay queryable without an extract job built just to join them; Silver/Gold still land as managed tables.

Design patterns

- One governed connection per source, created once and shared — the team's linked-server registry.
- Push filters into the inner query; target a read replica for workload isolation.

Anti-patterns

- Nightly extracts of operational tables just to join them — the job federation kills.
- Federation as the destination: read-only, no storage optimizations, every query taxes the operational store.

History & what's new

Cloud SQL federated queries entered beta August 2019 (BigQuery release notes); sources have since grown to Spanner, AlloyDB, and SAP Datasphere in Preview (2025 docs).

Flow

M8 just introduced federation; at 11:15 the same 'query where it lives' idea moves to files — DIN \$30 external tables.

Links

<https://cloud.google.com/bigquery/docs/federated-queries-intro>

<https://cloud.google.com/bigquery/docs/alloydb-federated-queries>

<https://cloud.google.com/bigquery/docs/cloud-sql-federated-queries>

Exploring external tables — Bronze made tangible (DIN §30)

20 minutes — 11:15–11:35 · BigQuery console — instructor leads, you follow; your class dataset

1. First browse the bucket: orders_partitioned/ is Hive-laid-out — order_date=YYYY-MM-DD/ Parquet
2. Create from GCS: pattern jwd-gcp-demos/orders_partitioned/* — Parquet, Source data partitioning on
3. URI prefix gs://jwd-gcp-demos/orders_partitioned/ ; table ext_part, type External, Auto detect
4. Query 1: SELECT * — full scan. Query 2: WHERE order_num=... — still a full scan (data column)
5. Query 3: WHERE order_date='2018-01-01' — bytes collapse: partition pruning on raw files

Why it matters — open-format files in a bucket, partitioned by date, queryable in SQL with zero loading — your architecture doc's Bronze layer is exactly this.

Custom slide · added by Imran Ahmad

Iowa DOM DoIT · Data Engineering on Google Cloud

INSTRUCTOR NOTES — PAGE 69 OF 208 · 11:15 · DIN — EXPLORING EXTERNAL TABLES (20 MIN)

Hook

Today a county file drop means an SSIS package and a staging table before anyone can query it; here the file is the table.

Say · cover

- Browse the bucket first: order_date=YYYY-MM-DD/ folders are Hive layout — the path is metadata.
- Two URI fields, different formats: file pattern jwd-gcp-demos/orders_partitioned/* (wildcard, no gs://) versus Source URI Prefix gs://jwd-gcp-demos/orders_partitioned/ (gs://, no wildcard).
- Tick Source data partitioning; type External; Auto detect is safe — Parquet carries its schema, unlike yesterday's CSV traps.
- Watch bytes processed: SELECT * full scan; WHERE order_num scans everything — a data-column filter saves nothing; WHERE order_date prunes partitions on raw files.
- Table details show no stored bytes — a metadata pointer; deleting it leaves every file untouched.

Iowa tie

Bronze made tangible — their architecture doc's landing layer; for M&O it's bucket IAM and lifecycle policies, not servers to patch.

Design patterns

- Partition Bronze by date at the folder level so pruning works from day one.
- Open formats for landing/exploration; managed tables for serving; BigLake adds fine-grained security.

Anti-patterns

- Reflex-copying files into the warehouse before anyone can query them.
- Assuming any WHERE saves money — non-partition filters still scan everything.

History & what's new

Hive-partitioned external data on GCS arrived March 2020 (release notes archive); CREATE EXTERNAL TABLE DDL followed that October. 2026 upgrade: Apache Iceberg managed tables — partitioning and multi-statement transactions now GA.

Flow

Same idea as the lab, now on files. Next: M9 partitioning/clustering at 11:35; ext_part returns in the 13:00 Iceberg lab.

Links

<https://cloud.google.com/bigquery/docs/external-data-cloud-storage>
<https://cloud.google.com/bigquery/docs/hive-partitioned-queries>
<https://cloud.google.com/bigquery/docs/biglake-intro>

WATCH FIRST · 2:02 · GOOGLE CLOUD TECH

BigLake in a Minute

- ✓ One storage engine unifying warehouse and lake: fine-grained security and fast queries over open files — a single copy of the data.
- ✓ This is the upgrade on this afternoon's Iceberg lab: BigLake tables are how Bronze files get governed like warehouse tables.



Scan to open on YouTube
youtu.be/UdpNYk0EP0g

Custom slide · added by Imran Ahmad

Iowa DOM DoIT · Data Engineering on Google Cloud

INSTRUCTOR NOTES — PAGE 70 OF 208 · 11:35 · MODULE 9 — MODERNIZING DATA WAREHOUSES

Hook

- Play to open M9 — 'BigLake' is in the module title; give it a face before the modernization slides.

Say

- "Yesterday's external tables were the bridge; BigLake is the destination — same files, warehouse-grade governance."

Cue

- Only 2:02 — play it twice if the room looks unconvinced.

Link

- <https://www.youtube.com/watch?v=UdpNYk0EP0g>

M9 — What this block gets you

- ✓ The 'traditional data warehouse challenges' on the next two slides are your SQL Server pain points— read them that way
- ✓ BigQuery fundamentals— serverless, storage/compute separation, slots and shuffle in plain terms
- ✓ Partitioning and clustering— your partition functions and clustered indexes, minus the maintenance jobs
- ✓ Iceberg partitioning and predicate pushdown— open metadata pruning files that live in your own bucket
- ✓ Both are a preview— the 3-day Data Warehousing with BigQuery course goes deep; today is the decision rules

Custom slide · added by Imran Ahmad

Iowa DOM DoIT · Data Engineering on Google Cloud

INSTRUCTOR NOTES — PAGE 71 OF 208 · 11:35 · MODULE 9 — MODERNIZING DATA WAREHOUSES

Hook

This is the heart of your migration case: the 'traditional data warehouse challenges' on the next two slides are your SQL Server pain points in disguise — read them as your own checklist.

Say · cover

- Frame slides 4-5: Cymbal is fictional; translate every bullet into DOM DoIT terms — nightly extract window, month-end, patching weekends.
- BigQuery fundamentals next: serverless, storage/compute separation, slots and shuffle — the vocabulary every later module reuses.
- Partitioning and clustering map to your partition functions and clustered indexes, minus the maintenance jobs.
- Iceberg partitioning and predicate pushdown preview what DIN §30 just showed you: metadata pruning files in your own bucket.
- Set expectation: today is decision rules; the 3-day Data Warehousing with BigQuery course goes deep on the knobs.

Iowa tie

This block argues for retiring the SQL Server warehouse: kill-the-extracts starts by deleting the infrastructure line items your M&O team owns today.

Design patterns

- Teach pain-to-removal pairs, one per infrastructure line item.
- Preview-and-defer deep tuning to the follow-on DW course.

Anti-patterns

- Do not pitch BigQuery as just a bigger SQL Server.
- Do not dive into slot math before the room feels the pain points.

History & what's new

BigQuery launched in 2010 on Google's internal Dremel engine (in production there since 2006) and went GA in November 2011; the VLDB 2020 retrospective 'Dremel: A Decade of Interactive SQL Analysis at Web Scale' documents how the engine matured.

Flow

Straight from DIN §30's external-table pruning demo into the module that explains the engine behind it.

Links

<https://cloud.google.com/bigquery/docs/introduction>
<https://cloud.google.com/blog/products/bigquery/bigquery-under-the-hood>
<https://www.vldb.org/pvldb/vol13/p3461-melnik.pdf>

Late morning into early afternoon

11:35–12:00	M9 — Modernizing DWs with BigQuery & BigLake (opens with your pain points)	
12:00–13:00	Lunch	
13:00–13:45	Lab: Querying External Data and Iceberg Tables — your Bronze pattern, hands-on	LAB
13:45–14:00	Iceberg deep-dive — what's under the table you just built	DEMO
14:00	M10 — governance + medallion: your Bronze/Silver/Gold in Google's own deck	

FACILITATOR NOTE

- Orient the room: M9 runs 11:35-12:00 and opens with their SQL Server pain points; after lunch the 13:00 lab makes partitioning hands-on with external data and Iceberg tables — their Bronze pattern.
- Then the 13:45 Iceberg deep-dive demo shows what is under the table they just built, and 14:00 M10 lands governance plus the Bronze/Silver/Gold medallion in Google's own deck.
- Flag the through-line: DIN §30 (just finished) showed partition pruning on files; M9 now explains the engine; the lab applies both.

Flow: Agenda divider between DIN §30 and M9 content; lunch follows the module.

Traditional data warehouse challenges (1 of 2)

Cymbal's data architects face significant challenges with traditional data warehouses:

01

Extensive time spent on capacity planning for events, such as the holiday shopping season.

03

Manual provisioning of hardware, software installation, and patching.

02

Difficulty handling unexpected traffic spikes.

04

Rebalancing data manually as it grows.

Google Cloud

INSTRUCTOR NOTES — PAGE 73 OF 208 · 11:35 · MODULE 9 — MODERNIZING DATA WAREHOUSES

Hook

Cymbal is a fiction — these four bullets are your SQL Server life. Ask the room: which one costs your team the most hours per month?

Say · cover

- 01 Capacity planning for peak events: your box is sized for the nightly extract window and month-end — the batch schedule dictates the whole server.
- 02 Unexpected traffic spikes: sized-for-average stalls; sized-for-peak idles most of the day. You have lived both.
- 03 Manual provisioning: procurement cycles, installation, patching weekends — work your M&O team owns today.
- 04 Manual rebalancing as data grows: new drives, filegroup shuffles, archiving old partitions by hand.
- Land the point: none of these are data problems — every one is an infrastructure problem.

Iowa tie

This is your current-state slide: the SQL Server warehouse behind the legacy extracts, sized for the nightly window, with M&O carrying provisioning.

Design patterns

- Inventory the room's own pain items live — they become migration justifications.

Anti-patterns

- Do not let the audience read these as generic retail problems; rename each one in their terms.

History & what's new

Google's 2010 BigQuery launch (GA November 2011) was aimed squarely at these on-prem pains; Google's 2016 'BigQuery under the hood' post explained the architecture built to erase them.

Flow

First of two pain slides — p071 adds the operational-overhead punchline, then p072 deletes each item.

Links

<https://cloud.google.com/bigquery/docs/introduction>

<https://cloud.google.com/architecture/dw2bq/dw-bq-migration-overview>

<https://cloud.google.com/blog/products/bigquery/bigquery-under-the-hood>

Traditional data warehouse challenges (2 of 2)

Operational overhead is **expensive** and **slows down** the ability of the analytics teams to get **timely insights**.

01

Extensive time spent on capacity planning for events, such as the holiday shopping season.

03

Manual provisioning of hardware, software installation, and patching.

02

Difficulty handling unexpected traffic spikes.

04

Rebalancing data manually as it grows.

Google Cloud

INSTRUCTOR NOTES — PAGE 74 OF 208 · 11:35 · MODULE 9 — MODERNIZING DATA WAREHOUSES

Hook

Same four pains, now with the bill attached: operational overhead is the line item that slows your analysts down.

Say · cover

- Read the takeaway line twice: overhead is expensive AND slows time-to-insight — the cost is dollars plus latency of answers.
- Quantify it in class: hours per month your team spends on capacity planning, patching, and rebalancing — put a rough number on the whiteboard.
- For a state agency, procurement cycles make every growth step a quarter-long project; that lead time is part of the pain.
- Bridge: each item is an infrastructure problem, so the fix is removing infrastructure — not tuning it harder.

Iowa tie

Legacy extracts persist partly because the warehouse is too precious to touch; serverless removes that scarcity, which is what unlocks kill-the-extracts.

Design patterns

- Convert the pain list into the migration business case: hours, dollars, risk.

Anti-patterns

- Do not promise BigQuery fixes badly modeled data — it deletes infrastructure, not design debt.

History & what's new

Even cloud capacity planning keeps shrinking: BigQuery Editions with slot autoscaling went GA on March 29, 2023, and flat-rate slot purchasing was retired on July 5, 2023.

Flow

Pain list complete — next is the custom slide that maps every pain to its BigQuery deletion.

Links

<https://cloud.google.com/architecture/dw2bq/dw-bq-migration-overview>

<https://cloud.google.com/blog/products/data-analytics/introducing-new-bigquery-pricing-editions>

<https://cloud.google.com/bigquery/docs/introduction>

Those five challenges are yours — this is the delete key

Each one is an infrastructure problem — the fix is to remove the infrastructure.

- Capacity planning for peak events— serverless slots, allocated per query and released when it finishes
- Sized for peak, idle the rest of the day— storage and compute scale and bill independently
- Procurement, installation, patching weekends— fully managed; that M&O; line item disappears
- Rebalancing data as it grows— storage scales automatically; no new drives, no filegroup shuffles
- The nightly window sized the whole box— this engine was built for exactly the scans you avoid
- Slow answers for analysts— thousands of slots per query: a wider machine, not a faster one

Custom slide · added by Imran Ahmad

Iowa DOM DoIT · Data Engineering on Google Cloud

INSTRUCTOR NOTES — PAGE 75 OF 208 · 11:35 · MODULE 9 — MODERNIZING DATA WAREHOUSES

Hook

Here is the whole migration pitch on one slide: each of your pains dies the same death — the infrastructure it lives on disappears.

Say · cover

- Walk the rows: capacity planning for peaks -> serverless slots are allocated per query and released when it finishes.
- Sized-for-peak-idle-all-day -> storage and compute scale and bill independently, so nothing sits idle.
- Procurement, installation, patching weekends -> fully managed; that M&O line item disappears.
- Rebalancing and filegroup shuffles -> storage scales automatically; no drives to add.
- Slow answers -> thousands of slots per query: a wider machine, not a faster one — have them write that phrase down.

Iowa tie

This is the ROI slide for kill-the-extracts: the M&O hours freed here get reallocated to Dataform, CI/CD, and Power BI work.

Design patterns

- Answer 'how big is the server?' with per-query allocation — there is no server.

Anti-patterns

- Do not claim zero operations — IAM, cost governance, and monitoring remain (M10 this afternoon).

History & what's new

Serverless warehousing was BigQuery's founding 2010 bet; by 2023 even capacity buying went autoscaling-first with Editions (GA March 29), removing the last manual capacity-planning step.

Flow

Pains deleted — next, the engine that does the deleting: BigQuery fundamentals.

Links

<https://cloud.google.com/bigquery/docs/introduction>

<https://cloud.google.com/bigquery/docs/editions-intro>

<https://cloud.google.com/blog/products/bigquery/bigquery-under-the-hood>

BigQuery

Enterprise-grade, fully managed, serverless data warehouse.



Google Cloud

INSTRUCTOR NOTES — PAGE 76 OF 208 · 11:35 · MODULE 9 — MODERNIZING DATA WAREHOUSES

Hook

One sentence to memorize — enterprise-grade, fully managed, serverless. Each adjective is a direct answer to a pain point the room just listed.

Say · cover

- Enterprise-grade: replication, high availability, and security controls a state agency can audit — not a startup toy.
- Fully managed: no hardware, no patching; the operational burden moves to Google.
- Serverless: there is no provisioning step at all — you load data and run SQL.
- Console anchor: they have been in BigQuery Studio all morning — the same place the federated-query lab ran; there is no instance or VM anywhere.
- It is a warehouse plus more (ML, geospatial, BI integration), but today its job is replacing their SQL Server DW and feeding Power BI.

Iowa tie

This is the target platform for the single source of truth: native BQ storage for Silver/Gold, Power BI on top.

Design patterns

- Repeat the three adjectives whenever a pain point resurfaces later today.

Anti-patterns

- Do not describe it as just a database — it is a query engine and a storage system sharing a SQL front door.

History & what's new

Launched 2010 and GA November 2011, BigQuery passed 15 years of service in 2025 and is still expanding — Google Cloud Next 2026 added native BigQuery Graphs with ISO GQL, well beyond classic warehousing.

Flow

Definition in hand — next slide unpacks fully managed and serverless word by word.

Links

<https://cloud.google.com/bigquery/docs/introduction>
<https://cloud.google.com/bigquery/docs/release-notes>
<https://cloud.google.com/bigquery/docs/editions-intro>

BigQuery fundamentals

Fully managed

The infrastructure (such as the hardware, the networking, and the low-level software) is all handled by Google. Your team doesn't need to worry about patches, updates, or hardware failures.

Serverless

Serverless takes things a step further. You don't have to provision or manage any servers at all. You simply load your data and start querying. BigQuery automatically allocates the necessary resources to run your queries and scales them up or down based on the complexity of your request.

Google Cloud

 INSTRUCTOR NOTES — PAGE 77 OF 208 · 11:35 · MODULE 9 — MODERNIZING DATA WAREHOUSES

Hook

Fully managed means Google's SREs become your warehouse M&O; serverless means you cannot even see the servers they are managing.

Say · cover

- Fully managed: hardware, networking, low-level software, patches, failures — all Google. Your M&O scope shrinks to data, access, and cost, not machines.
- Serverless goes further: no servers to provision or manage; load data and start querying — resources are allocated automatically per request.
- Scaling is per-query and automatic: a heavier query simply gets more resources; nothing to resize, no failover cluster to configure.
- Contrast with their world: Always On listener setup, patching windows, weekend maintenance — none of that exists here.
- Note the slide's typo 'Severless' if asked; the concept stands.

Iowa tie

For the M&O team: this is the line item that disappears — redeploy those hours to pipeline quality and CI/CD around Dataform.

Design patterns

- Treat the remaining ops surface as governance: IAM, cost visibility, monitoring.

Anti-patterns

- Do not promise 'no administration' — access control and cost governance become more important, not less.

History & what's new

Serverless was radical at the 2010 launch; Google doubled down in 2023 when Editions (GA March 29) made autoscaling the default capacity model and flat-rate purchasing ended July 5, 2023.

Flow

Managed and serverless defined — next, the architectural fact that makes both possible: storage/compute separation.

Links

<https://cloud.google.com/bigquery/docs/introduction>

<https://cloud.google.com/bigquery/docs/editions-intro>

<https://cloud.google.com/blog/products/data-analytics/introducing-new-bigquery-pricing-editions>

The magic behind BigQuery is its architecture. BigQuery separates storage from compute.

Google Cloud

INSTRUCTOR NOTES — PAGE 78 OF 208 · 11:35 · MODULE 9 — MODERNIZING DATA WAREHOUSES

Hook

This is the single architectural fact everything else today follows from — if they remember one M9 slide, make it this one.

Say · cover

- In SQL Server, growing data and growing horsepower are one purchase: one box, one license tier, one painful upgrade.
- BigQuery splits them: a replicated distributed storage system and a stateless compute fleet, joined by Google's petabit network.
- Consequence one: each side scales independently and bills independently — two separate dials.
- Consequence two: the same compute can read data that lives outside BQ storage — your GCS Bronze files (DIN \$30) and federated sources, without loading.
- Pause and draw the two boxes; this mental model is reused in every module after lunch.

Iowa tie

Your Bronze layer lives in Cloud Storage while Silver/Gold live in native BQ storage — one engine over both is exactly this separation.

Design patterns

- Keep compute stateless; let storage grow without compute conversations.

Anti-patterns

- Do not map BQ storage to 'the MDF file' — there is no file to size, grow, or rebalance.

History & what's new

Separation was Dremel's 2010-era bet, publicized in Google's 2016 'BigQuery under the hood'; it became the lakehouse industry default, and in January 2025 BigQuery extended it to open formats with BigQuery tables for Apache Iceberg (preview).

Flow

The fact stated — next slide shows the whole machine: Dremel, storage, shuffle, network.

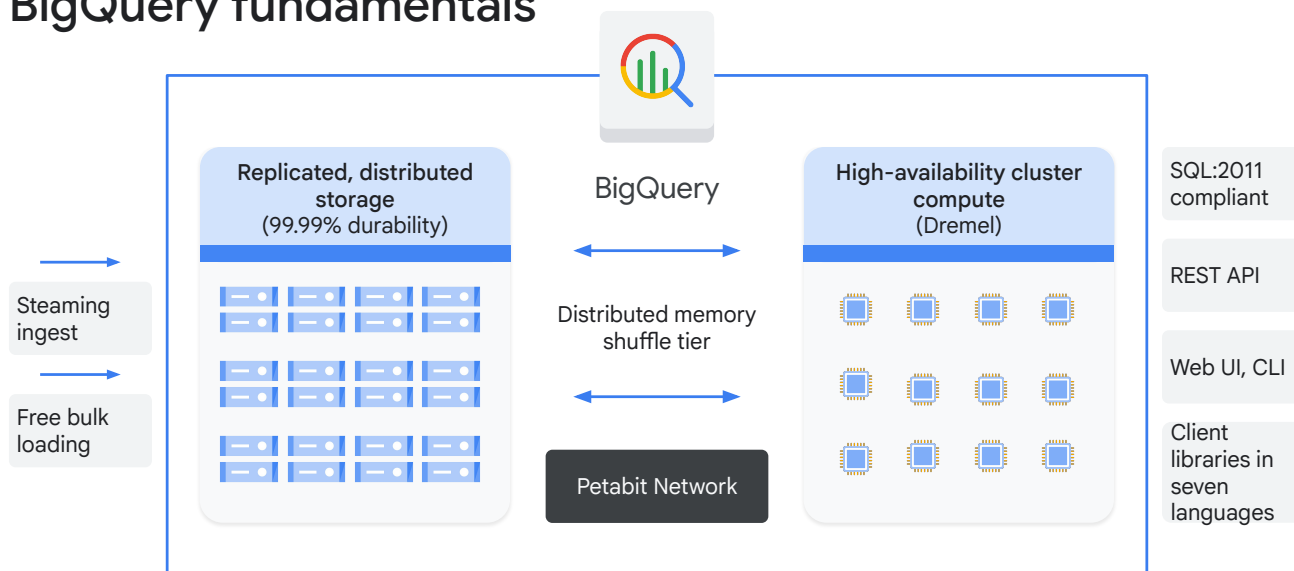
Links

<https://cloud.google.com/blog/products/bigquery/bigquery-under-the-hood>

<https://cloud.google.com/bigquery/docs/introduction>

<https://cloud.google.com/blog/products/data-analytics/bigquery-tables-for-apache-iceberg>

BigQuery fundamentals



Google Cloud

INSTRUCTOR NOTES — PAGE 79 OF 208 · 11:35 · MODULE 9 — MODERNIZING DATA WAREHOUSES

Hook

Every box on this diagram answers a question your team will ask in the first month — walk it slowly, top to bottom.

Say · cover

- Compute: high-availability Dremel clusters — thousands of slots per query, stateless, borrowed and released.
- Storage: replicated, distributed columnar storage; note free bulk loading — loading is not where the money goes.
- Shuffle tier over the petabit network: the in-memory fabric that redistributes intermediate results — why JOINS and GROUP BYs scale.
- Interfaces: web UI, bq CLI, REST API, client libraries in seven languages, SQL:2011-compliant GoogleSQL — your T-SQL habits mostly transfer.
- Streaming ingest exists alongside free batch loads; your pattern is mostly batch, with Datastream (yesterday) handling replication.

lowa tie

Free bulk loading matters for the migration itself: reloading history from SQL Server extracts costs query time, not load fees.

Design patterns

- Use batch loads for history; reserve streaming ingest for genuinely real-time feeds.

Anti-patterns

- Do not row-by-row INSERT for ELT — BigQuery is scan-optimized, not OLTP; batch the writes.

History & what's new

This is Dremel's 2010 architecture matured; Google's 2016 'under the hood' post named the pieces, and the VLDB 2020 'Decade of Dremel' paper details how the shuffle tier and storage evolved over the decade.

Flow

The machine shown — next, why its two halves scaling independently changes your cost math.

Links

<https://cloud.google.com/blog/products/bigquery/bigquery-under-the-hood>

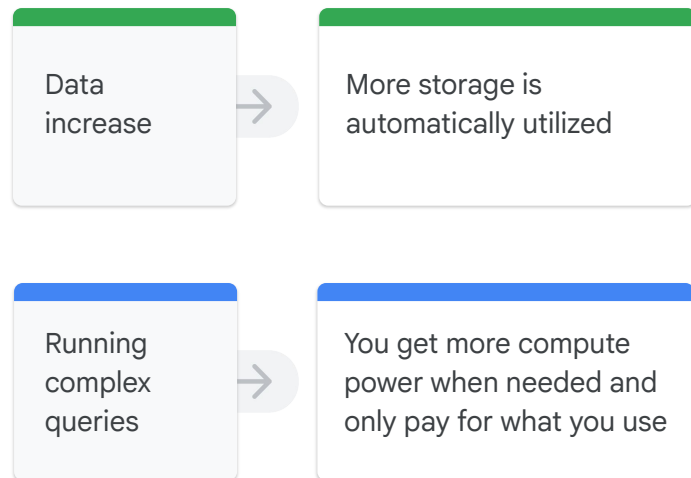
<https://www.vldb.org/pvldb/vol13/p3461-melnik.pdf>

<https://cloud.google.com/bigquery/docs/introduction>

BigQuery fundamentals

Because storage and compute are separate, they can scale independently.

This separation is a game changer for managing costs and ensuring performance.



Google Cloud

INSTRUCTOR NOTES — PAGE 80 OF 208 · 11:35 · MODULE 9 — MODERNIZING DATA WAREHOUSES

Hook

In your world, 'the server is full' means a budget request. Here it means nothing — storage and compute never ask each other for permission.

Say · cover

- Data increases -> more storage is automatically utilized; no drive purchase, no filegroup work.
- A complex query -> more compute for that query only; you pay for what you use, then it returns to the pool.
- This separation is the game changer: cost and performance become two problems you tune separately instead of one oversized box.
- Billing consequence: storage (\$/GB-month) and queries (\$/bytes-scanned on-demand, or slot-hours on editions) are independent lines.
- Month-end analogy: the query borrows thousands of slots for minutes instead of you owning peak capacity 24x7.

Iowa tie

Silver->Gold Dataform runs borrow big compute briefly; Power BI refreshes borrow little — same dataset, independent bills.

Design patterns

- Tune storage lifecycle (long-term pricing) and query bytes separately.

Anti-patterns

- Do not size a reservation on day one by copying the old server's core count — measure slot usage first.

History & what's new

On-demand bytes-scanned pricing embodied this split from 2011; BigQuery Editions with autoscaling (GA March 2023) added the capacity-side dial — two pricing models, same separation principle.

Flow

Scaling understood — the next slides define the two speed units: slots and shuffle.

Links

<https://cloud.google.com/bigquery/docs/editions-intro>
<https://cloud.google.com/bigquery/docs/best-practices-costs>
<https://cloud.google.com/bigquery/docs/introduction>

BigQuery achieves its speed through two core concepts: **slots and shuffle**.

Google Cloud

INSTRUCTOR NOTES — PAGE 81 OF 208 · 11:35 · MODULE 9 — MODERNIZING DATA WAREHOUSES

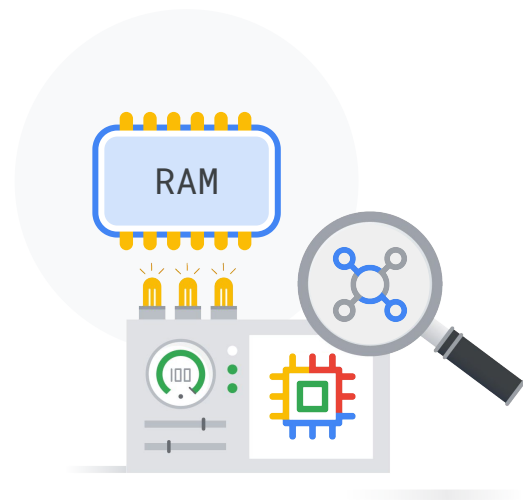
FACILITATOR NOTE

- Section divider: BigQuery's speed comes from two concepts — slots, the unit of compute, and shuffle, the re-sort between execution steps.
- Give the SQL Server mapping up front: slots are loosely your degree-of-parallelism workers; shuffle is the exchange/repartition operator in an execution plan — but both are elastic, per query, and gone when the query ends.
- Next four slides define each; keep the two terms paired in their notes.

Flow: Divider into the slots/shuffle pair (p079-082) after the independent-scaling slide.

What is a slot?

A small, self-contained unit of computational power that includes **CPU**, **RAM**, and **network bandwidth**.



Google Cloud

INSTRUCTOR NOTES — PAGE 82 OF 208 · 11:35 · MODULE 9 — MODERNIZING DATA WAREHOUSES

Hook

A slot is BigQuery's bundle of compute — the smallest piece of parallelism your query can rent.

Say · cover

- Definition: a small, self-contained unit of CPU, RAM, and network bandwidth.
- You never provision or count slots manually; BigQuery decides per query based on data volume and query complexity.
- SQL Server mapping: like MAXDOP workers, except elastic and drawn from a fleet — thousands available, not 8 or 16.
- Two ways to pay, name only: on-demand (per TiB scanned, with a slot cap plus bursting) or capacity reservations under Editions — details belong to the DW course.
- If a query wants more slots than are available, work units queue — that 'slot contention' is the BQ cousin of your server pegged at 100% CPU.

lowa tie

Month-end Power BI refresh versus a big Dataform rebuild: both simply ask for more slots — no resize ticket, no Change Advisory Board.

Design patterns

- Start on-demand; measure slot-ms in INFORMATION_SCHEMA before buying reservations.

Anti-patterns

- Do not translate slots 1:1 into on-prem cores — a slot-hour is a parallelism/billing unit, not a spec sheet.

History & what's new

Slots have been BigQuery's compute unit since launch; in 2023 Editions (GA March 29) made autoscaling slots the default capacity model and flat-rate purchasing ended July 5, 2023.

Flow

Slot defined — next, how the Dremel engine spends thousands of them on one query.

Links

<https://cloud.google.com/bigquery/docs/slots>

<https://cloud.google.com/bigquery/docs/editions-intro>

<https://cloud.google.com/bigquery/docs/reservations-intro>

Slots

01

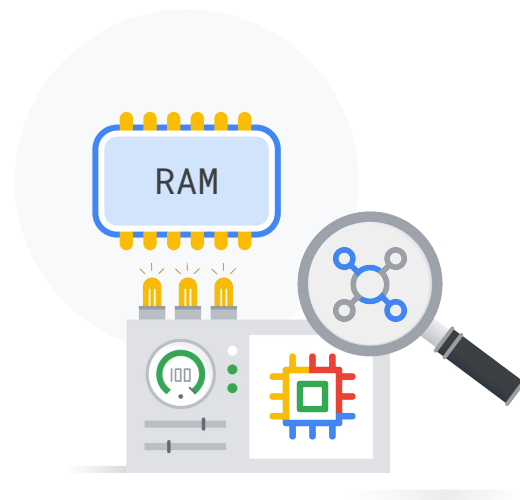
BigQuery's Dremel engine assigns thousands of slots to a query job.

02

Each slot processes a small piece of data simultaneously.

03

This massively parallel processing enables BigQuery to scan terabytes of data quickly.



Google Cloud

INSTRUCTOR NOTES — PAGE 83 OF 208 · 11:35 · MODULE 9 — MODERNIZING DATA WAREHOUSES

Hook

Here is the trick that scans your biggest table in seconds: not a faster machine — thousands of ordinary ones at once.

Say · cover

- The Dremel engine assigns thousands of slots to a single query job; each slot processes a small piece of the data simultaneously.
- That massively parallel processing is how terabyte scans return in seconds: the scan is spread wide, then reduced.
- Under the hood: SQL compiles to a plan of stages; stages run in parallel, and the plan is dynamic — BigQuery can repartition mid-flight and use speculative execution on stragglers.
- Demo hook for later: open a query's Execution details in the console and show slot-ms consumed per stage.
- Fair scheduling redistributes slots across concurrent queries — nobody's dashboard waits long behind your rebuild.

Iowa tie

This is why Gold-layer Dataform builds over Silver can be full set-based recomputes instead of fragile incremental SSIS logic — wide scans are cheap.

Design patterns

- Prefer set-based full transforms over row-by-row incremental plumbing when in doubt.

Anti-patterns

- Do not assume more slots rescue a badly filtered query — it just burns bytes faster.

History & what's new

Dremel's tree-of-servers execution was published at VLDB 2010; the 2020 decade retrospective details how dynamic query plans and speculative execution matured in production.

Flow

Slots covered — the other half of speed, shuffle, is next.

Links

<https://cloud.google.com/bigquery/docs/slots>

<https://www.vldb.org/pvldb/vol13/p3461-melnik.pdf>

<https://cloud.google.com/bigquery/docs/query-plan-explanation>

What is shuffle?

The process of redistributing the intermediate data that the slots have processed.



Google Cloud

INSTRUCTOR NOTES — PAGE 84 OF 208 · 11:35 · MODULE 9 — MODERNIZING DATA WAREHOUSES

Hook

Every JOIN and GROUP BY you have ever written had a hidden data-mover behind it — BigQuery just made that mover a first-class, in-memory service.

Say · cover

- Definition: shuffle redistributes the intermediate data that slots have processed, so the next stage can aggregate or join it.
- It is triggered by GROUP BY, JOIN, DISTINCT, window functions — the exchange operators you know from SQL Server execution plans.
- BigQuery's version is a dedicated distributed in-memory shuffle tier over a petabit network — not tempdb spilling to disk.
- Skew warning: a lopsided join key overloads a few slots — the same hotspot problem as skewed parallelism on-prem.
- Console habit: Execution details shows bytes shuffled per stage — your new 'tempdb contention' signal.

lowa tie

Your Silver->Gold joins (fact x dimension) will live or die by shuffle volume — this is why join-key choices resurface in M10's modeling talk.

Design patterns

- Filter early so joins move fewer rows; watch bytes-shuffled in execution details.

Anti-patterns

- Do not join two huge unfiltered tables and blame the engine — shuffle volume is self-inflicted.

History & what's new

Google's 2016 'under the hood' post introduced the in-memory shuffle tier publicly; the VLDB 2020 decade paper records its evolution into the petabit-scale fabric shown on the diagram.

Flow

Shuffle defined — next slide shows it inside the query lifecycle.

Links

<https://cloud.google.com/blog/products/bigquery/bigquery-under-the-hood>
<https://www.vldb.org/pvldb/vol13/p3461-melnik.pdf>
<https://cloud.google.com/bigquery/docs/query-plan-explanation>

Shuffle

- 01 Gathers and reorganizes data.
- 02 Sends it to the next set of slots for further processing, such as aggregation or joining.
- 03 Executes complex analytical queries efficiently at a massive scale.



Google Cloud

INSTRUCTOR NOTES — PAGE 85 OF 208 · 11:35 · MODULE 9 — MODERNIZING DATA WAREHOUSES

Hook

Same shuffle, now in sequence: gather, re-key, hand to the next set of slots — the conveyor belt between your query's stages.

Say · cover

- Step 1: slots finish a stage (scan + filter) holding partial results.
- Step 2: shuffle gathers and reorganizes rows by key — `customer_id`, `county_code` — so equal keys land together.
- Step 3: the next set of slots aggregates or joins those co-located rows; repeat per stage until the final result.
- This stage-shuffle-stage loop is what makes a GROUP BY over billions of rows a seconds-scale operation.
- Tie back to the architecture slide: the shuffle tier and petabit network are the box between Dremel and storage.

Iowa tie

When Power BI dashboards feel slow post-migration, bytes-shuffled in execution details is your first diagnostic — teach the M&O team where to look.

Design patterns

- Design Gold marts to shuffle less: pre-aggregate and denormalize hot dimensions.

Anti-patterns

- Avoid cross joins and joins on non-key expressions in big ELT — they explode shuffle.

History & what's new

The stage-shuffle-stage model is Dremel's 2010 design, documented in the VLDB 2020 retrospective; today's console exposes it directly — per-stage bytes-shuffled in the query execution graph.

Flow

Speed anatomy done — next slide pivots: the same separation also lets BigQuery query many sources.

Links

<https://cloud.google.com/bigquery/docs/query-plan-explanation>
<https://cloud.google.com/blog/products/bigquery/bigquery-under-the-hood>
<https://www.vldb.org/pvldb/vol13/p3461-melnik.pdf>

BigQuery's compute and storage separation enables versatile **multisource data querying**.

Google Cloud

INSTRUCTOR NOTES — PAGE 86 OF 208 · 11:35 · MODULE 9 — MODERNIZING DATA WAREHOUSES

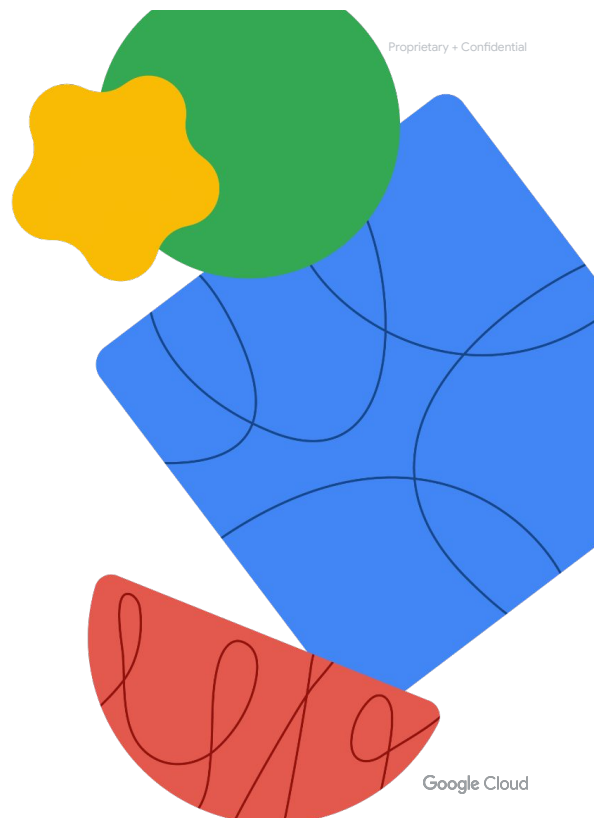
FACILITATOR NOTE

- Transition slide: because compute is separate from storage, the same Dremel engine can query native BigQuery storage, files in Cloud Storage, and federated sources in one SQL statement.
- Remind them they already did this today: the 10:45 federated-query lab and DIN §30's external tables — Bronze-in-GCS plus Silver/Gold-in-BQ under one SQL roof.
- This versatility is the bridge into partitioning and clustering, next.

Flow: Bridge from engine internals into the partitioning/clustering subsection.

Partitioning and clustering

Two powerful concepts you can use to allow BigQuery to process your data efficiently and cost-effectively.

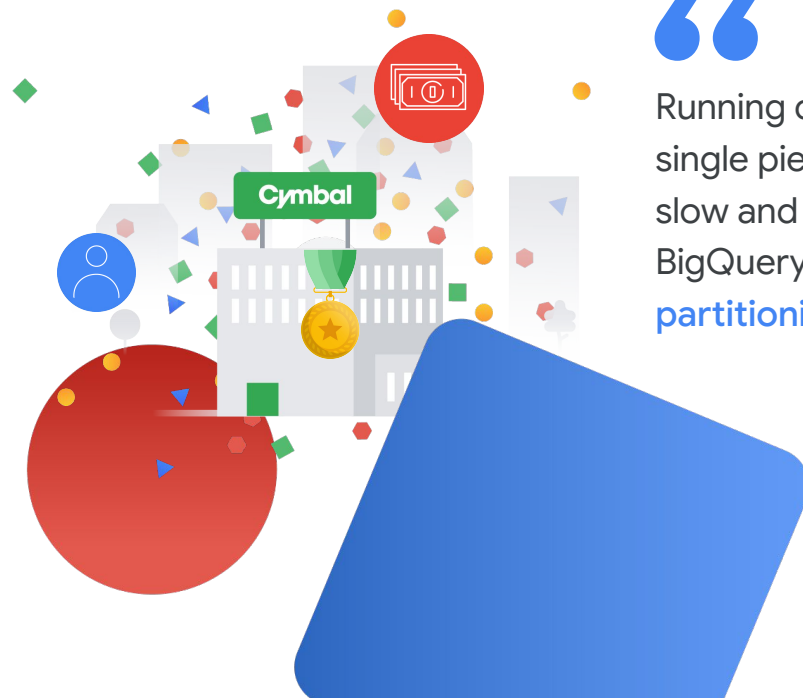


Google Cloud

FACILITATOR NOTE

- Section divider: partitioning and clustering are the two levers that keep big-table queries fast and cost-effective.
- Preview the mapping: partitioning is their SQL Server partition function and scheme; clustering is the clustered index sort order — but both are declared once at CREATE TABLE and then maintained automatically in the background.
- Next slides: motivation (full scans cost money), definitions, then the sales-table example.

Flow: Divider opening the partition/cluster subsection; motivation slide follows.



Running queries that scan every single piece of data can become slow and expensive, even for BigQuery. You can optimize this with **partitioning and clustering**.



Google Cloud

INSTRUCTOR NOTES — PAGE 88 OF 208 · 11:35 · MODULE 9 — MODERNIZING DATA WAREHOUSES

Hook

BigQuery will happily scan a petabyte for you — and bill you for a petabyte. This slide is where cost discipline starts.

Say · cover

- On-demand pricing is bytes-scanned: `SELECT *` without a filter is a financial event, not just a slow query.
- Even BigQuery slows when every query reads everything; the fix is reading less, not bigger hardware.
- The two levers: partitioning prunes whole date ranges; clustering prunes storage blocks within them.
- Console habit: the query validator shows bytes-to-be-processed before you hit Run — that number is your meter; dry runs make costs predictable.
- Partitioned tables give accurate pre-run cost estimates because pruned bytes are known up front.

Iowa tie

With ~10 developers plus Power BI service accounts hitting Gold marts daily, unpartitioned fact tables would be the most avoidable line in their GCP bill.

Design patterns

- Always filter on the partition column; check validator bytes before running.

Anti-patterns

- `SELECT *` with `LIMIT` is not cheap — `LIMIT` caps rows returned, not bytes scanned; without a filter you still pay for the full read.

History & what's new

Per-byte on-demand pricing dates to BigQuery's 2011 GA; Editions (GA March 2023) added a capacity alternative, but the bytes-scanned discipline on this slide applies under both models.

Flow

Motivation stated — next: partitioning, the first lever.

Links

<https://cloud.google.com/bigquery/docs/best-practices-costs>
<https://cloud.google.com/bigquery/docs/estimate-costs>
<https://cloud.google.com/bigquery/docs/partitioned-tables>

Partitioning

Partitioning is like organizing data into separate drawers for year, month, and day. In BigQuery, you can partition a table based on a date or an integer column.



Google Cloud

 INSTRUCTOR NOTES — PAGE 89 OF 208 · 11:35 · MODULE 9 — MODERNIZING DATA WAREHOUSES

Hook

You already know this concept — it is your SQL Server partition function and scheme, minus the sliding-window maintenance job.

Say · cover

- Partitioning divides a table into segments — the drawers — using one column: a DATE/TIMESTAMP/DATETIME column (hourly, daily, monthly, yearly), ingestion time, or integer ranges.
- Declared once at CREATE TABLE; rows route to the right partition automatically on write.
- Pruning: a qualifying filter on the partition column scans only matching partitions — fewer bytes, lower cost, faster queries.
- Operational freebies: partition expiration auto-deletes old partitions (your retention policy in one clause); loads and deletes can target single partitions.
- Limits to note: one partition column per table, and you cannot repartition in place — rebuild with CREATE TABLE AS SELECT.

Iowa tie

Gold marts: partition on the business date your reports filter on; partition expiration replaces the archive-and-purge SQL Agent jobs.

Design patterns

- Default to daily partitions on the reporting date column.

Anti-patterns

- Do not partition tiny tables or pick a column nobody filters on — metadata overhead with zero pruning benefit.

History & what's new

Integer-range partitioning landed in December 2019, completing the date/ingestion/integer trio; in January 2025 the same pruning concept reached open formats with BigQuery tables for Apache Iceberg (preview).

Flow

Definition done — the next two slides make it concrete with the sales table.

Links

<https://cloud.google.com/bigquery/docs/partitioned-tables>

<https://cloud.google.com/bigquery/docs/creating-partitioned-tables>

<https://cloud.google.com/blog/products/data-analytics/bigquery-tables-for-apache-iceberg>

Example: Sales data table

transaction_date	customer_id	product_category	sales_amount
2025-08-01	CUST002	Headsets	52
2025-08-01	CUST002	Keyboards	186
2025-08-02	CUST003	Headsets	465
2025-08-02	CUST005	Headsets	57
2025-08-03	CUST003	Shoes	413
2025-08-03	CUST001	Consoles	347

Google Cloud

INSTRUCTOR NOTES — PAGE 90 OF 208 · 11:35 · MODULE 9 — MODERNIZING DATA WAREHOUSES

Hook

A plain fact table — six rows standing in for your six hundred million. Watch what a simple date filter costs when nothing is partitioned.

Say · cover

- Walk the columns: transaction_date is the natural partition key — it is what every report filters on.
- Unpartitioned, WHERE transaction_date = '2025-08-01' still scans every storage block: full bytes, full bill, for three rows of answer.
- Ask the room: which column do YOUR Power BI reports filter on most? That is the partition-key candidate for each mart.
- Note customer_id repeats across dates — hold that thought; it becomes the clustering example in three slides.

Iowa tie

Map this to your biggest extract-fed table: the date the nightly job reloads is the column your Gold mart should partition on.

Design patterns

- Choose the partition key from actual report filters, not load convenience.

Anti-patterns

- Do not partition on a surrogate key or a column filters rarely use.

History & what's new

Support for loading and querying externally partitioned data arrived March 2020 — the Hive-layout pruning you saw in DIN §30 this morning; integer-range partitioning had landed three months earlier, in December 2019.

Flow

Problem shown — next slide: the same table, partitioned.

Links

<https://cloud.google.com/bigquery/docs/partitioned-tables>

<https://cloud.google.com/bigquery/docs/querying-partitioned-tables>

<https://cloud.google.com/bigquery/docs/release-notes>

Partitioned by `transaction_date`

transaction_date	customer_id	product_category	sales_amount
2025-08-01	CUST002	Headsets	52
2025-08-01	CUST002	Keyboards	186

transaction_date	customer_id	product_category	sales_amount
2025-08-02	CUST003	Headsets	465
2025-08-02	CUST005	Headsets	57

Google Cloud

INSTRUCTOR NOTES — PAGE 91 OF 208 · 11:35 · MODULE 9 — MODERNIZING DATA WAREHOUSES

Hook

Same table, one DDL clause different — now the date filter opens one drawer instead of the whole cabinet.

Say · cover

- Rows are physically grouped by `transaction_date`; each date is one partition in its own physical block.
- `WHERE transaction_date = '2025-08-02'` prunes to a single partition: bytes scanned collapse — the same behavior DIN \$30 showed on external files, now on native storage.
- Predictability: partitioned tables give accurate pre-run cost estimates because pruned bytes are known before execution.
- Granularity rule of thumb: daily is the default; aim for at least ~1-10 GB per partition — many tiny partitions just move the bottleneck into metadata.
- Lifecycle freebie: partition expiration auto-deletes old partitions — no archive/purge Agent jobs.

Iowa tie

Default shape for every Gold fact mart feeding Power BI: partition on the business date, expire partitions per the agency retention policy.

Design patterns

- Daily partition on the reporting date; dry-run to confirm pruning before scheduling.

Anti-patterns

- Do not shard tables by date suffix (`orders_20250801`) — Google explicitly recommends partitioned tables over sharding; one table beats N.

History & what's new

Date-sharded table suffixes were the workaround before partitioned tables existed; Google's docs now call partitioning the recommended pattern, and partition decorators (`table$20250802`) plus expiration cover the per-slice operations sharding was used for.

Flow

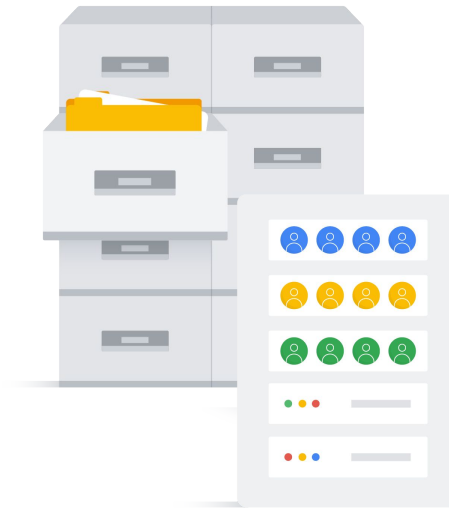
Pruning on native tables shown — next: clustering, which prunes within each partition.

Links

<https://cloud.google.com/bigquery/docs/partitioned-tables>
<https://cloud.google.com/bigquery/docs/managing-partitioned-tables>
<https://cloud.google.com/bigquery/docs/best-practices-costs>

Clustering

Clustering sorts the data within each partition. Think of it as organizing the files within each drawer of your filing cabinet alphabetically by customer name.



Google Cloud

 INSTRUCTOR NOTES — PAGE 92 OF 208 · 11:35 · MODULE 9 — MODERNIZING DATA WAREHOUSES

Hook

Clustering is your clustered index — the sort order — except BigQuery re-sorts in the background forever, and you never write the maintenance job.

Say · cover

- Clustering sorts data within each partition by up to four columns you choose, in priority order.
- Data lands in adaptively sized storage blocks; per-block metadata lets queries skip blocks that cannot contain matching rows — block pruning.
- Column order matters, like composite-index leftmost-prefix rules: a filter on the first clustering column prunes best.
- Automatic reclustering: as new data arrives, BigQuery maintains the sort in the background — no REBUILD, no REORGANIZE, ever.
- Cost caveat: unlike partitioning, clustering cannot give exact pre-run byte estimates; the savings show at execution.

lowa tie

Gold marts: after partitioning by date, cluster on the high-cardinality filters Power BI slicers use — county_id, customer_id.

Design patterns

- Partition by date, then cluster by the top filter/join columns (up to four, priority order).

Anti-patterns

- Do not cluster tiny partitions (under ~64 MB) or lead with low-cardinality flag columns — pruning gains are negligible.

History & what's new

Automatic background reclustering has replaced index maintenance since clustered tables launched; in January 2025 Google extended the same clustering machinery to open formats with BigQuery tables for Apache Iceberg (preview).

Flow

Clustering concept planted — the module continues after this with the clustering example, Iceberg partitioning, and BigLake before lunch.

Links

<https://cloud.google.com/bigquery/docs/clustered-tables>

<https://cloud.google.com/bigquery/docs/creating-clustered-tables>

<https://cloud.google.com/blog/products/data-analytics/bigquery-tables-for-apache-iceberg>

Clustering example

transaction_date	customer_id	product_category	sales_amount
2025-08-01	CUST002	Headsets	52
2025-08-01	CUST002	Keyboards	186
2025-08-01	CUST003	Headsets	465
2025-08-01	CUST005	Headsets	57
2025-08-01	CUST003	Shoes	413
2025-08-01	CUST001	Consoles	347

Google Cloud

INSTRUCTOR NOTES — PAGE 93 OF 208 · 11:35 · MODULE 9 — MODERNIZING DATA WAREHOUSES

Hook

Yesterday's clustered index, rebuilt as storage layout: inside one date partition, rows are physically sorted by customer_id.

Say · cover

- All rows share transaction_date 2025-08-01 — one partition, like your partition-function schemes.
- Within it, BigQuery sorted storage blocks by customer_id: CUST001, CUST002, CUST003, CUST005.
- Colocation is block-level, not row-level; each block carries min/max metadata per clustering column.
- New writes are sorted where possible; automatic background reclustering fixes the rest, free.
- Column order matters: cluster by the column you filter most first — the leftmost-key rule you know.

Iowa tie

Your Gold data marts: cluster each fact table on the agency, fund, and account columns Power BI filters and joins on.

Design patterns

- Cluster on high-cardinality filter/join columns, most-filtered first
- Partition + cluster together: date segment, then sort order inside it

Anti-patterns

- Clustering a low-cardinality flag column — few distinct values prune nothing
- Clustering tables or partitions under ~64 MB — overhead without gain, per Google's docs

History & what's new

Google documented changing a table's clustering specification in October 2020 (release-noted April 2021). In August 2025 Google described history-based optimization, which tunes execution using past query runs.

Flow

p088 split sales by date and p089 gave the filing-cabinet analogy — this is the sorted drawer; next, the query that exploits it.

Links

<https://cloud.google.com/bigquery/docs/clustered-tables>

<https://cloud.google.com/bigquery/docs/partitioned-tables>

<https://cloud.google.com/bigquery/docs/best-practices-performance-overview>

Query example with partition + clustering

```
SELECT *  
FROM cymbal_sales  
WHERE transaction_date = '2025-08-01'  
      AND customer_id = 'CUST002';
```

Google Cloud

INSTRUCTOR NOTES — PAGE 94 OF 208 · 11:35 · MODULE 9 — MODERNIZING DATA WAREHOUSES

Hook

One WHERE clause, two prunings — watch BigQuery skip 364 days of data, then skip the customers it doesn't need.

Say · cover

- The transaction_date filter triggers partition pruning: only the 2025-08-01 partition is scanned.
- The customer_id filter triggers block pruning inside that partition, using min/max block metadata.
- Bytes processed is your on-demand bill — the savings show up there; check it in the dry run.
- Keep the partition column bare in the filter; wrapping it in a function can defeat pruning.
- In Job information -> Execution details you can verify which partitions and blocks were read.

Iowa tie

This is the exact shape of your Power BI report queries — report date plus agency/fund — so both pruning layers fire.

Design patterns

- Filter the partition column first, then clustered columns
- Dry-run or check bytes processed before hitting Run

Anti-patterns

- Wrapping the partition column in DATE() or CAST() inside the predicate
- SELECT * full-table imports into Power BI, filtering afterward

History & what's new

Google's clustered-tables docs describe this two-level pruning (partition, then block min/max) as core behavior; the release-notes archive shows clustering-spec refinements in 2020–2021, and an August 2025 Google blog adds history-based optimization at execution.

Flow

The payoff query for the layout you just saw; next, a custom card translating all of this into your SQL Server toolkit.

Links

<https://cloud.google.com/bigquery/docs/clustered-tables>

<https://cloud.google.com/bigquery/docs/partitioned-tables>

<https://cloud.google.com/bigquery/docs/query-plan-explanation>

Partitioning & clustering — your tuning toolkit, translated

Same disciplines you use today, declared at CREATE TABLE time — then never maintained again.

- Partition functions and sliding windows— partitioned tables; pruning is automatic on a date filter
- Clustered indexes and index tuning— clustering columns: sort order inside storage blocks, up to four
- Index rebuilds in the maintenance plan— automatic background reclustering; there is no job to write
- No retroactive ALTER— repartition by rebuilding with CREATE TABLE AS SELECT
- Decision rule— partition on the date your reports filter; cluster on your top filter/join columns
- Aim for ~1–10 GB per partition— daily by default; tiny partitions slow metadata, not queries

Preview, not the full story — the 3-day Data Warehousing with BigQuery course being scheduled goes deep on both, hands-on.

Custom slide · added by Imran Ahmad

Iowa DOM DoIT · Data Engineering on Google Cloud

INSTRUCTOR NOTES — PAGE 95 OF 208 · 11:35 · MODULE 9 — MODERNIZING DATA WAREHOUSES

Hook

Every line on this custom slide is a skill you already own — partition functions, clustered indexes, index rebuilds — re-declared in DDL.

Say · cover

- Partition functions and sliding windows -> partitioned tables; pruning is automatic on a date filter, no SSIS switch-out.
- Clustered index -> up to four clustering columns defining sort order inside storage blocks.
- Index-rebuild maintenance plans vanish: background reclustering is automatic and free — nothing for M&O to schedule.
- No retroactive ALTER: repartition an existing table by rebuilding with CREATE TABLE AS SELECT.
- Decision rule: partition on the date reports filter; cluster on top filter/join columns; avoid tiny partitions.
- Preview only — the scheduled 3-day Data Warehousing with BigQuery course goes hands-on deep on both.

Iowa tie

Declare these once in your Dataform Gold-layer models; the platform maintains them — your CI/CD deploys DDL, not maintenance jobs.

Design patterns

- Require a partition filter on large fact tables to stop accidental full scans
- One partitioned table, not date-sharded table_20250801 copies

Anti-patterns

- Thousands of tiny partitions — metadata overhead slows planning, not queries
- Rebuilding SQL Server-style index maintenance jobs in Cloud Scheduler

History & what's new

Google's docs recommend partitioning over date-sharded tables, the older hand-rolled pattern; clustering-spec changes were release-noted in April 2021, and August 2025 added history-based optimization of query execution.

Flow

Recap card closing the native-BigQuery tuning section; next, the same ideas rebuilt on open Iceberg files — your Bronze layer.

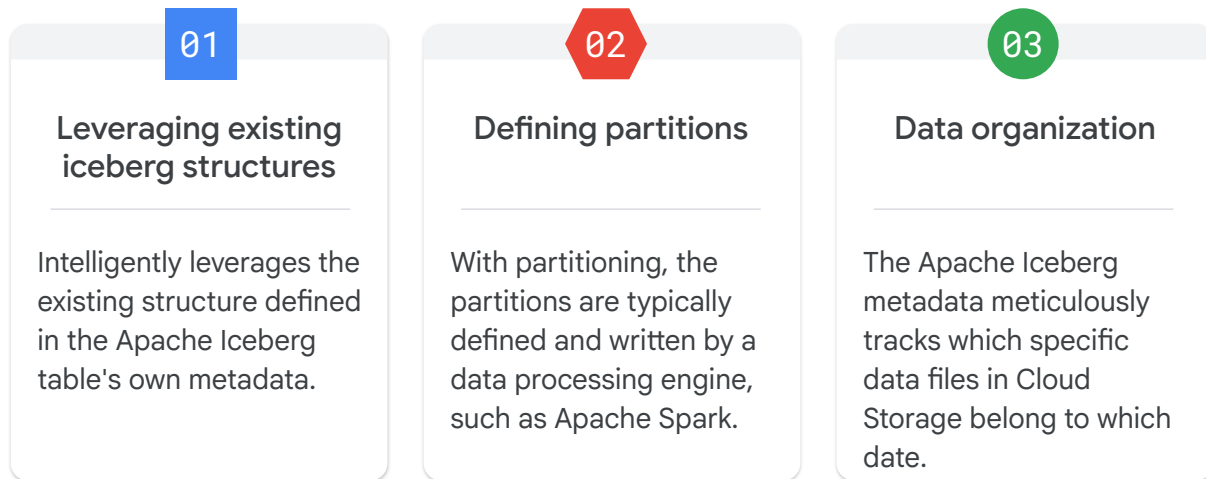
Links

<https://cloud.google.com/bigquery/docs/partitioned-tables>

<https://cloud.google.com/bigquery/docs/clustered-tables>

<https://cloud.google.com/bigquery/docs/best-practices-costs>

Partitioning in Iceberg



Google Cloud

INSTRUCTOR NOTES — PAGE 96 OF 208 · 11:35 · MODULE 9 — MODERNIZING DATA WAREHOUSES

Hook

Same partition-elimination idea, but the partition map is an open metadata file in your bucket — not a proprietary catalog.

Say · cover

- Iceberg's own metadata meticulously tracks which data files in Cloud Storage belong to which date partition.
- Partitions are typically defined and written by the processing engine — Apache Spark, for example.
- This is hidden partitioning: users filter a column; the metadata, not folder naming, resolves the files.
- BigQuery reads that metadata directly, so pruning works on files it never ingested.
- Managed Iceberg tables support DATE, DATETIME, and TIMESTAMP partitioning, hourly through yearly.

Iowa tie

This is your Bronze design: raw landings in GCS with partition metadata in the open — any engine can prune, nothing is locked in.

Design patterns

- Partition Bronze tables by the load/report date
- Let the engine and metadata manage partitions, not hand-built folder trees

Anti-patterns

- Hive-style directory partitioning that users must know to query correctly
- Assuming files-in-a-bucket alone give you pruning — the metadata is the table

History & what's new

Netflix built Iceberg and donated it to Apache in 2018; it became a top-level Apache project in May 2020. Google announced BigQuery tables for Apache Iceberg in preview in October 2024.

Flow

From native BigQuery storage to Iceberg on your own files; next, watch the planner use that metadata to skip files.

Links

<https://cloud.google.com/bigquery/docs/iceberg-tables>

<https://iceberg.apache.org/spec/>

<https://cloud.google.com/blog/products/data-analytics/announcing-bigquery-tables-for-apache-iceberg>

Query filtering example

```
WHERE transaction_date = '2025-08-15'
```

01

BigQuery (via Lakehouse) reads this metadata and identifies the exact set of files corresponding to that date and prunes away all others.

02

Ignores irrelevant data, drastically reducing the amount of data scanned, which lowers costs and accelerates query performance.

Google Cloud

INSTRUCTOR NOTES — PAGE 97 OF 208 · 11:35 · MODULE 9 — MODERNIZING DATA WAREHOUSES

Hook

WHERE transaction_date = '2025-08-15' never touches 99% of the bucket — the metadata answers before any file opens.

Say · cover

- BigQuery, via the lakehouse, consults the Iceberg metadata and identifies the exact files for that date.
- Every other file is pruned before reading — scan volume drops, latency drops.
- This is partition elimination from your SQL Server world, running over object storage.
- On managed Iceberg tables, BigQuery auto-scopes jobs to the right partitions — like Iceberg hidden partitioning.
- Billing follows bytes read, so pruned files are files you don't pay for.

Iowa tie

Bronze queries become cheap enough for interactive exploration — analysts profile raw landings without waiting for a load job.

Design patterns

- Always filter the partition column in Bronze queries
- Compare bytes processed with and without the filter — make pruning visible

Anti-patterns

- Dashboards full-scanning the raw bucket on every refresh
- Date filters hidden inside views nobody remembers to push through

History & what's new

Iceberg's hidden-partitioning design dates to its Netflix origins and Apache donation in 2018; Google's October 2024 preview of BigQuery Iceberg tables brought managed partitioning to GCS, with GA following in 2025.

Flow

You just saw date-level pruning over files; next, Iceberg's answer to clustering — sorted data plus per-file statistics.

Links

<https://cloud.google.com/bigquery/docs/query-iceberg-data>

<https://iceberg.apache.org/spec/>

<https://cloud.google.com/bigquery/docs/partitioned-tables>

Clustering in Apache Iceberg

01

Clustering in BigQuery is mirrored by data sorting and file-level statistics in Apache Iceberg.

02

The data within Apache Iceberg's Parquet or ORC files is often sorted by specific columns, such as `customer_id`.

03

The Apache Iceberg metadata stores statistics, such as the minimum and maximum `customer_id` values, for each individual data file.

Google Cloud

INSTRUCTOR NOTES — PAGE 98 OF 208 · 11:35 · MODULE 9 — MODERNIZING DATA WAREHOUSES

Hook

No clustered index to create here — the sort lives in the Parquet files themselves, and the metadata remembers each file's range.

Say · cover

- BigQuery clustering is mirrored by data sorting plus file-level statistics in Apache Iceberg.
- Writers sort rows within Parquet or ORC files by columns such as `customer_id`.
- Iceberg metadata records min/max values per column for every individual data file.
- BigQuery's managed Iceberg tables add automatic clustering and adaptive file sizing for you.
- Same mental model as a sorted, indexed staging table — but engine-neutral, in your bucket.

Iowa tie

Silver tables written by Dataform or Spark can be sorted by your join keys, so BigQuery and any open engine prune identically.

Design patterns

- Sort files by high-selectivity filter/join columns at write time
- Prefer managed Iceberg tables for automatic clustering and compaction

Anti-patterns

- Dumping unsorted small files and hoping pruning saves you
- Hand-optimizing files that BigQuery would auto-optimize

History & what's new

Iceberg's per-file metrics come from its 2018 Netflix design, refined as a top-level Apache project since 2020; by 2025 BigQuery's managed tables added automatic clustering, with history-based optimization reaching GA that year.

Flow

Partition pruning skips whole days; this is finer — file-level value ranges; next, the planner putting those stats to work.

Links

<https://cloud.google.com/bigquery/docs/iceberg-tables>

<https://iceberg.apache.org/>

<https://cloud.google.com/bigquery/docs/clustered-tables>

Predicate pushdown

- 01 If your BigQuery query filters for a specific customer, the query planner consults these statistics.
- 02 Skips reading any files whose min/max range doesn't contain the requested `customer_id`.
- 03 Predicate pushdown provides a finer-grained level of data pruning, ensuring maximum query efficiency.

Google Cloud

INSTRUCTOR NOTES — PAGE 99 OF 208 · 11:35 · MODULE 9 — MODERNIZING DATA WAREHOUSES

Hook

Ask for one customer, and the planner throws away every file whose min/max range can't contain them — before reading a byte.

Say · cover

- A filter like `customer_id = 'CUST002'` is evaluated against per-file min/max statistics first.
- Files whose range excludes the value are skipped entirely — finer-grained than partition pruning.
- The predicate travels down to the storage layer instead of filtering after the read — hence pushdown.
- It stacks: partition pruning, then file-level stats, then columnar reads of only the needed columns.
- Your index-seek intuition applies — selective predicates are cheap, full scans are not.

Iowa tie

Power BI reports filtering one agency or one voucher get fast answers straight from Bronze without a curated copy — extract-killer behavior.

Design patterns

- Write data sorted on selective filter columns so file stats stay tight
- Filter in SQL, not in the BI layer after import

Anti-patterns

- Importing whole tables into Power BI and filtering in DAX
- Expecting pushdown to save high-cardinality LIKE or contains searches

History & what's new

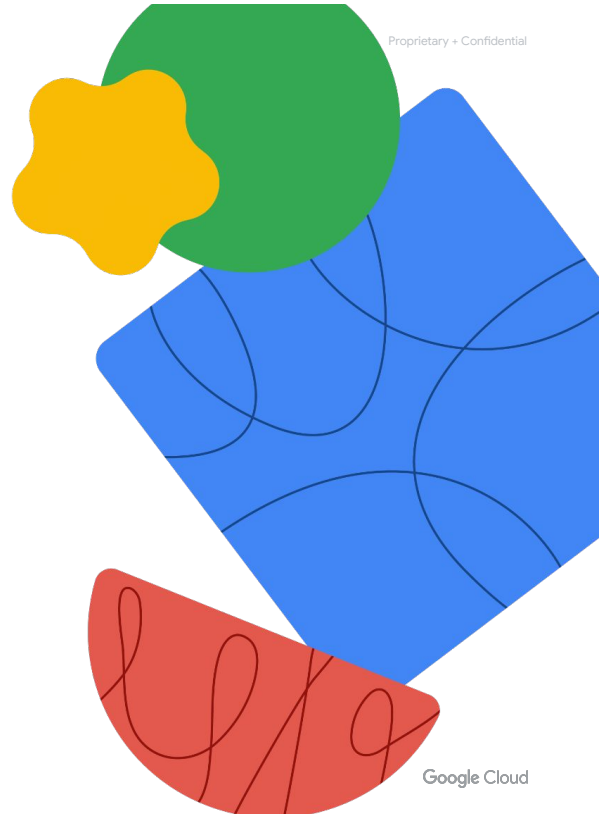
Predicate pushdown via manifest metrics is core Iceberg, designed at Netflix and open-sourced in 2018; in April 2026 Google renamed BigLake to Lakehouse for Apache Iceberg, keeping API, CLI, and IAM names unchanged.

Flow

The planner's third trick after partitions and sorting; next, why all three matter together as data grows.

Links

<https://iceberg.apache.org/spec/>
<https://cloud.google.com/bigquery/docs/query-iceberg-data>
<https://cloud.google.com/bigquery/docs/best-practices-performance-overview>



Google Cloud

Why it matters

Partitioning and clustering together give BigQuery fine-grained control over how your data is queried, leading to significant performance improvements and cost savings, especially as your data grows.

INSTRUCTOR NOTES — PAGE 100 OF 208 · 11:35 · MODULE 9 — MODERNIZING DATA WAREHOUSES

Hook

This is the slide that answers your CFO: the same query gets cheaper every time you prune, no matter how big the table grows.

Say · cover

- Partitioning and clustering together give fine-grained control over what a query touches.
- Performance improves because slots read fewer blocks; cost drops because billing follows bytes processed.
- Benefits compound with growth — the bigger the table, the more a full scan would have cost.
- It compounds with governance too: a required partition filter makes waste structurally impossible.
- Watch bytes processed per job; that one number is your tuning scoreboard.

lowa tie

Your nightly Dataform runs and every Power BI refresh multiply this saving — layout decisions in Gold models pay out daily.

Design patterns

- Set table layout once at CREATE; verify savings in job bytes processed
- Use `require_partition_filter` on the big fact tables

Anti-patterns

- Tuning SQL text while the table layout guarantees full scans
- Judging queries by elapsed time instead of bytes scanned

History & what's new

Google's cost-optimization docs have long pushed partition filters and dry-run byte checks; the August 2025 advanced-runtime blog shows the platform now also learns from past executions via history-based optimization.

Flow

The value case for everything since p084; next, the one-sentence thesis of this whole section.

Links

<https://cloud.google.com/bigquery/docs/clustered-tables>
<https://cloud.google.com/bigquery/docs/partitioned-tables>
<https://cloud.google.com/bigquery/docs/best-practices-costs>

BigQuery uses metadata to skip unnecessary data, so **queries run faster** and cost less.

Google Cloud

INSTRUCTOR NOTES — PAGE 101 OF 208 · 11:35 · MODULE 9 — MODERNIZING DATA WAREHOUSES

Hook

If you remember one line from Module 9, it's this: BigQuery wins by NOT reading data — the metadata is the index.

Say · cover

- Partition maps, block min/max, Iceberg manifests — all metadata that lets queries skip unnecessary data.
- Faster and cheaper are the same event here: skipped bytes are unbilled bytes.
- This is why there are no indexes to maintain — the metadata IS the index, and it maintains itself.
- Execution details expose the skip: compare bytes processed against table size after any query.

lowa tie

For M&O: cost governance becomes 'watch bytes processed' — a single metric in job history, not a tuning consultancy.

Design patterns

- Review bytes processed in job history after every model change
- Set maximum_bytes_billed as a guardrail on dev projects

Anti-patterns

- Assuming small result sets mean cheap queries — bytes scanned is not rows returned
- Ignoring dry-run estimates before running ad-hoc SQL

History & what's new

Metadata-driven pruning is BigQuery's longstanding design, per its clustered-tables docs (block metadata pruning); Google's August 2025 advanced-runtime blog describes history-based optimization that keeps improving repeat executions.

Flow

The thesis statement; next, a custom card on why this same pruning works on files you own outright.

Links

<https://cloud.google.com/bigquery/docs/query-plan-explanation>

<https://cloud.google.com/bigquery/docs/clustered-tables>

<https://cloud.google.com/blog/products/data-analytics/short-query-optimizations-in-bigquery-advanced-runtime>

Iceberg partitioning — the same pruning, on files you own

Why your Bronze layer cares: filtering works before anything is ever loaded.

- Iceberg metadata tracks which Parquet files belong to which partition— open metadata, in your bucket
- Per-file min/max statistics let the planner skip files that can't contain your value— predicate pushdown
- Same discipline as partition elimination and index seeks— but any Iceberg-speaking engine can read it
- This is the answer to 'will BigQuery hold our data hostage?'— open files plus open metadata
- Also a preview— Iceberg internals get the deep, hands-on treatment in the Data Warehousing course

Custom slide · added by Imran Ahmad

Iowa DOM DoIT · Data Engineering on Google Cloud

INSTRUCTOR NOTES — PAGE 102 OF 208 · 11:35 · MODULE 9 — MODERNIZING DATA WAREHOUSES

Hook

Leadership's lock-in question gets its answer here: the pruning magic runs on open files with open metadata in your bucket.

Say · cover

- Iceberg metadata in your GCS bucket tracks which Parquet files belong to which partition.
- Per-file min/max statistics let any Iceberg-speaking planner skip files — predicate pushdown included.
- Same discipline as partition elimination and index seeks — but Spark, Trino, or BigQuery can all read it.
- Open files plus open metadata is the direct answer to 'will BigQuery hold our data hostage?'
- Also a preview: Iceberg internals get deep, hands-on treatment in the Data Warehousing course.

Iowa tie

This is the architectural guarantee for your Bronze zone: state data stays in open formats, queryable by whatever engine Iowa chooses next.

Design patterns

- Standardize Bronze on Parquet + Iceberg metadata from day one
- Enable the REST catalog so non-Google engines see the same tables

Anti-patterns

- Copying files into proprietary storage 'for performance' before testing
- Letting each team invent its own file layout convention

History & what's new

Google announced GA of the Iceberg REST Catalog in BigLake metastore in November 2025, so third-party engines share one catalog; in April 2026 BigLake was renamed Lakehouse for Apache Iceberg.

Flow

The custom recap of the Iceberg half; next, the deck pivots to where valuable data actually lives — lakes and external tables.

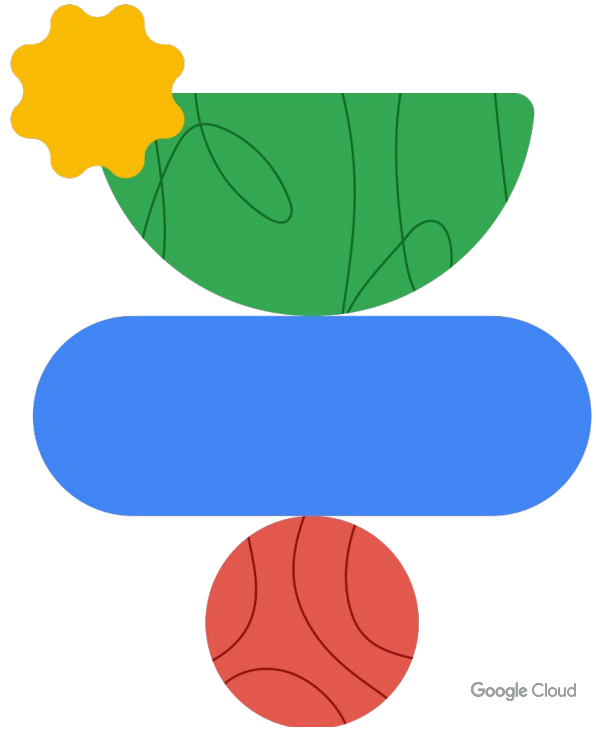
Links

<https://iceberg.apache.org/spec/>

<https://cloud.google.com/bigquery/docs/iceberg-tables>

<https://cloud.google.com/blog/products/data-analytics/biglake-metastore-now-supports-iceberg-rest-catalog>

Valuable data exists in many places and formats



Google Cloud

INSTRUCTOR NOTES — PAGE 103 OF 208 · 11:35 · MODULE 9 — MODERNIZING DATA WAREHOUSES

FACILITATOR NOTE

- Transition slide into the lakehouse/external-tables close of Module 9 — deliver it verbally: their 'many places' today is SQL Server plus network file drops, and GCP's answer is one query layer over all of it.
- Keep it to 30 seconds; the next three slides carry the content.

Flow: Section divider; next, the data warehouse vs. data lake framing.

Data warehouse vs. data lake

Traditional data storage

- Data warehouse: Ideal for structured, organized data used for business insights.
- Data lake: A cost-effective place for huge amounts of raw data in any format.

Challenges of separation

- Data silos: Hard to analyze different data types together.
- Complex extract, transform, load (ETL): Required moving and duplicating data, increasing costs and delays.
- Governance Issues: Managing security and access across two systems was complicated.

Google Cloud

 INSTRUCTOR NOTES — PAGE 104 OF 208 · 11:35 · MODULE 9 — MODERNIZING DATA WAREHOUSES

Hook

You've lived this split for a decade: the SQL Server warehouse for answers, the file shares for everything else — and the extracts between them.

Say · cover

- Warehouse: structured, curated, fast — your SQL Server DW today, BigQuery tomorrow.
- Lake: a cost-effective place for huge amounts of raw data in any format — Cloud Storage plays this role.
- Separated, they create silos: hard to analyze data types together, complex ETL, governance headaches.
- The ETL tax is the real pain: every cross-question meant moving and duplicating data first.
- Sound familiar? This is your nightly extract pattern, drawn on Google's own slide.

Iowa tie

Name the pain in the room: your legacy extracts exist because warehouse and file landings are separate worlds — that separation is what you're retiring.

Design patterns

- Land raw once in the lake; curate in place toward Gold
- One governance plane over both zones

Anti-patterns

- Copying the lake into the warehouse for every new question
- Two drifting security models — bucket ACLs vs. database grants

History & what's new

Databricks coined 'data lakehouse' in a January 2020 whitepaper, formalized in a CIDR 2021 paper by Armbrust, Ghodsi, Xin, and Zaharia; Google entered the space with BigLake in April 2022.

Flow

The problem statement; next, the one-line answer.

Links

<https://cloud.google.com/blog/products/data-analytics/unifying-data-lakes-and-data-warehouses-across-clouds-with-biglake>
<https://cloud.google.com/bigquery/docs/external-data-sources>
<https://cloud.google.com/bigquery/docs/biglake-intro>

A data lakehouse **solves these challenges.**

Google Cloud

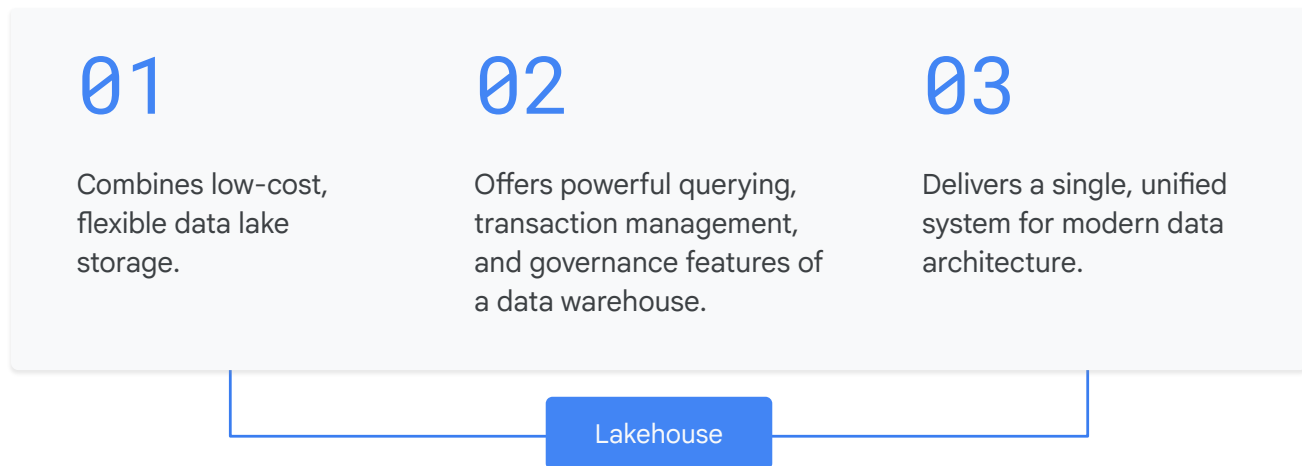
INSTRUCTOR NOTES — PAGE 105 OF 208 · 11:35 · MODULE 9 — MODERNIZING DATA WAREHOUSES

FACILITATOR NOTE

- One-breath slide: lake storage economics plus warehouse querying, transactions, and governance — a single system instead of two plus the plumbing between them.
- Bridge line: 'For you, this means the extract jobs die — the next slide shows the shape of the thing that replaces them.'

Flow: The answer in one sentence; next, the architecture slide that unpacks it.

Data lakehouse architecture



Google Cloud

INSTRUCTOR NOTES — PAGE 106 OF 208 · 11:35 · MODULE 9 — MODERNIZING DATA WAREHOUSES

Hook

Bronze-Silver-Gold on one platform: this slide is your architecture document wearing Google's logo.

Say · cover

- Low-cost, flexible lake storage — Cloud Storage holds raw landings in open formats.
- Warehouse-grade querying, transaction management, and governance ride on top — BigQuery plus Iceberg table semantics.
- One unified system: no copying between a 'lake team' and a 'warehouse team.'
- The medallion pattern — Bronze/Silver/Gold in M10 this afternoon — is this architecture zoned by data quality.
- Power BI, Dataform, and open engines all read the same governed copy — the single source of truth.

Iowa tie

This IS your target architecture: GCS Bronze, BigQuery/Iceberg Silver-to-Gold via Dataform, Power BI on top — hold this slide next to your design doc.

Design patterns

- Zone by data quality (Bronze/Silver/Gold), not by system
- Open table formats at the base so any engine can participate

Anti-patterns

- Building the lake and the warehouse as two projects with two teams
- Proprietary formats in Bronze that force copies later

History & what's new

The lakehouse concept was named by Databricks in 2020 (CIDR 2021 paper); Google launched BigLake in April 2022 and, in April 2026, renamed it Lakehouse for Apache Iceberg with cross-cloud catalog previews.

Flow

The architecture in three bullets; next, the mechanism that makes files queryable — Lakehouse and external tables.

Links

<https://cloud.google.com/bigquery/docs/biglake-intro>

<https://cloud.google.com/blog/products/data-analytics/unifying-data-lakes-and-data-warehouses-across-clouds-with-biglake>

<https://docs.cloud.google.com/lakehouse/docs/release-notes>

Lakehouse and external tables

- 01 Lakehouse extends BigQuery.
- 02 Create Lakehouse tables over data in object stores, such as Google Cloud Storage.
- 03 Reference the source data files in your data lake, and query them.



Google Cloud

INSTRUCTOR NOTES — PAGE 107 OF 208 · 11:35 · MODULE 9 — MODERNIZING DATA WAREHOUSES

Hook

Remember this morning's DIN where a bucket of files became a table in 60 seconds? This slide is that trick, formalized.

Say · cover

- Lakehouse extends BigQuery beyond native storage: create tables over data in object stores such as Cloud Storage.
- The table references the source files in place — you query the lake without loading it.
- Creation is metadata-only: a pointer registration, not a data-movement job — it returns in seconds.
- Upgrade path: plain external table -> BigLake table via WITH CONNECTION -> managed Iceberg table.
- Bucket and dataset locations must align — the classic first error in every lab.

Iowa tie

This is how your Bronze landings become queryable on day one: register, query, formalize later — no staging SSIS package required.

Design patterns

- Start with external tables for ad-hoc; upgrade to BigLake for governance
- Co-locate the bucket and dataset in the same region

Anti-patterns

- Loading files into native tables just to look at them
- Granting analysts raw bucket access instead of table-level grants

History & what's new

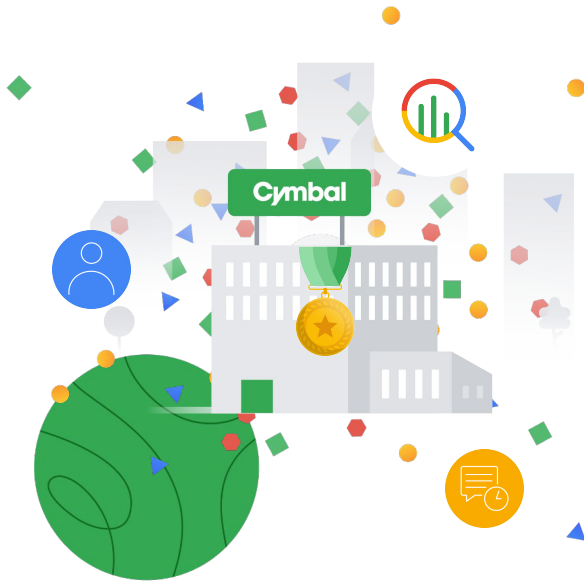
BigLake arrived in preview in April 2022 to unify lake and warehouse access; Google's docs now recommend BigLake tables over plain external tables for access delegation, and the family was renamed Lakehouse in April 2026.

Flow

The mechanism slide; next, Cymbal's concrete scenario — JSON logs without a pipeline.

Links

<https://cloud.google.com/bigquery/docs/external-data-cloud-storage>
<https://cloud.google.com/bigquery/docs/external-data-sources>
<https://cloud.google.com/bigquery/docs/biglake-intro>



How can Cymbal use Lakehouse external tables?

- Cymbal's data science team uses Cloud Storage to store raw web server log files in JSON format.
- Analyzing this clickstream data used to require a complex data pipeline to parse and load it into BigQuery.

Lakehouse simplifies this process.

Google Cloud

INSTRUCTOR NOTES — PAGE 108 OF 208 · 11:35 · MODULE 9 — MODERNIZING DATA WAREHOUSES

Hook

Swap 'Cymbal web logs' for 'your agency CSV/JSON drops' and this slide is about Iowa — the pipeline you almost built, deleted.

Say · cover

- Cymbal's data science team stores raw web server log files as JSON in Cloud Storage.
- Before: analyzing clickstream required a complex pipeline to parse and load it into BigQuery first.
- That pipeline is pure overhead when you don't yet know the data's value.
- Lakehouse move: point a table at the files, ask the question, decide if it's worth productizing.
- This is exploration-first engineering — schema-on-read where it earns its keep.

Iowa tie

Your kill-the-extracts pattern in miniature: a file landing in a bucket is immediately queryable — no SSIS package stands between landing and first question.

Design patterns

- Explore in place first; build the load pipeline only for proven data
- Prefer self-describing formats (Parquet, JSON) for schema-on-read

Anti-patterns

- Building a full production pipeline before anyone has asked a question
- Letting exploration tables silently become production dependencies

History & what's new

External tables over Cloud Storage are a long-standing BigQuery capability; the April 2022 BigLake launch added governed access delegation, and November 2025 brought the Iceberg REST Catalog to GA for engine-neutral metadata.

Flow

The scenario; next, the three steps that make it real.

Links

<https://cloud.google.com/bigquery/docs/external-data-sources>

<https://cloud.google.com/bigquery/docs/external-data-cloud-storage>

<https://cloud.google.com/bigquery/docs/biglake-intro>

Lakehouse external table

01

They create a Lakehouse table directly over the JSON files in Cloud Storage.

02

Use the familiar BigQuery SQL interface to query clickstream data instantly.

03

Join this external table with a native BigQuery sales table, to discover correlations between website behavior and purchasing habits.

Google Cloud

INSTRUCTOR NOTES — PAGE 109 OF 208 · 11:35 · MODULE 9 — MODERNIZING DATA WAREHOUSES

Hook

Three steps, zero load jobs — and step 3 is the one that should make your Power BI team lean forward.

Say + cover

- Step 1: create the Lakehouse table directly over the JSON files in Cloud Storage.
- Step 2: query clickstream instantly with the familiar BigQuery SQL interface — no parse stage, no staging schema.
- Step 3: JOIN the external table with a native BigQuery sales table — lake and warehouse, one query.
- That join is the silo-breaker: website behavior correlated with purchasing habits, no copies.
- External tables are read-only; for writes and governance you move up to BigLake or managed Iceberg.

Iowa tie

Step 3 is your end-state demo: Bronze file landings joined to Gold marts in one SQL statement — Power BI reads a single governed source.

Design patterns

- Join external Bronze with native Gold for correlation questions
- Promote hot external tables to managed once usage proves out

Anti-patterns

- Leaving heavy, repeated scans on raw JSON instead of converting to Parquet
- Duplicating external data per team because 'joins are slow'

History & what's new

Federated querying of GCS files predates the lakehouse brand; BigLake (April 2022) added access delegation and metadata caching, and the November 2025 Iceberg REST Catalog GA opened the same tables to Spark and Trino.

Flow

The full pattern on one slide; next, Google's intro to the lab where you'll do this yourself after lunch.

Links

<https://cloud.google.com/bigquery/docs/external-data-cloud-storage>
<https://cloud.google.com/bigquery/docs/reference/standard-sql/data-definition-language>
<https://cloud.google.com/bigquery/docs/biglake-intro>

Lab: Introduction to querying Apache Iceberg tables with BigQuery

Overview

Explore how BigQuery's Lakehouse tables can query data in open-source formats directly within Cloud Storage.



What you'll do

- Create a Lakehouse table that points to an existing Apache Iceberg table.
- Analyze data without moving or duplicating it.

Google Cloud

INSTRUCTOR NOTES — PAGE 110 OF 208 · 11:35 · MODULE 9 — MODERNIZING DATA WAREHOUSES

Hook

After lunch you stop watching and start typing — this lab is your Bronze pattern with your own hands.

Say · cover

- Goal: create a Lakehouse table that points to an existing Apache Iceberg table in Cloud Storage.
- Then analyze the data — filters, aggregations — with nothing moved or duplicated anywhere.
- Notice the DDL returns in seconds: it registers metadata, it does not load data.
- Compare with this morning's plain external table — same idea plus Iceberg's table semantics.
- Lab runs 13:00–13:45 after lunch on your Qwiklabs seat; the next slide has classroom logistics.

Iowa tie

When you design Bronze for the real migration, this 45 minutes is your reference experience — files that answer SQL from minute one.

Design patterns

- Type the DDL once yourself — the muscle memory is the point
- Confirm the project picker shows your lab project before creating

Anti-patterns

- Treating a slow CREATE as a load problem — it is metadata-only; errors mean typos
- Skipping the name suffix; sixteen identical table names collide

History & what's new

BigQuery tables for Apache Iceberg entered preview in October 2024 and reached GA in 2025 with catalog integration; the Iceberg REST Catalog in BigLake metastore went GA in November 2025.

Flow

Google's lab intro; next, our custom card with seat logistics and dialect warnings — then Module 9 wraps before lunch.

Links

<https://cloud.google.com/bigquery/docs/query-iceberg-data>

<https://cloud.google.com/bigquery/docs/iceberg-tables>

<https://cloud.google.com/blog/products/data-analytics/announcing-bigquery-tables-for-apache-iceberg>

Lab: Querying External Data and Iceberg Tables

45 minutes · Qwiklabs seat — BigQuery console

1. Create a Lakehouse (Iceberg) table pointing at the existing Apache Iceberg data in Cloud Storage
2. Suffix your table name with your initials — 16 identical DDL statements in one dataset will collide
3. Query it: filter and aggregate — plain SQL over files that were never loaded anywhere
4. Watch the dialect: backticks not brackets, LIMIT not TOP, dataset in the fully qualified name
5. Notice the DDL is metadata-only and returns instantly — nothing moved, nothing copied

Why it matters — this is your Bronze pattern, hands-on — raw data stays in open formats in Cloud Storage and is queryable from minute one. Bronze is not a landing zone you copy out of.

Custom slide · added by Imran Ahmad

Iowa DOM DoIT · Data Engineering on Google Cloud

INSTRUCTOR NOTES — PAGE 111 OF 208 · 11:35 · MODULE 9 — MODERNIZING DATA WAREHOUSES

Hook

This is the lab that matters most for your Bronze design — raw files, plain SQL, zero loads.

Say · cover

- Runs 13:00–13:45 after lunch, Qwiklabs seat, BigQuery console — seats pre-warmed this morning.
- Create a Lakehouse (Iceberg) table over the existing Iceberg data in GCS; suffix the name with your initials.
- Query it with filters and aggregates, then open Job information -> bytes processed to watch pruning work.
- Dialect gotchas: backticks not brackets, LIMIT not TOP — decoder table in your lab companion.
- The DDL is metadata-only and returns instantly; if it hangs, it's a URI typo, not a 'slow load.'
- Roam targets: T-SQL reflexes and the project-picker mistake — the two biggest time sinks.

Iowa tie

Debrief line: 'Bronze is not a landing zone you copy out of' — that sentence belongs in their architecture doc verbatim.

Design patterns

- Verify the first SELECT returns rows with no load job in history
- Compare bytes processed with and without the partition filter

Anti-patterns

- Re-running CREATE in a loop while seats provision slowly at 13:00
- Browsing the GCS file listing to answer questions instead of querying the table

History & what's new

The lab's Lakehouse-table pattern rides the Iceberg support Google previewed in October 2024 and GA'd in 2025; in April 2026 the BigLake family was renamed Lakehouse for Apache Iceberg.

Flow

Lab logistics card — lunch (p111) follows Module 9's summary, lab at 13:00; this module's concepts all get exercised there.

Links

<https://cloud.google.com/bigquery/docs/query-iceberg-data>
<https://cloud.google.com/bigquery/docs/reference/standard-sql/data-definition-language>
<https://cloud.google.com/bigquery/docs/query-plan-explanation>

Iceberg deep-dive — what's under the table you just built

15 minutes · Instructor demo — BigQuery console (from Jeff's Iceberg tutorial, gcp-demos)

1. Frame the one idea that matters: three Iceberg flavors differ by who owns the metadata and who can write
2. Debrief the lab: what your Iceberg table has that this morning's external table didn't — DML, time travel, snapshots
3. Upgrade the morning's ext_part external table to a BigLake table — same files, richer metadata
4. Dwell on WITH CONNECTION — the connection's service account reads the bucket; when it fails, it's that IAM grant
5. Payoff: column-level security and row access policies on files that never left the lake

Why it matters — files became a table this morning, got ACID semantics in the lab, and get governance now — three rungs of one ladder, and the rungs are the Bronze section of your architecture doc. Fallback if the tutorial is still under construction: extended lab debrief plus this BigLake demo.

Custom slide · added by Imran Ahmad

Iowa DOM DoIT · Data Engineering on Google Cloud

INSTRUCTOR NOTES — PAGE 112 OF 208 · 11:35 · MODULE 9 — MODERNIZING DATA WAREHOUSES

Hook

At 13:45 I take the table you just built apart — and upgrade this morning's external table into something governable.

Say · cover

- Frame the one idea that matters: the three Iceberg flavors differ by who owns the metadata and who can write.
- Debrief the lab: what your Iceberg table has that this morning's external table didn't — DML, time travel, snapshots.
- Live: upgrade the morning's ext_part external table to a BigLake table — same files, richer metadata, one DDL.
- Dwell on WITH CONNECTION: the connection's service account reads the bucket; failures are that IAM grant.
- Payoff: column-level security and row access policies on files that never left the lake.
- Fallback if Jeff's tutorial isn't ready: extended lab debrief plus this BigLake demo.

Iowa tie

The governance rung is your M&O story: one service account mediates bucket access, so analysts get auditable table grants, not storage ACLs.

Design patterns

- Upgrade external -> BigLake with CREATE OR REPLACE ... WITH CONNECTION
- Grant the connection's service account storage.objectUser on the bucket

Anti-patterns

- Handing every analyst storage.objectViewer on raw buckets
- Editing Iceberg-managed table files outside BigQuery — untracked files get garbage-collected

History & what's new

Google launched BigLake in April 2022, GA'd the Iceberg REST Catalog in its metastore in November 2025, and renamed the family Lakehouse for Apache Iceberg in April 2026 alongside cross-cloud previews.

Flow

Demo preview card (runs 13:45 after the lab); next, Module 9's summary, then lunch.

Links

<https://cloud.google.com/bigquery/docs/external-data-cloud-storage>
<https://cloud.google.com/bigquery/docs/create-cloud-resource-connection>
<https://cloud.google.com/bigquery/docs/column-level-security-intro>

M9 — What you now have

- ✓ Your SQL Server pain points named on Google's own slides— and the answer: delete the infrastructure layer
- ✓ Slots and shuffle in plain terms— a wider machine, not a faster one; 'bytes shuffled' is where slow queries hide
- ✓ Partitioning and clustering decision rules— flagged as a preview of the Data Warehousing course's deep dive
- ✓ Hands-on Bronze: an Iceberg/Lakehouse table over files in Cloud Storage, queried without loading anything
- ✓ Next: M10 at 14:00— Google's own Bronze/Silver/Gold zone slides, held up next to your architecture doc

Custom slide · added by Imran Ahmad

Iowa DOM DoIT · Data Engineering on Google Cloud

INSTRUCTOR NOTES — PAGE 113 OF 208 · 11:35 · MODULE 9 — MODERNIZING DATA WAREHOUSES

Hook

An hour ago 'slots' and 'shuffle' were jargon; now you can read a BigQuery bill like an execution plan.

Say · cover

- Your SQL Server pain points were named on Google's own slides — the answer: delete the infrastructure layer.
- Slots and shuffle in plain terms: a wider machine, not a faster one; bytes shuffled is where slow queries hide.
- Partitioning and clustering decision rules — a preview of the Data Warehousing course's deep dive.
- Hands-on Bronze: an Iceberg/Lakehouse table over files in Cloud Storage, queried without loading anything.
- Next: M10 at 14:00 — Google's own Bronze/Silver/Gold zone slides next to your architecture doc.

Iowa tie

You now hold the full vocabulary for the migration design review — Bronze lakehouse, partition/cluster tuning, bytes-processed cost model; bring it to the 15:20 capstone.

Design patterns

- Use this vocabulary in design reviews — it maps 1:1 to Google's docs
- Carry the bytes-processed habit into every future query review

Anti-patterns

- Treating today's preview as the final word on tuning — the DW course is the deep dive
- Designing Silver/Gold without the partition and cluster decision rules

History & what's new

The module's arc mirrors the industry's: Iceberg open-sourced 2018, Google's managed Iceberg tables previewed 2024 and GA'd 2025, REST Catalog GA November 2025, BigLake renamed Lakehouse April 2026.

Flow

Module 9 done — lunch now (p111), LAB External Data & Iceberg at 13:00, Iceberg demo at 13:45, M10 governance at 14:00.

Links

<https://cloud.google.com/bigquery/docs/partitioned-tables>

<https://cloud.google.com/bigquery/docs/iceberg-tables>

<https://cloud.google.com/blog/products/data-analytics/biglake-metastore-now-supports-iceberg-rest-catalog>

Lunch

60 minutes · 12:00–13:00

Custom slide · added by Imran Ahmad

Iowa DOM DoIT · Data Engineering on Google Cloud

INSTRUCTOR NOTES — PAGE 114 OF 208 · 12:00 · LUNCH (60 MIN)

FACILITATOR NOTE

- 60 minutes. Unlike yesterday, no Datastream trick runs while they eat — Day 2 labs were pre-warmed this morning, so seats should provision fast at 13:00.
- Wrap M9's partitioning/clustering discussion cleanly before you release them; if M9 ran long, protect the after-lunch lab, not the slides.
- Coming back they go straight onto keyboards for the Bronze-layer Iceberg lab, so set the expectation now: lunch ends, hands on.

Flow: M9 (Modernizing Data Warehouses — partitioning, clustering, Iceberg partitioning) just ended; after lunch comes the 45-min External Data & Iceberg lab, then your 13:45 Iceberg/BigLake demo.

Lab — Querying External Data & Apache Iceberg Tables

45 minutes — 13:00–13:45 · Google Skills classroom

1. Launch the lab; check the project picker; suffix table names with your initials (16 seats)
2. Type the DDL yourself: a Lakehouse table over the existing Iceberg data in the bucket
3. Metadata-only — returns in seconds; nothing loads; a hang means a typo in the URI
4. Run filters and aggregations over data that was never loaded — note bytes processed
5. Dialect deltas: backticks not [brackets], LIMIT not TOP, IFNULL not ISNULL

Why it matters — your Bronze layer, hands-on: raw data stays in open formats in object storage, and BigQuery queries it in place — queryable from minute one.

Custom slide · added by Imran Ahmad

Iowa DOM DoIT · Data Engineering on Google Cloud

INSTRUCTOR NOTES — PAGE 115 OF 208 · 13:00 · LAB — EXTERNAL DATA & APACHE ICEBERG TABLES (45 MIN)

Hook

Yesterday they loaded files into BigQuery; today they query files they never load — the SSIS staging package they stop writing.

Say · cover

- Check the project picker before every create; suffix table names with initials — 16 seats share the source.
- The CREATE is metadata-only: a pointer to existing Iceberg data, returning in seconds. A hang means a typo in the URI, because nothing loads.
- Type the DDL by hand once, then filter and aggregate; compare bytes processed in Execution details — M9's metadata pruning made visible.
- Contrast this morning's \$30 external table: same query-in-place idea, but Iceberg adds ACID and snapshots — Bronze is Iceberg-family, not bare external tables.
- Roam for dialect errors: backticks not [brackets], LIMIT not TOP, IFNULL not ISNULL (companion's T-SQL->GoogleSQL box).

Iowa tie

Their architecture doc's Bronze layer, hands-on: raw landings in open formats on GCS, queryable from minute one — no copy-to-staging before anyone can query.

Design patterns

- Land raw on GCS in open formats and query in place — Bronze is not a copy-out landing zone
- Verify pruning by comparing bytes processed with and without the partition filter

Anti-patterns

- A metadata-only CREATE that hangs is a wrong statement, not slow data — nothing loads
- Copy-pasted gs:// URIs lose characters to line-wrap — creates fine, queries return 0 rows

History & what's new

Google announced BigQuery tables for Apache Iceberg in October 2024; the BigLake metastore's Iceberg REST catalog went GA in November 2025.

Flow

From lunch and M9's Iceberg slides into 45 minutes of hands-on Bronze; you debrief it in the 13:45 demo.

Links

<https://cloud.google.com/bigquery/docs/iceberg-external-tables>

<https://cloud.google.com/bigquery/docs/iceberg-tables>

<https://cloud.google.com/bigquery/docs/reference/standard-sql/functions-and-operators>

Iceberg deep-dive + the BigLake table upgrade

15 minutes — 13:45–14:00 · Instructor-led — Jeff's Iceberg tutorial (gcp-demos)

1. Iceberg in one sentence: an open metadata layer that gives Parquet files table behavior
2. What it buys: ACID, schema evolution, time travel, hidden partitioning — over files in your bucket
3. Who writes, who keeps metadata: Lakehouse over existing Iceberg / BigQuery-managed / open engines
4. The upgrade: this morning's external-table pattern gains governance as a BigLake table

Why it matters — the no-lock-in answer for M&O: open Parquet + open metadata in your buckets — detach and walk away anytime.

Custom slide · added by Imran Ahmad

Iowa DOM DoIT · Data Engineering on Google Cloud

INSTRUCTOR NOTES — PAGE 116 OF 208 · 13:45 · DEMO — ICEBERG DEEP-DIVE + BIGLAKE UPGRADE (15 MIN)

Hook

Fifteen instructor-led minutes: what the lab did under the hood, plus M&O's lock-in answer.

Say · cover

- Iceberg: an open metadata layer giving bucket Parquet files table behavior — snapshots track which files make up the table.
- SQL Server terms: ACID over files, schema evolution without rewrites, FOR SYSTEM_TIME AS OF over restore cycles, hidden partitioning.
- Three flavors, one difference — who owns metadata and who writes: external/Lakehouse (Spark owns, BQ reads), managed (BQ writes Parquet to your bucket), open engines via REST catalog.
- Upgrade beat: this morning's external table gains column security, row policies, masking as a BigLake table — governance at Bronze, before Dataform.
- Pre-flight: Jeff's tutorial is 'under construction' (Aug 2026); managed/external flows verified, teardown buggy — fallback: extended lab debrief plus a BigLake demo.

Iowa tie

The M&O no-lock-in answer: open Parquet and metadata in their buckets — detach and walk away; time travel replaces restore cycles for Dataform replays.

Design patterns

- Open formats and open metadata in Bronze — a second engine reads the same files, no copy

Anti-patterns

- 'Iceberg' is not one product — ask who owns the metadata and who may write
- Pinning an external table to a stale metadata JSON — queries diverge from other engines

History & what's new

Built at Netflix in 2017, Apache top-level by 2020; Databricks' June 2024 Tabular buy ended the format war. October 2026 BigQuery release notes: partitioning, multi-statement transactions, and advanced runtime for Iceberg managed tables now GA.

Flow

The Bronze lab just ended; this explains it and adds governance — M10 (14:00) brings Google's Bronze/Silver/Gold zones.

Links

<https://cloud.google.com/bigquery/docs/iceberg-tables>

<https://cloud.google.com/bigquery/docs/biglake-intro>

<https://iceberg.apache.org/>

WATCH FIRST · 4:39 · GOOGLE CLOUD TECH

De-identifying Data with Sensitive Data Protection

- ✓ Finds and classifies PII, then redacts, masks, or tokenizes it — before it ever reaches a dashboard. The governance half of M10: your state data has PII in it, and this is the service that knows
- ✓ exactly where.



Scan to open on YouTube
youtu.be/MHEnuw9if3Y

Custom slide · added by Imran Ahmad

Iowa DOM DoIT · Data Engineering on Google Cloud

INSTRUCTOR NOTES — PAGE 117 OF 208 · 14:00 · MODULE 10 — GOVERNANCE + THE MEDALLION**Hook**

- Play as the M10 opener — governance lands better as a demo than as a policy slide.

Say

- “Citizen data, PII, audits — this is the tool your security office will ask about by name.”

Cue

- All 4:39; masking vs. tokenization (~2:30) is the distinction worth pausing on.

Link

- <https://www.youtube.com/watch?v=MHEnuw9if3Y>

M10 — What this block gets you

- ✓ The governance tooling behind your single source of truth— Knowledge Catalog, Sensitive Data Protection, fine-grained security
- ✓ The moment: Google's own Bronze/Silver/Gold zone slides held up next to your architecture doc— zone for zone
- ✓ A GCP home for each zone— GCS/BigLake Bronze, Dataform Silver-to-Gold, native BigQuery Gold marts
- ✓ A phased migration path off the SQL Server DW— no big-bang cutover, extracts retired last
- ✓ The cost levers and best practices you set on day one— storage class, bytes scanned, budgets and alerts

Custom slide · added by Imran Ahmad

Iowa DOM DoIT · Data Engineering on Google Cloud

INSTRUCTOR NOTES — PAGE 118 OF 208 · 14:00 · MODULE 10 — GOVERNANCE + THE MEDALLION

Hook

The Iceberg demo just showed governed files; now the leadership question — how does BigQuery become the single source of truth instead of copy number fifteen?

Say · cover

- Next 35 minutes: governance tooling, then Google's own Bronze/Silver/Gold zone slides — their architecture doc, zone for zone, in Google's deck.
- 'Single source of truth' is a governance claim: one cataloged, classified, access-controlled copy that every team serves from.
- Each zone already has a GCP home from today: GCS/BigLake Bronze, Dataform Silver-to-Gold, native BigQuery Gold marts behind Power BI.
- Migration is phased by use case — no big-bang cutover; the SQL Server DW and its extracts retire last.
- Cost levers get set on day one: storage class, bytes-scanned discipline, budgets and alerts in Cloud Billing.

Iowa tie

This block is their project charter in Google tooling — the 'single source of truth' objective from the migration brief gets its mechanism here.

Design patterns

- Name a zone owner and serving layer per zone before migrating anything
- Govern Bronze in place with BigLake policy tags, not just Gold

Anti-patterns

- Big-bang cutover off the SQL Server DW
- Governance as a phase-2 retrofit after the marts are built

History & what's new

Catalog branding moved fast: Data Catalog deprecated in 2025, Dataplex Universal Catalog renamed Knowledge Catalog on April 10, 2026, data products GA that May.

Flow

Iceberg/BigLake demo just ended; this opens M10 — a timing card next, then metadata.

Links

<https://docs.cloud.google.com/knowledge-catalog/docs/overview>

<https://cloud.google.com/sensitive-data-protection/docs>

<https://docs.cloud.google.com/lakehouse/docs/key-concepts>

M10 — 14:00 to 14:35, then break

14:00–14:10	Governance as your single-source-of-truth requirement — Knowledge Catalog, Sensitive Data Protection
14:10–14:15	Fine-grained security — column-level, row-level, dynamic masking
14:15–14:25	The Medallion — Google's Bronze/Silver/Gold zones next to your architecture doc
14:25–14:35	Migration strategy, cost management, best practices
14:35–14:50	Break
14:50–15:20	M11 — the BQML direction and the Power BI last mile

FACILITATOR NOTE

- Timing card for the block: governance and the single-source-of-truth requirement 14:00–14:10, fine-grained security 14:10–14:15, the medallion reveal 14:15–14:25, migration/cost/best practices 14:25–14:35.
- Hold the schedule — the Bronze/Silver/Gold zone slides are the moment, so keep the DLP demo under five minutes.
- Break at 14:35, then M11: the BQML direction and the Power BI last mile.

Flow: Block opener done; this is the pacing contract — next is metadata ("What is metadata?").

What is metadata?

Metadata is data about data, which can include:

01 Who created the data

04 How it relates to other data

02 When it was created

05 Who owns it

03 What it contains

06 Its security sensitivity

Google Cloud

INSTRUCTOR NOTES — PAGE 120 OF 208 · 14:00 · MODULE 10 — GOVERNANCE + THE MEDALLION

Hook

In your estate this knowledge lives in a wiki nobody maintains, extended properties, and Dave's head. Metadata is what turns a pile of tables into a governable asset.

Say · cover

- Walk the six facets — creator, creation time, contents, relationships, owner, sensitivity. Each is a question someone asks before trusting a table.
- Split technical metadata (schema, types, partitions) from business metadata (owner, glossary terms, sensitivity); a source of truth needs both.
- 'Which table is real?' is a metadata question — without a system, the answer is archaeology across extracts.
- Sensitivity as metadata is the hinge of this module: it drives classification, masking, and access control downstream.

Iowa tie

Their tribal knowledge about the SQL Server DW — which extracts feed which reports — is exactly the metadata that must be captured in GCP before the DW retires.

Design patterns

- Capture ownership and sensitivity at table creation, not in an annual audit

Anti-patterns

- Metadata as tribal knowledge or a stale wiki — it dies with the next reorg

History & what's new

Google's 2026 framing treats metadata as 'active context': Knowledge Catalog (renamed from Dataplex Universal Catalog in April 2026) auto-ingests technical metadata and uses Gemini to draft descriptions.

Flow

Timing card done; this is the foundational definition before Knowledge Catalog.

Links

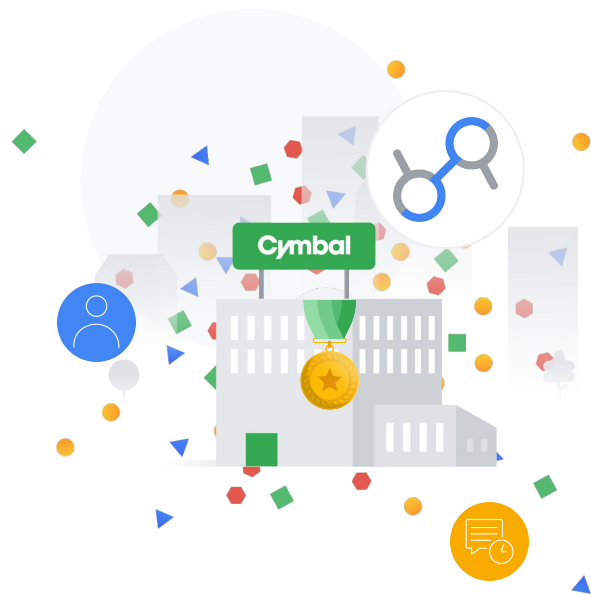
<https://docs.cloud.google.com/knowledge-catalog/docs/concepts>

<https://docs.cloud.google.com/knowledge-catalog/docs/overview>

<https://docs.cloud.google.com/knowledge-catalog/docs/discover-data>

Knowledge Catalog for Cymbal

Without a centralized metadata system, data management can be difficult. Knowledge Catalog provides a unified metadata hub.



Google Cloud

INSTRUCTOR NOTES — PAGE 121 OF 208 · 14:00 · MODULE 10 — GOVERNANCE + THE MEDALLION

Hook

The SQL Server world answers 'what do we have?' with data dictionaries and asking around. Knowledge Catalog is that answer as a managed service.

Say · cover

- Cymbal's problem is theirs: without a centralized metadata system, data management degrades as assets multiply.
- Knowledge Catalog auto-crawls BigQuery datasets and Cloud Storage buckets — schemas and attributes appear without manual registration.
- One hub lets stewardship scale: owners, descriptions, and glossary terms attach to the asset, not to a person.
- Console pointer: search 'Knowledge Catalog' in the console nav — API, CLI, and IAM roles still say 'dataplex'.

Iowa tie

This is the tool that makes 'which table is the real one?' answerable after migration — a single source of truth needs a single catalog of truth.

Design patterns

- Adopt a business glossary early so county and program terms mean one thing across marts

Anti-patterns

- Cataloging only BigQuery while Bronze files in Cloud Storage stay invisible

History & what's new

Dataplex Universal Catalog was renamed Knowledge Catalog on April 10, 2026 (API and IAM names unchanged); legacy Data Catalog shut down June 1, 2026 — expect stale screenshots in older blog posts.

Flow

Metadata defined; now the hub that operationalizes it — capabilities next.

Links

<https://docs.cloud.google.com/knowledge-catalog/docs/overview>
<https://docs.cloud.google.com/knowledge-catalog/docs/release-notes>
<https://docs.cloud.google.com/knowledge-catalog/docs/concepts>

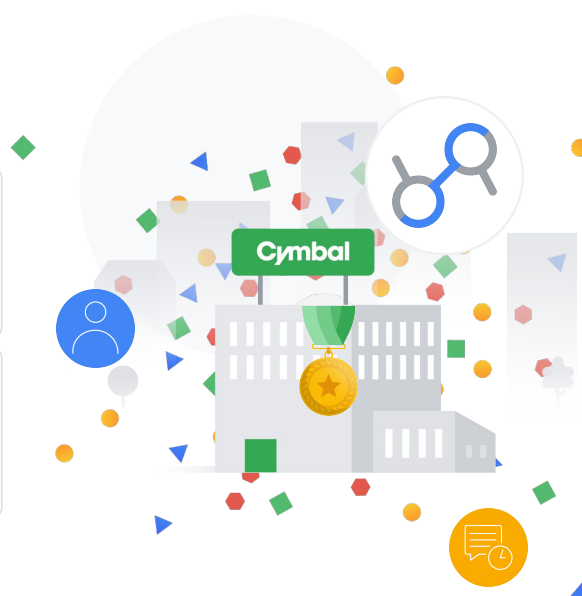
Knowledge Catalog for Cymbal

01

Acts as a universal catalog for all data assets, whether they reside in BigQuery, Cloud Storage, or Lakehouse.

02

Use to discover data, understand data lineage, and manage and augment metadata.



Google Cloud

INSTRUCTOR NOTES — PAGE 122 OF 208 · 14:00 · MODULE 10 — GOVERNANCE + THE MEDALLION

Hook

Three verbs on this slide — discover, lineage, augment — map to three pains you have today with extracts.

Say · cover

- Universal: one catalog spans BigQuery, Cloud Storage, and lakehouse assets — Bronze files and Gold marts in the same index.
- Discover: natural-language semantic search — analysts find the certified table instead of copying an extract.
- Lineage: column-level traces across pipelines — change a Silver table and see exactly which Gold marts moved.
- Augment: attach aspects, owners, glossary terms; Gemini drafts descriptions from query history for steward review.

Iowa tie

Lineage is their SSIS-package-archaeology killer: the extract-to-report chain that lives in someone's head becomes a queryable graph before the DW retires.

Design patterns

- Certify Gold marts with an ownership/trust aspect so Power BI builders stop guessing

Anti-patterns

- Letting analysts search raw Bronze and certified Gold with no trust marker between them

History & what's new

Lineage keeps deepening: column-level lineage for Dataproc went GA in May 2026, and a Datastream metadata integration with Knowledge Catalog entered public preview in 2026.

Flow

The hub's capabilities; next the payoff slide — consistency and quality at scale.

Links

<https://docs.cloud.google.com/knowledge-catalog/docs/discover-data>
<https://docs.cloud.google.com/knowledge-catalog/docs/track-lineage>
<https://docs.cloud.google.com/knowledge-catalog/docs/overview>

By providing a single reference source,
Knowledge Catalog helps Cymbal manage
data at scale while **ensuring consistency
and quality**.

Google Cloud

INSTRUCTOR NOTES — PAGE 123 OF 208 · 14:00 · MODULE 10 — GOVERNANCE + THE MEDALLION

Hook

Scale is the operative word — you are about to have more tables than any team can hold in their heads.

Say · cover

- One reference source ends the 'two extracts, two answers' meeting: everyone resolves to the same certified asset.
- Consistency is enforced by glossary terms and quality rules attached to entries, not by convention documents.
- Quality scans profile data and flag anomalies or stale freshness — trust signals sit next to the table in search results.
- For the M&O team: this is where 'who owns this table?' gets a standing answer after go-live.

Iowa tie

Their kill-the-extracts goal fails without this — if the catalog cannot say which copy is authoritative, teams keep exporting their own.

Design patterns

- Publish Gold marts as certified data products; keep Bronze/Silver discoverable but marked unverified

Anti-patterns

- Treating the catalog as documentation you write once — it must be crawled and refreshed

History & what's new

Data products — bundling tables with owners, SLAs, and usage contracts — went GA in Knowledge Catalog in May 2026, formalizing exactly this 'certified, serveable asset' idea.

Flow

Knowledge Catalog wrapped; next the custom slide tying all four governance tools to their requirement.

Links

<https://docs.cloud.google.com/knowledge-catalog/docs/overview>

<https://docs.cloud.google.com/knowledge-catalog/docs/concepts>

<https://docs.cloud.google.com/knowledge-catalog/docs/release-notes>

Governance = your single source of truth

Your problem is not storing data — extracts scatter it. One governed copy serves everyone.

- Knowledge Catalog— one catalog across BigQuery, Cloud Storage, lakehouse; which table is real has an answer
- Sensitive Data Protection— finds and classifies PII before it reaches a dashboard; your Data Discovery analogue
- Column-level security + masking— purchase history yes, contact info no; your column GRANT/DENY plus masking
- Row access policies— the regional manager sees only their region, like your RLS predicates
- Lineage— change a Silver table, see exactly which Gold marts and Power BI reports you touched

Custom slide · added by Imran Ahmad

Iowa DOM DoIT · Data Engineering on Google Cloud

INSTRUCTOR NOTES — PAGE 124 OF 208 · 14:00 · MODULE 10 — GOVERNANCE + THE MEDALLION

Hook

This slide is your architecture review in one picture — every line is a tool you have met, or will meet in the next fifteen minutes.

Say · cover

- Read the premise aloud: their problem is not storing data — extracts scatter it; one governed copy serves everyone.
- Knowledge Catalog answers 'which table is real?'; Sensitive Data Protection finds PII before it reaches a dashboard — their SSMS Data Discovery analogue.
- Column-level security plus masking equals their column GRANT/DENY and Dynamic Data Masking, delivered as policy tags instead of per-table grants.
- Row access policies equal their RLS predicates — the regional manager sees only their region's rows.
- Lineage closes the loop: change a Silver table, see which Gold marts — and which Power BI reports — you touched.

Iowa tie

Point at each line and name their component: catalog, classification, masking, row filters, lineage — the tooling behind the 'single source of truth' line in their migration objective.

Design patterns

- Map every SQL Server control (GRANT/DENY, DDM, RLS) to its GCP twin in migration planning so nothing is dropped in translation

Anti-patterns

- Recreating scatter by granting each agency its own extract 'just for now'

History & what's new

The stack kept converging in 2026: BigQuery data governance tags — Resource Manager tags enforcing column security and masking — entered preview in July 2026.

Flow

Custom anchor slide for the block; next, Sensitive Data Protection gets three slides.

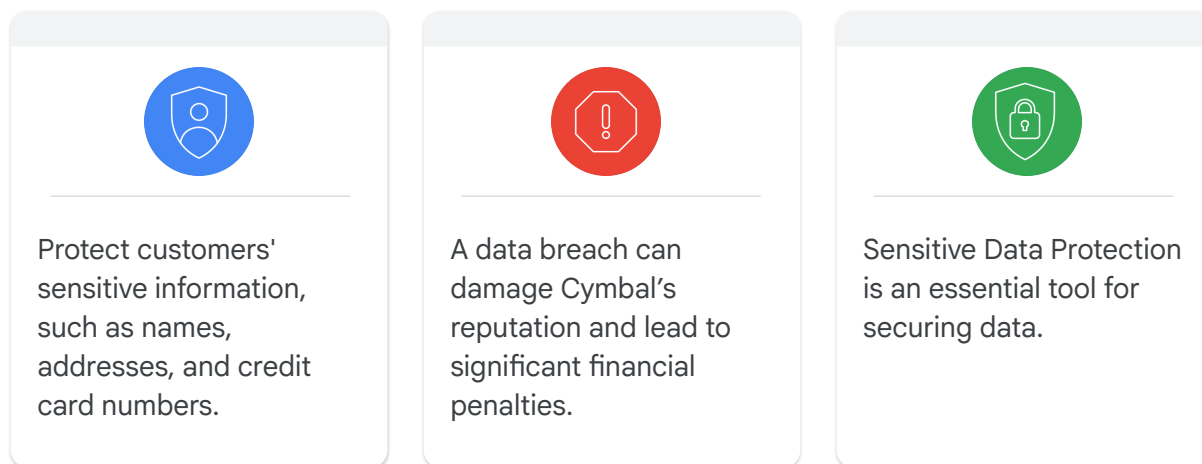
Links

<https://cloud.google.com/bigquery/docs/column-level-security>

<https://docs.cloud.google.com/bigquery/docs/row-level-security-intro>

<https://docs.cloud.google.com/knowledge-catalog/docs/overview>

Sensitive Data Protection



Google Cloud

INSTRUCTOR NOTES — PAGE 125 OF 208 · 14:00 · MODULE 10 — GOVERNANCE + THE MEDALLION

Hook

SSMS Data Discovery & Classification tells you where sensitive columns are. This is that idea as a fleet-wide, continuously running service.

Say · cover

- The Cymbal framing is theirs: state data means names, addresses, SSNs — a breach is a headline and a penalty, not just an incident.
- Sensitive Data Protection is fully managed: discovery, inspection, de-identification, and risk analysis — the underlying DLP API keeps its old name.
- It scans BigQuery, Cloud SQL, and Cloud Storage — Bronze files are scannable before any transformation runs.
- Findings stream to BigQuery, Pub/Sub, or Security Command Center — audit-ready for a government compliance shop.

Iowa tie

County and program data landing in their Bronze zone will contain PII; SDP tells them exactly where before Power BI ever sees it.

Design patterns

- Scan Bronze on landing — classify before Dataform touches anything

Anti-patterns

- Assuming PII lives only in obvious columns; free-text comment fields leak the worst surprises

History & what's new

Cloud DLP became Sensitive Data Protection in 2023; Google added built-in tokenization in January 2025, and 2026 discovery profiles scan BigQuery, Cloud SQL, and Cloud Storage continuously.

Flow

Custom governance map done; now SDP proper — the discover/classify/protect loop next.

Links

<https://cloud.google.com/sensitive-data-protection/docs>

<https://cloud.google.com/sensitive-data-protection/docs/concepts>

<https://docs.cloud.google.com/sensitive-data-protection/docs/infotypes-reference>

Sensitive Data Protection

Discover

Run scans on data lakehouse and other data to identify multiple types of sensitive field.

Classification

Data is classified by sensitivity level. This ensures the right security controls are applied.

Protection

Use techniques, such as masking or tokenization, to de-identify the data.

Example: *****9787

Google Cloud

INSTRUCTOR NOTES — PAGE 126 OF 208 · 14:00 · MODULE 10 — GOVERNANCE + THE MEDALLION

Hook

Three stages, and the order matters: you cannot mask what you have not found.

Say · cover

- Discover: scan configurations profile tables across the org against infoTypes — EMAIL_ADDRESS, PHONE_NUMBER, US_SOCIAL_SECURITY_NUMBER, and hundreds more.
- Classify: each asset gets a sensitivity level and risk score, so controls follow the data, not someone's memory of the schema.
- Protect: masking, redaction, tokenization, bucketing, date-shifting — the *****9787 example is format-preserving masking.
- Detectors read free text too — the survey-comment column with a phone number typed into it gets caught.

Iowa tie

Their pipeline gets this staged treatment at zone boundaries: discover in Bronze, classify before Silver, protect before Gold marts reach Power BI.

Design patterns

- Start with org-level discovery profiles, then targeted inspection jobs on flagged datasets

Anti-patterns

- Masking everything 'to be safe' — over-de-identification destroys analytical utility

History & what's new

The 2026 discovery service generates continuous data profiles with sensitivity scores per table — classification is now a standing posture, not a one-off scan project.

Flow

SDP intro done; this is its working loop — the proactive-use payoff next.

Links

<https://docs.cloud.google.com/sensitive-data-protection/docs/data-profiles>
<https://docs.cloud.google.com/sensitive-data-protection/docs/infotypes-reference>
<https://docs.cloud.google.com/sensitive-data-protection/docs/deidentify-sensitive-data>

This proactive approach to data protection allows Cymbal to **use their data for analytics and ML** without exposing sensitive information.

Google Cloud

INSTRUCTOR NOTES — PAGE 127 OF 208 · 14:00 · MODULE 10 — GOVERNANCE + THE MEDALLION

Hook

The goal is not locking data away — it is making PII safe enough that analysts stop needing the raw copy.

Say · cover

- De-identification preserves utility: analysts aggregate masked or tokenized columns without ever seeing plaintext PII.
- Tokenization is reversible with the key — re-identification stays available, under controlled roles, for the few who legitimately need it.
- This kills the 'give me a full extract, I will be careful' request: the governed, de-identified copy becomes the self-serve copy.
- For their Power BI last mile: dashboards point at data that is safe by construction, not by policy memo.

Iowa tie

This is how one governed copy actually serves everyone — agency analytics teams get useful data while DOM provably holds the breach risk down.

Design patterns

- De-identify in a controlled pipeline step and grant raw access by exception only

Anti-patterns

- Copying raw PII into analyst sandboxes 'temporarily' — those copies never die

History & what's new

Google shipped built-in tokenization for Sensitive Data Protection in January 2025 — format-preserving, reversible tokens without standing up your own vault.

Flow

SDP loop complete; fine-grained security inside BigQuery is next.

Links

<https://docs.cloud.google.com/sensitive-data-protection/docs/deidentify-sensitive-data>

<https://cloud.google.com/blog/products/identity-security/get-started-with-built-in-tokenization-for-sensitive-data-protection>

<https://cloud.google.com/sensitive-data-protection/docs/concepts>

Fine-grained security

Column-level security

Restricts access to specific columns in a table. For example, a marketing analyst might be able to access a customer's purchase history but not their contact information. This is effective for protecting Personally Identifiable Information (PII).

Row-level security

Filters which rows a user can access. A regional sales manager for North America, for instance, would only have access to sales data for that region. This is particularly useful for large, multinational companies, such as Cymbal.

Google Cloud

 INSTRUCTOR NOTES — PAGE 128 OF 208 · 14:00 · MODULE 10 — GOVERNANCE + THE MEDALLION

Hook

You know both of these: column GRANT/DENY and row-level security predicates. BigQuery has the same two ideas — with different mechanics.

Say · cover

- Column-level: policy tags from a taxonomy attach to columns; users need the Fine-Grained Reader role on the tag — PII protection without splitting tables.
- The slide's example is the classic one: purchase history yes, contact info no — one table, two audiences.
- Row-level: CREATE ROW ACCESS POLICY with a filter_expression that works like a WHERE clause per grantee list.
- Critical gotcha: users in no grantee list see zero rows — add a TRUE-filter policy for admins or the table looks empty.
- Both compose with dataset/table IAM and each other — least privilege at three layers.

Iowa tie

County-scoped reporting is their row-filter case; SSN and contact columns are their policy-tag case — both land on the Gold marts Power BI reads.

Design patterns

- Centralize policy tags in one taxonomy managed with the data platform, not per-project improvisation

Anti-patterns

- Duplicating tables per audience instead of filtering — the extract problem rebuilt inside BigQuery

History & what's new

Row access policies arrived in 2020, dynamic masking preview in 2022; in July 2026 data governance tags entered preview — Resource Manager tags enforcing column security and masking uniformly.

Flow

SDP done; this is the 14:10 segment — a quote slide, then masking on lakehouse files.

Links

<https://cloud.google.com/bigquery/docs/column-level-security>

<https://docs.cloud.google.com/bigquery/docs/row-level-security-intro>

<https://docs.cloud.google.com/bigquery/docs/column-data-masking-intro>



Hook

Read the quote as two clauses — 'secure' is the easy half; 'accessible to the correct people' is the engineering.

Say · cover

- Security that blocks everyone fails as surely as no security: the goal is governed self-service, not a vault.
- The stack they just saw — catalog, classification, policy tags, row filters — is what makes 'correct people, appropriate time' operational.
- Contrast with today: accessibility is currently achieved by handing out extract copies, which instantly destroys the 'secure' half of the sentence.
- For M&O: access changes become tag and policy edits, not table rebuilds and new extract jobs.

Iowa tie

This is the operating promise of their target state: agencies self-serve from governed Gold marts while DOM provably controls PII exposure.

Design patterns

- Default-deny with group-based grantee lists; audit access through the catalog, not ticket archaeology

Anti-patterns

- Granting broad table access because fine-grained setup felt slow — the exception becomes the architecture

History & what's new

Google's June 2022 blog introduced dynamic data masking on top of BigQuery column-level security; masking reached GA in 2023.

Flow

Fine-grained security stated; the payoff quote — next, masking reaches files still in the lake.

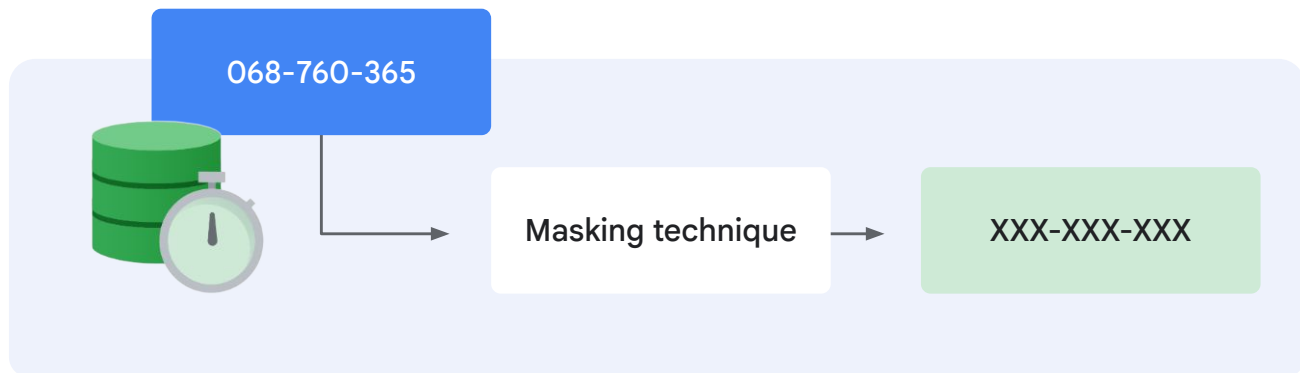
Links

<https://cloud.google.com/blog/products/identity-security/announcing-new-bigquery-capabilities-to-help-secure-sensitive-data>

<https://cloud.google.com/bigquery/docs/column-level-security>

<https://docs.cloud.google.com/bigquery/docs/row-level-security-intro>

For Lakehouse tables in Cloud Storage, Sensitive Data Protection can apply **dynamic data masking**.



Google Cloud

INSTRUCTOR NOTES — PAGE 130 OF 208 · 14:00 · MODULE 10 — GOVERNANCE + THE MEDALLION

Hook

Your Dynamic Data Masking in SQL Server — same idea, except here it reaches files that never left the bucket.

Say · cover

- Masking is a data policy on a policy tag: same table, different view per role — Masked Reader sees XXX-XXX-XXX, Fine-Grained Reader sees the number.
- Masking routines range from full nulling to format-preserving patterns like the phone example — no view layer to maintain.
- On BigLake/lakehouse tables over Cloud Storage the same policies apply — Bronze stays in open formats yet is governed.
- This is why the 13:00 lab mattered: Iceberg/BigLake tables are not governance-free raw files.

Iowa tie

Their Bronze zone can land county data with PII and still be broadly queryable — masking travels with the column, so nobody needs an extract to 'clean it first'.

Design patterns

- Mask at the policy tag so every current and future tagged table inherits the rule

Anti-patterns

- Building parallel 'safe' copies with PII columns stripped — drift and double storage

History & what's new

Dynamic data masking was announced for BigQuery in June 2022 and reached GA in 2023; BigLake tables then carried the same column and row controls onto open-format files in Cloud Storage.

Flow

Column/row controls plus masking complete; next is the live Sensitive Data Protection demo.

Links

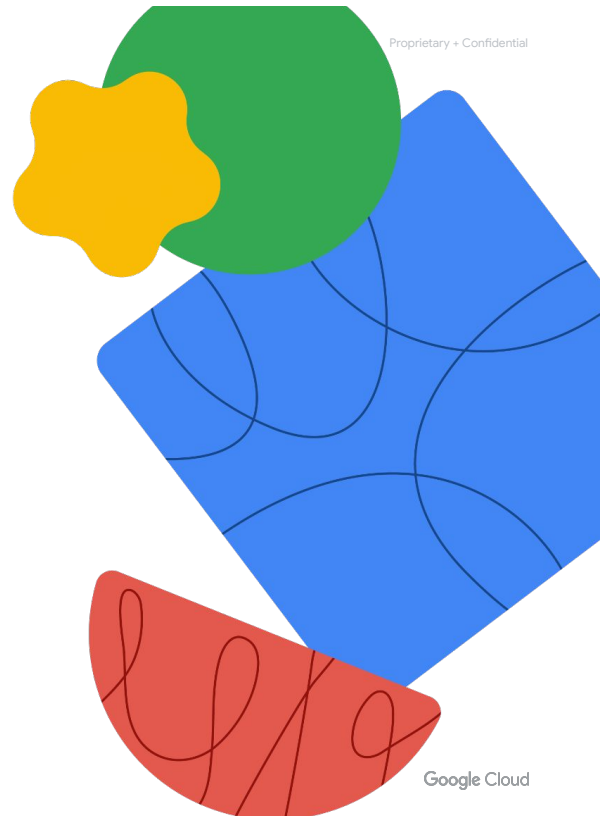
<https://docs.cloud.google.com/bigquery/docs/column-data-masking-intro>

<https://cloud.google.com/bigquery/docs/biglake-intro>

<https://cloud.google.com/sensitive-data-protection/docs>

Data loss prevention

Demonstration



INSTRUCTOR NOTES — PAGE 131 OF 208 · 14:00 · MODULE 10 — GOVERNANCE + THE MEDALLION

Hook

Enough slides — watch the service find PII in a real table. Keep it tight: the medallion slides at 14:15 are the moment we are protecting time for.

Say · cover

- Demo path: console > Security > Sensitive Data Protection > create an inspection job against the demo BigQuery table; walk the findings by infoType.
- Then the fix: a de-identification job (masking/tokenization) writing a safe copy — or policy tags plus masking if the data stays in BigQuery.
- Narrate the SQL Server analogue while clicking: Data Discovery & Classification plus masking, run as a fleet-wide job.
- Fallback if the console misbehaves: screenshots plus the pre-run findings table; cap the demo at five minutes.

Iowa tie

Tell them to picture this job pointed at their first migrated Bronze dataset — same clicks, their county data, before any Power BI repointing.

Design patterns

- Save inspection findings to a BigQuery table so classification evidence is itself queryable and auditable

Anti-patterns

- Running inspection ad hoc once — schedule scan configs so new tables get profiled automatically

History & what's new

The demo workflow is stable, but the product around it moved: continuous discovery profiles became the recommended entry point, and the DLP API kept its name through the 2023 Sensitive Data Protection rename.

Flow

Fine-grained security wrapped; this is the demo card — the loyalty-program scenario setup follows.

Links

<https://cloud.google.com/sensitive-data-protection/docs>

<https://docs.cloud.google.com/sensitive-data-protection/docs/data-profiles>

<https://docs.cloud.google.com/sensitive-data-protection/docs/infotypes-reference>

Scenario: Cymbal loyalty program

01

New customer information has been collected and loaded into a BigQuery table named `loyalty_program_customers`.

02

This table contains standard information, such as names and purchase histories.

03

Includes potentially sensitive data, such as email addresses, phone numbers, and free-text comments from customer feedback surveys.

Google Cloud

INSTRUCTOR NOTES — PAGE 132 OF 208 · 14:00 · MODULE 10 — GOVERNANCE + THE MEDALLION

Hook

Read the scenario as your own: a new customer dataset lands, it mixes harmless and sensitive fields, and someone wants it in a dashboard by Friday.

Say · cover

- The `loyalty_program_customers` table mirrors their reality: standard columns plus email, phone, and free-text survey comments.
- Free text is the trap — inspection finds the phone number typed into a comment field that schema review never would.
- Before marketing analytics gets access, the questions are exactly theirs: what is sensitive, where, and who may see it unmasked?
- Demo clicks: inspect the table, show infoType findings including inside comments, then apply masking.

Iowa tie

Swap 'Cymbal loyalty' for any county program intake file — the shape is identical: structured PII columns plus a comments field nobody audited.

Design patterns

- Treat free-text columns as sensitive until inspection proves otherwise

Anti-patterns

- Classifying by column name only — 'comments' and 'notes' fields defeat name-based review

History & what's new

infoTypes now number in the hundreds, with ML-assisted detectors for unstructured text; the 2026 inspection service reports each finding's exact location — what the demo shows live.

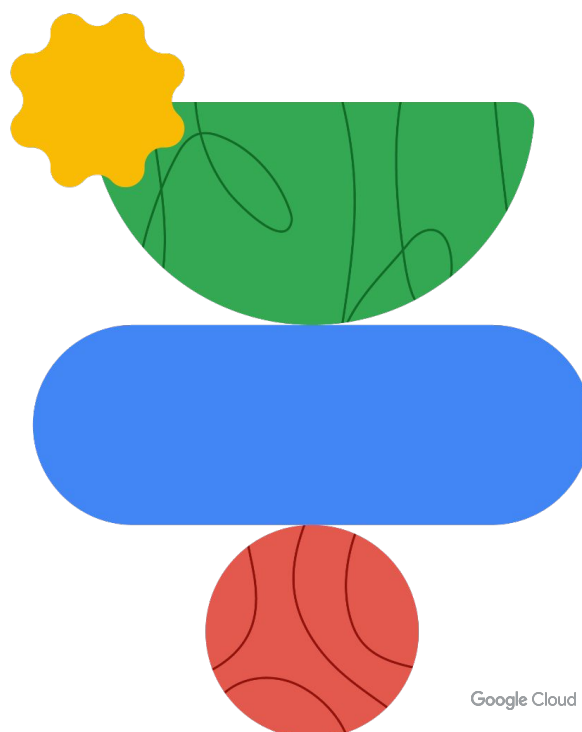
Flow

Demo card shown; this sets the scenario — the exposure-risk statement is next.

Links

<https://cloud.google.com/sensitive-data-protection/docs/concepts>
<https://docs.cloud.google.com/sensitive-data-protection/docs/infoTypes-reference>
<https://docs.cloud.google.com/sensitive-data-protection/docs/data-profiles>

Before this data is made available to the marketing analytics team, **any PII must be properly handled** to avoid exposing sensitive customer details in analytics dashboards.



INSTRUCTOR NOTES — PAGE 133 OF 208 · 14:00 · MODULE 10 — GOVERNANCE + THE MEDALLION

Hook

This sentence is your governance gate in one line: nobody gets the data until PII handling is decided and enforced.

Say · cover

- 'Made available' is the key phrase — the gate sits between the landing table and every consumer, including dashboards.
- Enforcement order: inspect and classify; mask or tokenize the sensitive columns; grant masked vs. clear access via policy tags.
- Dashboards are the highest-risk surface: one extract into a report dataset bypasses every control if the gate is skipped.
- With the gate in place, the marketing-analytics team self-serves — the extract request disappears instead of being refused.

Iowa tie

This is the end-state for their Power BI estate: reports repoint at governed Gold marts where PII handling already happened, so publishing a dashboard stops being a risk review.

Design patterns

- Make de-identification a Dataform pipeline step — enforced in code, reviewable in their CI/CD

Anti-patterns

- Gating by policy memo: 'the dashboard team knows not to select those columns'

History & what's new

The 2022–2023 wave adding masking to column-level security made this gate enforceable in the engine; the July 2026 data governance tags preview pushes the same rules into org-level Resource Manager tags.

Flow

Scenario fully framed — run the demo, then part B: the Bronze/Silver/Gold medallion slides, their architecture in Google's deck.

Links

<https://docs.cloud.google.com/sensitive-data-protection/docs/deidentify-sensitive-data>
<https://cloud.google.com/bigquery/docs/column-level-security>
<https://docs.cloud.google.com/bigquery/docs/column-data-masking-intro>

Your architecture doc, in Google's deck

Hold the next six slides up next to the Iowa architecture doc — they line up zone for zone.

- Retire legacy extracts— the nightly SSIS extract jobs end; data lands in BigQuery continuously
- SQL Server DW to BigQuery— one governed copy becomes the analytics source of truth
- Bronze, Silver, Gold— your medallion lakehouse, in Google's own vocabulary
- Dataform ELT Silver to Gold— versioned, tested SQL pipelines replacing SSIS data flows
- Gold marts feed Power BI— the reports your users already have, repointed at BigQuery

Your design, in their deck.

Custom slide · added by Imran Ahmad

Iowa DOM DoIT · Data Engineering on Google Cloud

INSTRUCTOR NOTES — PAGE 134 OF 208 · 14:00 · MODULE 10 — GOVERNANCE + THE MEDALLION

Hook

Hold the Iowa architecture doc up next to the screen - for the next six slides, Google's own curriculum draws your design, zone for zone.

Say · cover

- Walk the five lines: retiring SSIS extracts, one governed copy as truth, Bronze/Silver/Gold in Google's vocabulary, Dataform Silver-to-Gold ELT, Gold marts feeding Power BI reports.
- Nothing here is new vocabulary - the value is validation: the pattern you committed to is the pattern Google teaches.
- The next six slides are Google's stock Cymbal retail case; every box maps to a named service in your design.
- Have an attendee match a line from their doc to the slide - 60 seconds that anchors the afternoon.

Iowa tie

All five lines are literally your project: kill the extracts, SQL Server DW to BigQuery, medallion zones, Dataform ELT, Power BI repointed at Gold.

Design patterns

- Use identical zone names in your doc, datasets, and the deck - shared vocabulary is free governance
- One governed copy in BigQuery replacing per-report extracts

Anti-patterns

- Lift-and-shifting every extract job to the cloud unchanged
- Standing up a second, ungoverned copy of the warehouse

History & what's new

Databricks coined the medallion vocabulary around 2019-2020; Google Cloud now teaches it as the canonical lakehouse pattern (an October 2025 community series even showed data-engineering agents building medallion pipelines in BigQuery).

Flow

Straight from the 13:45 Iceberg/BigLake demo; next are the six medallion slides, then migration strategy and cost before the 14:35 break.

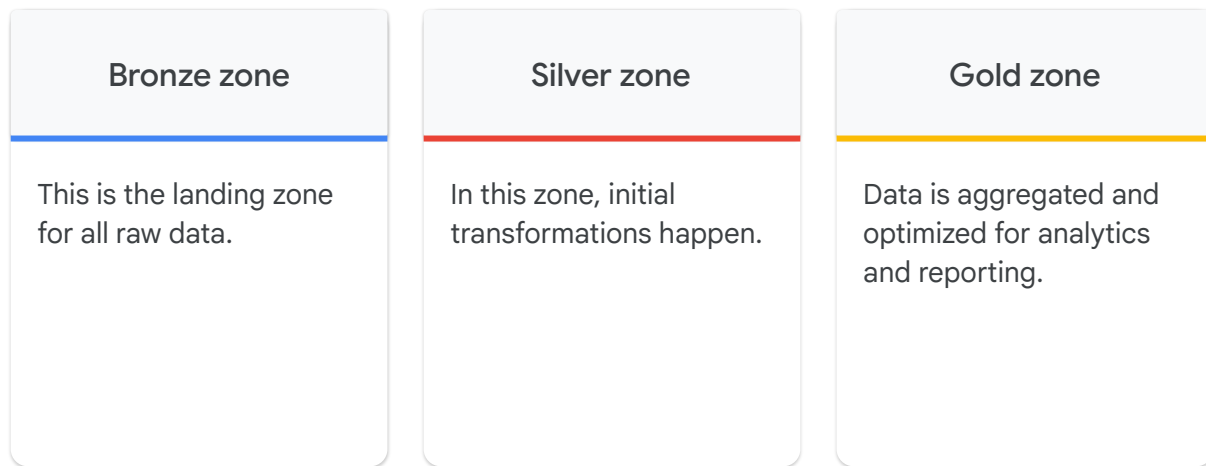
Links

<https://cloud.google.com/discover/what-is-medallion-architecture>

<https://cloud.google.com/dataform/docs/overview>

<https://cloud.google.com/bigquery/docs/introduction>

The medallion architecture (1 of 3)



Google Cloud

INSTRUCTOR NOTES — PAGE 135 OF 208 · 14:00 · MODULE 10 — GOVERNANCE + THE MEDALLION

Hook

You already know this shape from your SQL Server estate - staging tables, cleansed integration layer, the star schema Power BI reads. Google just names the layers Bronze, Silver, Gold.

Say · cover

- Bronze is the landing zone for raw data: everything as received, nothing edited - your extract output plus staging, kept forever.
- Silver is where initial transformations happen: typing, dedupe, standardized codes and dates - what your SSIS data flows produce.
- Gold is aggregated and optimized for analytics and reporting - the marts and star schemas reports sit on.
- Key property: data flows forward only. If Gold logic changes, rebuild Silver and Gold from Bronze - the raw record is never destroyed.
- Draw the trust ladder: Bronze is received, Silver is verified, Gold is decision-grade.

Iowa tie

This is the zone-for-zone match to your architecture doc - the deck confirms the layering you already chose for the BigQuery migration.

Design patterns

- Forward-only flow; Silver and Gold always rebuildable from Bronze
- One trust label per zone so consumers know what they may rely on

Anti-patterns

- Editing or compacting Bronze - you lose replay and audit
- Letting reports read straight from Bronze

History & what's new

Databricks popularized the medallion pattern around 2019-2020 as the lakehouse organizing scheme; Google Cloud adopted the same vocabulary across BigQuery lakehouse docs and its own 'What is medallion architecture' guide.

Flow

After the Iowa anchor slide; the next three slides walk Google's Cymbal example zone by zone - Bronze first.

Links

<https://cloud.google.com/discover/what-is-medallion-architecture>
<https://docs.databricks.com/gcp/en/lakehouse/medallion>
<https://cloud.google.com/bigquery/docs/biglake-intro>

Cymbal: Bronze zone (raw data)

01

Clickstream data from their website.

02

Batch exports of transactional data from their e-commerce database.

03

JSON data from their social media marketing campaigns.

Google Cloud

INSTRUCTOR NOTES — PAGE 136 OF 208 · 14:00 · MODULE 10 — GOVERNANCE + THE MEDALLION

Hook

Cymbal lands three raw sources untouched - clickstream, database exports, social JSON. Swap in your county file drops and SQL Server change feeds and this is your Bronze.

Say · cover

- Cymbal's three landings - clickstream, e-commerce database exports, social JSON - deliberately mix streaming, batch, and semi-structured sources.
- Translate each: clickstream is your file-drop traffic, batch exports are your nightly extracts, social JSON is the semi-structured feed every agency inherits.
- Bronze rule: land as received, add ingest timestamps, never edit - the historical record you replay when logic changes.
- For continuous landing, Datastream reads the SQL Server transaction log and streams changes - the extract jobs stop polling the source.
- Console pointer: Bronze lives in Cloud Storage buckets as BigLake/Iceberg tables queried in place - the 13:00 lab exactly.

Iowa tie

Bronze is where your retired extract output lands from now on - Datastream CDC from SQL Server plus batch exports from source systems, immutable in GCS.

Design patterns

- Land raw with ingest timestamps and source metadata
- Keep Bronze immutable and query-in-place via BigLake/Iceberg

Anti-patterns

- Cleansing on the way in - land first, judge later
- Overwriting Bronze files, which destroys replay

History & what's new

Datastream's SQL Server source went GA in July 2024 - serverless CDC from Iowa's exact source estate into BigQuery and Cloud Storage is now a first-class, supported path.

Flow

First of the three Cymbal zone slides; next is Silver, where the cleansing they do in SSIS today moves to Dataform.

Links

<https://cloud.google.com/datastream/docs/overview>

<https://cloud.google.com/blog/products/data-analytics/datastream-makes-it-easy-to-replicate-sql-server-databases>

<https://cloud.google.com/bigquery/docs/iceberg-tables>

Cymbal: Silver zone (cleansed and conformed data)

01

Parsed clickstream data with structured sessions.

02

Augmented transactional data.

03

Standardized date and time formats.

Google Cloud

INSTRUCTOR NOTES — PAGE 137 OF 208 · 14:00 · MODULE 10 — GOVERNANCE + THE MEDALLION

Hook

Parsed sessions, augmented transactions, standardized dates - this is the layer your SSIS data flows produce today. In the target design, Dataform builds it.

Say · cover

- Cymbal's Silver does three things: parses raw clickstream into structured sessions, augments transactional data, and standardizes date and time formats.
- Name the work in their terms: type-casting, dedupe, surrogate keys, conformed county and agency codes across sources.
- Silver is the verified layer - clean enough for analysts building marts, still detailed enough for questions no one planned for.
- In GCP this is Dataform territory: incremental SQLX models for big tables, assertions acting as the data-quality gates SSIS never enforced.
- Point back to Jeff's Day-1 demo: compiled graph, assertions, release configs - the CI/CD discipline they already have translates directly.

Iowa tie

Silver is your SSIS transformation layer re-homed: Dataform incremental models with assertions, keys conformed across every source you migrate.

Design patterns

- Assertions on every Silver model - quality gates, not hopes
- Incremental models for large tables instead of full rebuilds

Anti-patterns

- Cleansing per report in Power Query - logic diverges instantly
- Skipping key conformance, so Gold joins never reconcile

History & what's new

Dataform was a London startup (founded 2018) that Google acquired in December 2020; its engine, Dataform core, is open source - the 3.0.0-beta CLI docs were refreshed in September 2025.

Flow

Cymbal zone two of three; next is Gold - the marts your Power BI reports will repoint to.

Links

<https://cloud.google.com/dataform/docs/overview>
<https://cloud.google.com/dataform/docs/assertions>
<https://cloud.google.com/dataform/docs/use-dataform-cli>

Cymbal: Gold zone (curated business-level data)

01

Aggregated daily sales tables.

02

Customer 360-degree view tables.

03

Inventory performance summaries.

Google Cloud

INSTRUCTOR NOTES — PAGE 138 OF 208 · 14:00 · MODULE 10 — GOVERNANCE + THE MEDALLION

Hook

Daily sales, customer 360, inventory summaries - Gold is the layer your Power BI reports already think they're reading. The migration just makes it true.

Say · cover

- Cymbal's Gold is business products: daily sales aggregates, customer 360, inventory summaries - marts and star schemas, not tables of record.
- Gold tables serve one audience each: aggregate once, serve many reports - two reports needing the same number means one Gold table.
- Physically, Gold is normally native BigQuery storage, partitioned and clustered with the M9 levers - Power BI feels the difference.
- The last mile is repointing: the native connector reads Gold (Import or DirectQuery); the report doesn't change.
- Warn about scope: Gold is curated per business process - don't rebuild the whole warehouse schema as 'Gold'.

Iowa tie

Gold is what your existing Power BI reports repoint to - same reports, new governed source. This is the visible payoff of the whole migration.

Design patterns

- One mart per business process; aggregate once, read many times
- Partition and cluster Gold marts for report query patterns

Anti-patterns

- Pointing Power BI at Silver or raw data - bypasses all governance
- Duplicating aggregation logic in DAX across reports

History & what's new

Power BI ships a native BigQuery connector (Import and DirectQuery); Microsoft added Entra ID (Azure AD) single sign-on for DirectQuery in public preview March 2023 - enterprise auth like your SSAS experience.

Flow

Cymbal zone three of three; next, medallion part 2 covers where each zone physically lives - open formats vs native tables.

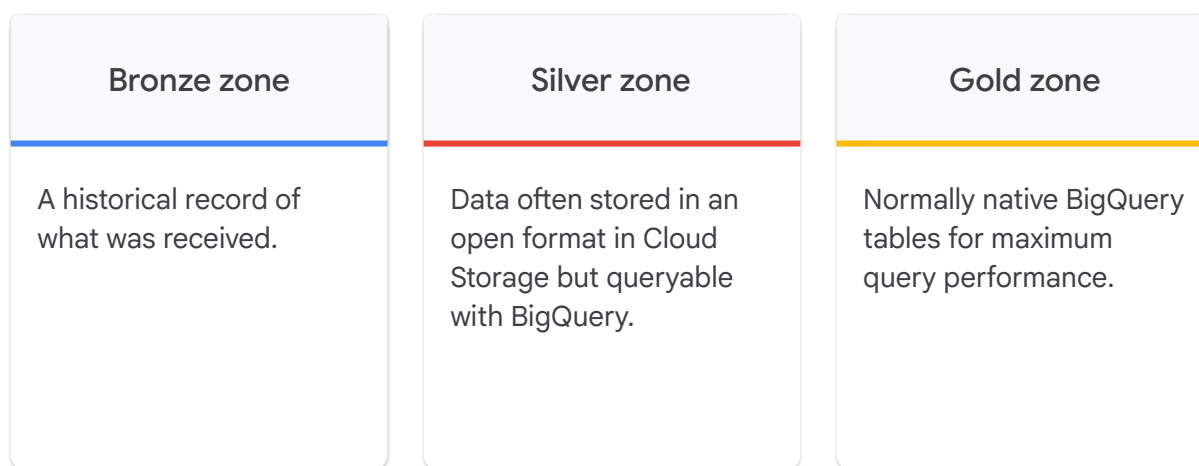
Links

<https://learn.microsoft.com/en-us/power-query/connectors/google-bigquery>

<https://cloud.google.com/bigquery/docs/partitioned-tables>

<https://cloud.google.com/bigquery/docs/clustered-tables>

The medallion architecture (2 of 3)



Google Cloud

INSTRUCTOR NOTES — PAGE 139 OF 208 · 14:00 · MODULE 10 — GOVERNANCE + THE MEDALLION

Hook

Same three zones, now a storage decision: which live in open files on Cloud Storage, and which live inside BigQuery itself?

Say · cover

- Bronze: open format in Cloud Storage, queryable by BigQuery - Parquet or Iceberg files, the permanent record of what was received.
- Silver: often still open format on GCS through BigQuery - Dataform reads and writes across that boundary without moving data.
- Gold: normally native BigQuery tables for maximum query performance - the Power BI hot path, so it gets the warehouse's own storage.
- Open formats are the anti-lock-in clause: Iceberg files BigQuery writes stay readable by Spark and other engines - one copy, many readers.
- Tie to their day: the 13:00 lab queried external data in place; the demo's BigLake upgrade makes those files governed tables.

Iowa tie

Your Bronze design choice - GCS files as BigLake/Iceberg - is the deck's recommendation, not a compromise: governed, open, and replayable.

Design patterns

- Open table formats (Iceberg) for Bronze/Silver; native storage for hot Gold
- One physical copy readable by multiple engines

Anti-patterns

- Copying the same data into three proprietary formats across zones
- Assuming bucket files can't carry table-level governance - BigLake closes that gap

History & what's new

BigLake tables for Apache Iceberg in BigQuery reached GA in May 2025 - full read/write Iceberg with BigQuery governance (RLS, masking) on files in your own bucket.

Flow

Storage placement done; medallion part 3 next - who is allowed to work in each zone, then back to your zone map.

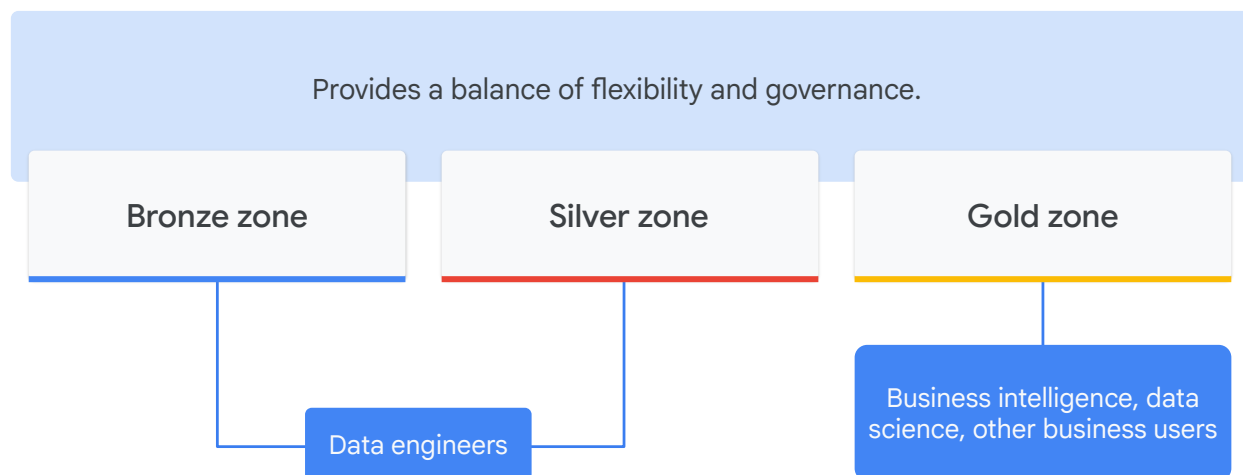
Links

<https://cloud.google.com/bigquery/docs/iceberg-tables>

<https://cloud.google.com/bigquery/docs/biglake-intro>

<https://cloud.google.com/blog/products/data-analytics/enhancing-biglake-for-iceberg-lakehouses>

The medallion architecture (3 of 3)



Google Cloud

INSTRUCTOR NOTES — PAGE 140 OF 208 · 14:00 · MODULE 10 — GOVERNANCE + THE MEDALLION

Hook

Last medallion slide is about people, not data: who works in Bronze, and who is only ever allowed to touch Gold?

Say · cover

- Data engineers own Bronze and Silver; BI, data science, and business users consume Gold - the audience narrows as trust rises.
- That split is the balance: flexibility (raw data never destroyed, engineers can always rebuild) plus governance (consumers touch only curated data).
- Enforce it structurally: dataset-level IAM per zone, column and row security from the governance slides, Power BI service accounts that see only Gold.
- For their org: DE and M&O live in Bronze/Silver; report consumers never get Bronze credentials - no exceptions 'for a quick check.'
- This kills extract sprawl: nobody needs a private copy when Gold is governed.

Iowa tie

Your operating model on one slide: engineers and M&O in Bronze/Silver, Power BI and business users reading only Gold - the end of per-user extracts.

Design patterns

- Least privilege by zone; consumers get Gold, read-only
- Column- and row-level security carried onto BigLake tables too

Anti-patterns

- Granting broad Bronze access 'for convenience' - governance dies there
- Tolerating personal extracts of Gold - recreates the sprawl you are migrating away from

History & what's new

Google's catalog keeps evolving: legacy Data Catalog was deprecated in 2025, Dataplex Catalog became BigQuery universal catalog in April 2025, now Knowledge Catalog - the zone-security concepts stay stable as labels move.

Flow

Medallion trilogy complete; next slide maps each zone to its GCP service in your design, then migration strategy.

Links

<https://cloud.google.com/bigquery/docs/column-level-security-intro>
<https://cloud.google.com/bigquery/docs/row-level-security-intro>
<https://cloud.google.com/dataplex/docs/transition-to-dataplex-catalog>

Where each of your zones lives in GCP

Same zones as your doc — here is the service behind each box.

- Bronze— raw landings on Cloud Storage as BigLake/Iceberg tables, queried in place (your 13:00 lab)
- Silver— cleansed, conformed tables built by Dataform incremental models, keys conformed across sources
- Gold— native BigQuery marts, partitioned and clustered with this morning's M9 levers
- Consumers— engineers work in Bronze and Silver; Power BI and business users read only Gold
- Trust— Bronze is received, Silver is verified, Gold is decision-grade

Custom slide · added by Imran Ahmad

Iowa DOM DoIT · Data Engineering on Google Cloud

INSTRUCTOR NOTES — PAGE 141 OF 208 · 14:00 · MODULE 10 — GOVERNANCE + THE MEDALLION

Hook

Same zones as your doc - this slide just names the Google service behind each box, so there is no translation left to do after class.

Say · cover

- Bronze: raw landings on Cloud Storage as BigLake/Iceberg tables, queried in place - the 13:00-lab pattern, in production vocabulary.
- Silver: cleansed, conformed tables from Dataform incremental models, keys conformed across sources - the SSIS data-flow layer, versioned in git.
- Gold: native BigQuery marts, partitioned and clustered with the M9 levers - decision-grade, Power BI-ready.
- Consumers: engineers work in Bronze and Silver; Power BI and business users read only Gold - the trust ladder: received, verified, decision-grade.
- Console anchors: GCS buckets for Bronze, Dataform repositories in BigQuery Studio, one dataset per zone as the naming convention.

Iowa tie

This is your target architecture on one slide: GCS/BigLake Bronze, Dataform Silver-to-Gold, BigQuery Gold marts for Power BI, M&O owning the platform zones.

Design patterns

- Dataset-per-zone naming so IAM and lineage stay obvious
- Dataform release configs as the CI/CD promotion path Silver-to-Gold

Anti-patterns

- Mixing zones inside one dataset - access control becomes unmanageable
- Letting ad-hoc tables accumulate outside any zone

History & what's new

Both enablers went GA within two years: Datastream's SQL Server source (July 2024) feeds Bronze continuously; BigLake Iceberg tables (May 2025) make it governed - the design is current, not aspirational.

Flow

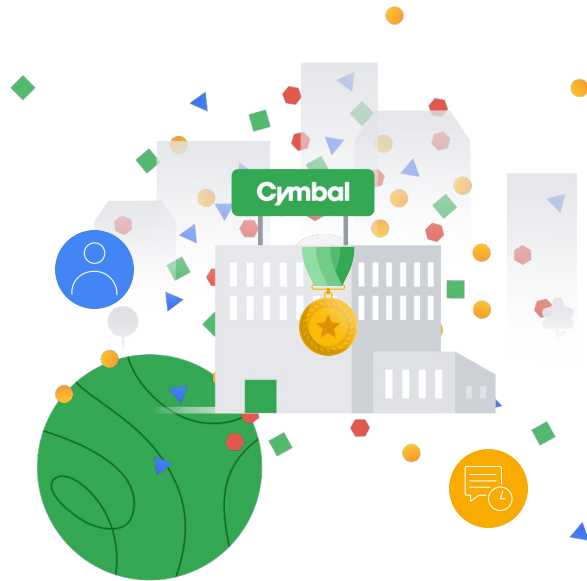
Iowa zone map complete; the deck now pivots to how you get there - migration strategy, two slides, then cost.

Links

<https://cloud.google.com/bigquery/docs/biglake-intro>

<https://cloud.google.com/dataform/docs/release-configurations>

<https://cloud.google.com/bigquery/docs/iceberg-tables>



Cymbal migration strategies

For a company, such as Cymbal, which might be running on a traditional on-premises data warehouse (such as Teradata or Hadoop), migrating to a cloud-native data lakehouse is a significant project.

A complete, one-time migration is often too risky and disruptive. Instead, a phased, use-case-driven approach is usually more successful.

Google Cloud

INSTRUCTOR NOTES — PAGE 142 OF 208 · 14:00 · MODULE 10 — GOVERNANCE + THE MEDALLION

Hook

Google's slide says Teradata and Hadoop; read it as your SQL Server warehouse. And its verdict matches your instinct: big-bang migrations fail.

Say · cover

- Moving off a traditional on-prem warehouse to a cloud lakehouse is a significant program - anyone who has lived a platform cutover knows why.
- A one-time migration is usually too risky and disruptive: every report, every job, every user changes on one weekend.
- The recommended shape is phased and use-case-driven: pick a slice, migrate it end to end, learn, repeat.
- Phase by use case, not by table - a use case ends in a report and a consumer, so every phase delivers something visible.
- For this room: the extracts are the phasing unit - each retired extract chain is one completed phase.

Iowa tie

Your SQL Server DW to BigQuery program is exactly this slide: phased by extract chain and report, each phase ending in a Power BI report that now reads Gold.

Design patterns

- Phase by use case with a consumer-visible deliverable
- Carry lessons from each phase into the next pipeline template

Anti-patterns

- Big-bang cutover of warehouse, pipelines, and reports at once
- Migrating everything including dead tables and unopened reports

History & what's new

Google institutionalized this phased motion in the BigQuery Migration Service, launched in October 2022, which automates the four phases: assessment, SQL translation, data transfer, and validation.

Flow

Migration framing set; next slide is the five-step plan itself, then the slide that restates it in Iowa terms.

Links

<https://cloud.google.com/bigquery/docs/migration-intro>

<https://cloud.google.com/architecture/dw2bq/dw-bq-migration-overview>

<https://cloud.google.com/blog/products/data-analytics/simplify-data-warehouse-migrations-using-bigquery-migration-service>

Migration strategy

- 01 Set up the core infrastructure on Google Cloud
- 02 Start with a high-impact use case
- 03 Migrate relevant data
- 04 Build the new pipelines and reports
- 05 Decommission and iterate



Google Cloud

INSTRUCTOR NOTES — PAGE 143 OF 208 · 14:00 · MODULE 10 — GOVERNANCE + THE MEDALLION

Hook

Five steps on the slide - and you will recognize the shape, because it is the same discipline as any controlled deployment you run today.

Say · cover

- Step 1, core infrastructure first: projects, datasets, IAM, billing guardrails - nothing migrates until the landing zone is governed.
- Step 2, pick one high-impact use case: painful enough to matter, small enough to survive being first.
- Steps 3 and 4, migrate the relevant data, then build new pipelines and reports: load history, stand up Dataform Silver-to-Gold, replot the report.
- Step 5, decommission and iterate: retire the old path, capture lessons, pick the next use case.
- Before all five: run the BigQuery migration assessment against the SQL Server estate - it inventories schemas, query patterns, and effort so the pilot is chosen on evidence.

Iowa tie

This is your migration runbook skeleton: landing zone, one extract chain as the pilot, Dataform rebuild, Power BI replot, retire, repeat per use case.

Design patterns

- Assessment-driven choice of the first use case
- Defined exit criteria per phase - the numbers must match

Anti-patterns

- Skipping assessment and guessing scope from tribal knowledge
- Never decommissioning - paying for both estates indefinitely

History & what's new

The BigQuery Migration Service (October 2022) wraps these steps in tooling: automated assessment, SQL translation between dialects, managed data transfer, and validation - with ongoing assessment improvements documented through 2025.

Flow

The five steps, generic; next slide restates them as Iowa's own phased plan, word for word.

Links

<https://cloud.google.com/bigquery/docs/migration-assessment>

<https://cloud.google.com/bigquery/docs/migration-intro>

<https://cloud.google.com/blog/products/data-analytics/simplify-data-warehouse-migrations-using-bigquery-migration-service>

Their phased migration is your phased migration

A one-time move off the warehouse is too risky — phase it by use case.

- Stand up the core first— projects, datasets, IAM, billing guardrails
- Pick one painful extract chain— Bronze to Dataform Silver to Gold mart to the existing Power BI report
- Parallel-run, then cut over— old job and new pipeline side by side until the numbers match
- Decommission last— retire the extract only after the Gold mart feeds the report
- Repeat per use case— this afternoon's capstone starts that exercise on paper

Custom slide · added by Imran Ahmad

Iowa DOM DoIT · Data Engineering on Google Cloud

INSTRUCTOR NOTES — PAGE 144 OF 208 · 14:00 · MODULE 10 — GOVERNANCE + THE MEDALLION

Hook

This is the previous slide wearing an Iowa badge - same five steps, now with your extract chains, your reports, and your cutover discipline.

Say · cover

- Stand up the core first: projects, datasets, IAM, billing guardrails - M&O owns this; it happens once.
- Pick one painful extract chain and run it end to end: Bronze landing, Dataform Silver-to-Gold, the existing Power BI report repointed at Gold.
- Parallel-run, then cut over: old SSIS job and new pipeline side by side until the numbers match - reconciliation queries, not vibes.
- Decommission last: the extract retires only after the Gold mart feeds the report. Then repeat per use case.
- Trailer the capstone: at 15:20 each group starts this exercise on paper with a real DOM DoIT extract job.

Iowa tie

Your cutover playbook on a slide: parallel-run old and new until numbers match, retire the extract only when the report reads Gold - then the capstone drafts yours.

Design patterns

- Parallel-run with automated reconciliation before any cutover
- Retire the source extract as the definition of done

Anti-patterns

- Cutting over before reconciliation passes
- Parallel-running forever - dual cost, dual truth

History & what's new

The parallel-run is cheap now: Datastream's SQL Server source (GA July 2024) keeps the new side continuously in sync from the transaction log while you compare numbers, instead of nightly re-loads.

Flow

Iowa's phased plan stated; the deck closes M10 with cost management - two best-practice slides, recap, then the 14:35 break.

Links

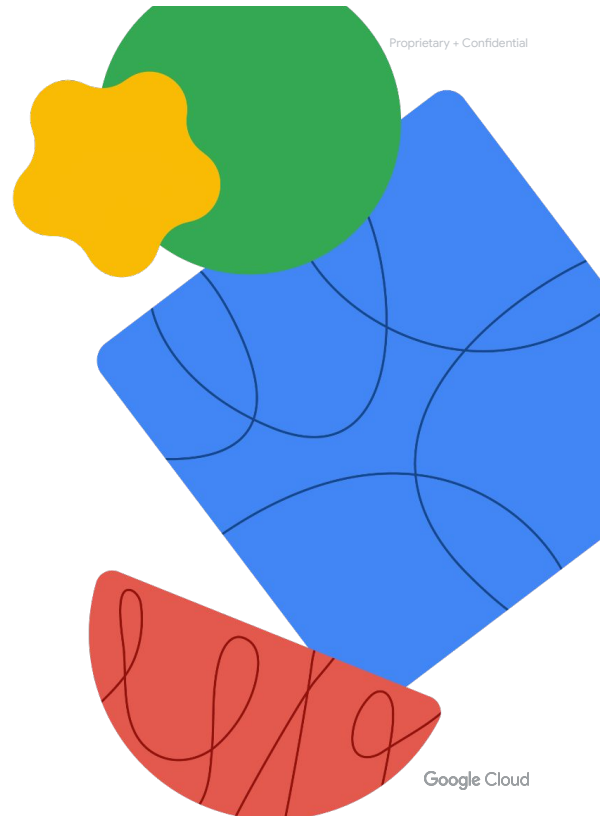
<https://cloud.google.com/architecture/dw2bq/dw-bq-migration-overview>

<https://cloud.google.com/blog/products/data-analytics/datastream-makes-it-easy-to-replicate-sql-server-databases>

<https://cloud.google.com/dataform/docs/release-configurations>

Cost management and optimization best practices

A proactive approach to cost management.



INSTRUCTOR NOTES — PAGE 145 OF 208 · 14:00 · MODULE 10 — GOVERNANCE + THE MEDALLION

Hook

On SQL Server, cost was a license and a server you already owned. On BigQuery, cost is metered by usage - so cost management becomes an engineering habit, not an invoice surprise.

Say · cover

- Frame the shift: from fixed capital cost (licenses, hardware) to metered operational cost - storage per GB, queries per byte scanned or slot.
- The slide's word is proactive: cost is designed in (storage class, partitioning, pricing model), not rescued after the bill.
- Split ownership: developers own query efficiency and zone hygiene; M&O owns budgets, alerts, capacity decisions.
- Reassure them: controls map to existing instincts - bytes scanned is the new rows-touched, budgets the new monitoring alerts.
- Next two slides: four levers - storage class, query optimization, pricing model, budgets and alerts.

Iowa tie

M&O gets billing and budgets; the DE team gets bytes-scanned discipline - both before the first production pipeline.

Design patterns

- Cost review as a step in every pipeline release, like code review
- Named owners: DE for query cost, M&O for budgets and capacity

Anti-patterns

- Treating cost as finance's problem alone
- Discovering metered-pricing surprises from the invoice

History & what's new

The pricing model itself changed recently: Google announced BigQuery editions (Standard, Enterprise, Enterprise Plus) in March 2023 and closed new flat-rate purchases July 5, 2023 - capacity pricing is now editions with autoscaling slots.

Flow

Cost block opener; next two slides take the four levers one by one - storage class and query cost first.

Links

<https://cloud.google.com/bigquery/docs/best-practices-costs>
<https://cloud.google.com/bigquery/pricing>
<https://cloud.google.com/bigquery/docs/editions-intro>

Best practices (1 of 2)

01

Choose the right storage class

Not all data needs to be accessed with the same frequency. Consider storage class in Cloud Storage.

02

Optimize BigQuery queries

Use features, such as partitioning and clustering, as well as writing efficient SQL queries.

Google Cloud

INSTRUCTOR NOTES — PAGE 146 OF 208 · 14:00 · MODULE 10 — GOVERNANCE + THE MEDALLION

Hook

First two levers: pay the right price for where data sleeps, and pay less every time a query runs. Both are versions of what you already do in SQL Server.

Say · cover

- Lever 1, storage class: Standard, Nearline, Coldline, Archive trade storage price for access price, with 30/90/365-day minimums - choose by access frequency, not age.
- Bronze has a natural lifecycle: recent landings in Standard, older history drifting colder; Autoclass automates the transitions per bucket.
- Lever 2, optimize queries: partitioning and clustering from M9 now have a dollar sign - every report that prunes partitions scans fewer bytes.
- The visceral version: SELECT * across ten unpartitioned years is the new table scan on prod - visible in bytes scanned.
- Efficient SQL habits carry over: named columns, filter early, dry-run to see bytes scanned before executing.

Iowa tie

Bronze retention becomes lifecycle rules; Gold marts get partitioned and clustered so Power BI queries stay cheap as data grows.

Design patterns

- Lifecycle or Autoclass rules moving aged Bronze to colder classes
- Partition plus cluster every large Gold table around its report filters

Anti-patterns

- Archiving data you still re-query - retrieval fees erase the savings
- Unpartitioned fact tables scanned in full by every report

History & what's new

Cloud Storage Autoclass arrived in late 2022 - a bucket-level setting that transitions objects between classes by observed access, removing manual lifecycle guesswork.

Flow

Levers one and two done; next slide covers the pricing model itself and the budget alerts M&O will own.

Links

<https://cloud.google.com/storage/docs/storage-classes>

<https://cloud.google.com/storage/docs/autoclass>

<https://cloud.google.com/bigquery/docs/best-practices-costs>

Best practices (2 of 2)

03

Use BigQuery's flat-rate pricing

Great for predictable workloads.

04

Set up budgets and alerts

In Google Cloud's billing tools.

Google Cloud

INSTRUCTOR NOTES — PAGE 147 OF 208 · 14:00 · MODULE 10 — GOVERNANCE + THE MEDALLION

Hook

Last two levers: how you buy BigQuery compute, and how you make sure nothing on the bill ever surprises you.

Say · cover

- Lever 3 is labeled flat-rate, but translate to 2026: capacity pricing through BigQuery editions - you buy slot capacity instead of paying per byte scanned.
- Decision rule: start on-demand while workloads are unknown; move to capacity once usage is steady and predictable.
- Lever 4, budgets and alerts: in Cloud Billing, set a per-project budget with threshold alerts - 50%, 90%, forecasted overage - emailed to M&O before month-end.
- Console pointer: Billing > Budgets and alerts for guardrails; BigQuery > Capacity management for reservations and editions.
- Migration link: on-demand makes the parallel-run cheap - you pay for real usage while old and new run side by side.

Iowa tie

M&O owns this slide: budgets with alert thresholds from day one, and the on-demand-to-editions decision once your migrated workloads stabilize.

Design patterns

- On-demand first, capacity edition once usage is measured and steady
- Per-project budgets with threshold alerts routed to M&O

Anti-patterns

- Buying capacity before measuring real workloads
- No budget alerts on a metered platform - the invoice becomes the monitoring

History & what's new

The slide's 'flat-rate' is historical: Google announced BigQuery editions in March 2023 and closed new flat-rate commitments on July 5, 2023 - autoscaling slot capacity under editions is the modern equivalent.

Flow

Four cost levers complete; next is the M10 recap slide, then the 14:35 break.

Links

<https://cloud.google.com/bigquery/docs/editions-intro>
<https://cloud.google.com/billing/docs/how-to/budgets>
<https://cloud.google.com/bigquery/pricing>

M10 — What you now have

- ✓ Governance mapped to your single-source-of-truth requirement— catalog, PII protection, column and row security
- ✓ Your medallion design confirmed in Google's own teaching deck— Bronze, Silver, Gold, zone for zone
- ✓ A GCP home for each zone— GCS/BigLake Bronze, Dataform Silver-to-Gold, BigQuery Gold marts for Power BI
- ✓ A phased migration path and the cost levers— storage class, bytes scanned, budgets and alerts
- ✓ Next: M11— the BQML direction and the Power BI last mile on your Gold marts

Custom slide · added by Imran Ahmad

Iowa DOM DoIT · Data Engineering on Google Cloud

INSTRUCTOR NOTES — PAGE 148 OF 208 · 14:00 · MODULE 10 — GOVERNANCE + THE MEDALLION

Hook

Close the module by inventorying what this half hour added to their project - then send them to break with the Power BI question hanging.

Say · cover

- Recap the four acquisitions: governance mapped to single-source-of-truth; medallion confirmed in Google's deck; a GCP service per zone; a phased migration path plus cost levers.
- Land the beat: their architecture doc survived contact with the vendor's curriculum - validated, not revised.
- Name the open thread: Gold marts exist for a consumer - Power BI - and M11 closes that last mile after the break.
- Flag the direction item: M11 opens with BQML, Google's in-warehouse ML - not in their brief, but know it exists.
- Logistics: back at 14:50 sharp - the 15:20 capstone needs one real extract job per group; have them return with one in mind.

Iowa tie

Deliverables: truth governed, zones service-mapped, migration phased, cost levers split between DE and M&O.

Design patterns

- End each module by restating deliverables in the customer's project vocabulary
- Leave one open thread that the next module closes

Anti-patterns

- Recapping as product names instead of project outcomes
- Skipping capstone setup - cold groups burn five of their thirty minutes choosing a job

History & what's new

Console labels keep shifting: Dataplex Catalog became BigQuery universal catalog in April 2025, now Knowledge Catalog - the governance concepts stay stable as surfaces rename.

Flow

M10 done; break now (14:35-14:50), then M11 compressed - BQML direction plus the Power BI last mile on Gold marts.

Links

<https://cloud.google.com/dataplex/docs/transition-to-dataplex-catalog>
<https://docs.cloud.google.com/knowledge-catalog/docs/release-notes>
<https://cloud.google.com/discover/what-is-medallion-architecture>

Afternoon Break

15 minutes · 14:35–14:50

Custom slide · added by Imran Ahmad

Iowa DOM DoIT · Data Engineering on Google Cloud

INSTRUCTOR NOTES — PAGE 149 OF 208 · 14:35 · AFTERNOON BREAK (15 MIN)

FACILITATOR NOTE

- 15 minutes, back at 14:50 sharp. M10 just landed the two big anchors - their medallion design confirmed in Google's own deck, and a phased migration path with the cost levers assigned.
- Send them out with one instruction: come back with one painful, real extract job in mind - the 15:20 capstone maps it to GCP in groups, and M11 right after the break (BQML direction plus the Power BI last mile on Gold marts) sets up the consumer end of that exercise.

Flow: M10 (governance + medallion + migration + cost) just ended; after the break: M11 compressed at 14:50, capstone workshop at 15:20, course summary at 15:50.

WATCH FIRST · 1:41 · GOOGLE CLOUD TECH

BigQuery ML in a Minute

- ✓ Train and run models with plain SQL — CREATE MODEL, no Python, no data movement, no new platform. Ninety seconds that reframes 'ML project' for a SQL team — then watch the live
- ✓ three-statement demo.



Scan to open on YouTube
youtu.be/0RMT8uEplbM

Custom slide · added by Imran Ahmad

Iowa DOM DoIT · Data Engineering on Google Cloud

INSTRUCTOR NOTES — PAGE 150 OF 208 · 14:50 · MODULE 11 — BQML + THE POWER BI LAST MILE

Hook

- Play to open M11 — 101 seconds, and the room believes SQL can do ML before you prove it live.

Say

- “Your analysts already know the syntax — the demo shows the other two statements.”

Cue

- Full play; it pairs directly with the three-statement live demo later in the block.

Link

- <https://www.youtube.com/watch?v=0RMT8uEplbM>

M11 — What this block gets you

- ✓ BQML seen, not drilled— a model trained by CREATE MODEL, living next to your tables
- ✓ The end-to-end closed: extracts → BigQuery → Bronze/Silver/Gold → the Power BI you already have
- ✓ The Import vs DirectQuery call, per mart— default: Import + refresh aligned to Dataform releases
- ✓ Auth the right way— author OAuth on the desktop, a least-privilege service account in the service
- ✓ The gateway answer for M&O— BigQuery is a cloud source; nothing new to run

Custom slide · added by Imran Ahmad

Iowa DOM DoIT · Data Engineering on Google Cloud

INSTRUCTOR NOTES — PAGE 151 OF 208 · 14:50 · MODULE 11 — BQML + THE POWER BI LAST MILE

Hook

Back from break: the last teaching block, and half of it is the part the Google course never mentions — your Power BI. First 15 minutes: BQML, shown not drilled.

Say · cover

- BQML: a model trained by CREATE MODEL, living next to your tables — show it exists, don't drill.
- Then close the end-to-end: extracts to BigQuery to Bronze/Silver/Gold to the Power BI they already have.
- The Import vs DirectQuery call is per mart; default is Import with refresh aligned to Dataform releases.
- Auth splits in two: author OAuth on Desktop, a least-privilege service account in the service.
- Gateway answer for M&O: BigQuery is a cloud source — nothing new to run on-prem.

Iowa tie

This block finishes their stated end-to-end: one source of truth in BigQuery, surfaced in the Power BI Iowa already runs.

Design patterns

- Import + refresh pinned to the Dataform release schedule.
- Least-privilege non-human identity for the Power BI service.

Anti-patterns

- Turning this into an ML deep-dive — nothing in their brief asks for ML.
- One shared personal login across every report author.

History & what's new

BigQuery ML went from Next '18 beta to GA at Next '19 (April 2019); the Power BI BigQuery connector gained a faster ADBC-based V2 implementation in May 2025.

Flow

First slide after the 14:35 break; next is the block's timing card, then the vendor BQML walkthrough.

Links

<https://cloud.google.com/bigquery/docs/bqml-introduction>

<https://learn.microsoft.com/en-us/power-query/connectors/google-bigquery>

<https://learn.microsoft.com/en-us/power-bi/connect-data/desktop-directquery-about>

This block, and what follows

14:35–14:50 Break

14:50–15:05 BQML live: three SQL statements, one trained model — show it exists, don't drill

DEMO

15:05–15:20 The Power BI last mile: connect, Import vs DirectQuery, auth (custom segment)

15:20–15:50 Capstone workshop: map one of your real extract jobs onto the GCP target

WORKSHOP

Custom slide · added by Imran Ahmad

Iowa DOM DoIT · Data Engineering on Google Cloud

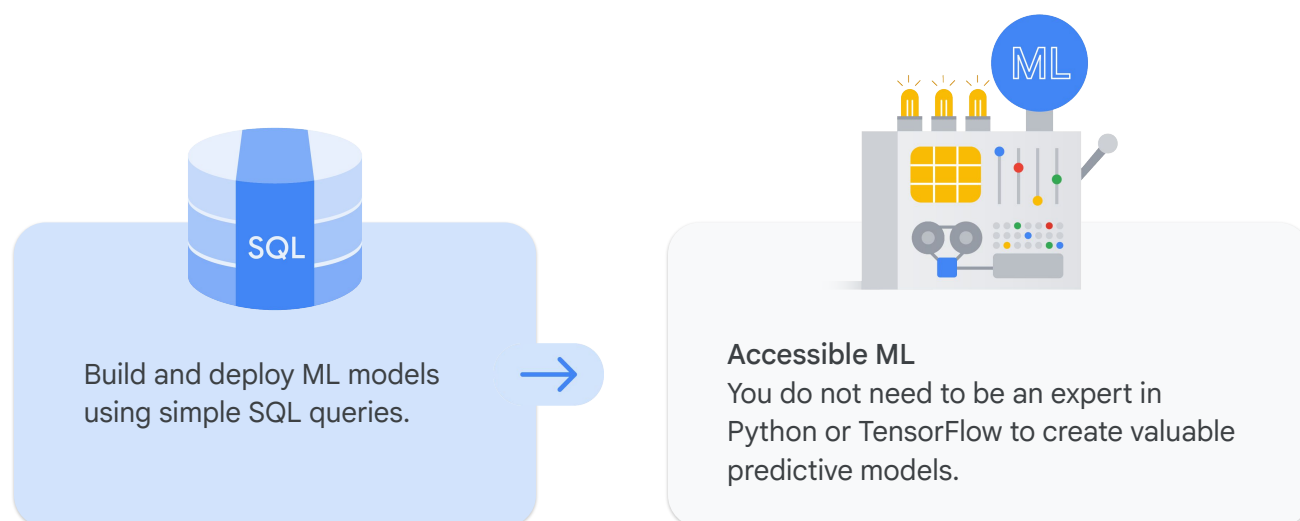
INSTRUCTOR NOTES — PAGE 152 OF 208 · 14:50 · MODULE 11 — BQML + THE POWER BI LAST MILE

FACILITATOR NOTE

- Pacing card: guard the 15/15 split. The demo is the flexible part — if CREATE MODEL runs long, narrate from prepared results and protect the Power BI segment, which is their stated objective.
- The 15:20 capstone is the payoff of both days; do not let this block eat it.

Flow: Next: the vendor's BigQuery ML intro slide.

BigQuery ML



Google Cloud

INSTRUCTOR NOTES — PAGE 153 OF 208 · 14:50 · MODULE 11 — BQML + THE POWER BI LAST MILE

Hook

Two days of moving data into BigQuery — this is the one place we show what else living there buys you. Frame it as a stored procedure that trains a model.

Say · cover

- CREATE MODEL is DDL, like CREATE TABLE: one statement, no Python, no TensorFlow, no separate server.
- The model menu covers the classics: linear and logistic regression, boosted trees, K-means, ARIMA and TimesFM time series.
- Training runs as a BigQuery job on data that never leaves the warehouse — the 'ML where the data lives' pitch.
- The intended audience is SQL analysts — which in this room means the ten people who write the ETL today.

Iowa tie

Nothing in Corey's brief requires ML. Position it as a door left open: when a data-science team arrives, their Gold marts are the training data.

Design patterns

- Logistic-regression baseline before anything fancier.
- Models inherit dataset IAM — no parallel security model to run.

Anti-patterns

- Promising ML capability DOM DoIT hasn't asked for.
- Implying Python is exiled — Colab Enterprise notebooks integrate with BQML.

History & what's new

Beta at Next '18, GA at Next '19 (April 2019); the 2025 addition is TimesFM forecasting — AI.FORECAST reached GA in November 2025.

Flow

First of the compressed C2-M4 BQML slides; next is a custom framing slide, then the churn walkthrough.

Links

<https://cloud.google.com/bigquery/docs/bqml-introduction>
<https://cloud.google.com/bigquery/docs/reference/standard-sql/bigqueryml-syntax-create>
<https://cloud.google.com/bigquery/docs/reference/standard-sql/bigqueryml-syntax-forecast>

ML where the data lives

Same rule as everything since 9 AM: don't move the data to the tool.

- No export, no notebook— the model trains on data that never leaves BigQuery
- Models are first-class objects— they sit in your dataset, take IAM, appear in Information Schema
- ML.PREDICT is a function call— predictions are a SQL query, schedulable like any other
- Honest scope— ML is a possible future for DOM DoIT, not the priority; the warehouse is
- Why show it at all— when a data-science team arrives, your Gold marts are their training data

Custom slide · added by Imran Ahmad

Iowa DOM DoIT · Data Engineering on Google Cloud

INSTRUCTOR NOTES — PAGE 154 OF 208 · 14:50 · MODULE 11 — BQML + THE POWER BI LAST MILE

Hook

Same rule as everything since 9 AM: don't move the data to the tool. In SSIS terms — no export package to a separate ML server.

Say · cover

- No export, no notebook required: the model trains on data that never leaves BigQuery.
- Models are first-class objects: they sit in the dataset, take IAM, and appear in Information Schema and the Explorer pane.
- ML.PREDICT is a function call — predictions are a SQL query, schedulable like any other.
- Honest scope: ML is a possible future for DOM DoIT, not the priority; the warehouse is.
- Why show it at all: when a data-science team arrives, your Gold marts are their training data.

Iowa tie

Governance extends to models for free: same dataset IAM and audit that M&O will already administer — no new platform to secure.

Design patterns

- Score with scheduled SQL right after the Gold release.
- Keep models inside the same dataset/IAM boundary as the marts.

Anti-patterns

- Exporting data to a separate ML stack — recreates the extract problem they're killing.
- Shipping an ML roadmap before the warehouse lands.

History & what's new

The in-database philosophy keeps growing: BigQuery vector search went GA in January 2025, and BQML models can now be registered to Vertex AI Model Registry.

Flow

Custom framing slide; next: the vendor's four-step churn use case.

Links

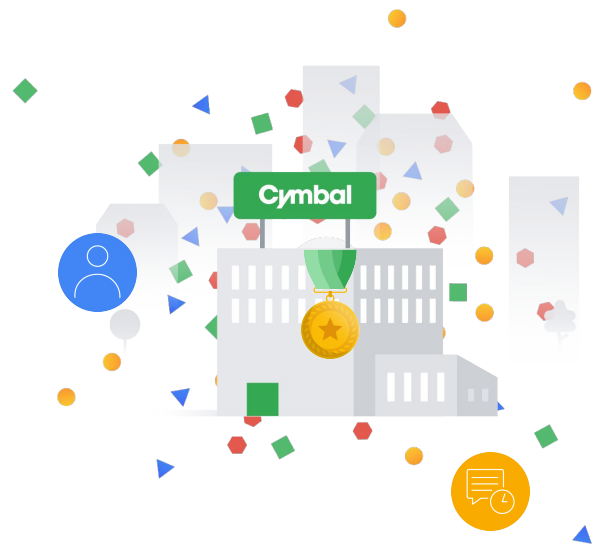
<https://cloud.google.com/bigquery/docs/information-schema-intro>

<https://cloud.google.com/bigquery/docs/managing-models>

<https://cloud.google.com/bigquery/docs/scheduling-queries>

Use case

To identify customers who are at risk of not making another purchase, the marketing analytics team can use BigQuery ML and a few SQL statements.



Google Cloud

INSTRUCTOR NOTES — PAGE 155 OF 208 · 14:50 · MODULE 11 — BQML + THE POWER BI LAST MILE

Hook

Swap 'customer' for 'program participant' or 'case' and the shape should look familiar: RFM features, a yes/no label, a scored list at the end.

Say · cover

- Scenario: a marketing analytics team flags customers at risk of not returning, using a few SQL statements.
- Four steps ahead: feature engineering, training, evaluation, prediction — all inside the warehouse.
- The deliverable is a scored table, not a service to deploy — a list marketing can act on.
- For binary churn-or-not, the documented choices are logistic regression or a boosted tree.

Iowa tie

Their analog: which providers, employers, or cases are about to go dormant — the same SQL shape, run on the Gold marts they are already building.

Design patterns

- Binary classification on tabular data: logistic baseline, boosted-tree challenger.
- Deliver scores as a table analysts can join anywhere.

Anti-patterns

- Reaching for deep learning on tabular state-agency data.
- Presenting churn as a DOM DoIT requirement — it is a walkthrough.

History & what's new

Google's canonical tutorial (predicting repeat visitors on the Analytics sample) has taught this pattern for years; Q1 2025 added remote models over Claude, but tabular churn still defaults to logistic/boosted tree.

Flow

Scene-setter; the next slides walk the four steps, then the live demo.

Links

<https://cloud.google.com/bigquery/docs/logistic-regression-prediction>

<https://cloud.google.com/bigquery/docs/reference/standard-sql/bigqueryml-syntax-create>

<https://cloud.google.com/bigquery/docs/reference/standard-sql/bigqueryml-syntax-create-boosted-tree>

Use case: Predicting customer churn (1 of 4)

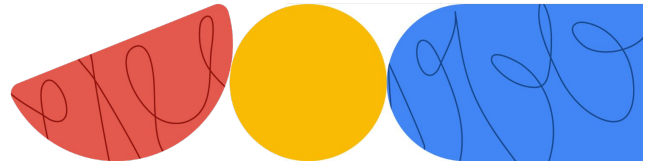
01

Feature engineering

The first step is to prepare the data. The analysts write a SQL query to create features (or signals) that might predict churn.

Fields

- `recency`: days since last purchase
- `frequency`: purchases in the last year
- `monetary_value`: total spent
- `days_since_first_purchase`: customer tenure



INSTRUCTOR NOTES — PAGE 156 OF 208 · 14:50 · MODULE 11 — BQML + THE POWER BI LAST MILE

Hook

This is the step your team already knows cold: it is a `SELECT` with date math and `GROUP BY`. Recency, frequency, monetary — classic RFM.

Say · cover

- Features: `recency`, `frequency`, `monetary_value`, `tenure` — aggregates over purchase history.
- The label is derived in SQL too: `(total_purchases > 1) AS will_return`.
- A `TRANSFORM` clause can bundle preprocessing into the model so scoring repeats it automatically.
- Walk the field list: `recency`, `frequency`, `monetary_value`, `days_since_first_purchase` — every one is one aggregate over the orders table.
- Treat the feature query as a Gold-style aggregate: versioned, repeatable, reviewable.

Iowa tie

Feature engineering is exactly their Silver-to-Gold Dataform work — the feature table is one more `.sqlx` in the same repo and release pipeline.

Design patterns

- Feature definitions in version-controlled SQL, not notebooks.
- Compute features as of the prediction date to avoid leakage.

Anti-patterns

- One-off hand-built feature queries nobody can re-run.
- Leaking future information into training features.

History & what's new

`TRANSFORM`-clause preprocessing is long-documented; through 2025 Google added the `AI.GENERATE_*` SQL functions, but classic numeric features are still plain SQL.

Flow

Step 1 of 4; next: the single `CREATE MODEL` statement.

Links

<https://cloud.google.com/bigquery/docs/reference/standard-sql/bigqueryml-syntax-transform>
<https://cloud.google.com/bigquery/docs/reference/standard-sql/bigqueryml-syntax-create>
<https://cloud.google.com/bigquery/docs/logistic-regression-prediction>

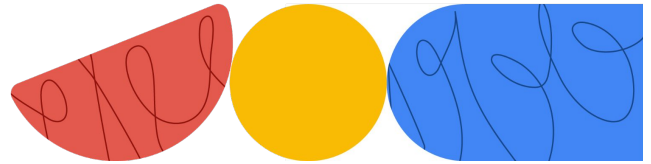
Use case: Predicting customer churn (2 of 4)

02

Model training

After the features are ready, they train a model with a single `CREATE MODEL` statement in SQL.

For this binary classification problem (churn or no churn), a logistic regression or a boosted tree model is a good choice.



INSTRUCTOR NOTES — PAGE 157 OF 208 · 14:50 · MODULE 11 — BQML + THE POWER BI LAST MILE

Hook

One DDL statement replaces what on-prem would be an SSIS package, a Python script, and a deployment step.

Say · cover

- `CREATE OR REPLACE MODEL ... OPTIONS(model_type=...) AS SELECT` — the `SELECT` supplies features and label.
- Binary classification: start with `LOGISTIC_REG`; a boosted-tree classifier is the usual accuracy challenger.
- Training is a BigQuery job: slots, bytes processed, visible in job history like any query.
- `OR REPLACE` retrains in place — model refresh fits a release cadence.

Iowa tie

Retraining can ride their Dataform release pipeline: new Gold build, `CREATE OR REPLACE`, new scores — through the same CI/CD discipline they run for SQL Server today.

Design patterns

- Baseline logistic first, then compare boosted tree.
- Train on curated Gold, never on raw Bronze.

Anti-patterns

- Hyperparameter tuning before a baseline exists.
- Training from console clicks with nothing in source control.

History & what's new

New model types have arrived every year since GA in 2019 (boosted trees among the first); at Next 2025 Google added TimesFM, and `AI.FORECAST` went GA in November 2025 — forecasting with no training step at all.

Flow

Step 2; the next slide shows the actual SQL.

Links

<https://cloud.google.com/bigquery/docs/reference/standard-sql/bigqueryml-syntax-create>

<https://cloud.google.com/bigquery/docs/reference/standard-sql/bigqueryml-syntax-create-boosted-tree>

<https://cloud.google.com/bigquery/docs/reference/standard-sql/bigqueryml-syntax-forecast>

Sample code

```
SQL
CREATE OR REPLACE MODEL cymbal_ecommerce.customer_churn_predictor
OPTIONS(model_type='LOGISTIC_REG') AS
SELECT
customer_id,
recency,
frequency,
monetary_value,
(total_purchases > 1) AS will_return -- This is our label
FROM
cymbal_ecommerce.customer_purchase_summary;
```

Google Cloud

INSTRUCTOR NOTES — PAGE 158 OF 208 · 14:50 · MODULE 11 — BQML + THE POWER BI LAST MILE

Hook

Read it top to bottom out loud: every line is SQL they could write today. Ask the room to find the label.

Say · cover

- Fully-qualified name — project.dataset.model — the model lands in a dataset like a table.
- OPTIONS(model_type='LOGISTIC_REG') picks the algorithm; defaults are sane for a first run.
- The AS SELECT defines features inline; the inline comment flags will_return as the label.
- No training server, no export, no endpoint: the statement is the whole deployment.

Iowa tie

This file can sit in their Git repo beside Dataform .sqlx — same PR, review, and deploy pipeline as their SQL Server code today.

Design patterns

- CREATE MODEL lives in source control, deployed like any other SQL.
- Dataset name acts as the schema boundary — same convention as their marts.

Anti-patterns

- Label logic living only in the training query, undocumented.
- Ad-hoc console models with no path to reproduce.

History & what's new

The statement shape has been stable since the 2018 beta; Google keeps extending it — Q1 2025 added remote models that call Anthropic Claude through the same CREATE MODEL.

Flow

The code behind step 2; next: evaluation with ML.EVALUATE.

Links

<https://cloud.google.com/bigquery/docs/reference/standard-sql/bigqueryml-syntax-create>

<https://cloud.google.com/bigquery/docs/logistic-regression-prediction>

<https://cloud.google.com/bigquery/docs/bqml-introduction>

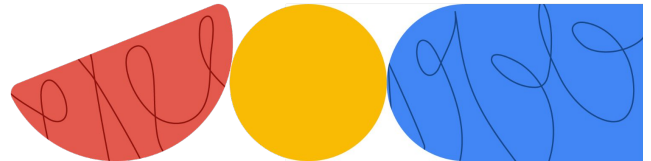
Use case: Predicting customer churn (3 of 4)

03

Model evaluation

After the model is trained, the analysts evaluate its performance using the `ML.EVALUATE` function.

- ✓ Accuracy
- ✓ Precision
- ✓ Recall



INSTRUCTOR NOTES — PAGE 159 OF 208 · 14:50 · MODULE 11 — BQML + THE POWER BI LAST MILE

Hook

Evaluation is a `SELECT` that returns one row of metrics — present it as a unit test for the model.

Say · cover

- `ML.EVALUATE(model)` returns accuracy, precision, recall, and friends in a single row.
- For churn, recall is the honest metric: the cost is missing an at-risk customer, not a false alarm.
- BigQuery auto-splits held-out data by default; you can also evaluate against your own labeled set.
- The decision threshold is tunable — `ML.EVALUATE` accepts a threshold option, which shifts the precision/recall balance.

Iowa tie

A metrics row per training run is an auditable artifact — store it next to the release so M&O can answer 'how good was the model we shipped?'

Design patterns

- Pick the metric that matches the business cost — recall for churn.
- Persist evaluation output per release, alongside the model version.

Anti-patterns

- Quoting accuracy on imbalanced churn data — the majority class games it.
- Evaluating on the training rows.

History & what's new

`ML.EVALUATE` has grown past classic models: the Q1 2025 releases let it score generative remote models such as Claude as well.

Flow

Step 3; next: `ML.PREDICT` produces the scored list.

Links

<https://cloud.google.com/bigquery/docs/reference/standard-sql/bigqueryml-syntax-evaluate>

<https://cloud.google.com/bigquery/docs/logistic-regression-prediction>

<https://cloud.google.com/bigquery/docs/reference/standard-sql/bigqueryml-syntax-create>

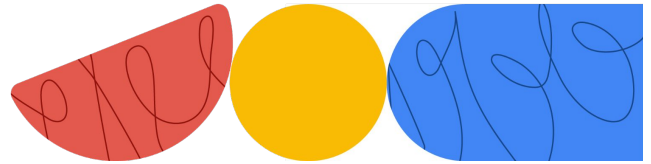
Use case: Predicting customer churn (4 of 4)

04

Prediction

With the `ML.PREDICT` function, you can get a list of all customers and their probability of churning.

This list can then be used to create targeted marketing campaigns to re-engage at-risk customers.



INSTRUCTOR NOTES — PAGE 160 OF 208 · 14:50 · MODULE 11 — BQML + THE POWER BI LAST MILE

Hook

The payoff is a table: `customer_id` plus churn probability, ready to join to anything — including a Power BI dataset.

Say · cover

- `ML.PREDICT(model, (SELECT ...))` returns the predicted label plus per-class probabilities.
- Filter to the top decile by probability and you have the campaign list marketing actually uses.
- Output is a normal table: persist it dated, so each scoring run stays comparable.
- Schedule the scoring — a scheduled query or a Dataform step right after the Gold release.

Iowa tie

Scores land back in the warehouse, so Power BI reads them through the same connector as every other mart — the loop Gold, model, scores, report stays inside one platform.

Design patterns

- Batch scoring on a schedule tied to the Gold release.
- Persist prediction output as a dated table.

Anti-patterns

- Ad-hoc scoring with nothing persisted.
- `ML.PREDICT` behind a DirectQuery dashboard — batch it instead.

History & what's new

Batch inference in SQL was the 2018 launch promise; the November 2025 GA of `AI.FORECAST` extends the same `SELECT`-based pattern to zero-training forecasting.

Flow

End of the walkthrough; a vendor takeaway slide, then the live demo.

Links

<https://cloud.google.com/bigquery/docs/reference/standard-sql/bigqueryml-syntax-predict>

<https://cloud.google.com/bigquery/docs/scheduling-queries>

<https://cloud.google.com/bigquery/docs/reference/standard-sql/bigqueryml-syntax-forecast>

BigQuery ML allows the teams who are closest to the data to **build predictive models**, which leads to faster and more relevant insights.

Google Cloud

INSTRUCTOR NOTES — PAGE 161 OF 208 · 14:50 · MODULE 11 — BQML + THE POWER BI LAST MILE

FACILITATOR NOTE

- Translate the quote for the room: 'teams closest to the data' means their ten SQL developers, not a separate data-science org.
- Leave one idea before the demo: the model is an object in the dataset and prediction is a query — nothing new to govern, deploy, or secure.
- Bridge line: 'Watch it happen in three statements.'

Flow: Next is the live BQML demo run-sheet, then the custom Power BI segment.

Demo: a live BQML model in three SQL statements

15 minutes · BigQuery console — instructor's demo dataset

1. Peek at `bigquery-public-data.ml_datasets.penguins` — 344 penguins; we predict `body_mass_g`
2. `CREATE OR REPLACE MODEL class.penguins_model` — linear regression, trains in ~1–2 minutes
3. `ML.EVALUATE` — mean absolute error in plain speech: a unit test for a model
4. `ML.PREDICT` — actual vs predicted side by side, sorted by worst error
5. Point at the Explorer pane — the model sits in the dataset like a table, with IAM

Why it matters — nothing in your brief points at ML — this exists so you know the door is there: the day someone asks about forecasting, the data and the tool already live in the same place.

Custom slide · added by Imran Ahmad

Iowa DOM DoIT · Data Engineering on Google Cloud

INSTRUCTOR NOTES — PAGE 162 OF 208 · 14:50 · MODULE 11 — BQML + THE POWER BI LAST MILE

Hook

Three statements, one trained model — deliberately boring. The point is that there is nothing to install, provision, or deploy.

Say · cover

- Peek `bigquery-public-data.ml_datasets.penguins`: 344 penguins; predict `body_mass_g` from the other measurements.
- `CREATE OR REPLACE MODEL class.penguins_model` with `LINEAR_REG` — trains in about 1–2 minutes; narrate while it runs.
- `ML.EVALUATE`: read mean absolute error in plain speech — 'on average we are off by this many grams.'
- `ML.PREDICT`: actual vs predicted side by side, sorted by worst error.
- Land on the Explorer pane: the model sits in the dataset like a table, with IAM.

Iowa tie

Message for M&O: nothing new to run — 'deployment' is a dataset object under IAM, and scoring is a scheduled query.

Design patterns

- Narrate console locations as you go — Explorer, dataset, model; the room is console-junior.
- Keep prepared results open in a second tab in case training stalls.

Anti-patterns

- Drifting into hyperparameters or algorithm theory.
- Letting the demo eat the Power BI segment's 15 minutes.

History & what's new

Linear regression was the original 2018 launch model type; the newest sibling, TimesFM, went GA via `AI.FORECAST` in November 2025 — the penguins demo still runs on the classics.

Flow

The 15-minute live demo (14:50–15:05); the next slide starts the custom Power BI segment.

Links

<https://cloud.google.com/bigquery/public-data>

<https://cloud.google.com/bigquery/docs/reference/standard-sql/bigqueryml-syntax-create>

<https://cloud.google.com/bigquery/docs/reference/standard-sql/bigqueryml-syntax-predict>

The last mile: Power BI ↔ BigQuery

Your reports don't change — their source does. The course never mentions this; your project lives on it.

- Get Data → Google BigQuery— the connector ships in Power BI Desktop; the same one Excel uses
- Sign in with your own Google account (OAuth)— BigQuery sees the person, not a shared login
- Navigator shows project → dataset → table— the console hierarchy, inside your existing BI tool
- IAM on the Gold dataset is the gatekeeper— row/column security applies per report author
- From Load onward it's the Power BI you know— DAX, visuals, publish; nothing report-side changes
- Bookmark Microsoft's connector docs page— the pattern is stable, the dialogs evolve

Custom slide · added by Imran Ahmad

Iowa DOM DoIT · Data Engineering on Google Cloud

INSTRUCTOR NOTES — PAGE 163 OF 208 · 14:50 · MODULE 11 — BQML + THE POWER BI LAST MILE

Hook

Everything so far was plumbing you chose to change; this slide is the part of your world that does not change. The course never mentions it — your project lives on it.

Say · cover

- Get Data, Google BigQuery: the connector ships in Power BI Desktop — the same one Excel uses.
- Each author signs in with their own Google account over OAuth — BigQuery sees the person, not a shared login.
- Navigator mirrors the console hierarchy: project, dataset, table.
- IAM on the Gold dataset is the gatekeeper; row and column security apply per report author.
- From Load onward it is the Power BI they know: DAX, visuals, publish — nothing report-side changes.

Iowa tie

The kill-the-extracts payoff: Gold marts surface in the tool Iowa's authors already use; report rework is approximately zero.

Design patterns

- Show one live connection plus screenshots; bookmark Microsoft's connector page — dialogs evolve.
- Point existing .pbix files at BigQuery via data-source settings rather than rebuilding.

Anti-patterns

- Rebuilding reports from scratch before switching the source.
- One shared OAuth identity across all authors — the audit trail lies.

History & what's new

The connector is GA and stable; in May 2025 Microsoft introduced a faster ADBC-based V2 implementation, now recommended for new workloads.

Flow

First slide of the custom 15:05–15:20 segment; next: the Import vs DirectQuery decision.

Links

<https://learn.microsoft.com/en-us/power-query/connectors/google-bigquery>
<https://learn.microsoft.com/en-us/power-bi/connect-data/desktop-use-directquery>
<https://cloud.google.com/bigquery/docs/access-control>

Import vs DirectQuery — decide per mart

The one choice in the connector dialog that matters.

- Import— data cached in Power BI; fastest reports, full DAX, one query per refresh
- Import fits— nightly-built Gold marts; Dataform rebuilds on a schedule, refresh right after
- Import cost— small and predictable; a post-build refresh matches pipeline reality
- DirectQuery— a live query per visual, per user, per click; current on every interaction
- DirectQuery cost— bills BigQuery scan bytes continuously; on-demand pricing needs watching
- DirectQuery fits— intra-day executive freshness, or a dataset too large to cache

Default: Import + scheduled refresh aligned to the Dataform release schedule. DirectQuery buys freshness the pipeline doesn't have yet — revisit per mart when streaming arrives.

Custom slide · added by Imran Ahmad

Iowa DOM DoIT · Data Engineering on Google Cloud

INSTRUCTOR NOTES — PAGE 164 OF 208 · 14:50 · MODULE 11 — BQML + THE POWER BI LAST MILE

Hook

The one choice in the connector dialog that matters — the same call they make today between cached tabular models and live connections.

Say · cover

- Import: data cached in Power BI — fastest reports, full DAX, one BigQuery query per refresh.
- Import fits nightly Gold marts: Dataform rebuilds on a schedule, refresh fires right after.
- DirectQuery: a live query per visual, per user, per click — current on every interaction.
- DirectQuery bills BigQuery scan bytes continuously; on-demand pricing needs watching, and Microsoft notes cold-start latency.
- Default: Import plus scheduled refresh aligned to the Dataform release; revisit per mart when streaming arrives.

Iowa tie

Cost governance lives here: DirectQuery on on-demand pricing turns every slicer click into billed bytes — an M&O conversation, not a report-author one.

Design patterns

- Decide per mart, not per workspace.
- Composite models when one fact table needs freshness the rest don't.

Anti-patterns

- DirectQuery against nightly-built marts — buying freshness the pipeline doesn't have.
- DirectQuery as the whole-model default 'to be safe'.

History & what's new

Microsoft's current guidance is Import-first, with hybrid tables and Direct Lake as modern middle grounds; the May 2025 V2 connector also improved DirectQuery performance against BigQuery.

Flow

Next: auth and the gateway question M&O will ask.

Links

<https://learn.microsoft.com/en-us/power-bi/connect-data/desktop-directquery-about>
<https://learn.microsoft.com/en-us/power-bi/connect-data/refresh-scheduled-refresh>
<https://cloud.google.com/bigquery/pricing>

Auth and gateways — two identities, one quiet gateway

Desktop and the Power BI service are different stories. Keep them distinct.

- Desktop— each author signs in with their own Google account; the audit trail is real people
- Service— stored credentials are a service account whose only job is reading the Gold dataset
- Minimum roles— `bigquery.dataViewer` on the Gold dataset, `bigquery.jobUser` on the project
- Same discipline as SQL Agent proxies— least-privilege, non-human identity, one job
- Gateway answer for M&O— BigQuery is a cloud source; no on-premises data gateway required
- Gateway shrinks, not grows— only if a report mashes BigQuery with on-prem SQL Server

Custom slide · added by Imran Ahmad

Iowa DOM DoIT · Data Engineering on Google Cloud

INSTRUCTOR NOTES — PAGE 165 OF 208 · 14:50 · MODULE 11 — BQML + THE POWER BI LAST MILE

Hook

Desktop and the Power BI service are different stories — the same discipline they already apply with SQL Agent proxies, just with Google identities.

Say · cover

- Desktop: each author signs in with their own Google account — the audit trail is real people.
- Service: the stored credential is a dedicated service account whose only job is reading the Gold dataset — the connector supports service-account login directly.
- Minimum roles: `bigquery.dataViewer` on the Gold dataset, `bigquery.jobUser` on the project.
- Gateway answer for M&O: BigQuery is a cloud source — no on-premises data gateway required.
- The gateway shrinks, not grows: needed only if a report mashes BigQuery with on-prem SQL Server.

Iowa tie

M&O owns exactly one new thing: a service account plus a refresh schedule. Least-privilege, non-human, one job — same rules as their SQL Agent proxies.

Design patterns

- Non-human least-privilege identity for the service.
- Per-person OAuth on Desktop for a real audit trail.

Anti-patterns

- A person's OAuth stored in the service — breaks on attrition and audits as that person.
- Project-level owner/editor for the refresh identity.

History & what's new

The connector supports organizational-account and service-account sign-in, and Microsoft's May 2025 V2 (ADBC) refresh kept both — the auth pattern is stable.

Flow

Next: the one-line close of the whole end-to-end.

Links

<https://learn.microsoft.com/en-us/power-query/connectors/google-bigquery>
<https://learn.microsoft.com/en-us/power-bi/connect-data/service-gateway-onprem>
<https://cloud.google.com/bigquery/docs/access-control>

Your Gold marts land in the Power BI you already have. Same authors, same workspaces, same DAX — only the connection string changes.

The close of the Power BI segment — the whole migration, end to end

Custom slide · added by Imran Ahmad

Iowa DOM DoIT · Data Engineering on Google Cloud

INSTRUCTOR NOTES — PAGE 166 OF 208 · 14:50 · MODULE 11 — BQML + THE POWER BI LAST MILE

Hook

Read the slide verbatim, then land it: two days of architecture end at a tool they have had for years.

Say · cover

- Retrace in one breath: extracts to Datastream/DTS to Bronze on GCS and Iceberg, Dataform Silver-to-Gold, scheduled releases, Power BI connector.
- For report authors, the change is one connection dialog; workspaces, DAX, and visuals carry over.
- For M&O, the change is a cloud source, a service account, and a refresh schedule — no gateway fleet.
- Tee up the capstone: in ten minutes each group maps one real extract job onto exactly this path.

Iowa tie

This is the migration charter in one sentence: a single source of truth in BigQuery behind the reports Iowa already runs.

Design patterns

- Close with the one-line summary, then pause for objections before the workshop.
- Keep this card on screen while tables form for the capstone.

Anti-patterns

- Adding new content here — the teaching is done.
- Skipping the questions pause; the workshop lands better with objections aired.

History & what's new

The pattern is mature: the BigQuery connector is GA, refreshed by Microsoft's May 2025 ADBC-based V2 — and their follow-on 3-day BigQuery warehousing course goes deeper.

Flow

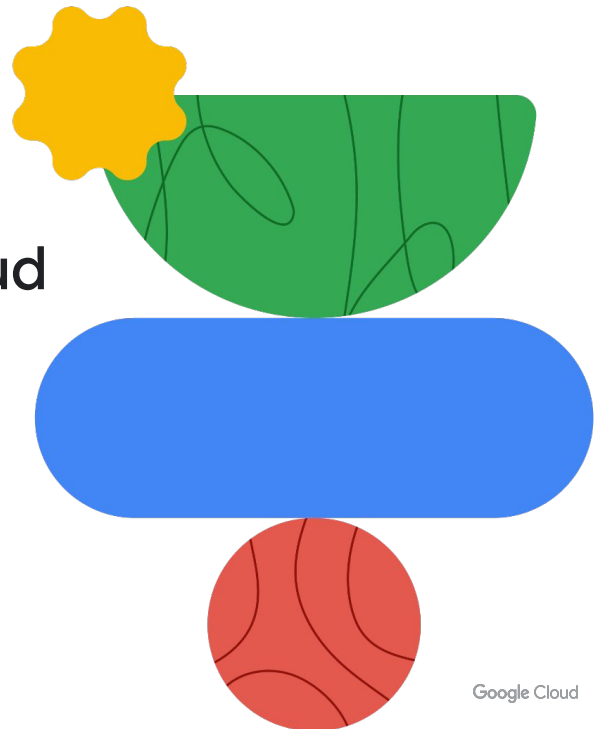
End of Part A teaching; next the deck shifts to the lakehouse recap and AI-direction slides before the 15:20 capstone.

Links

<https://learn.microsoft.com/en-us/power-query/connectors/google-bigquery>
<https://learn.microsoft.com/en-us/power-bi/connect-data/desktop-directquery-about>
<https://learn.microsoft.com/en-us/power-bi/connect-data/refresh-scheduled-refresh>

Building a modern data lakehouse on Google Cloud

Leveraging powerful, open-source-based services.



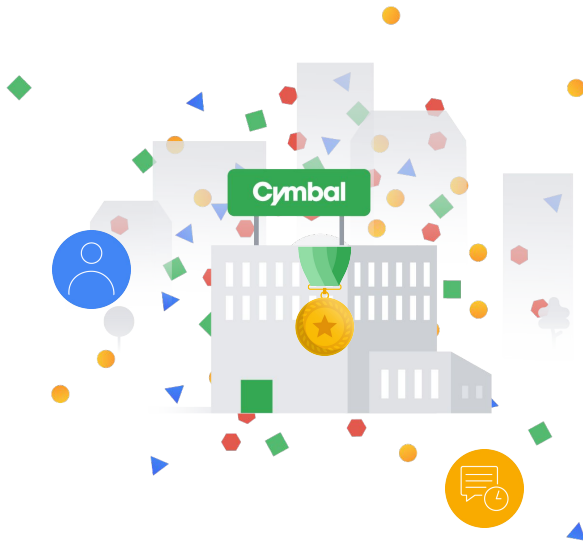
Google Cloud

INSTRUCTOR NOTES — PAGE 167 OF 208 · 14:50 · MODULE 11 — BQML + THE POWER BI LAST MILE

FACILITATOR NOTE

- Vendor section divider opening the lakehouse recap module — the slides ahead stitch GCS, BigQuery, and the open-source stack back together and gesture at Google's AI direction.
- Keep it moving: the teaching already happened; this is consolidation before the capstone, not new material.

Flow: Part B continues with the lakehouse core-components recap and AI-direction slides, landing at the 15:20 capstone workshop.



Cymbal deals with a massive amount of data daily in various formats. A data lakehouse architecture helps bring it together for powerful analytics and AI-powered customer experiences.

Google Cloud

INSTRUCTOR NOTES — PAGE 168 OF 208 · 14:50 · MODULE 11 — BQML + THE POWER BI LAST MILE

Hook

Part A put Power BI and a BQML model on the table; now the deck rewinds once to show the whole platform as one system. Cymbal is the stand-in for DOM DoIT: many sources, many formats, one destination.

Say · cover

- Map Cymbal to yourselves: dozens of source systems, CSV/database extracts flying around, one analytics destination wanted.
- A lakehouse = one copy of the data (Cloud Storage + BigQuery) serving SQL, BI and AI - no extract-per-consumer.
- 'AI-powered experiences' is why the deck insists on open formats: models will read the same tables Power BI does.
- Their version of this slide: one BigQuery-based source of truth replacing per-agency SQL Server extracts.

Iowa tie

This is your target operating model in one slide: extracts die, data lands once in the lakehouse, every consumer - Power BI included - reads from there.

Design patterns

- One storage layer, many engines; raw and curated live in the same platform.

Anti-patterns

- Copying data out per tool - an extract per consumer, which is the exact thing you are retiring.

History & what's new

Databricks evangelized the 'lakehouse' concept around 2020-2021 as the warehouse-plus-lake answer; by 2026 Google's own docs formalize a 'borderless lakehouse' on BigQuery + Cloud Storage.

Flow

We just closed the Power BI connector segment; this section reassembles the full platform, then recaps it, before the capstone.

Links

<https://cloud.google.com/discover/what-is-a-data-lakehouse>

<https://cloud.google.com/bigquery/docs/biglake-intro>

<https://cloud.google.com/customers/impulsogov>

Core components of a Google Cloud data lakehouse

A robust and scalable data lakehouse on Google Cloud is built on a foundation of several key services that work together seamlessly.



INSTRUCTOR NOTES — PAGE 169 OF 208 · 14:50 · MODULE 11 — BQML + THE POWER BI LAST MILE

Hook

Read this as your SQL Server estate re-drawn: Cloud Storage is the file share that became a lake, BigQuery is the warehouse engine, and the catalog is what msdb metadata always wished it were.

Say · cover

- Four components the deck walks next: Cloud Storage (the lake), BigQuery (the engine), Knowledge Catalog (governance), AlloyDB (the OLTP edge) - with Iceberg as the open table format.
- Console orientation: Storage browser, BigQuery Studio, and the catalog all live in the same console, same project, same IAM.
- None of this replaces Power BI - these components sit behind it.
- You already chose this stack: GCS/BigQuery for the lakehouse, Dataform for transforms, Power BI on top.

Iowa tie

Your architecture document already names these components; this slide is the vendor deck catching up to your design.

Design patterns

- Managed services over self-managed VMs; one IAM plane across storage, warehouse and catalog.

Anti-patterns

- Bolting on a separate metastore/governance product later instead of the built-in catalog from day one.

History & what's new

Cloud Storage (2010) and BigQuery (GA November 2011) were separate products for a decade; Google fused them into a documented lakehouse stack via Iceberg tables (2024) and 2026 'borderless lakehouse' guidance.

Flow

Section opener - the next four slides take one component each, so park deep questions; each gets its own page.

Links

<https://cloud.google.com/storage/docs/introduction>
<https://cloud.google.com/bigquery/docs/introduction>
<https://cloud.google.com/dataplex/docs/overview>

Cloud Storage

This is the heart of your data lakehouse, providing a highly scalable and cost-effective object storage service. Think of it as the foundational layer where all your data, in its raw and processed forms, resides.

Cymbal's data

- ✓ Website clickstream data
- ✓ Product images
- ✓ Supplier inventory files (CSV/Parquet)
- ✓ Historical data (Apache Iceberg)

Google Cloud

INSTRUCTOR NOTES — PAGE 170 OF 208 · 14:50 · MODULE 11 — BQML + THE POWER BI LAST MILE

Hook

This is the landing zone your extracts die into: instead of CSV drops on a file share picked up by SSIS, files land in a bucket and stay useful forever.

Say · cover

- Buckets are the Bronze landing zone - clickstream, images, supplier CSV/Parquet; GCS holds any format at any scale.
- Yesterday's CSV-vs-Parquet DIN applies here: columnar, typed Parquet is cheaper and faster for every downstream read.
- Storage classes (Standard/Nearline/Coldline/Archive) are your retention lever - Bronze history ages to colder classes without leaving the platform.
- BigLake lets BigQuery query these files in place, with table-level security - no load step.
- Iceberg on top turns the bucket into database-style tables with updates and deletes.

Iowa tie

This is your Bronze zone: Datastream output and file loads land here, and BigLake/Iceberg make the bucket queryable without moving data into the warehouse.

Design patterns

- Land raw first, transform later; lifecycle rules age Bronze to cheaper storage classes automatically.

Anti-patterns

- Treating the bucket like an FTP drop with no naming or partition layout - that builds a swamp, not a lake.

History & what's new

Cloud Storage launched in May 2010 as Google's second cloud service; the 2024-2026 BigLake and Iceberg work turned it from file store into the lakehouse table layer.

Flow

Component one of four; next is BigQuery, which queries this storage without moving it.

Links

<https://cloud.google.com/storage/docs/introduction>
<https://cloud.google.com/storage/docs/storage-classes>
<https://cloud.google.com/bigquery/docs/biglake-intro>

BigQuery

This powerful, serverless data warehouse acts as the analytics engine for your data lakehouse. What makes BigQuery unique is its ability to directly query data in Cloud Storage without needing to move it.

The separation of storage and compute is a core principle. BigQuery provides:

- ✓ Flexibility
- ✓ Cost savings

Google Cloud

INSTRUCTOR NOTES — PAGE 171 OF 208 · 14:50 · MODULE 11 — BQML + THE POWER BI LAST MILE

Hook

You have lived in BigQuery for two days; this slide names the architectural reason it feels different from SQL Server: storage and compute are divorced.

Say · cover

- Serverless means no instance sizing, no tempdb, no index maintenance - you write SQL, slots do the rest.
- Separation of storage and compute: pay for stored bytes once, scanned bytes per query; scaling one never touches the other.
- Federated queries reach GCS files, Cloud SQL/AlloyDB and more - no SSIS package needed to move data first.
- That scan-bytes model is exactly why part A defaulted Power BI to Import mode, not DirectQuery.
- For you, this is the single source of truth replacing the SQL Server warehouse; Power BI is just another client.

Iowa tie

This is the warehouse you are migrating to; the Import-mode default for your marts exists because of this slide's scan-bytes pricing.

Design patterns

- Partition and cluster the big tables so the scan-bytes model stays cheap; one warehouse, many datasets.

Anti-patterns

- Pointing DirectQuery dashboards at unpartitioned tables - every visual click is a billed full scan.

History & what's new

BigQuery was announced in May 2010 and went GA in November 2011; in 2026 ObjectRefs reached GA, letting SQL treat unstructured files in Cloud Storage as first-class data alongside Iceberg tables.

Flow

Component two done; next, the catalog that keeps everything findable and governed.

Links

<https://cloud.google.com/bigquery/docs/introduction>

<https://cloud.google.com/blog/products/data-analytics/separation-of-storage-and-compute-in-bigquery>

<https://cloud.google.com/bigquery/docs/federated-queries-intro>

Knowledge Catalog

As your data grows, managing it can become a significant challenge. Knowledge Catalog provides an intelligent data fabric that allows you to discover, manage, monitor, govern, and describe your data across your entire data lakehouse.

Cymbal's data

- Knowledge Catalog automatically catalogs data in Cloud Storage buckets, making it easily searchable for data analysts and scientists.
- It enforces data quality rules and security policies, protecting sensitive customer information.

Google Cloud

INSTRUCTOR NOTES — PAGE 172 OF 208 · 14:50 · MODULE 11 — BQML + THE POWER BI LAST MILE

Hook

In SQL Server land, 'what does this column mean?' lives in someone's head or a wiki from 2014. Knowledge Catalog is Google's answer: automatic metadata over everything you built today.

Say · cover

- Automatic cataloging: it scans BigQuery datasets and Cloud Storage buckets, so tables are searchable minutes after creation.
- Data quality rules and security policies attach to the data itself - the governance half of Module 10 resurfacing.
- Cymbal angle: analysts find supplier files without asking engineering; sensitive customer data gets policy-tagged.
- Naming warning: this is Dataplex Universal Catalog renamed (April 2026) - older docs say Dataplex; API and IAM names are unchanged.
- Sixteen people cannot govern what they cannot find - the catalog is what makes 'single source of truth' real.

Iowa tie

Single source of truth means cataloged source of truth: this is how agencies discover your Gold marts without emailing the team.

Design patterns

- Turn on automatic data-quality scans per zone from day one - Bronze schema drift, Silver null rates.

Anti-patterns

- Governance as phase two: retrofitting a catalog after 200 tables exist is the classic failure mode.

History & what's new

Lineage: Data Catalog (deprecated February 2025) became Dataplex Universal Catalog, renamed Knowledge Catalog on April 10, 2026 - now repositioned as the context layer for Google's agents.

Flow

Component three; next is AlloyDB - the one piece that is not analytics at all.

Links

<https://cloud.google.com/dataplex/docs/overview>
<https://docs.cloud.google.com/knowledge-catalog/docs/release-notes>
<https://cloud.google.com/dataplex/docs/auto-data-quality-overview>

AlloyDB for PostgreSQL

Although the data lakehouse is excellent for analytical workloads, many applications still rely on transactional databases.

AlloyDB is a fully managed, PostgreSQL-compatible database service that offers superior performance and availability.

Cymbal's data

- AlloyDB for e-commerce backend handles real-time order processing and customer account management.
- Integrate AlloyDB data into data lakehouse for broader analytics via federated queries.

Google Cloud

INSTRUCTOR NOTES — PAGE 173 OF 208 · 14:50 · MODULE 11 — BQML + THE POWER BI LAST MILE

Hook

Your line-of-business systems keep running on OLTP databases - nobody migrates the order-entry app into a lakehouse. AlloyDB is Google's PostgreSQL answer, and the lakehouse reads it without extracts.

Say · cover

- AlloyDB = fully managed, PostgreSQL-compatible OLTP: the transactional edge the lakehouse does not replace.
- The pattern that matters to you: BigQuery federated queries SELECT against the operational database live - no nightly extract for reference data.
- Cymbal runs real-time orders and accounts on it, then federates into the lakehouse for analytics.
- Iowa translation: agency systems that keep their own databases still feed Gold marts - federate the small lookups, Datastream-replicate the heavy history (Day 1, M2).
- Do not confuse roles: AlloyDB is not the warehouse; BigQuery stays the analytics source of truth.

Iowa tie

This answers 'what about systems that keep their databases?' - federate the small stuff, replicate the big stuff, extract nothing.

Design patterns

- Federation for low-volume current-as-of-now lookups; Datastream CDC for high-volume history.

Anti-patterns

- Federating million-row joins behind a dashboard - the OLTP box pays for every report refresh.

History & what's new

AlloyDB was announced at Google I/O in May 2022 and went GA that December; at Next '25 (April 2025) it gained an AI query engine with natural-language and vector capabilities.

Flow

Last component; next is Iceberg - the open table format tying storage and warehouse together.

Links

<https://cloud.google.com/alloydb/docs/overview>

<https://cloud.google.com/bigquery/docs/federated-queries-intro>

<https://cloud.google.com/alloydb/docs/omni>

Apache Iceberg

An open-source table format that brings the reliability and performance of a traditional database to your data lake.

Using Apache Iceberg with BigQuery and Cloud Storage, you can perform efficient updates and deletes on your data, a capability that was traditionally difficult in data lakes.

This is crucial for Cymbal when they need to update product information or manage customer data privacy requests.

Google Cloud

INSTRUCTOR NOTES — PAGE 174 OF 208 · 14:50 · MODULE 11 — BQML + THE POWER BI LAST MILE

Hook

Remember the CSV-vs-Parquet DIN? Iceberg is the next rung: Parquet files plus an open table format that gives them ACID, schema evolution and time travel.

Say · cover

- Iceberg = open table format over Parquet in Cloud Storage: updates, deletes, schema changes - things raw data lakes cannot do.
- BigQuery's Iceberg managed tables: BigQuery owns the metadata while data stays open in your bucket - Spark and other engines can read the same tables.
- Cymbal's use - product updates and privacy-request deletes - is exactly your 'correct a record in Bronze/Silver' problem.
- Legitimate zone split: Bronze/Silver as Iceberg tables, Gold as native BigQuery marts.
- For you it is also lock-in insurance: the bytes stay readable even if you ever leave BigQuery.

Iowa tie

This is how you correct or delete records in the lake (privacy requests!) without extract-reload gymnastics - and the format stays open by design.

Design patterns

- Iceberg for zones needing updates/deletes or cross-engine access; native BigQuery tables for Gold serving.

Anti-patterns

- Hand-rolling 'delete this person's rows' as raw Parquet file rewrites - error-prone and unauditible.

History & what's new

BigQuery tables for Apache Iceberg entered preview in October 2024; 2026 releases brought GA partitioning and multi-statement transactions for managed Iceberg tables plus cross-engine ObjectRefs.

Flow

Components complete; now the deck zooms out to the pattern you already chose - the medallion.

Links

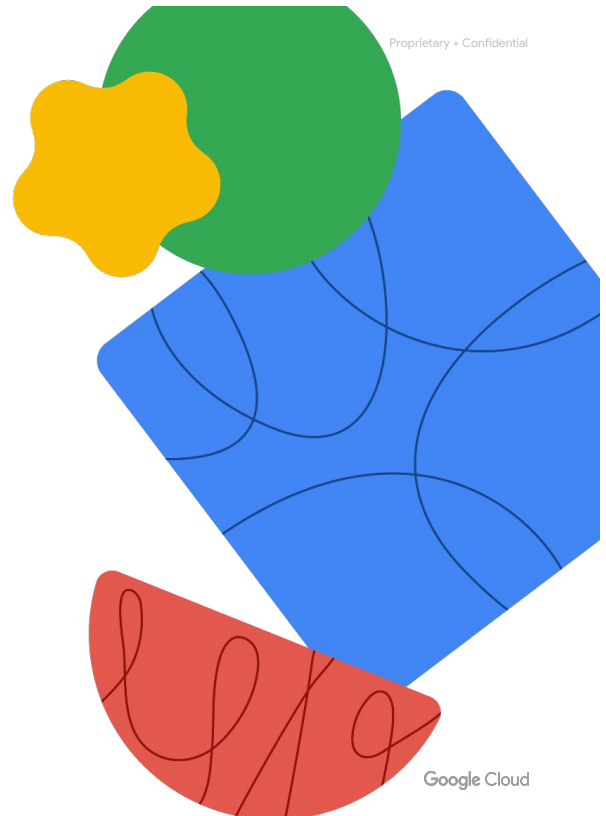
<https://cloud.google.com/bigquery/docs/iceberg-tables>

<https://cloud.google.com/blog/products/data-analytics/announcing-bigquery-tables-for-apache-iceberg>

<https://iceberg.apache.org>

Architectural patterns and best practices

The most common and effective architectural pattern for a data lakehouse is the medallion architecture, which organizes data into three distinct zones.



INSTRUCTOR NOTES — PAGE 175 OF 208 · 14:50 · MODULE 11 — BQML + THE POWER BI LAST MILE

Hook

Here is the slide where the deck endorses your architecture document: of all the ways to organize a lakehouse, one pattern won - and you are already building it.

Say · cover

- The medallion architecture - Bronze, Silver, Gold - is the industry-default way to zone a lakehouse.
- Why it won: each zone has one job (land, conform, serve), so pipelines stay debuggable and quality is measurable per hop.
- It is not a Databricks-only idea: Microsoft's Fabric uses the same three names, and Google's lakehouse docs recommend it outright.
- Next slide is your design in the deck's words - use it as validation, not new material.

Iowa tie

Validation moment: the target architecture your team chose is the one the industry converged on - nothing proprietary about it.

Design patterns

- One responsibility per zone; quality gates at zone boundaries, not inside one mega-pipeline.

Anti-patterns

- Zones in name only - if Silver tables are just Bronze with a prefix, you have a swamp with medals.

History & what's new

Databricks coined 'medallion' around 2020-2021 as lakehouse vocabulary; Microsoft adopted the same zone names in Fabric, and Google's 2026 lakehouse documentation recommends the pattern.

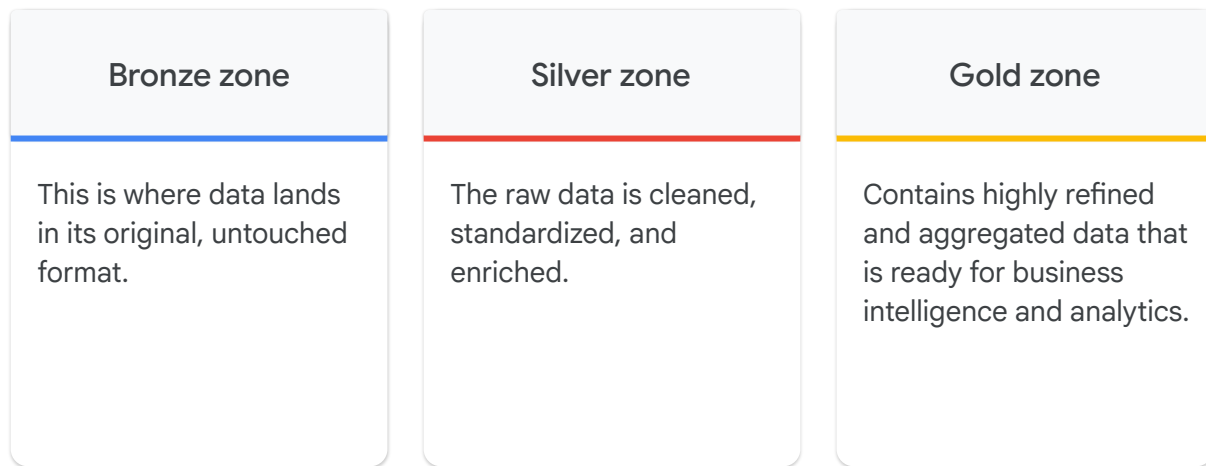
Flow

One slide of concept, then the zones themselves - the recap your design document deserves.

Links

<https://cloud.google.com/discover/what-is-medallion-architecture>
<https://www.databricks.com/glossary/medallion-architecture>
<https://docs.cloud.google.com/lakehouse/docs/lakehouse-basics>

The medallion architecture



Google Cloud

 INSTRUCTOR NOTES — PAGE 176 OF 208 · 14:50 · MODULE 11 — BQML + THE POWER BI LAST MILE

Hook

Read this slide as your design doc with Google's logo: Bronze is where extracts land, Silver is where Dataform conforms, Gold is what Power BI imports.

Say · cover

- Bronze: raw and untouched - Datastream CDC output and file drops in GCS/BigLake; never edit in place.
- Silver: cleaned, standardized, enriched - BigQuery tables built by Dataform; conformed dimensions live here.
- Gold: refined aggregates - your data marts, one per subject area, shaped for reports.
- And the last mile from part A: Gold marts land in the Power BI you already have - Import mode, refresh aligned to the Dataform release that built them.
- Trace it backwards for the room: a Power BI visual <- Gold mart <- Dataform release <- Silver tables <- Bronze files <- the dead extract's source system.

Iowa tie

This is your project on one slide: kill-the-extracts feeds Bronze, Dataform owns Silver-to-Gold, Power BI consumes Gold - nothing left to invent.

Design patterns

- Immutable Bronze; Dataform (not ad-hoc queries) owns every Silver/Gold table, so lineage is automatic.

Anti-patterns

- Letting analysts report directly on Bronze/Silver - you will never be able to refactor upstream.

History & what's new

The zone vocabulary came from Databricks (~2020) and is now platform-neutral: Google customers run Bronze in Cloud Storage with Silver/Gold in BigQuery - ImpulsoGov's 2026 case study is exactly that layout.

Flow

Zones recapped; next, four practices that keep this from decaying, then the pitfalls - before the capstone.

Links

<https://cloud.google.com/dataform/docs/overview>
<https://cloud.google.com/discover/what-is-medallion-architecture>
<https://cloud.google.com/bigquery/docs/iceberg-tables>

Best practices

- 01 Embrace open standards
- 02 Govern your data from the start
- 03 Optimize for cost and performance
- 04 Automate your data pipelines

Google Cloud

 INSTRUCTOR NOTES — PAGE 177 OF 208 · 14:50 · MODULE 11 — BQML + THE POWER BI LAST MILE

Hook

Four practices, translated for this room: open standards = never trapped; governance early = your single-source-of-truth promise; cost/performance = watch scans like query plans; automate = your CI/CD instinct applied to data.

Say · cover

- 01 Open standards: Parquet plus Iceberg means the bytes outlive any vendor decision - lock-in insurance.
- 02 Govern from the start: catalog and policy tags on day one, not after the first audit finding - Module 10's message.
- 03 Optimize cost and performance: partition and cluster the big Silver tables, let slots autoscale, monitor scan bytes.
- 04 Automate pipelines: Dataform release + workflow configurations run Silver-to-Gold on a schedule - the discipline your SQL Server CI/CD already enforces for code.
- For M&O: automation includes monitoring - pipeline failures should page the ops team, not the report consumers.

Iowa tie

Your CI/CD muscle transfers directly: a Dataform release is the data-side equivalent of your SQL Server deployment pipeline - schedule the build, refresh Power BI right after.

Design patterns

- Give every practice an owner: formats (engineers), governance (catalog steward), cost (billing-export owner), automation (M&O).

Anti-patterns

- Manual console clicks as the deployment mechanism - fine for class, disqualifying in production.

History & what's new

Google productized the automation half through 2023: Dataform release and workflow configurations, plus service-account delegation (September 2023), turned 'schedule the Silver-to-Gold build' into a managed feature.

Flow

Practices stated; the next two slides are the mirror image - four ways this goes wrong.

Links

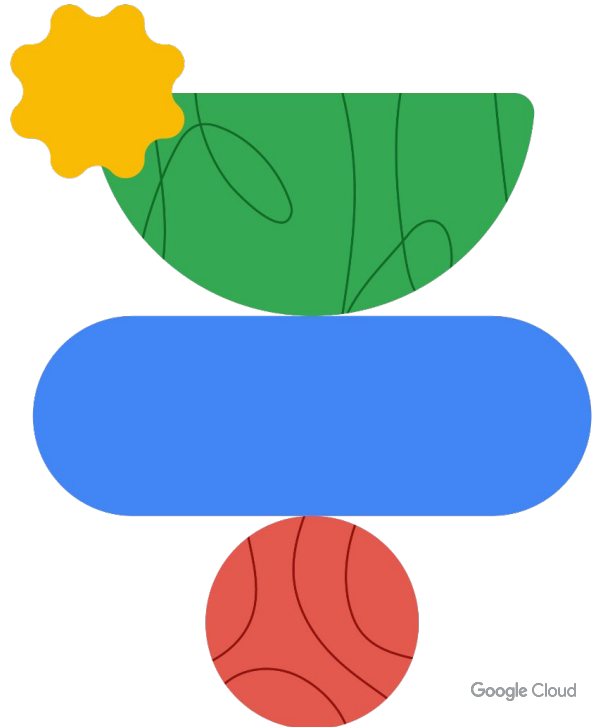
<https://cloud.google.com/bigquery/docs/best-practices-costs>

<https://cloud.google.com/architecture/framework/cost-optimization>

<https://cloud.google.com/dataform/docs/overview>

Common pitfalls to avoid

Implementing a data lakehouse is a journey, and there are some common challenges to be aware of.



Google Cloud

INSTRUCTOR NOTES — PAGE 178 OF 208 · 14:50 · MODULE 11 — BQML + THE POWER BI LAST MILE

FACILITATOR NOTE

- Section breather before the four pitfalls.
- Say it plainly: none of these are technical failures - they are program failures, and a large state IT organization is exactly where they happen.
- Next slide names them one by one; keep it conversational and ask which one feels most familiar.

Flow: Best practices done; two slides of pitfalls, then the AI-direction close before the capstone.

Common pitfalls to avoid

01 Neglecting governance

02 Ignoring data quality

03 Vendor lock-in

04 Lack of clear strategy

Google Cloud

INSTRUCTOR NOTES — PAGE 179 OF 208 · 14:50 · MODULE 11 — BQML + THE POWER BI LAST MILE

Hook

You have seen these four in slow motion on SQL Server projects - same movie, new platform, same ending unless someone intervenes early.

Say · cover

- 01 Neglecting governance: without catalog and policy tags from day one, your single source of truth becomes sixteen versions again.
- 02 Ignoring data quality: garbage upstream becomes confident dashboards downstream - put automatic quality scans on the Silver gates.
- 03 Vendor lock-in: the fear that brought you to open formats - mitigate with Parquet/Iceberg in Bronze/Silver, not with architecture paralysis.
- 04 No clear strategy: 'migrate everything' is not a strategy - sequence mart by mart and prove the pattern on one extract first (the capstone's whole point).
- Reframe for this room: your phased migration with Power BI continuity already de-risks #4.

Iowa tie

Pitfall #4 is your superpower: you arrive with a defined target - kill extracts, medallion, Power BI last mile - while most organizations build the lake first and ask why later.

Design patterns

- One mart end-to-end as the pilot; quality gates and governance wired in before mart two starts.

Anti-patterns

- Big-bang migration with governance 'to follow' - how lakehouses become swamps with better branding.

History & what's new

The 2026 Knowledge Catalog rename bundled automatic data quality into the same service as the catalog - governance and quality are now one product surface, not two projects.

Flow

Pitfalls named; the deck closes with the AI direction, then the capstone.

Links

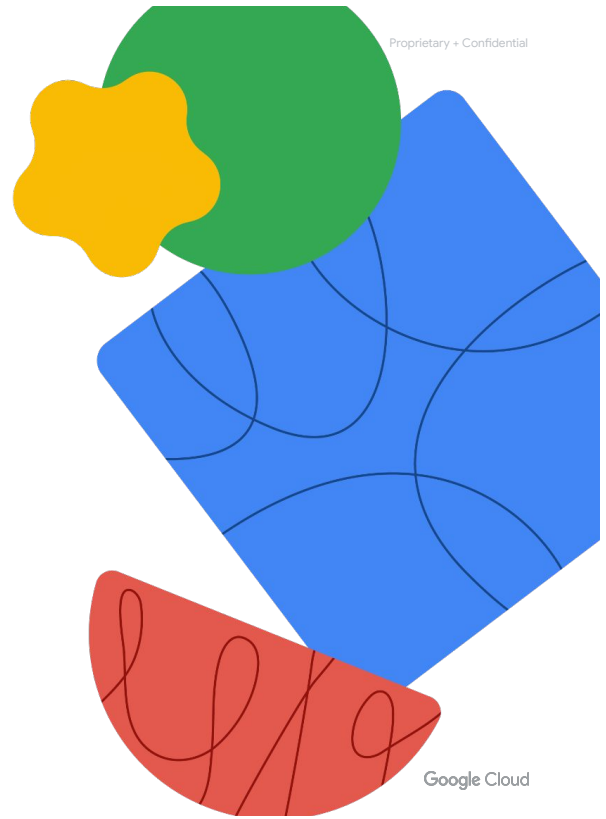
<https://cloud.google.com/dataplex/docs/auto-data-quality-overview>

<https://cloud.google.com/architecture/framework>

<https://iceberg.apache.org>

The future: AI and data modernization

The true power of a modern data platform lies in its ability to support advanced analytics and AI.



Google Cloud

INSTRUCTOR NOTES — PAGE 180 OF 208 · 14:50 · MODULE 11 — BQML + THE POWER BI LAST MILE

FACILITATOR NOTE

- Divider for the two AI slides - frame it in ten seconds.
- ML now lives where the data lives: first the BQML you saw in part A, then the agent and generative direction.
- Nothing here changes the migration plan; it justifies the open, governed lakehouse you just designed.

Flow: Pitfalls done; two AI-direction slides, then the M11 recap and the 15:20 capstone.

AI and data modernization (1 of 2)

01

Advanced analytics can be done quickly using BigQuery ML.

02

Services, such as Agent Platform, can be used to build and train ML models directly on the data in your data lakehouse.

03

Cymbal could develop a recommendation engine for product suggestions or a fraud detection model for suspicious transactions.

Google Cloud

INSTRUCTOR NOTES — PAGE 181 OF 208 · 14:50 · MODULE 11 — BQML + THE POWER BI LAST MILE

Hook

Part A showed CREATE MODEL on penguins; this is the idea at platform scale - and why clean, governed Gold marts matter more than any algorithm.

Say · cover

- 01 BQML: training and prediction in SQL, where the data lives - your team already has the core skill, they write SQL for a living.
- 02 'Agent Platform' is Vertex AI renamed: the Gemini Enterprise Agent Platform (April 2026) builds models and agents on lakehouse data.
- 03 Translate Cymbal to government: recommendations become 'next best service'; fraud detection becomes improper-payment scoring.
- Honest framing: nothing in Corey's brief requires ML - direction, not homework; the migration earns the option.
- What makes it cheap later: medallion plus catalog means models train on governed, deduplicated data instead of extracts.

Iowa tie

Payoff line: the same Gold marts that feed Power BI are the training set - no second pipeline for AI, ever.

Design patterns

- Start ML on Gold marts (clean, documented), not Bronze files; BQML for tabular problems, Agent Platform for agents/gen-AI.

Anti-patterns

- A separate 'AI data stack' with its own copies - that is just extracts wearing a new hat.

History & what's new

BQML launched in beta at Next 2018 with only linear and logistic regression; in April 2026 Vertex AI was renamed the Gemini Enterprise Agent Platform (GA April 22) - hence this deck's wording.

Flow

AI slide one of two; next the generative half, then the recap card.

Links

<https://cloud.google.com/bigquery/docs/bqml-introduction>

<https://cloud.google.com/gemini-enterprise-agent-platform>

<https://cloud.google.com/bigquery/docs/reference/standard-sql/bigqueryml-syntax-create>

AI and data modernization (2 of 2)

The rise of generative AI also opens up exciting new possibilities. Cymbal could use large language models to create engaging product descriptions automatically or to power a chatbot that provides personalized customer support.



Google Cloud

INSTRUCTOR NOTES — PAGE 182 OF 208 · 14:50 · MODULE 11 — BQML + THE POWER BI LAST MILE

Hook

Generative AI for a data team is not a chatbot demo - it is SQL functions that call Gemini on columns you already govern, with your audit logs watching.

Say · cover

- BigQuery AI functions (the AI.GENERATE family) call Gemini from SQL: classify, summarize, extract fields - in place, on Bronze/Silver/Gold data.
- Cymbal's examples translated: auto product descriptions become plain-language program summaries; the support chatbot becomes a constituent-service assistant grounded in cataloged data.
- Governance carry-over: prompts run inside your project under your IAM and audit logs - not data pasted into an outside service.
- Keep the part-A discipline: seen, not drilled - file it under 'after the migration lands.'
- Realistic first candidates for you: document processing and plain-language summarization, once the lakehouse is the source of truth.

Iowa tie

Where this touches your roadmap: AI functions on governed Gold data let agencies get summarization/classification wins without a new platform or new copies.

Design patterns

- Call AI functions inside Dataform models so outputs are versioned and rebuildable like any other table.

Anti-patterns

- Pasting agency data into consumer chatbots outside your project - ungoverned, unaudited, a records-request nightmare.

History & what's new

Google made AI.GENERATE and AI.GENERATE_TABLE generally available on January 27, 2026, with Gemini 3.0 support in the same BigQuery AI line - 'LLM over warehouse data' is now standard SQL.

Flow

AI direction closed; one recap card for the whole module, then the capstone.

Links

<https://cloud.google.com/bigquery/docs/generate-text>

<https://cloud.google.com/blog/products/data-analytics/new-bigquery-gen-ai-functions-for-better-data-analysis>

<https://cloud.google.com/vertex-ai/generative-ai/docs/learn/overview>

M11 — What you now have

- ✓ BQML seen, not drilled— CREATE MODEL → ML.PREDICT in SQL, where your data already lives
- ✓ The end-to-end closed— kill the extracts → BigQuery → Bronze/Silver/Gold → Gold marts → Power BI
- ✓ A default for the marts— Import + scheduled refresh aligned to your Dataform release schedule
- ✓ An auth pattern— author OAuth on the desktop, least-privilege service account in the service
- ✓ Next: the capstone— map one of your real extract jobs onto everything you've just seen

Custom slide · added by Imran Ahmad

Iowa DOM DoIT · Data Engineering on Google Cloud

INSTRUCTOR NOTES — PAGE 183 OF 208 · 14:50 · MODULE 11 — BQML + THE POWER BI LAST MILE

Hook

This card folds two days into your project plan - if anyone photographs one slide, make it this one.

Say · cover

- BQML seen, not drilled: CREATE MODEL to ML.PREDICT in SQL, beside your tables - an option you own, not an assignment.
- The end-to-end is closed: kill the extracts -> BigQuery -> Bronze/Silver/Gold -> Gold marts -> the Power BI you already have.
- Mart default: Import mode plus scheduled refresh aligned to your Dataform release schedule - the report refreshes right after its build.
- Auth pattern: authors use OAuth on Desktop (real people in the audit trail); the service uses a least-privilege service account reading only Gold.
- Gateway answer for M&O: BigQuery is a cloud source - nothing new to run on-prem.

Iowa tie

Every line is a decision from your brief: extracts retire, Dataform owns Silver-to-Gold, Power BI authors keep their tool - only the connection string changes.

Design patterns

- Chain refresh to the Dataform release, not an arbitrary clock time - data and reports can never disagree by a day.

Anti-patterns

- DirectQuery by default 'because it is current' - per-visual scan billing is the quiet budget-killer.

History & what's new

Microsoft shipped a revised BigQuery connector in the May 2025 Power BI release, with an Entra ID variant via Workforce Identity Federation - a living, supported path, not a hack.

Flow

Module 11 complete - next, the 15:20 capstone: groups map one real extract job onto everything on this card.

Links

<https://learn.microsoft.com/en-us/power-query/connectors/google-bigquery>
<https://cloud.google.com/dataform/docs/schedule-runs>
<https://learn.microsoft.com/en-us/power-bi/connect-data/refresh-data>

Capstone — map one of YOUR real extract jobs

30 minutes — 15:20–15:50 (20 build + 10 share-out) · Groups of 3–4 — the filled map is the deliverable

1. Pick one real, typical extract job — the one that pages someone when it fails; keep it to one
2. Fill the map left to right: job → source system → ingest tool → Bronze → Silver → Gold
3. ... then orchestration & CI/CD → Power BI serving → first thing that breaks (be honest)
4. Aids: database → Datastream or batch; files → Storage Transfer + LOAD DATA, pinned schema
5. Defaults: Dataform owns Silver→Gold in git; Scheduler stays boring; Power BI Import-first
6. Share-out: a box you can't fill is the most valuable thing to raise — then photograph your map

Why it matters — this page is backlog draft zero for your migration — the artifact you take back to work.

Custom slide · added by Imran Ahmad

Iowa DOM DoIT · Data Engineering on Google Cloud

INSTRUCTOR NOTES — PAGE 184 OF 208 · 15:20 · WORKSHOP — CAPSTONE: MAP ONE OF YOUR EXTRACT JOBS (30 MIN)

Hook

Every team has the SQL Agent job that pages someone at 2 AM — each group now maps that job onto the two days' GCP target.

Say · cover

- 15:20–15:50: 20 minutes build in groups of 3–4, 10 share-out — the filled map, not a discussion, is the deliverable.
- Pick the most typical job, not the biggest or simplest — the one whose failure ruins someone's morning.
- Fill left to right: job → source → ingest → Bronze → Silver → Gold → orchestration & CI/CD → Power BI serving.
- Every box has an answer from the last two days; a box you cannot fill is the most valuable thing to raise — flag it.
- End with 'first thing that breaks' — schema drift, PII, 10x volume, a late county file; honesty starts the risk register.

Iowa tie

Kill-the-extracts made personal: one real job mapped onto their Bronze→Silver→Gold, Dataform, Power BI target — backlog draft zero for the migration.

Design patterns

- One tool per box — an 'or' marks an open question for the migration
- Bronze stays raw and replayable; cleaning belongs in Silver

Anti-patterns

- Designing a whole platform instead of mapping one job
- Picking the showcase job instead of the typical one

History & what's new

Datastream's SQL Server source went GA July 22, 2024 — the 'database → Datastream' box is a production answer, not preview; Dataform (acquired December 2020) is BigQuery's native ELT layer.

Flow

M11 just closed the last mile (BQML, Power BI connector) — the workshop assembles everything; the 15:50 summary retraces the two days.

Links

<https://docs.cloud.google.com/datastream/docs>

<https://cloud.google.com/blog/products/data-analytics/datastream-makes-it-easy-to-replicate-sql-server-databases>

<https://docs.cloud.google.com/dataform/docs/overview>

Closing the course — what this last hour gets you

- ✓ Capstone first— your group maps one real DOM DoIT extract job onto the GCP target architecture
- ✓ Then the fast retrace— two days against your migration map; recall, not new content
- ✓ One honest look at what changes Monday— your worksheet is backlog draft zero
- ✓ Where to go next— the 3-day Data Warehousing with BigQuery course and free practice paths
- ✓ Open questions and evaluations— the last 20 minutes are yours

Custom slide · added by Imran Ahmad

Iowa DOM DoIT · Data Engineering on Google Cloud

INSTRUCTOR NOTES — PAGE 185 OF 208 · 15:50 · COURSE SUMMARY & NEXT STEPS

Hook

Two days ago this room had barely opened the Google Cloud console; tonight they leave with their own migration map — this last hour is about locking it in.

Say · cover

- Order matters: capstone first while energy is high, then the fast retrace — recall, not new content.
- The retrace deck (lakehouse -> GCS -> Iceberg -> BigQuery -> governance -> ML) is the services walk: narrate each stop as 'your Bronze, your Silver->Gold, your marts'.
- Be plain about Monday: the worksheet is backlog draft zero — photograph and email it before leaving.
- Point forward: the 3-day Data Warehousing with BigQuery course is being scheduled and goes deep on everything Day 2 previewed.
- Hold the last 20 minutes for open questions and evaluations — they shape what gets scheduled next.

Iowa tie

The handoff from training to migration: the capstone map, the pack's reference cards, and the follow-on course are what they carry out.

Design patterns

- Close with an artifact plus a retrace, never new material
- Name the next training step explicitly so momentum survives the weekend

Anti-patterns

- Teaching new features in the summary hour
- Skipping evaluations — they drive the follow-on scheduling

History & what's new

BigQuery went GA November 2011 — 15 years old at this delivery — and this 2026 deck already reflects the April 2026 renames (BigLake -> Lakehouse, Dataplex -> Knowledge Catalog).

Flow

Agenda card for the closing block — p181 gives the minute-by-minute, p182 kicks off the capstone itself.

Links

<https://docs.cloud.google.com/bigquery/docs/introduction>
<https://cloud.google.com/discover/what-is-a-data-lakehouse>
<https://cloud.google.com/learn/training>

The last 70 minutes

15:20–15:50

Capstone workshop — groups of 3–4 map one real extract job: source → ingest → Bronze → Silver/Gold → Power BI

WORKSHOP

15:50–16:10

Course summary — the two days retraced against your migration map, one more time, fast

16:10–16:30

Flex / open questions / evaluations — plus what comes next (the 3-day BigQuery data warehousing course)

FACILITATOR NOTE

- Timing card for the close: capstone 15:20–15:50, summary retrace 15:50–16:10, then flex/Q&A/evaluations 16:10–16:30.
- Protect the share-out — if groups run long building, cut build time, never the share-out; the flex block is the buffer that keeps a 16:30 finish.
- Keep the retrace under ten minutes: recall, not reteach, and the capstone maps are still fresh on the tables.

Flow: Capstone kickoff slide follows at p182; the vendor retrace deck starts at p185 after the workshop.

Capstone: map one of your real extract jobs

30 minutes — 20 build + 10 share-out · Groups of 3–4 · whiteboard or paper · worksheet template in your pack

1. Pick ONE real extract job your team owns — the most typical one, not the biggest or simplest
2. Fill the template left to right: job → source → ingest → Bronze → Silver → Gold → orchestration → serving
3. Decide: one tool per box; being stuck on a box is the valuable part — flag it for share-out
4. Answer 'first thing that breaks' honestly — schema drift, PII, 10× volume, a late county file
5. Share-out: 2–2.5 min per group — read the job aloud, then walk the map left to right

Why it matters — Single-org room, so real systems are safe to name. This page is the artifact you take back to work — backlog draft zero for the migration project.

INSTRUCTOR NOTES — PAGE 187 OF 208 · 15:50 · COURSE SUMMARY & NEXT STEPS

Hook

Single-org room means they can name real systems aloud — say so; it makes this worth more than a generic case study.

Say · cover

- Read the rules as written: one real, typical job; worksheet in their pack; 20 minutes build, then 2–2.5 minutes per group.
- Share-out format: read the job aloud, then walk the map left to right — hold them to it or the last group gets squeezed.
- Enforce one tool per box; when a group stalls, coach them to flag it, not stuff it with three options.
- Push hard on 'first thing that breaks' — schema drift, PII, 10x volume, a late county file; 'nothing breaks' means not finished.
- Close with: photograph the map — it is backlog draft zero for the migration project.

Iowa tie

Their estate: SQL Server extracts, SSIS sweeps, county file drops — expect groups to land on the Datastream CDC path or the Storage Transfer + LOAD DATA path from the worksheet examples.

Design patterns

- Timebox share-outs to 2–2.5 minutes so every group speaks
- Treat unfilled boxes as open questions to raise, not failures to hide

Anti-patterns

- Letting groups design a platform instead of one job
- Allowing 'Datastream or DTS or Dataflow' cells with no open-question mark

History & what's new

Google acquired Dataform in December 2020 and made it BigQuery's native ELT layer; release configurations are the deploy surface their SQL Server CI/CD discipline maps onto — git, PR, scheduled release.

Flow

Kicks off 30 minutes of group work; p183 walks the template row by row while they build, p184 is your share-out rubric.

Links

<https://docs.cloud.google.com/dataform/docs/overview>

<https://docs.cloud.google.com/dataform/docs/configure-compilation#create-release-configuration>

<https://docs.cloud.google.com/scheduler/docs/overview>

The template: one real job, left to right

Every box has an answer from the last two days.

- The job— name it: one real SQL Server extract you own or babysit
- Source system— tables or files? Size? Change rate? PII?
- Ingest— Datastream CDC for databases; Storage Transfer + LOAD DATA for files
- Bronze— raw and replayable: GCS files, BigLake, or Iceberg; no cleaning here
- Silver→Gold— Dataform models dedupe and conform, then marts partitioned + clustered
- Serve— release configs + Scheduler; Power BI Import-first, refresh after the release

Stuck on ingest? Database → Datastream; files → Storage Transfer + LOAD DATA. Nightly tolerance → batch; always-current → CDC.

Custom slide · added by Imran Ahmad

Iowa DOM DoIT · Data Engineering on Google Cloud

INSTRUCTOR NOTES — PAGE 188 OF 208 · 15:50 · COURSE SUMMARY & NEXT STEPS

Hook

Walk this slide slowly during the build — each row is a decision they already practiced: Datastream, LOAD DATA, Dataform.

Say · cover

- Job row: force real job and table names — specificity makes it backlog draft zero, not a course exercise.
- Source row: tables or files, size, change rate, PII — those answers pick the ingest tool for you.
- Ingest: database -> Datastream (log CDC, no extract window); files -> Storage Transfer + LOAD DATA with pinned schema — the autodetect N/A pitfall from DIN \$25.
- Bronze stays raw and replayable — GCS files, BigLake, or Iceberg; no cleaning rules allowed in that box.
- Silver->Gold: Dataform models dedupe and conform, marts partitioned + clustered; serve Power BI Import refreshed after the release event.

Iowa tie

The rows are their migration: SQL Server source -> Datastream or file load -> Bronze on GCS -> Dataform Silver->Gold -> the Power BI they already own.

Design patterns

- Refresh Power BI Import on the pipeline event ('after the Dataform release'), not a wall-clock guess
- Pinned schemas and declared null markers so bad files fail loudly

Anti-patterns

- Cleaning data in Bronze — it destroys replayability
- Autodetect on CSV loads — silent STRING columns, the N/A trap

History & what's new

Storage Transfer Service made event-driven transfers GA in July 2023 — a file landing in a bucket can trigger its own move; the scheduled-sweep SSIS pattern has an event-driven answer.

Flow

Groups are building now; next slide (p184) is the rubric you grade their maps against in the share-out.

Links

<https://docs.cloud.google.com/datastream/docs>

<https://docs.cloud.google.com/storage-transfer/docs/overview>

https://docs.cloud.google.com/bigquery/docs/reference/standard-sql/data-definition-language#load_data_statement

What a good map looks like — debrief

- Every box decided— one tool per cell; an 'or' means the decision got deferred — say so
- Bronze stays raw— it answers 'what did the source say'; Silver answers 'what does it mean'
- Dataform owns Silver→Gold, in git— transforms anywhere else need a stated reason
- Orchestration is boring— Scheduler + release configs; Composer when pipelines multiply
- Power BI Import-first— DirectQuery only with a stated freshness need
- 'First thing that breaks' is honest— those answers start your migration risk register

Custom slide · added by Imran Ahmad

Iowa DOM DoIT · Data Engineering on Google Cloud

INSTRUCTOR NOTES — PAGE 189 OF 208 · 15:50 · COURSE SUMMARY & NEXT STEPS

Hook

Your share-out scorecard — as each group walks its map, check it against these six lines and challenge what's missing.

Say · cover

- One tool per cell: an 'or' is a deferred decision — fine for draft zero, but mark it as an open question so it survives.
- Bronze answers 'what did the source say'; Silver 'what does it mean' — that separation makes reprocessing possible, the thing current extract jobs cannot do.
- Dataform owns Silver→Gold, in git; transforms anywhere else (heavy scheduled queries, Power Query) need a stated reason.
- Orchestration stays boring: Scheduler + release configs; Composer only when they can name the interdependency that needs it.
- Import-first Power BI — DirectQuery needs a stated intra-day freshness need; the honest 'breaks' row seeds their migration risk register.

Iowa tie

Six checks = their architecture document in miniature: medallion discipline, Dataform in git under CI/CD, Power BI last mile, an honest risk list for M&O.

Design patterns

- Refresh Import after the release event; DirectQuery only beside a stated freshness need
- Feed 'first thing that breaks' answers straight into the migration risk register

Anti-patterns

- Composer on day one — orchestration theatre without a named interdependency
- Hiding transforms in Power Query inside Power BI, where CI/CD cannot see them

History & what's new

The connector kept moving after this deck: in May 2025 Microsoft announced an ADBC-based Google BigQuery connector for Power BI, replacing the Simba ODBC implementation — worth knowing before their first refresh.

Flow

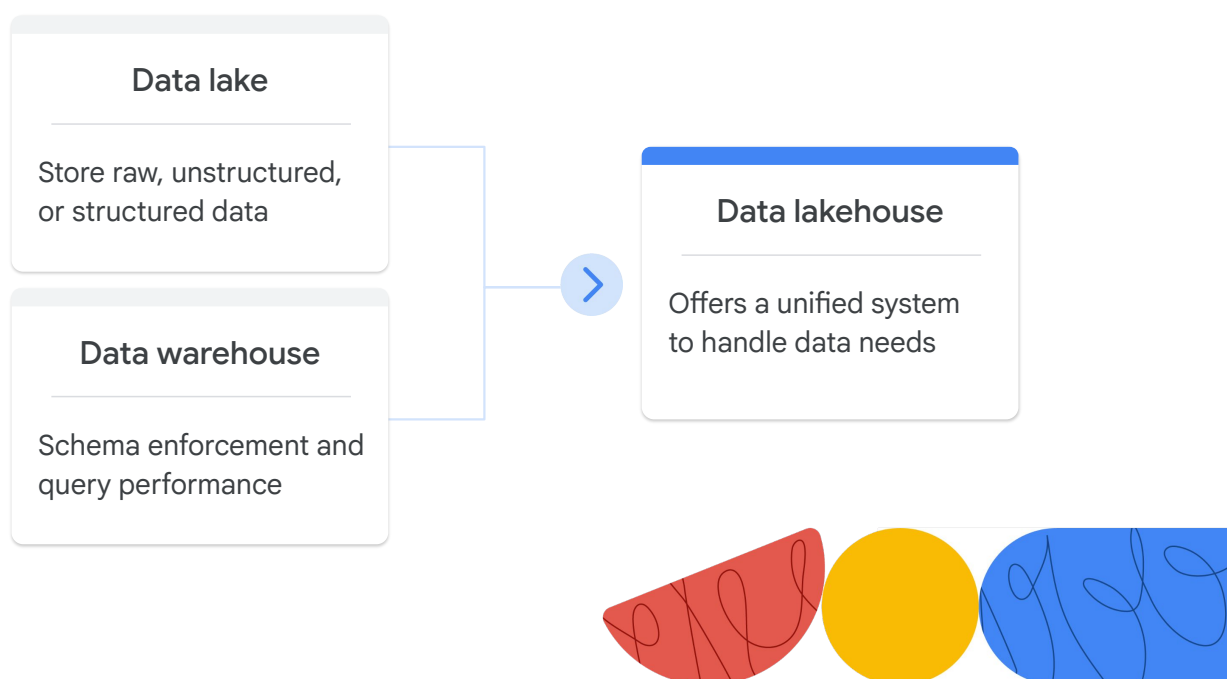
Share-out rubric; after the last group presents, p185 starts the fast services-come-together retrace.

Links

<https://learn.microsoft.com/en-us/power-query/connectors/google-bigquery>

<https://docs.cloud.google.com/dataform/docs/configure-compilation#create-release-configuration>

<https://docs.cloud.google.com/composer/docs>



INSTRUCTOR NOTES — PAGE 190 OF 208 · 15:50 · COURSE SUMMARY & NEXT STEPS

Hook

Start the retrace here: ask the room which of the three shapes their SQL Server warehouse is — then which shape their architecture document targets.

Say · cover

- Lake: cheap, schema-on-read, anything lands — but discipline is optional. Warehouse: schema enforcement and query performance. Lakehouse: both, on one copy.
- Their current state is a classic warehouse plus file drops; the target is lakehouse — GCS Bronze under the same governance as BigQuery Gold.
- The point for their migration: one copy of data serving both the raw-and-replayable requirement and the mart-performance requirement.
- This is 60-second recall, not a reteach — M7 and M8 this morning made the vocabulary concrete.

Iowa tie

Their Bronze->Silver->Gold lakehouse pattern is exactly this diagram: Bronze is the lake side (GCS, open formats), Gold is the warehouse side (BigQuery marts).

Design patterns

- One physical copy, two access modes — raw files and governed tables
- Warehouse semantics (schema, partitioning) for marts; lake semantics (open formats, immutability) for Bronze

Anti-patterns

- Two copies — lake for files, warehouse for tables — recreates the silo they are escaping
- Forcing warehouse schema-on-write onto raw county file drops

History & what's new

Databricks coined 'lakehouse' in a 2020 blog and the CIDR 2021 paper gave it academic footing; six years on, Google's 2026 docs title this same architecture 'Lakehouse for Apache Iceberg' — the term won.

Flow

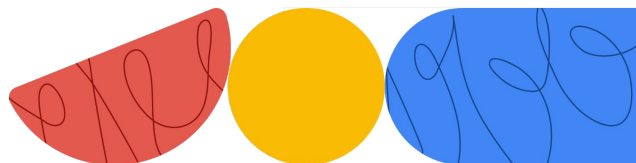
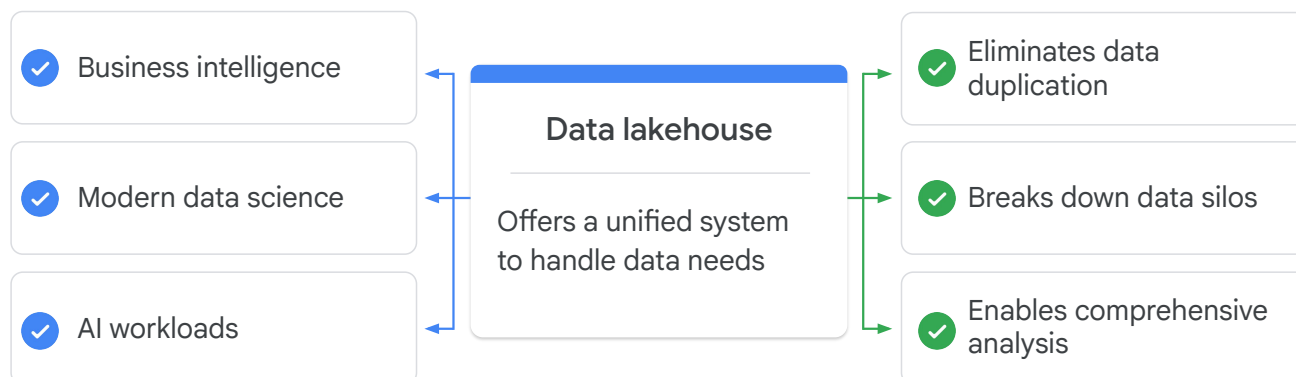
First retrace stop (vendor deck, C2-M6); next slide adds the lakehouse benefit bullets, then the per-service flyover.

Links

<https://cloud.google.com/discover/what-is-a-data-lakehouse>

<https://docs.cloud.google.com/lakehouse/docs>

<https://docs.cloud.google.com/bigquery/docs/introduction>



INSTRUCTOR NOTES — PAGE 191 OF 208 · 15:50 · COURSE SUMMARY & NEXT STEPS

Hook

Six benefit bubbles — translate each into their world: 'eliminates data duplication' is the death of the nightly reporting copy.

Say · cover

- BI, data science, and AI workloads off one copy: Power BI authors and future ML work read the same Gold tables — no export jobs.
- 'Eliminates data duplication' = the reporting copy dies; BigQuery becomes the single source of truth their charter asks for.
- 'Breaks down data silos' = counties and agencies land in one governed place instead of per-team extracts.
- 'Enables comprehensive analysis' = SQL over files and tables alike — the federated and BigLake queries from this morning's lab.
- Reality-check for the room: the benefits are real, but M10 governance is what keeps one copy from becoming one swamp.

Iowa tie

This is the business-case slide for their single-source-of-truth objective — quote 'eliminates data duplication' when someone asks why the extract jobs have to die.

Design patterns

- Serve BI and ML from the same governed Gold tables
- Let the lake layer absorb new sources without spawning new silos

Anti-patterns

- One copy without governance becomes a swamp — pair lakehouse with M10 controls
- Keeping per-team export copies 'just in case' — recreates the duplication

History & what's new

The lakehouse keeps widening: Google's Lakehouse release notes show cross-cloud Lakehouse adding AWS Glue as a remote catalog in preview (June 2026) — one SQL layer reaching into another cloud without moving data.

Flow

Benefits done; p187 turns to the Google Cloud services that deliver this — the stack flyover.

Links

<https://cloud.google.com/discover/what-is-a-data-lakehouse>

<https://docs.cloud.google.com/lakehouse/docs>

<https://docs.cloud.google.com/lakehouse/docs/release-notes>

How Google Cloud services come together to form a modern, efficient, and scalable data architecture.

Google Cloud

 INSTRUCTOR NOTES — PAGE 192 OF 208 · 15:50 · COURSE SUMMARY & NEXT STEPS

Hook

The two-day stack on one wall — as each service appears, have the room call out which layer of their capstone map it fills.

Say · cover

- Run it as call-and-response: GCS = Bronze, Iceberg = open table format, BigQuery = warehouse engine, lakehouse tables = one governance layer, catalog + protection = discover and defend, BQML/Agent = after go-live.
- Console geography one last time: Cloud Storage browser for buckets, BigQuery Studio for datasets and models, Knowledge Catalog and Sensitive Data Protection under governance.
- Keep pace fast — every service had a module, DIN, or lab; this is recall, and capstone maps are still warm.
- Land the punchline: nothing on this wall required them to buy new BI — Power BI sits on top unchanged.

Iowa tie

Each stop is a named piece of their target: GCS/Iceberg Bronze, Dataform Silver->Gold, BigQuery marts, Knowledge Catalog for the single source of truth, Power BI serving.

Design patterns

- Retrace services against the audience's own architecture, not in vendor order
- Say where each service lives in the console — their platform anxiety is console-level

Anti-patterns

- Rushing the retrace into a feature list with no 'your layer' mapping
- Introducing preview features here — stick to what they actually touched

History & what's new

Fresh names: in April 2026 Google renamed BigLake to Lakehouse for Apache Iceberg (Apr 20) and Dataplex Universal Catalog to Knowledge Catalog (Apr 10); APIs/CLI keep old names, so docs in two vocabularies are both right.

Flow

Section header for the flyover; p188–p198 take one service each, then the custom two-day retrace at p200.

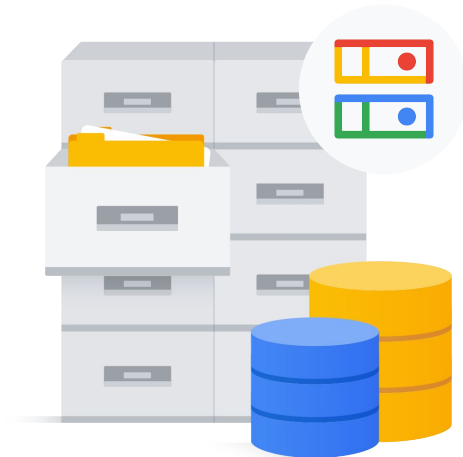
Links

<https://docs.cloud.google.com/bigquery/docs/introduction>
<https://docs.cloud.google.com/bigquery/docs/biglake-intro>
<https://docs.cloud.google.com/knowledge-catalog/docs/introduction>

Cloud Storage

The foundational, cost-effective object storage for all raw data, including

- ✓ Structured
- ✓ Semistructured
- ✓ Unstructured
- ✓ Multimodal



Google Cloud

INSTRUCTOR NOTES — PAGE 193 OF 208 · 15:50 · COURSE SUMMARY & NEXT STEPS

Hook

Their Bronze layer in one sentence: an object store that costs almost nothing, keeps files immutable, and every GCP data service can read.

Say · cover

- GCS is the foundational object store — structured, semi-structured, unstructured; county CSV drops and Datastream snapshots both land here.
- Console: Cloud Storage -> Buckets. Layout matters more than the bucket — Hive-style county=X/period=YYYY-MM/ prefixes make partitions queryable.
- Storage classes (Standard -> Nearline -> Coldline -> Archive) are the cost dial for Bronze retention; originals stay immutable and replayable.
- Files here are one step from BigQuery: external table for immediate query, LOAD DATA for the conformed Bronze copy — Day 1's DIN block.

lowa tie

This is where the file-drop extracts (the SSIS SFTP-sweep pattern) retire to: counties drop to a bucket, or Storage Transfer sweeps the existing share — no server, no package.

Design patterns

- Hive-style key prefixes (county=/period=) so loads and external tables partition naturally
- Keep original files immutable — replayability is Bronze's whole job

Anti-patterns

- One flat bucket with no prefix discipline — an unqueryable dumping ground
- Overwriting raw files in place — kills replay and audit

History & what's new

Cloud Storage launched May 2010 — Google's second cloud service — and still ships monthly: the April 2, 2026 release notes added bucket-level control over which encryption types are allowed for new objects.

Flow

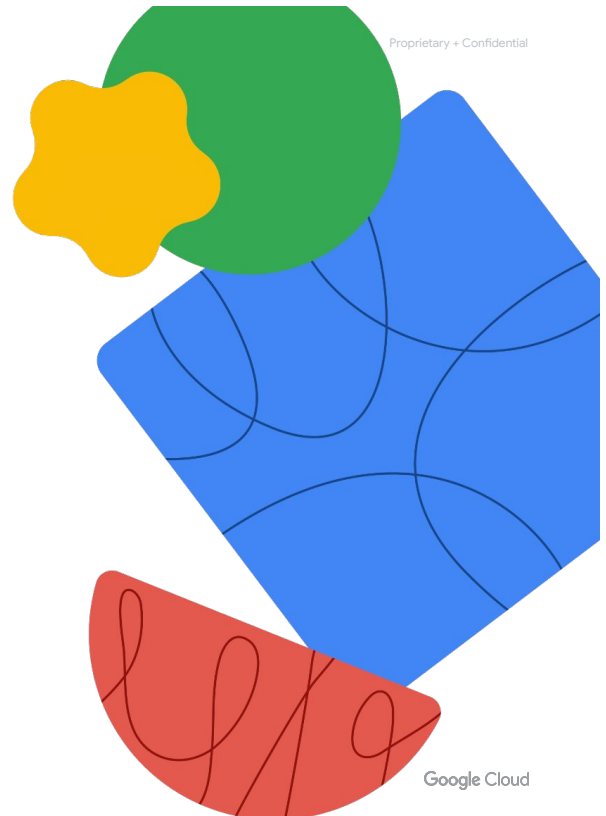
First service in the flyover; p189 layers Apache Iceberg table structure on top of these files.

Links

<https://docs.cloud.google.com/storage/docs/introduction>
<https://docs.cloud.google.com/storage/docs/storage-classes>
<https://docs.cloud.google.com/lakehouse/docs>

Apache Iceberg

A data lakehouse needs structure and performance. Open table formats, such as Apache Iceberg, provide this.



INSTRUCTOR NOTES — PAGE 194 OF 208 · 15:50 · COURSE SUMMARY & NEXT STEPS

Hook

Files alone are a swamp waiting to happen; Iceberg is the open table format that gives a GCS bucket database manners — this was the 13:45 BigLake upgrade demo.

Say · cover

- Iceberg = open table format over Parquet files in GCS: schema, partitions, and snapshots tracked as metadata, so engines see a table, not a pile of files.
- Open is the point: BigQuery, Spark, and other engines read the same tables — no format lock-in, matching their multi-tool future.
- In their stack Iceberg belongs at the Bronze/Silver boundary: raw files gain ACID behavior and schema evolution without leaving the bucket.
- Recall the demo: converting a BigLake table to a managed Iceberg table without rewriting any data.

Iowa tie

This is their open Bronze option: county files or Datastream exports in GCS, queryable and governed as Iceberg tables before Dataform ever runs.

Design patterns

- Open formats (Parquet + Iceberg) at Bronze so any engine can read later
- Let the catalog track partitions and files instead of hand-maintained manifests

Anti-patterns

- Querying raw CSV/Parquet directories forever — no ACID, no schema evolution
- Proprietary-only formats at Bronze — locks the raw layer to one engine

History & what's new

Netflix created Iceberg in 2017 and it became an Apache top-level project in 2020; Google shipped BigLake managed Iceberg tables in 2024 and they matured to GA through 2025 with automatic maintenance.

Flow

Iceberg introduced; p190 enumerates the four properties it buys them.

Links

<https://iceberg.apache.org/>
<https://docs.cloud.google.com/bigquery/docs/biglake-iceberg-tables-in-bigquery>
<https://docs.cloud.google.com/lakehouse/docs>

What does Apache Iceberg provide?

01

Schema evolution

Allows schema changes (such as adding/renaming columns) without impacting existing data or applications.

02

Hidden partitioning

Automatically manages how data is partitioned for efficient querying.

03

Time travel

Enables querying past data versions for auditing and debugging, allowing users to reproduce exact data states from previous points in time.

04

Atomic transactions

Enable reliable, concurrent data operations, preventing corruption when multiple processes read/write to the same tables.

Google Cloud

INSTRUCTOR NOTES — PAGE 195 OF 208 · 15:50 · COURSE SUMMARY & NEXT STEPS

Hook

Four features on the slide — each answers a pain their SSIS/SQL Server world handles with heroics or quietly ignores.

Say · cover

- Schema evolution: add or rename columns without rewriting data or breaking downstream — the county-that-changed-its-CSV problem, answered.
- Hidden partitioning: partitioning declared as metadata transforms, not folder conventions users must memorize — queries prune automatically.
- Atomic transactions: concurrent readers and writers never see a half-loaded table — replaces the stage-then-swap SSIS dance.
- Time travel: query any past snapshot for audit and debugging — their auditors will like this one.
- BigQuery speaks all four through managed Iceberg tables, including `FOR SYSTEM_TIME AS OF`.

Iowa tie

Schema evolution and time travel are the two they will quote back: county file drift and state audit requirements are their daily reality.

Design patterns

- Use time travel for audit and debug instead of snapshot copies
- Rely on atomic commits instead of stage-and-swap load patterns

Anti-patterns

- Emulating schema evolution with drop-and-reload — loses history
- Encoding partitions in folder names users must hard-code in `WHERE` clauses

History & what's new

Iceberg spec v2 (2021) added row-level deletes for CDC-style upserts; in 2025 Google's managed Iceberg tables added automatic compaction, clustering, and garbage collection — maintenance M&O doesn't have to schedule.

Flow

Iceberg properties done; p191 pivots from the lake side to the warehouse engine — BigQuery itself.

Links

<https://iceberg.apache.org/>
<https://docs.cloud.google.com/bigquery/docs/biglake-iceberg-tables-in-bigquery>
<https://docs.cloud.google.com/lakehouse/docs/release-notes>

BigQuery

Enterprise-grade, fully managed, serverless data warehouse.



Google Cloud

INSTRUCTOR NOTES — PAGE 196 OF 208 · 15:50 · COURSE SUMMARY & NEXT STEPS

Hook

The destination for every extract they kill: an enterprise warehouse with no instances, no patching, no capacity planning — the lightest page of M&O's future runbook.

Say · cover

- Serverless means no cluster to size: storage and compute scale independently; the analog of their SQL Server box simply does not exist to manage.
- Fully managed: HA, backups, and patching are Google's job — M&O watches freshness metrics and bytes-scanned dashboards instead of restarting services.
- Enterprise-grade: the same engine from their first Day-1 load to petabyte marts; SQL:2011, so T-SQL instincts transfer with dialect notes.
- Cost model: on-demand bytes scanned (1 TiB/month free) or flat-rate editions; bulk loads are always free — batch is the cheap default.

Iowa tie

This is the single source of truth their charter names: the SQL Server warehouse's replacement, which they already loaded twice yesterday (console load and a Datastream stream).

Design patterns

- Default to batch loads (free) unless freshness demands streaming
- Watch bytes scanned with custom quotas and dry-run before turning analysts loose

Anti-patterns

- Sizing conversations carried over from the VM world — there is nothing to size
- SELECT * habits on on-demand pricing — bytes scanned is the bill

History & what's new

Announced May 2010, GA November 2011 — 15 years old; at Next 2025 (April) Google folded in the BigQuery AI wave: LLM inference and vector search beside SQL in the same engine.

Flow

BigQuery introduced; p192–193 unpack the architecture diagram the vendor deck shows for it.

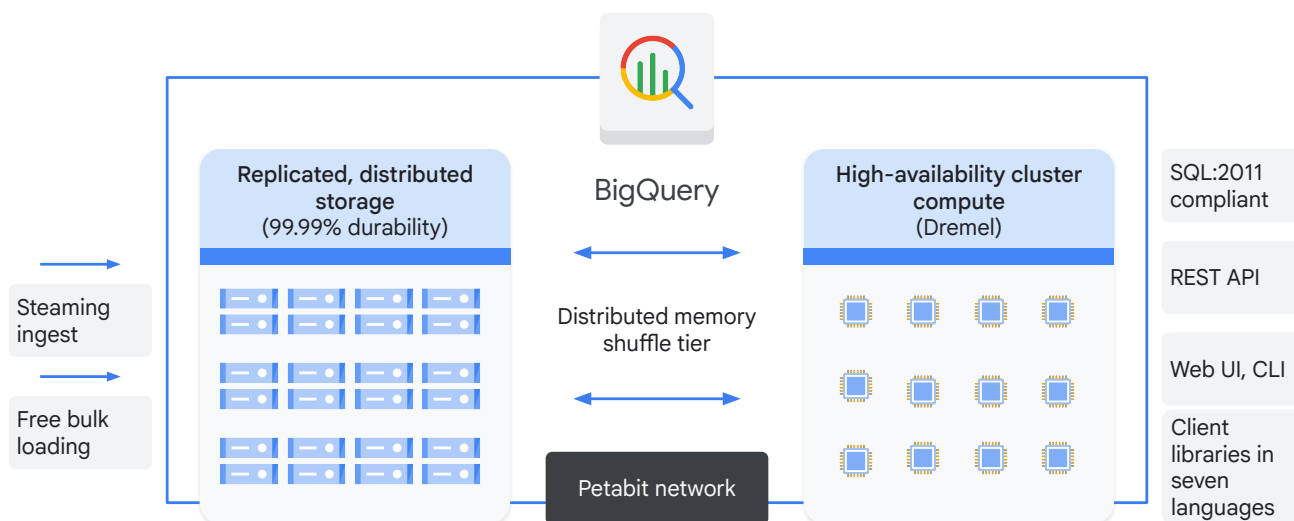
Links

<https://docs.cloud.google.com/bigquery/docs/introduction>

<https://cloud.google.com/blog/products/data-analytics/new-blog-series-bigquery-explained-overview>

<https://cloud.google.com/bigquery/pricing>

BigQuery fundamentals (1 of 2)



Google Cloud

INSTRUCTOR NOTES — PAGE 197 OF 208 · 15:50 · COURSE SUMMARY & NEXT STEPS

Hook

Why no indexes and still fast: the pieces on this slide — Dremel compute, replicated storage, shuffle, petabit network — answer every 'but how does it scale' question.

Say · cover

- Compute is Dremel: thousands of slots tree-execute your SQL in parallel; you rent them per query, there is no cluster.
- Storage is Colossus — replicated, distributed, with durability measured in nines you never manage; separation from compute is why the two scale independently.
- A distributed in-memory shuffle tier moves intermediate results between stages — the piece that makes big joins tractable.
- Ingest doors: free bulk loading plus the streaming Storage Write API. Front doors: web UI, bq CLI, REST, and client libraries in seven languages.

Iowa tie

Free bulk loading is why their nightly batch window translates so cheaply; the seven-language client libraries are for the .NET services sitting around the warehouse.

Design patterns

- Bulk load for batch; reserve streaming ingest for genuinely real-time feeds
- Write set-based SQL and let the engine tree-execute — no cursor loops

Anti-patterns

- Row-by-row inserts as a load pattern — wrong door, slow, and billed like streaming
- Assuming indexes exist — design with partition and cluster keys instead

History & what's new

The architecture was published as 'Dremel: A Decade of Interactive SQL Analysis at Web Scale' (VLDB 2020) — and the 2026 docs still describe the same disaggregated design this slide shows.

Flow

Architecture half one; p193 shows how the same engine reaches out to open-format data in GCS.

Links

<https://cloud.google.com/blog/products/data-analytics/new-blog-series-bigquery-explained-overview>
<https://www.vldb.org/pvldb/vol13/p3461-melnik.pdf>
<https://docs.cloud.google.com/bigquery/docs/introduction>

BigQuery fundamentals (2 of 2)

Through Lakehouse technology, BigQuery can directly query open-format data (such as Apache Iceberg tables) in Cloud Storage or other clouds, applying BigQuery's powerful optimizations, such as partitioning and clustering, by leveraging Apache Iceberg's metadata.



Google Cloud

INSTRUCTOR NOTES — PAGE 198 OF 208 · 15:50 · COURSE SUMMARY & NEXT STEPS

Hook

The plot twist of Day 2: the warehouse engine queries open Iceberg tables in GCS directly — lake and warehouse stop being two systems.

Say · cover

- BigQuery reads Iceberg metadata (snapshots, manifests) and applies its own optimizations — partition pruning, cluster-aware scans — to data it does not store.
- So Bronze files in a bucket get Dremel-speed SQL without a load step; loading becomes a choice about performance tier, not about access.
- Metadata-driven acceleration narrows managed-vs-external performance toward parity, per Google's 2025 lakehouse write-ups.
- Multi-cloud reach exists too (Omni, cross-cloud catalogs) — worth naming, not needed for their migration.

Iowa tie

This is the technical proof behind their architecture document: Bronze on GCS and Gold in BigQuery are one queryable estate — the Dataform Silver->Gold flow can start from either.

Design patterns

- Query in place first; load to managed storage when access patterns justify it
- Keep Iceberg metadata healthy (compaction) so external scans stay fast

Anti-patterns

- Loading everything into managed storage on day one — pays for a tier before the need exists
- Treating external tables as second-class — BigLake governance applies there too

History & what's new

Cross-engine momentum: June 2026 brought GA of Iceberg REST catalog integration from Snowflake into Google's Lakehouse runtime catalog — third-party engines now read the same tables BigQuery manages.

Flow

BigQuery architecture done; p194 names the table types that make this one estate — lakehouse and external tables.

Links

<https://docs.cloud.google.com/bigquery/docs/biglake-iceberg-tables-in-bigquery>
<https://docs.cloud.google.com/bigquery/docs/biglake-intro>
<https://docs.cloud.google.com/lakehouse/docs>

Lakehouse

01

Allows you to create Lakehouse tables over data in object stores (such as Cloud Storage).

02

External tables reference data lake files, enabling unified governance and performance without duplicating data storage.



Google Cloud

 INSTRUCTOR NOTES — PAGE 199 OF 208 · 15:50 · COURSE SUMMARY & NEXT STEPS

Hook

Two table types, one governance umbrella — the mechanism that lets Bronze live in cheap object storage without escaping IAM.

Say · cover

- Lakehouse (BigLake) tables: BigQuery tables over GCS files — one security and governance model, fine-grained column/row IAM, without copying data.
- External tables reference lake files directly — query without duplicating storage; this morning's federated lab proved it.
- The upgrade path from the 13:45 demo: external -> BigLake -> managed Iceberg, each step adding governance and maintenance without data rewrites.
- Connection model: a cloud resource connection holds the service account; file access flows from IAM, not shared keys.

Iowa tie

Their Bronze answer: county files stay in GCS under BigLake governance — M&O grants access with the same IAM roles as BigQuery tables, no SFTP permissions sprawl.

Design patterns

- Prefer BigLake tables over legacy external tables — governance without duplication
- One connection and service account per source, least-privilege IAM

Anti-patterns

- Copying files into managed storage just to get security — the duplication the lakehouse exists to kill
- Legacy external tables with broad bucket ACLs — bypasses fine-grained control

History & what's new

BigLake launched April 2022 at Google's Data Cloud Summit to unify lake and warehouse governance; on April 20, 2026 it was renamed Lakehouse for Apache Iceberg — same APIs, new name on the console door.

Flow

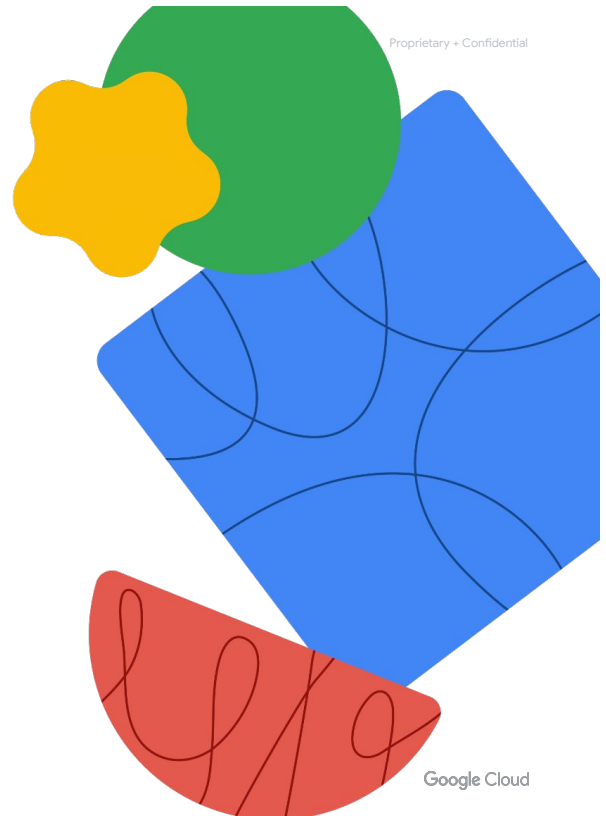
Storage and table layers done; p195–p196 cover the governance pair — Knowledge Catalog and Sensitive Data Protection.

Links

<https://docs.cloud.google.com/bigquery/docs/external-tables>
<https://docs.cloud.google.com/bigquery/docs/biglake-intro>
<https://docs.cloud.google.com/lakehouse/docs>

Knowledge Catalog

Unified data fabric and centralized metadata hub for discovering, managing, monitoring, and governing data across the data lakehouse.



INSTRUCTOR NOTES — PAGE 200 OF 208 · 15:50 · COURSE SUMMARY & NEXT STEPS

Hook

A single source of truth needs a single place to find it: the catalog is how 16 people — then 160 — discover which Gold mart is the official one.

Say · cover

- Centralized metadata hub across the lakehouse: automatic discovery of BigQuery datasets, tables, and lake files, searchable with business context.
- Aspects and tags carry the semantics — owner, sensitivity class, mart tier — the difference between a table existing and being findable.
- Data products (GA 2026) package governed, shareable data with contracts — the grown-up version of their 'official reporting copy'.
- Console: it lives with BigQuery governance; their Day-1 class dataset is already indexed there.

Iowa tie

This backs their single-source-of-truth requirement: M&O can answer 'who owns this mart, what is in it, is it current' without asking the developer who built it.

Design patterns

- Tag sensitivity and tier (Bronze/Silver/Gold) at dataset creation, not after audit asks
- Publish Gold marts as data products with named owners

Anti-patterns

- Catalog as afterthought — six months in, nobody knows which table is official
- Metadata owned by one hero developer instead of the platform

History & what's new

Dataplex launched May 2021; Data Catalog was deprecated February 2025; the service became Dataplex Universal Catalog (April 2025) and then Knowledge Catalog (April 10, 2026), with data products GA May 2026.

Flow

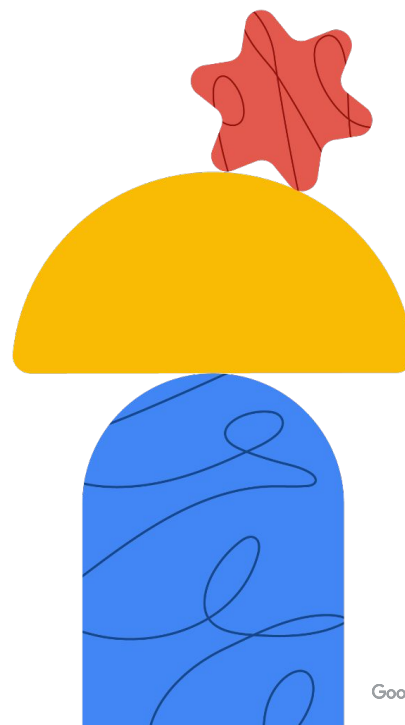
Discovery covered; p196 covers the protection half — Sensitive Data Protection.

Links

<https://docs.cloud.google.com/knowledge-catalog/docs/introduction>
<https://docs.cloud.google.com/knowledge-catalog/docs/release-notes>
<https://docs.cloud.google.com/knowledge-catalog/docs/transition-to-dataplex-catalog>

Sensitive Data Protection

Automates the discovery, classification, and protection of sensitive data using techniques, such as masking, tokenization, or redaction.



Google Cloud

INSTRUCTOR NOTES — PAGE 201 OF 208 · 15:50 · COURSE SUMMARY & NEXT STEPS

Hook

State government data means PII is a when, not an if — this is the scanner that finds the SSN column nobody documented before it reaches a mart.

Say · cover

- Discovery: scan BigQuery tables and GCS for 100+ built-in infoTypes — SSNs, names, addresses — with findings reported per column.
- Protection: masking, tokenization, redaction — de-identify before data reaches a wider audience or a lower tier.
- It pairs with the catalog: sensitivity findings become tags, tags become policy tags, policy tags enforce column-level access — discover, classify, enforce.
- In the medallion: scan at the Bronze/Silver boundary so Gold marts only ever hold what Power BI should see.

Iowa tie

Directly serves their compliance posture: counties send what they send; SDP finds the PII so Silver->Gold Dataform models can mask it deliberately, not hopefully.

Design patterns

- Scan Bronze on arrival; tag findings in the catalog; mask in Dataform Silver models
- Use tokenization when downstream needs joins without raw values

Anti-patterns

- Assuming sources are clean — county files always surprise
- Masking only in the BI layer — too late; Gold already leaked

History & what's new

Cloud DLP went beta March 2017 and GA March 2018, then was folded into the Sensitive Data Protection family in 2023 — inspection, de-identification, and discovery under one name.

Flow

Governance pair complete; p197–p198 close the flyover with the ML/agent layer they will grow into.

Links

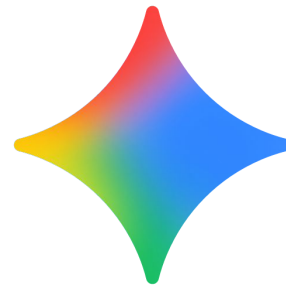
<https://docs.cloud.google.com/sensitive-data-protection/docs>

<https://cloud.google.com/security/products/dlp>

<https://docs.cloud.google.com/sensitive-data-protection/docs/deidentify-sensitive-data>

BigQuery ML and Agent Platform

Enable advanced analytics and ML directly on data lakehouse data.



Google Cloud

INSTRUCTOR NOTES — PAGE 202 OF 208 · 15:50 · COURSE SUMMARY & NEXT STEPS

Hook

Everything so far serves the reports they have; this slide is the door to the analytics they don't have yet — and it opens with plain SQL.

Say · cover

- BigQuery ML: CREATE MODEL -> ML.PREDICT inside SQL — their analysts' existing skill reaches regression, classification, forecasting, anomaly detection.
- Agent Platform (the renamed Vertex AI): the full custom-model and MLOps path for when BQML's built-ins are not enough — integrated with BigQuery data.
- Nothing moves: models train where the data lives — no export-to-notebook pipeline, matching the anti-duplication theme.
- Position for this room: show the door exists; their migration project does not need it on day one.

Iowa tie

Future tense for DOM DoIT: caseload forecasting or anomaly flags on the same Gold marts Power BI reads — no new platform, no new copy.

Design patterns

- Start with BQML for tabular problems; graduate to the agent/Vertex path only when needed
- Train on Gold marts — governed inputs, auditable models

Anti-patterns

- Exporting data to notebooks for ML — recreates the extract problem
- Chasing ML before the medallion is stable — models amplify data debt

History & what's new

BigQuery ML launched July 2018; at Cloud Next 2026 (April 22) Google consolidated Vertex AI and Agentspace into the Gemini Enterprise Agent Platform — the slide's 'Agent Platform' wording is this year's name.

Flow

ML layer introduced; p198 details the split between the SQL-simple and full-platform paths.

Links

<https://docs.cloud.google.com/bigquery/docs/bqml-introduction>
<https://docs.cloud.google.com/gemini-enterprise-agent-platform>
<https://cloud.google.com/blog/topics/google-cloud-next/google-cloud-next-2025-wrap-up>

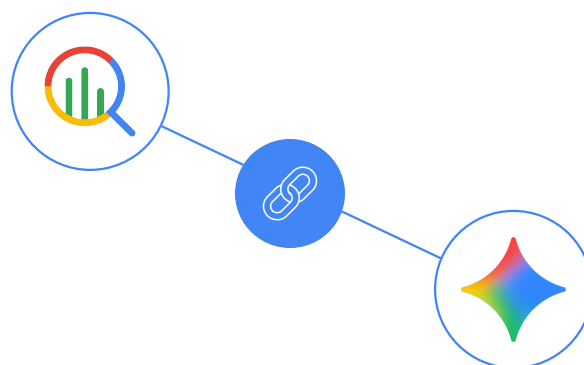
BigQuery ML and Agent Platform

01

BigQuery ML allows data analysts to build and deploy ML models using familiar SQL queries.

02

Agent Platform offers a comprehensive platform for complex custom models and MLOps, fully integrated with BigQuery.



Google Cloud

INSTRUCTOR NOTES — PAGE 203 OF 208 · 15:50 · COURSE SUMMARY & NEXT STEPS

Hook

Two rungs on one ladder: analysts stay in SQL for the common cases; data science gets the full platform for the hard ones — same governed data underneath.

Say · cover

- Rung 1 — BQML: the model lifecycle in SQL (CREATE MODEL, ML.EVALUATE, ML.PREDICT), versioned alongside the Dataform pipeline that builds its inputs.
- Rung 2 — Agent Platform: custom training, tuning, deployment, and MLOps, plus the 2026 agent stack for AI applications over their data.
- LLMs joined the SQL party: AI.GENERATE-style functions and remote models (Gemini, Claude, Llama) callable from queries — announced through 2025.
- For the migration, the only takeaway needed: Gold marts are ML-ready inputs — nothing to rebuild later.

Iowa tie

Keeps their options open: when leadership asks about 'AI', the answer is 'our marts already feed it' — BQML for quick wins, the agent platform for anything bespoke.

Design patterns

- Version models next to the Dataform release that produced the training data
- Call remote LLMs via SQL functions for text tasks before building bespoke pipelines

Anti-patterns

- Bespoke ML infrastructure before a first BQML baseline
- Training on ungoverned extracts — inherits every data-quality sin

History & what's new

At Next 2025 (April) BigQuery ML gained AI.GENERATE_TABLE in preview and support for open and Anthropic models (Claude, Llama, Mistral) in SQL; Next 2026 then rebranded Vertex AI as the Gemini Enterprise Agent Platform.

Flow

Flyover complete — p199 is the vendor divider, then p200 lands the custom two-day retrace.

Links

<https://docs.cloud.google.com/bigquery/docs/bqml-introduction>

<https://cloud.google.com/blog/topics/google-cloud-next/google-cloud-next-2025-wrap-up>

<https://docs.cloud.google.com/gemini-enterprise-agent-platform>



Course summary

Google Cloud

INSTRUCTOR NOTES — PAGE 204 OF 208 · 15:50 · COURSE SUMMARY & NEXT STEPS

FACILITATOR NOTE

- Vendor section divider for the Course-1 summary module — click past quickly; your custom retrace at p200 does this job keyed to their migration.
- Keep energy up: the room has been building capstone maps for 30 minutes, so the retrace should run under ten minutes, not become a third lecture.

Flow: Divider between the services flyover and the custom 'two days, end to end' slide at p200.

The two days, end to end

- ✓ Day 1 AM— console, projects, IAM, your class dataset; your first BigQuery load in a new console
- ✓ Day 1 PM— Datastream CDC killed the nightly extract; three load paths; pin schemas; CSV vs Parquet
- ✓ Day 1 close— Dataform owns Silver→Gold in git; Scheduler + release configs = SQL Agent, translated
- ✓ Day 2— lakehouse + Iceberg Bronze; governance and the medallion matched your architecture doc
- ✓ The spine— kill the extracts → land in BigQuery → Bronze/Silver/Gold → Dataform → Power BI

Custom slide · added by Imran Ahmad

Iowa DOM DoIT · Data Engineering on Google Cloud

INSTRUCTOR NOTES — PAGE 205 OF 208 · 15:50 · COURSE SUMMARY & NEXT STEPS

Hook

One slide, whole course: read it top to bottom and you have retold their migration story — the recall pass they will repeat to their manager on Wednesday.

Say · cover

- Day 1 AM: console, projects, IAM, the class dataset — platform fear retired with a first load in minutes.
- Day 1 PM: Datastream CDC killed the nightly extract; three load paths; pin schemas; CSV vs Parquet — the loading discipline.
- Day 1 close: Dataform owns Silver→Gold in git; Scheduler + release configs = SQL Agent translated — their CI/CD discipline applies.
- Day 2: lakehouse + Iceberg Bronze, governance and the medallion matched their architecture document, and the Power BI last mile closed.
- Say the spine aloud: kill the extracts -> land in BigQuery -> Bronze/Silver/Gold -> Dataform -> Power BI.

Iowa tie

Every bullet is a piece of their project plan: ingest (Datastream or loads), Bronze/Silver/Gold, Dataform under CI/CD, M&O's Scheduler, and the Power BI they already own.

Design patterns

- Retell the course as their migration story, not as module numbers
- End on the spine sentence — it is the one they will reuse upward

Anti-patterns

- A chronological feature dump with no through-line
- Skipping the honest 'what changes Monday' beat

History & what's new

The spine is also the product timeline: BigQuery GA 2011, Dataform acquired 2020, Datastream SQL Server GA 2024, managed Iceberg GA through 2025 — the course exists because the pieces finished maturing.

Flow

Custom retrace delivered; p201 sends them off with the follow-on course and free practice paths.

Links

<https://cloud.google.com/blog/products/data-analytics/datastream-makes-it-easy-to-replicate-sql-server-databases>

<https://docs.cloud.google.com/dataform/docs/overview>

<https://roitraining.github.io/gcp-demos/bigquery.html>

Where you go from here

- Data Warehousing with BigQuery— 3 days, being scheduled; goes deep on everything Day 2 previewed
- Your capstone map— photograph it tonight; it's backlog draft zero for the migration project
- Reference cards in your student pack— keep-forever desk aids, incl. the Power BI connector guide
- Practice free— 1 TiB/month query free tier, bigquery-public-data, the DIN page, dataform-demo repo
- Optional homework— the serverless Spark lab from Day 1, still waiting in your Skills classroom
- Evaluations— two minutes before you leave; they shape what we schedule for you next

Custom slide · added by Imran Ahmad

Iowa DOM DoIT · Data Engineering on Google Cloud

INSTRUCTOR NOTES — PAGE 206 OF 208 · 15:50 · COURSE SUMMARY & NEXT STEPS

Hook

The course ends but the project starts: everything on this slide keeps momentum alive until migration kickoff — and most of it costs nothing.

Say · cover

- Data Warehousing with BigQuery (3 days, being scheduled): partitioning/clustering, optimization and cost, Iceberg, governance — everything Day 2 previewed, in depth; tell your coordinator you want seats.
- Photograph the capstone map tonight — backlog draft zero; the pack's reference cards (including the Power BI connector guide) are the keep-forever desk aids.
- Practice free: 1 TiB/month query free tier, bigquery-public-data, the DIN page's untouched sections (time travel, caching, clustering), and the public dataform-demo repo to rebuild in a scratch project.
- Optional homework: the serverless Spark lab still waiting in their Skills classroom — the 'when SQL is not enough' escape hatch.
- Two minutes for evaluations before they leave — they shape what gets scheduled next.

Iowa tie

Their continuity plan: follow-on course for depth, the capstone map for the backlog, and free-tier practice so 16 people stay warm until kickoff.

Design patterns

- Assign practice against the free tier before skills decay
- One scratch project per developer for safe experimentation

Anti-patterns

- A six-month gap before the next training — skills evaporate
- Practicing in the production project — use scratch or sandbox

History & what's new

The free path is real: Google introduced the BigQuery sandbox in February 2019 (1 TB queries + 10 GB storage monthly, no credit card) — still current in 2026, and public datasets mean practice storage costs nothing.

Flow

Final content slide; p202 is the thank-you/evaluation card, p203 the back cover.

Links

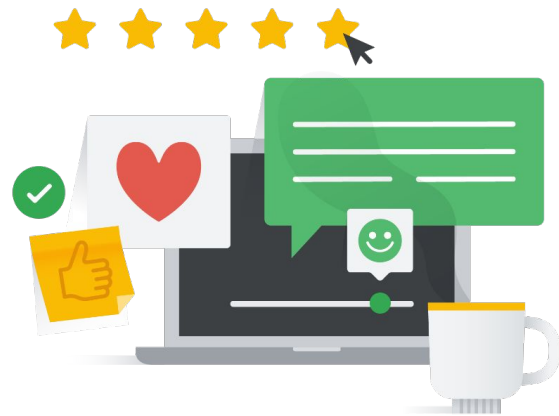
<https://cloud.google.com/learn/training>

<https://cloud.google.com/blog/products/data-analytics/query-without-a-credit-card-introducing-bigquery-sandbox>

<https://github.com/jwdavis/dataform-demo>

Thank you for attending this training!

We love your feedback! Please take a minute to complete the survey and help us improve our courses.



Google Cloud

INSTRUCTOR NOTES — PAGE 207 OF 208 · 15:50 · COURSE SUMMARY & NEXT STEPS

FACILITATOR NOTE

- Vendor thank-you card with the course survey — give the evaluations two honest minutes and say plainly that responses shape what gets scheduled for DOM DoIT next (the 3-day Data Warehousing with BigQuery course is being scheduled now).
- One last reminder before they log off: photograph the capstone map and email it to yourself — it is backlog draft zero.

Flow: Thank-you and survey; p203 is the blank back cover — the session is done.



INSTRUCTOR NOTES — PAGE 208 OF 208 · 15:50 · COURSE SUMMARY & NEXT STEPS

FACILITATOR NOTE

- Back cover with the confidentiality mark only — nothing to teach.
- Use any remaining minutes for open questions; the flex block officially runs to 16:30. End by pointing to the student pack's reference folder and the follow-on Data Warehousing with BigQuery course.

Flow: End of deck and end of the two-day delivery.